INTERVIEW PREP · 2026-10-13

# Interview Prep Book

Cheat sheet up front. Depth on every page.

55 study pages

6 design problems

# Contents


**At a Glance** ..... 5

**Chapter 1 · Python**

Python Overview ..... 13
Async / Await ..... 16
Class vs Instance Attributes & Methods ..... 19
Concurrency Models (GIL, Threads, Processes, asyncio) ..... 21
Generators & Iterators ..... 26
Closures, Decorators & Context Managers ..... 31
Python Gotchas (Predict the Output) ..... 36
OOP Internals & Memory Model ..... 42

**Chapter 2 · Data Structures**

Data Structures Overview ..... 52
Python Data Structures Map ..... 53

**Chapter 3 · Algorithm Patterns**

Algorithm Patterns Overview ..... 66
Pattern Recognition Map ..... 68
Big-O Cheat Sheet ..... 80
Two Sum / Complement Lookup ..... 84
How to Solve a Coding Question ..... 86

**Chapter 4 · System Design**

System Design Overview ..... 90
How to Run a System Design Interview ..... 92
Kafka vs RabbitMQ ..... 96
The 4-Step Framework (Alex Xu, with a News Feed Example) ..... 100
Redis Pub/Sub vs RabbitMQ vs Kafka ..... 105
Celery with Redis as Broker ..... 107

**Chapter 5 · Databases**

Databases Overview ..... 109
Indexes ..... 111
Transactions & Isolation Levels ..... 116
Locking & Concurrency Control ..... 121
SQL vs NoSQL & Scaling Databases ..... 126

**Chapter 6 · Networking**

Networking Overview ..... 132
HTTP/1.1 vs HTTP/2 vs HTTP/3 ..... 133

Web & Networking Fundamentals ... 137

**Chapter 7 · Go**

Go Overview ... 142
Go Essentials & Gotchas ... 143

**Chapter 8 · Django**

Django Overview ... 149
Django Interview Questions ... 152
Django ORM in Depth ... 158
Request Lifecycle, Middleware & Signals ... 166
Transactions & Migrations ... 172
Django REST Framework ... 178
Django in Production: Performance, Security, Async ... 185

**Chapter 9 · System Design Case Studies**

System Design Case Studies Overview ... 192
V1·4 Rate Limiter ... 201
V1·5 Consistent Hashing ... 207
V1·6 Key-Value Store ... 211
V1·7 Unique ID Generator ... 218
V1·8 URL Shortener ... 222
V1·9 Web Crawler ... 227
V1·10 Notification System ... 232
V1·11 News Feed System ... 237
V1·12 Chat System ... 242
V1·13 Search Autocomplete ... 248
V1·14 YouTube ... 253
V1·15 Google Drive ... 259
V2·1 Proximity Service ... 266
V2·2 Nearby Friends ... 273
V2·3 Google Maps ... 279
V2·4 Distributed Message Queue ... 285
V2·5 Metrics Monitoring and Alerting ... 293
V2·6 Ad Click Event Aggregation ... 300
V2·7 Hotel Reservation System ... 307
V2·8 Distributed Email Service ... 313
V2·9 S3-like Object Storage ... 321
V2·10 Real-time Gaming Leaderboard ... 329
V2·11 Payment System ... 335

V2·12 Digital Wallet ..... 342
V2·13 Stock Exchange ..... 349

**Design Problems**

Design Problems Overview ..... 356
Chat System ..... 358
Notification System ..... 364
Order Book and Matching Engine ..... 369
Rate Limiter ..... 376
URL Shortener ..... 382
Wallet and Ledger (Crypto Exchange) ..... 388

**Index of Terms** ..... **394**

# At a Glance

CHAPTER 1 · PYTHON

### Async / Await

- `await` waits; it doesn't start new work. Two `await`s in a row run **sequentially** (time = sum).
- Concurrent = schedule first: `asyncio.gather(...)` / `create_task(...)` (time = max).
- Calling a coroutine function runs nothing until it's awaited or turned into a task (JS `async` starts immediately).

### Class vs Instance Attributes & Methods

- Class attribute = one shared object; instance attribute (`self.x` in `__init__`) = one per object.
- Lookup: instance → class → MRO. **Assign** writes to the instance; **mutate** changes the shared object.
- Mutable class attr (`l = []`) is shared by every instance, the same bug as `def f(x=[])`.

### Concurrency Models (GIL, Threads, Processes, asyncio)

- GIL = only **one thread runs Python bytecode at a time** in a process. Threads still help I/O (GIL is released while waiting).
- I/O-bound, many connections → **asyncio**. I/O-bound with blocking libs → **threads**. CPU-bound → **processes**.
- `counter += 1` is **not atomic** across threads (read, add, write). Protect shared state with `threading.Lock`.

### Generators & Iterators

- **Iterable** has `__iter__` (list, dict, str). **Iterator** has `__next__` too and remembers its position.
- A function with `yield` returns a **generator**; its body runs only on `next()`, and pauses at each `yield`.
- Generators are **one-time**: after full consumption, a second loop gives nothing (`sum(g)` twice → second is `0`).

### Closures, Decorators & Context Managers

- Closure = inner function that **remembers variables** of the outer function (by reference, not by value).
- Decorator = `f = deco(f)`. It runs **once, at definition time**; the wrapper runs on every call.
- Decorator with args = 3 levels: `deco(arg)` → `decorator(func)` → `wrapper(*args, **kwargs)`. Always add `@functools.wraps(func)`.

### Python Gotchas (Predict the Output)

- `is` = same object, `==` = same value. Small ints (-5..256) are cached, so `is` "works" by accident. Use `is` only for `None`.
- `def f(x=[])`: the default list is created **once** and shared across calls. Use `x=None`.
- `[[0]*3]*3` = 3 references to **one** row. Use `[[0]*3 for _ in range(3)]`.

### OOP Internals & Memory Model

- Interface = **ABC** (explicit, runtime `TypeError`) or **Protocol** (implicit like Go, checked by mypy). A **mixin** adds behavior, it's not an interface; put it **left** of the base.
- `super()` = **next class in the MRO** of the object, not "my parent". Diamond `D(B, C)` → D, B, C, A.
- Define `__eq__` → `__hash__` becomes `None` (unhashable) unless you also define `__hash__`. Equal objects must have equal hashes.

CHAPTER 2 · DATA STRUCTURES

### Python Data Structures Map

- Queue/deque: `collections.deque`, with `popleft()` O(1). Never `list.pop(0)` (O(n)).
- Min-heap: `heapq` on a list, min at `h[0]`; max-heap: push `-x`; top-k: heap of size k.
- "Next free / earliest / k smallest" → heap of `(time, id)` tuples (car wash).

CHAPTER 3 · ALGORITHM PATTERNS

### Pattern Recognition Map

- "Have I seen X before?" / pair with sum → **hash set/dict**, O(n).
- Contiguous subarray/substring + "longest/shortest/at most k" → **sliding window**.
- "Next free / earliest / k largest" → **heap** (`heapq`); shortest path, unweighted → **BFS** with `deque`.

### Big-O Cheat Sheet

- ~$10^8$ simple ops/s in C, ~$10^7$ in Python → $n \leq 10^5$ needs O(n log n) or better.
- `x in list` is O(n); `x in set/dict` is O(1) avg. `list.pop(0)` / `insert(0)` are O(n) → use `deque`.
- Slicing copies: `a[i:j]` is O(j − i). `s += ch` in a loop can be $O(n^2)$ → `"".join(parts)`.

### Two Sum / Complement Lookup

- Give the answers in order: brute force $O(n^2)$ → sort + two pointers O(n log n) → **hash set O(n)**.
- For each `x`, check whether `target - x` is in `seen`, then add `x`. Use a dict `value → index` if indices are needed.
- Already sorted input: two pointers is O(n) with O(1) space.

### How to Solve a Coding Question

- Routine: **clarify → examples → brute force + Big-O → optimise (pattern map) → code → test → complexity**.
- Always ask: input size? duplicates? negatives? empty? sorted? return value or index?
- Say the brute force and its complexity **before** optimising. It is never wasted.

CHAPTER 4 · SYSTEM DESIGN

### How to Run a System Design Interview

- **Drive it in order:** requirements → estimates → API → data model → high-level design → deep dives → wrap-up.
- Spend the first ~5–8 min on **questions**, not boxes. Write down the functional + non-functional requirements and agree on them.
- Estimates: **1M requests/day ≈ 12 QPS**, peak ≈ 2–3× average; storage = items/day × size × retention.

### Kafka vs RabbitMQ

- RabbitMQ = **queue** (routed, deleted on ack). Kafka = **log** (retained, replayable, consumers track offsets).
- Task/job queue, complex routing, per-message retry/TTL/priority → **RabbitMQ**.
- Event stream, many independent consumers, replay, high throughput → **Kafka**.

### The 4-Step Framework (Alex Xu, with a News Feed Example)

- **4 steps in 45 min:** scope (3–10) → high-level design + buy-in (10–15) → deep dive (10–25) → wrap-up (3–5).
- **Don't be "Jimmy":** ask about features, users, growth and tech stack first; write the assumptions down.
- Get **buy-in** on the box diagram before going deep; walk through 1–2 **concrete use cases** to find edge cases.

### Redis Pub/Sub vs RabbitMQ vs Kafka

- Redis Pub/Sub = **live broadcast, nothing stored**: offline or slow subscribers miss messages (at-most-once, no ack).
- Every subscriber gets every message, so it's **not a work queue**.
- Use it for signals that are fine to lose: WebSocket fan-out across servers, typing/online indicators, cache invalidation, config reload.

### Celery with Redis as Broker

- Celery + Redis uses **Lists** for tasks (`LPUSH` to send, `BRPOP` to receive → one worker per task), **not** Pub/Sub.
- Pub/Sub is used only for broadcast control (`inspect` / `revoke`), monitoring events, and result notifications.
- No native acks → Kombu uses an `unacked` hash + `visibility_timeout` **(1h)**: a crashed worker's tasks come back only after ~1h.

CHAPTER 5 · DATABASES

### Indexes

- B-tree = sorted tree, O(log n): serves `=`, ranges, `ORDER BY`, prefix `LIKE 'abc%'`.
- Composite `(a, b, c)` works for `a`, `a,b`, `a,b,c` (leftmost prefix). Put **equality columns first, range column last**.
- Index is skipped when you wrap the column in a function, use `LIKE '%x'`, mismatch types, or the filter matches a big part of the table.

### Transactions & Isolation Levels

- ACID: all-or-nothing, rules kept, concurrent txns don't see each other's mess, committed = survives a crash (WAL).
- Postgres default = **Read Committed**: each statement sees a fresh snapshot.
- Postgres **Repeatable Read** = snapshot isolation: no phantoms either, but **write skew** is still possible.

### Locking & Concurrency Control

- Pessimistic: `SELECT ... FOR UPDATE` locks the row until commit; others wait. Good under **high contention**.
- Optimistic: `UPDATE ... SET version = version + 1 WHERE id = ? AND version = ?`; 0 rows updated → conflict, retry. Good under **low contention**.
- Simplest and best for counters/balances: one atomic `UPDATE ... SET balance = balance - 10 WHERE id = 1 AND balance ⩾ 10 RETURNING balance`.

### SQL vs NoSQL & Scaling Databases

- Default to **Postgres**; pick NoSQL for a specific need (huge write volume, flexible schema, simple key lookups, graph traversal).
- Scale reads with **replicas**; replicas are usually **async**, so expect **replication lag** → read-your-writes from the primary.
- **Partitioning** = split a table inside one server; **sharding** = split data across servers.

CHAPTER 6 · NETWORKING

### HTTP/1.1 vs HTTP/2 vs HTTP/3

- HTTP/1.1: keep-alive, but **one request at a time per connection** → browsers open ~6 connections per host.
- HTTP/2: binary frames, **many streams on one TCP connection**, HPACK headers. Still **TCP head-of-line blocking** on packet loss.
- HTTP/3: HTTP over **QUIC (UDP)**: per-stream loss recovery, TLS 1.3 built in (1-RTT, 0-RTT resume), **connection migration**.

### Web & Networking Fundamentals

- TCP = reliable, ordered, connection (3-way handshake). UDP = no handshake, no order, no retransmit (DNS, video, QUIC).
- URL → DNS → TCP handshake → TLS → HTTP request → server/app/DB → response → render.
- Safe = no side effects (GET, HEAD, OPTIONS). Idempotent = same result if repeated (safe + PUT, DELETE). **POST and PATCH are not.**

CHAPTER 7 · GO

### Go Essentials & Gotchas

- Goroutine = ~2 KB growable stack, scheduled by Go (GMP: many G on few OS threads M via P = GOMAXPROCS).
- Unbuffered send blocks until a receiver is ready; receive from a **closed** chan → zero value, `ok=false`; **nil** chan blocks forever.
- Only the **sender** closes a channel. Send on closed chan → panic.

CHAPTER 8 · DJANGO

### Django Interview Questions

- Count queries: lazy QuerySet, cache per object, FK in a loop = N+1, `.filter()` on a prefetched manager = new query.
- `update()` / `bulk_*` skip `save()` and save-signals; `on_commit` for Celery; `F()` for counters.
- `select_related` = JOIN (FK), `prefetch_related` = extra `IN` query (M2M / reverse FK).

### Django ORM in Depth

- A QuerySet is **lazy**: it hits the DB only when evaluated (iterate, `list()`, `len()`, `bool()`, step-slice, `repr`). Its results are cached on that object.
- `select_related` = SQL **JOIN** (FK / one-to-one, 1 query). `prefetch_related` = **separate query + Python join** (M2M / reverse FK, 1 + 1 per relation).
- N+1 shows up in loops and serializers. Catch it with `assertNumQueries` / debug-toolbar.

### Request Lifecycle, Middleware & Signals

- Flow: server (gunicorn/uvicorn) → WSGI/ASGI handler → **middleware (top→down)** → URL resolver → `process_view` → **view** → response → **middleware (bottom→up)**.
- Middleware is an **onion**: order in `MIDDLEWARE` matters (Session before Auth, Security first).
- CBV: `as_view()` creates a new instance per request, `dispatch()` picks the method handler.

### Transactions & Migrations

- Default is **autocommit**: every query commits alone. `atomic()` groups them; nested `atomic()` = **savepoint**.
- `select_for_update()` locks rows until the transaction ends; only works **inside** `atomic()`. `skip_locked=True` for job queues.
- Start Celery tasks with `transaction.on_commit(...)`, never directly inside a transaction.

### Django REST Framework

- Validation order: each field (`to_internal_value` + field validators) → `validate_<field>` → Meta/class `validators` → `validate(attrs)`.
- `ModelSerializer` does **not** write nested data; override `create()` / `update()`.
- `SerializerMethodField` that queries = **N+1**. Fix in `get_queryset()` with `select_related` / `prefetch_related` / `annotate`.

### Django in Production: Performance, Security, Async

- Cache layers: per-view (`cache_page`), template fragment (`{% cache %}`), low-level (`cache.get_or_set`). Hard part = **invalidation** (versioned keys, delete in `on_commit`).
- `CONN_MAX_AGE=0` (default) = new DB connection per request. Use `CONN_MAX_AGE=60` + `CONN_HEALTH_CHECKS`, or pgbouncer / Django 5.1 `"pool"`. Total connections = instances × workers × threads.
- Security: `DEBUG=False`, `ALLOWED_HOSTS`, secrets from env, HTTPS settings, never f-strings in `raw()`, `|safe` only on trusted HTML. Run `check --deploy`.

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

### V1·4 Rate Limiter

- **Server-side middleware + Redis counters:** check the counter, reject with **HTTP 429** if over the limit, else forward and increment.
- **5 algorithms:** token bucket (bursts, Amazon/Stripe), leaking bucket (fixed outflow, Shopify), fixed window (edge bursts up to 2×), sliding log (exact, memory-heavy), sliding counter (approx, cheap).
- **Sliding window counter:** current + previous × overlap%; Cloudflare saw only **0.003%** wrong of 400M requests.

### V1·5 Consistent Hashing

- `hash(key) % N` **breaks on resize:** change N and most keys move → a storm of cache misses.
- **Hash ring:** hash servers and keys onto the same ring; a key lives on the **first server clockwise**. Resizing moves only **~k/n keys**.
- **Virtual nodes** fix uneven partitions and skewed keys: 100–200 vnodes → std dev **10%–5%** of the mean. Trade-off: more metadata.

### V1·6 Key-Value Store

- **CAP:** partitions happen, so pick **CP** (block writes, e.g. banks) or **AP** (serve stale, sync later). CA can't exist in practice.
- **Partition** with consistent hashing; **replicate** to the first **N unique servers** clockwise, across data centers.
- **Quorum:** W + R > N → strong consistency (usually **N = 3, W = R = 2**). R = 1, W = N → fast reads; W = 1, R = N → fast writes.

### V1·7 Unique ID Generator

- **Snowflake 64-bit ID:** 1 sign bit + 41 bits timestamp (ms) + 5 datacenter + 5 machine + 12 sequence.
- **Numbers:** 41 bits of ms ≈ **69 years** from the epoch; 12 bits = **4,096 IDs per ms per machine**.
- Rejected: **multi-master** (step by k, not time-ordered), **UUID** (128-bit, not sortable), **ticket server** (single point of failure).

### V1·8 URL Shortener

- **Short code = base 62 of a unique ID.** 62^7 ≈ 3.5 trillion, so **7 chars** cover 365 billion URLs.
- **Numbers:** 100M new URLs/day → ~1,160 writes/s, ~11,600 reads/s (10:1), 365B records in 10 years.
- **301 vs 302:** 301 is cached by the browser (less server load); 302 hits you every time (better analytics).

### V1·9 Web Crawler

- **Loop:** seed URLs → frontier → download → parse → content seen? → extract links → filter → URL seen? → frontier.
- **Numbers:** 1B pages/month → **~400 QPS** (peak 800); 500 KB/page → **500 TB/month**, **30 PB** for 5 years.
- **BFS, not DFS**, but plain FIFO is impolite and ignores priority → **URL frontier**: front queues = priority, back queues = politeness (one host per queue).

### V1·10 Notification System

- **Servers → one queue per channel → workers → third party** (APNs, FCM, Twilio, SendGrid). One channel's outage does not block the others.
- Scale: **10M push + 1M SMS + 5M email per day**. Soft real-time: a small delay under load is fine.
- **Never lose a notification:** persist it in a notification log, and retry from the queue on failure. Delay or reorder is OK.

### V1·11 News Feed System

- **Two flows:** feed publishing (post → DB/cache → fanout to friends) and feed retrieval (read IDs from cache → hydrate).
- **Fanout on write (push)** = fast reads, but slow for users with many friends and wasted on inactive users. **Fanout on read (pull)** = no waste, but slow reads.
- **Hybrid:** push for most users; followers **pull celebrities'** posts on demand. Consistent hashing spreads hot keys.

### V1·12 Chat System

- **WebSocket** for both send and receive; plain **HTTP** for login, signup and profile. Chat servers are **stateful**, so pick one via **service discovery (ZooKeeper)**.
- Scale: **50M DAU**, groups ≤ **100**, text < **100,000 chars**, history kept **forever**. ~10 KB per connection → 1M connections ≈ 10 GB.
- History lives in a **KV store** (HBase, Cassandra): easy horizontal scaling, low latency, and a 1:1 read/write ratio.

### V1·13 Search Autocomplete

- **Trie with top-k cached at every node** + max prefix length → lookup is O(1). Trade space for time.
- **Numbers:** 10M DAU × 10 searches × 20 chars ≈ **24K QPS** (peak ~48K), 0.4 GB new data/day, reply in < **100 ms**.
- **Don't update the trie on every query:** logs → aggregators → workers rebuild the trie **weekly** → Trie DB → Trie Cache.

### V1·14 YouTube

- **Use the cloud:** blob storage for videos, **CDN** for streaming. API servers do everything else.
- **Numbers:** 5M DAU, 5 videos/day, 10% upload 300 MB → **150 TB/day** storage; CDN at $0.02/GB ≈ **$150K/day**.
- Upload: client → **pre-signed URL** → original storage → **transcoding (DAG)** → transcoded storage → CDN; a completion queue updates metadata.

### V1·15 Google Drive

- **Split files into ≤ 4 MB blocks** (Dropbox): block servers compress, encrypt, upload; **delta sync** sends only changed blocks.
- **Numbers:** 50M users × 10 GB = **500 PB** allocated; 10M DAU × 2 uploads → **~240 QPS**, peak 480.
- **Strong consistency** for metadata → relational DB (ACID) + invalidate cache on write.

### V2·1 Proximity Service

- **Map 2D to 1D:** a lat/lng index only helps one dimension. Split the map into grids (**geohash**, **quadtree**, **S2**) and index the grid key.
- Scale: **100M DAU**, **200M businesses**, 5 searches/user/day → **~5,000 search QPS**. Read-heavy; writes are rare and take effect **next day**.
- **Geohash** = easy, fixed grid size, easy updates, needs **8 neighbor cells** for boundary cases. **Quadtree** = in-memory tree (~**1.71 GB**), adapts to density, good for **k-nearest**, harder to update.

### V2·2 Nearby Friends

- Unlike proximity search, **locations change all the time**, so this is a **message-routing** problem, not an index problem.
- Scale: **100M DAU**, **10M concurrent**, update every **30 s** → **~334K updates/s**; × 400 friends × 10% online and nearby → **~13M pushes/s**.
- **WebSocket** servers + **Redis location cache** (user_id → lat/lng/ts, **TTL** = inactive timeout) + **Redis pub/sub, one channel per user**. Friends' handlers subscribe and filter by distance.

### V2·3 Google Maps

- Three features: **location updates**, **navigation + ETA**, **map rendering**. Mobile first, so **data and battery** matter.
- Scale: **1B DAU**; navigation ~**7,200 QPS** (peak 36K); GPS **batched every 15 s** → **200K QPS** (peak **1M**). Map tiles ≈ **100 PB** across 21 zoom levels.
- **Map tiles:** pre-generated, static, one per geohash per zoom level, served from a **CDN**. ~**1.25 MB/min** per navigating user.

### V2·4 Distributed Message Queue

- **Topic → partitions → brokers:** each partition is an append-only **FIFO log**; order holds only inside a partition; `hash(key) % numPartitions` picks it.
- **Storage = WAL segments on disk:** sequential I/O + OS page cache + **batching** everywhere + no message copying = high throughput.
- **Consumer group:** one partition → one consumer per group (keeps order). All consumers in one group = point-to-point; many groups = pub/sub.

### V2·5 Metrics Monitoring and Alerting

- **Scale:** 1,000 pools × 100 machines × 100 metrics ≈ **10M metrics**; keep raw **7 days**, 1-min rollup **30 days**, 1-hour rollup **1 year**.
- **Workload:** constant heavy writes, spiky reads → use a **time-series DB** (InfluxDB, Prometheus), not MySQL or a generic NoSQL.
- **Pull vs push:** no winner. Pull (Prometheus) = easy debug + health check; push (CloudWatch, Graphite) = short-lived jobs, firewalls. Big orgs support both.

### V2·6 Ad Click Event Aggregation

- **Pipeline:** log watcher → Kafka → **aggregation service (Map → Aggregate → Reduce)** → Kafka → DB writer → aggregation DB. Raw events also go to a raw DB.
- **Numbers:** 1B clicks/day → **10K QPS avg, 50K peak**; 0.1 KB/event → **100 GB/day, ~3 TB/month**; 2M ads; +30%/year.
- Use **event time + watermark** (e.g. 15 s) for accuracy; very late events are fixed by **end-of-day reconciliation**.

### V2·7 Hotel Reservation System

- **Book a room type, not a room.** Core table: `room_type_inventory(hotel_id, room_type_id, date, total_inventory, total_reserved)`, one row per date, 2 years ahead (~73M rows).
- **Numbers:** 5,000 hotels, 1M rooms, 70% occupied, 3-day stays → ~240K bookings/day ≈ **3 TPS**; funnel 300 → 30 → 3 QPS.
- **Same user double-clicks:** `reservation_id` as **idempotency key** (primary key, unique constraint).

### V2·8 Distributed Email Service

- **Numbers:** 1B users; send **100K QPS**; 40 emails/day received, 50 KB metadata → **730 PB/year**; 20% with 500 KB attachments → **1,460 PB/year**.
- Webmail → **web servers (HTTPS)** + **real-time servers (WebSocket, long-poll fallback)** → storage: **metadata DB, attachment store (S3), Redis cache, search store**.
- Send/receive both go through **message queues** (outgoing / incoming) so SMTP workers and mail processing (spam, virus) scale on their own.

### V2·9 S3-like Object Storage

- **Split metadata (mutable) from data (immutable):** metadata store maps name → UUID; data store only knows UUIDs.
- **Numbers:** 100 PB/year, 6 nines durability, 4 nines availability → **~0.68B objects**, ~0.68 TB metadata.
- **Pack small objects into big append-only files** (WAL-style); per-node SQLite maps UUID → (file, offset, size).

### V2·10 Real-time Gaming Leaderboard

- **Redis sorted set per month:** `ZINCRBY` on a win, `ZREVRANGE 0 9` for top 10, `ZREVRANK` for a user's rank, all **O(log n)**.
- **Numbers:** 5M DAU, 25M MAU, 10 games/day → **~500 score QPS**, peak **2,500**; **~650 MB** in Redis (26 B × 25M).
- SQL rank needs a sort over millions of changing rows: **10s of seconds**, and caching does not help.

### V2·11 Payment System

- **Pay-in** (buyer card → site's bank) and **pay-out** (site's bank → seller). A PSP moves the money; we never store card numbers.
- Only **10 TPS** (1M tx/day). The interview is about **correctness**, not throughput. Pick a proven **ACID relational DB**.
- **Exactly-once = at-least-once (retry) + at-most-once (idempotency key).** The payment_order_id is the PSP's dedupe key.

### V2·12 Digital Wallet

- **1M transfers/s = 2M DB ops/s.** At 1,000 TPS per node that is **2,000 nodes**, so raise per-node TPS.
- **Redis shards** are fast but not atomic across two nodes. A crash between "A−$1" and "C+$1" loses money.
- Distributed tx: **2PC** (DB-level, holds locks, coordinator SPOF) vs **TC/C** and **Saga** (app-level compensation). Always **deduct first**, then add.

### V2·13 Stock Exchange

- **1B orders/day over 6.5 h ≈ 43K QPS, peak ~215K.** 100 symbols, tens of thousands of users, limit orders only.
- Critical path: **gateway → order manager (risk, wallet) → sequencer → matching engine**. Market data and reporting are off the path.
- **Sequencer** stamps every order and fill with a gap-free ID, so the **deterministic** matching engine can replay and recover.

DESIGN PROBLEMS

### Chat System

- Separate the **stateful gateway** (WebSockets) from **stateless** chat logic; estimate connections per server.
- Store the message **before** acking "sent"; real-time push is best-effort and backed by **sync from last seq**.
- **Per-conversation sequence numbers** for ordering, not client clocks; no need for global order.

### Notification System

- Accept with **202** and process async; make the **idempotency key required** for callers.
- **Queue per channel and per priority** so OTPs never wait behind a campaign.
- Retries with **exponential backoff + jitter**, separate retryable vs non-retryable errors, **DLQ** with replay.

### Order Book and Matching Engine

- Define **price-time priority** and trade at the **maker's price**.
- Data structures: **sorted price levels + FIFO doubly linked list per level + order_id hash map** (O(1) cancel); give complexities.
- **Single-threaded engine per market** fed by **one ordered log** (sequencer) → no locks, deterministic.

### Rate Limiter

- Compare **all five algorithms** in one table and name the fixed-window **edge burst** problem.
- Pick **token bucket** (bursts OK, 2 fields per key) and say why not sliding log (memory).
- Show the **race condition** and solve it with an **atomic Lua script**; mention `EVALSHA` and single-slot keys in Redis Cluster.

### URL Shortener

- Do the math: **7 base62 chars ≈ 3.5 trillion codes**; 100:1 read:write; hot set fits in RAM.
- Compare ID strategies (hash / random / counter / Snowflake) and explain why **counter ranges + shuffle** avoid both collisions and guessable codes.
- Explain **301 vs 302** and connect it to the analytics requirement.

### Wallet and Ledger (Crypto Exchange)

- **Double-entry**: every transfer sums to zero; balances are a cache of entries and can be rebuilt.
- **Never use float**; integer smallest units or `NUMERIC`.
- Prevent double spend with **atomic conditional UPDATE** (or `SELECT FOR UPDATE` in fixed order) + `CHECK (amount ⩾ 0)`.

# 01
# Python

7 pages. Each card says what the page covers and its key lines; open the page for the full depth.

### Async / Await · p. 16

cooperative concurrency on **one thread**. An *event loop* runs many tasks by switching between them only at `await` points.

- `await` waits; it doesn't start new work. Two `await`s in a row run **sequentially** (time = sum).
- Concurrent = schedule first: `asyncio.gather(...)` / `create_task(...)` (time = max).
- Calling a coroutine function runs nothing until it's awaited or turned into a task (JS `async` starts immediately).
- One thread, interleaving only at `await`: good for I/O, useless for CPU work (use processes).
- Any blocking call (`time.sleep`, `requests`) freezes the whole loop.
- `await` frees the loop **only if it actually suspends**. Awaiting a coroutine with no real wait runs synchronously, and code between two awaits is atomic.

Read the full page →

### Class vs Instance Attributes & Methods · p. 19

Python objects have two storage levels:

- Class attribute = one shared object; instance attribute (`self.x` in `__init__`) = one per object.
- Lookup: instance → class → MRO. **Assign** writes to the instance; **mutate** changes the shared object.
- Mutable class attr (`l = []`) is shared by every instance, the same bug as `def f(x=[])`.
- `@classmethod` gets `cls` (alternative constructors); `@staticmethod` gets nothing (a utility).

Read the full page →

## Concurrency Models (GIL, Threads, Processes, asyncio) · p. 21

Python has three ways to do many things "at once": **threads**, **processes**, and **asyncio**. Which one helps depends on whether the work is waiting (I/O-bound) or computing (CPU-bound), and on the **GIL**.

- GIL = only **one thread runs Python bytecode at a time** in a process. Threads still help I/O (GIL is released while waiting).
- I/O-bound, many connections → **asyncio**. I/O-bound with blocking libs → **threads**. CPU-bound → **processes**.
- `counter += 1` is **not atomic** across threads (read, add, write). Protect shared state with `threading.Lock`.
- `concurrent.futures`: same API for `ThreadPoolExecutor` and `ProcessPoolExecutor`; just swap the class.
- Blocking call inside async code → `await asyncio.to_thread(fn, ...)`.
- 3.13+ has an optional **free-threaded** build (`python3.13t`, no GIL); default CPython still has the GIL.

Read the full page →

## Generators & Iterators · p. 26

an **iterator** gives values one at a time with `next()`. A **generator** is the easy way to write an iterator: a function that uses `yield`.

- **Iterable** has `__iter__` (list, dict, str). **Iterator** has `__next__` too and remembers its position.
- A function with `yield` returns a **generator**; its body runs only on `next()`, and pauses at each `yield`.
- Generators are **one-time**: after full consumption, a second loop gives nothing (`sum(g)` twice → second is `0`).
- `(x for x in ...)` is lazy, O(1) memory; `[x for x in ...]` builds the whole list.
- `yield from sub` = yield every item of `sub` (and get its `return` value).
- `groupby` only groups **consecutive** equal keys: sort first.

Read the full page →

## Closures, Decorators & Context Managers · p. 31

a **closure** is a function that keeps access to variables from the scope where it was created. A **decorator** uses a closure to wrap a function with extra behavior. A **context manager** wraps a block of code with setup/cleanup (`with`).

- Closure = inner function that **remembers variables** of the outer function (by reference, not by value).
- Decorator = `f = deco(f)`. It runs **once, at definition time**; the wrapper runs on every call.
- Decorator with args = 3 levels: `deco(arg)` → `decorator(func)` → `wrapper(*args, **kwargs)`. Always add `@functools.wraps(func)`.
- Stacked `@a @b def f` = `a(b(f))`: `a`'s wrapper runs first on call.
- `with` calls `__enter__`, then `__exit__` **always**, even on exception. `__exit__` returning `True` swallows the exception.
- Lambdas in a loop see the **last** value of `i` → `[2, 2, 2]`. Fix: `lambda i=i: i`.

Read the full page →

## Python Gotchas (Predict the Output) · p. 36

short snippets where Python does something surprising. For each: the code, the answer, and one line of why.

- `is` = same object, `==` = same value. Small ints (-5..256) are cached, so `is` "works" by accident. Use `is` only for `None`.
- `def f(x=[])`: the default list is created **once** and shared across calls. Use `x=None`.
- `[[0]*3]*3` = 3 references to **one** row. Use `[[0]*3 for _ in range(3)]`.
- `round(2.5) == 2` (banker's rounding); `-7 // 2 == -4` (floor, not truncate).
- `return` in `finally` **overrides** the `try`'s return and even swallows exceptions.
- `t = ([1],); t[0] += [2]` raises `TypeError` **and** the list is changed.

Read the full page →

## OOP Internals & Memory Model · p. 42

how Python classes work under the hood: method lookup (MRO), special "dunder" methods, attribute control (`property`, `__slots__`), object creation (`__new__`), and how memory is managed.

- Interface = **ABC** (explicit, runtime `TypeError`) or **Protocol** (implicit like Go, checked by mypy). A **mixin** adds behavior, it's not an interface; put it **left** of the base.
- `super()` = **next class in the MRO** of the object, not "my parent". Diamond `D(B, C)` → D, B, C, A.
- Define `__eq__` → `__hash__` becomes `None` (unhashable) unless you also define `__hash__`. Equal objects must have equal hashes.
- `__new__` **creates** the object (gets `cls`), `__init__` **fills** it (gets `self`). Override `__new__` for singletons / immutable subclasses.
- `__slots__` = no per-instance `__dict__`: less memory, no new attributes.
- Memory: names are references. Refcount frees most objects at once; the cyclic **GC** handles cycles (`a.x = b; b.x = a`).
- Args are passed by **object reference**: mutating a param is visible to the caller, reassigning it isn't.

Read the full page →

CHAPTER 1 · PYTHON

# Async / Await

async await Future asyncio.sleep(0) asyncio.to_thread run_in_executor asyncio.Lock asyncio coroutine event loop asyncio.gather asyncio.create_task Promise.all concurrency vs parallelism blocking call

SEE ALSO Python Data Structures Map · Concurrency Models (GIL, Threads, Processes, asyncio) · Closures, Decorators & Context Managers · Generators & Iterators · Celery with Redis as Broker · Go Essentials & Gotchas · Django in Production: Performance, Security, Async

**What:** cooperative concurrency on **one thread**. An *event loop* runs many tasks by switching between them only at `await` points.

> **i WHY IT MATTERS**
> it's the standard way to handle lots of I/O (HTTP, DB, sockets) without threads. Interviewers test whether you know what actually runs concurrently.

### The pieces:

| Term | Meaning |
|---|---|
| `async def f()` | Defines a coroutine function. Calling `f()` **runs nothing**; it only returns a coroutine object. |
| coroutine | A paused-able function. It runs only when awaited or wrapped in a task. |
| `await x` | "Run `x` and pause **me** until it finishes." The loop is freed **only if** `x` **actually suspends** (reaches an unfinished Future: I/O, timer, lock). |
| Task | A coroutine scheduled on the loop (`asyncio.create_task`). It starts running at the next `await` of the caller, without waiting to be awaited. |
| event loop | The scheduler. Picks a ready task, runs it until it hits an `await`, then switches. |

### Sequential vs concurrent (the core trap):

PYTHON

```python
# Sequential: ~13s, output 1 2 3 4
await f1()            # f2 doesn't exist yet
await f2()

# Concurrent: ~10s, output 1 3 2 4
await asyncio.gather(f1(), f2())

# Same thing with tasks
t1 = asyncio.create_task(f1())
t2 = asyncio.create_task(f2())
await t1; await t2
```

Rule: `await` **doesn't start new work. It just waits.** Work only overlaps if it was *scheduled* (task/gather) before you wait.

JS equivalent: `await a(); await b();` is sequential, `await Promise.all([a(), b()])` is concurrent. One difference: a JS `async` function starts running **immediately** when called (up to its first `await`). A Python coroutine does nothing until it's awaited or turned into a task.

## Does `await` free the event loop?

↑ TOP

**Only if the awaited thing actually suspends.** `await` means "run this and wait for it". Control goes back to the loop only when the chain of awaits reaches a **Future that isn't done yet** (network I/O, `asyncio.sleep`, a lock, `queue.get`).

```
your coroutine ─await─► fetch() ─await─► reader.read() ─await─► Future (socket not ready)
                                                                        │
                                                          yields up the whole chain to the Task
                                                                        ▼
                                          event loop: "this task is parked, run other ready tasks"
                                                                        │
                                             socket becomes readable → Future done → task resumed
```

- Awaiting a **coroutine** is just a call into it: **no switch** until something deeper really waits.
- Awaiting a Future/Task that is **already done** returns immediately: no switch.
- Everything **between two awaits** runs without interruption.

PYTHON

```python
async def add(a, b):
    return a + b                  # no real await inside

async def handler():
    x = await add(1, 2)           # runs synchronously: no other task runs here
    data = await db.fetch()       # ✅ suspends: the loop runs other tasks meanwhile
    result = heavy_cpu(data)      # ❌ blocks the loop: no await here
    time.sleep(1)                 # ❌ blocks the loop: not awaitable
    await asyncio.sleep(1)        # ✅ suspends
```

**Letting others run during long work:**

- A CPU loop: `await asyncio.sleep(0)` every so often (an explicit yield).
- Blocking I/O or a sync library: `await asyncio.to_thread(fn, ...)`.
- Real CPU work: `await loop.run_in_executor(ProcessPoolExecutor(), fn)`.

**Side effect: code between awaits is atomic** with respect to other tasks:

```python
counter += 1          # safe: no await in between, nothing can interleave

v = counter
await something()     # ❌ another task may change counter here
counter = v + 1       # lost update → use asyncio.Lock or avoid the await in between
```

**❝ INTERVIEW LINE**

*"`await` yields to the event loop only when the awaited operation actually suspends, i.e. it reaches an unfinished Future like I/O or a timer. Awaiting a coroutine that never suspends runs synchronously, and CPU work or blocking calls between awaits block the whole loop."*

**✓ KEY POINTS**

- Concurrency ≠ parallelism. Everything runs on one thread, and tasks just interleave at `await`s.
- Good for **I/O-bound** work. Useless for CPU-bound work; use processes for that (`ProcessPoolExecutor`).
- A blocking call (`time.sleep`, `requests.get`, heavy loop) inside async code **freezes the whole loop**. Use `await asyncio.sleep`, `httpx` / `aiohttp`, or `run_in_executor`.
- Total time: sequential is the **sum** of the waits, concurrent is the **max**.

**⚠ COMMON TRAP**

thinking two `await` lines in a row run in parallel. They don't.

↑ Back to top ☰ Contents

CHAPTER 1 · PYTHON

# Class vs Instance Attributes & Methods

class attribute  instance attribute  __dict__  MRO  attribute lookup  shadowing  classmethod  staticmethod  mutable default argument

SEE ALSO  Python Gotchas (Predict the Output) · OOP Internals & Memory Model

**What:** Python objects have two storage levels:

- **Class attribute:** defined in the class body and stored once in `A.__dict__`. **Shared** by all instances.
- **Instance attribute:** set on `self` (usually in `__init__`) and stored in `obj.__dict__`. **Per object**.

```python
class A:
    shared = []              # class attr: one list for everyone
    def __init__(self):
        self.own = []        # instance attr: new list per object
```

**Lookup rule (reading `obj.x`):** instance `__dict__` → class → parent classes (MRO). The first match wins.

**Writing rule:**

| Operation | Effect |
|---|---|
| `obj.x = 5` (assign) | Always writes to the **instance**, which shadows the class attr. The class is untouched. |
| `obj.x.append(5)` (mutate) | Changes **whatever object the lookup found**. If that's the class list, every instance sees it. |
| `A.x = 5` | Changes the class attr for every instance that hasn't shadowed it. |

```python
class A: l = []
a, b = A(), A()
a.l.append(1)    # mutates shared list → b.l == [1]
a.l = [9]        # assign → a now has its own l; b.l still [1]
```

**When class attrs are fine:** immutable values like constants, defaults and counters (`MAX = 10`, `kind = "dog"`). Assigning to them through an instance just shadows them, so nothing leaks. **When they bite:** mutable values (`list`, `dict`, `set`) that you meant to be per-instance.

**Methods (three kinds):**

| Kind | First arg | Access | Use for |
|---|---|---|---|
| Instance method `def m(self)` | the object | instance + class data | normal behavior |
| `@classmethod def m(cls)` | the class | class data, works with subclasses | alternative constructors (`from_json`), class-wide state |
| `@staticmethod def m()` | none | nothing implicit | utility logically grouped with the class |

PYTHON

```python
class User:
    count = 0
    def __init__(self, name):
        self.name = name
        User.count += 1          # use the class name: self.count += 1 would create an instance attr!
    @classmethod
    def from_dict(cls, d): return cls(d["name"])
    @staticmethod
    def valid(name): return bool(name)
```

Methods themselves are class attributes too. `a.m()` finds `m` on the class and binds `a` as `self`, so `a.m()` is the same as `A.m(a)`.

⚠ COMMON TRAP

a mutable class attribute is shared state, the same bug as a mutable default argument (`def f(x=[])`, where the list is created once at definition time). Fix: create it in `__init__` (or default to `x=None`, then `if x is None: x = []`).

↑ Back to top ☰ Contents

CHAPTER 1 · PYTHON

# Concurrency Models (GIL, Threads, Processes, asyncio)

GIL · global interpreter lock · threading · multiprocessing · ProcessPoolExecutor · ThreadPoolExecutor · concurrent.futures · asyncio.to_thread · race condition · threading.Lock · I/O bound · CPU bound · free-threaded · PEP 703 · python3.13t · concurrency vs parallelism

SEE ALSO Async / Await · OOP Internals & Memory Model · Django in Production: Performance, Security, Async · V2·2 Nearby Friends · V2·13 Stock Exchange

ON THIS PAGE

1. The GIL · p. 21
2. Choosing a model · p. 22
3. concurrent.futures: one API for both pools · p. 22
4. asyncio and blocking code · p. 22
5. Race condition with threads · p. 23
6. Free-threaded Python (3.13+) · p. 24

**What:** Python has three ways to do many things "at once": **threads**, **processes**, and **asyncio**. Which one helps depends on whether the work is waiting (I/O-bound) or computing (CPU-bound), and on the **GIL**.

> **i WHY IT MATTERS**
> "Why don't threads make my CPU code faster in Python?" is a classic question. You must explain the GIL and pick the right model for a given task.

## The GIL

↑ TOP

The **Global Interpreter Lock** is a lock inside CPython. A thread must hold it to run Python bytecode, so **only one thread runs Python code at a time** per process.

- It is **released** during blocking I/O (`socket.recv`, `time.sleep`, file reads, DB drivers) and by many C extensions (NumPy, hashlib on big data).
- So threads **do** give real overlap for I/O-bound work, but **no speedup** for pure-Python CPU work.
- Why it exists: it makes reference counting (memory management) simple and fast, and C extensions easy to write.
- The GIL does **not** make your code thread-safe. It protects the interpreter's internals, not your `counter += 1`.

↑ TOP

## Choosing a model

| Model | Parallel CPU? | Cost | Best for |
|---|---|---|---|
| `threading` / `ThreadPoolExecutor` | No (GIL) | Medium (OS thread, ~MBs stack) | I/O with **blocking** libs (`requests`, old DB drivers), tens–hundreds of tasks |
| `multiprocessing` / `ProcessPoolExecutor` | **Yes** | High (new process, data is pickled between processes) | **CPU-bound** work: image processing, parsing, math |
| `asyncio` | No (one thread) | Very low per task | **Many** I/O tasks (thousands of sockets), async libs (`httpx`, `asyncpg`) |

Rule of thumb: **waiting → threads or asyncio; computing → processes.**

↑ TOP

## concurrent.futures: one API for both pools

PYTHON

```python
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor
import time

def io_task(n):
    time.sleep(0.2)          # blocking I/O: releases the GIL
    return n

def cpu_task(n):
    return sum(i * i for i in range(n))   # pure Python: holds the GIL

if __name__ == "__main__":   # required for processes (spawn re-imports this file)
    start = time.perf_counter()
    with ThreadPoolExecutor(max_workers=5) as pool:
        print(list(pool.map(io_task, range(5))))   # [0, 1, 2, 3, 4]
    print(round(time.perf_counter() - start, 1))   # 0.2  (not 1.0: the sleeps overlap)

    with ProcessPoolExecutor() as pool:
        print(list(pool.map(cpu_task, [10, 100])))  # [285, 328350]  (runs on several
cores)
```

- `pool.map` keeps input order. `pool.submit(fn, x)` returns a `Future`; use `as_completed(futures)` to get results as they finish.
- Process pools need picklable functions and arguments (no lambdas, no open sockets).

↑ TOP

## asyncio and blocking code

asyncio runs everything on **one thread**. A blocking call freezes all tasks. Move it to a thread:

```python
import asyncio, time

def blocking_read():
    time.sleep(0.2)          # a sync library call
    return "data"

async def main():
    start = time.perf_counter()
    results = await asyncio.gather(
        asyncio.to_thread(blocking_read),
        asyncio.to_thread(blocking_read),
    )
    print(results)                                  # ['data', 'data']
    print(round(time.perf_counter() - start, 1))    # 0.2  (run in parallel threads)

asyncio.run(main())
```

For CPU work inside async code: `await loop.run_in_executor(ProcessPoolExecutor(), fn, arg)`. See Async / Await.

## Race condition with threads

↑ TOP

**Predict the output:**

```python
import threading, time

counter = 0

def work():
    global counter
    for _ in range(1000):
        v = counter          # read
        time.sleep(0)        # let another thread run here (like a real context switch)
        counter = v + 1      # write: may overwrite another thread's update

threads = [threading.Thread(target=work) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(counter == 4000)   # False: lost updates
```

`counter += 1` has the same problem: it is several bytecodes (load, add, store), and a thread switch can happen between them. The bug is just rarer without the `sleep(0)`.

**Fix: a Lock makes read-modify-write one unit.**

PYTHON

```python
import threading, time

counter = 0
lock = threading.Lock()

def work():
    global counter
    for _ in range(1000):
        with lock:                # only one thread inside at a time
            v = counter
            time.sleep(0)
            counter = v + 1

threads = [threading.Thread(target=work) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(counter)   # 4000
```

Other safe options: `queue.Queue` (thread-safe by design), or give each thread its own data and combine at the end.

## Free-threaded Python (3.13+)

↑ TOP

- PEP 703: CPython 3.13 ships an **optional** build without the GIL (`python3.13t`). In 3.14 it is officially supported but still **not the default**.
- Threads can then run Python code on many cores at once.
- Costs: single-thread code is somewhat slower, and C extensions must be rebuilt/marked safe.
- Races become **more** likely, not less. Locks are still your job.

✓ KEY POINTS

- GIL: one thread runs Python bytecode at a time; released on I/O and in many C extensions.
- Threads = concurrency for I/O. Processes = real parallelism for CPU. asyncio = cheap concurrency for lots of I/O on one thread.
- Processes don't share memory; arguments and results are pickled (overhead for big data).
- Shared mutable state + threads → use `Lock`, `Queue`, or avoid sharing.

❝ INTERVIEW LINE

*"Because of the GIL, only one thread executes Python bytecode at a time, so threads help with I/O-bound work, where the GIL is released while waiting, but not CPU-bound work. For CPU-bound tasks I use a `ProcessPoolExecutor`, and for many concurrent network calls I use asyncio."*

> ⚠ **COMMON TRAP**
> saying "the GIL makes Python code thread-safe, so I don't need locks." It only protects the interpreter; `counter += 1` can still lose updates.

---

↑ Back to top ☰ Contents

CHAPTER 1 · PYTHON

# Generators & Iterators

iterator iterable iterator protocol __iter__ __next__ StopIteration generator yield yield from lazy evaluation generator expression list comprehension itertools islice chain groupby batched tee memory

SEE ALSO Python Gotchas (Predict the Output) · Closures, Decorators & Context Managers · Async / Await · OOP Internals & Memory Model

ON THIS PAGE

1. Iterator protocol · p. 26
2. Generators: lazy, pause at yield · p. 27
3. Generator expression vs list comprehension · p. 28
4. One-time consumption trap · p. 28
5. yield from · p. 29
6. itertools highlights · p. 30

**What:** an **iterator** gives values one at a time with `next()`. A **generator** is the easy way to write an iterator: a function that uses `yield`.

> **i WHY IT MATTERS**
> generators let you process huge data (files, DB rows, API pages) with constant memory. Interviewers like "predict the output" questions about when generator code runs and why the second loop is empty.

## Iterator protocol

↑ TOP

| Term | Has | Example |
|---|---|---|
| Iterable | `__iter__()` returning an iterator | `list`, `dict`, `str`, `range`, file |
| Iterator | `__next__()` and `__iter__()` that returns `self` | `iter([1, 2])`, generator, `map`, `zip`, file |

`for x in obj` does: `it = iter(obj)`, then calls `next(it)` until `StopIteration`.

PYTHON

```python
class Countdown:
    def __init__(self, n):
        self.n = n
    def __iter__(self):
        return self             # an iterator is its own iterator
    def __next__(self):
        if self.n == 0:
            raise StopIteration
        self.n -= 1
        return self.n + 1

c = Countdown(3)
print(list(c))   # [3, 2, 1]
print(list(c))   # []  (same object, already at the end)

nums = [1, 2]    # a list is iterable, not an iterator: each loop gets a new iterator
print(list(nums), list(nums))   # [1, 2] [1, 2]
```

## Generators: lazy, pause at yield

↑ TOP

**Predict the output:**

PYTHON

```python
def gen():
    print("start")
    yield 1
    print("middle")
    yield 2
    print("end")

g = gen()                 # nothing printed: body has not started
print("created")
print(next(g))            # start, 1
print(next(g))            # middle, 2
print(next(g, "done"))    # end, done  (StopIteration → default value)
```

Output: `created`, `start`, `1`, `middle`, `2`, `end`, `done`.

- Calling `gen()` runs **nothing**. Code runs only when someone asks for the next value.
- Local variables are kept between `yield`s; that is how a generator "remembers" its state.
- A `return x` inside a generator ends it (`StopIteration.value == x`).

**Real use: stream a big file.**

```python
def read_lines(lines):
    for line in lines:          # with a real file: for line in open(path)
        if line.strip():
            yield line.strip()

def parse(lines):
    for line in lines:
        yield line.split(",")

data = ["a,1", "", "b,2"]
for row in parse(read_lines(data)):   # a pipeline: one line in memory at a time
    print(row)                         # ['a', '1'] then ['b', '2']
```

## Generator expression vs list comprehension

↑ TOP

```python
import sys

squares_list = [x * x for x in range(1_000_000)]
squares_gen = (x * x for x in range(1_000_000))

print(sys.getsizeof(squares_list) > 8_000_000)   # True  (~8 MB: all values stored)
print(sys.getsizeof(squares_gen) < 500)          # True  (~200 bytes: just the paused
state)
print(sum(x * x for x in range(4)))              # 14  (no extra brackets needed as the
only argument)
```

| | **List comprehension `[...]`** | **Generator expression `(...)`** |
|---|---|---|
| Evaluation | Eager, all at once | Lazy, on demand |
| Memory | O(n) | O(1) |
| Reuse / `len()` / index | Yes | No: one pass only |
| Use when | You need the data many times or random access | You consume once (`sum`, `max`, `any`, `join`, a loop) |

## One-time consumption trap

↑ TOP

**Predict the output:**

```python
nums = (x * x for x in range(3))
print(sum(nums))   # 5
print(sum(nums))   # 0  (already exhausted)

m = map(str, [1, 2, 3])
print(list(m))     # ['1', '2', '3']
print(list(m))     # []

it = iter([1, 2, 3, 4])
print(3 in it)     # True  (consumed 1, 2, 3 to find it)
print(list(it))    # [4]
```

Fix: store the result in a list if you need it twice, or create a new generator each time.

## yield from

↑ TOP

```python
def flatten(items):
    for x in items:
        if isinstance(x, list):
            yield from flatten(x)    # same as: for y in flatten(x): yield y
        else:
            yield x

print(list(flatten([1, [2, [3, 4]], 5])))   # [1, 2, 3, 4, 5]

def sub():
    yield 1
    return "sub done"

def outer():
    result = yield from sub()        # also receives sub's return value
    print(result)

print(list(outer()))   # prints "sub done", then [1]
```

↑ TOP

## itertools highlights

PYTHON

```python
from itertools import islice, chain, count, groupby, accumulate, product, tee

print(list(islice(count(10), 3)))            # [10, 11, 12]  (take 3 from an infinite iterator)
print(list(chain([1, 2], (3, 4))))           # [1, 2, 3, 4]
print(list(accumulate([1, 2, 3, 4])))        # [1, 3, 6, 10]  (running sum)
print(list(product("ab", [0, 1])))           # [('a', 0), ('a', 1), ('b', 0), ('b', 1)]

words = ["apple", "avocado", "banana", "apricot"]
print([(k, list(g)) for k, g in groupby(words, key=lambda w: w[0])])
# [('a', ['apple', 'avocado']), ('b', ['banana']), ('a', ['apricot'])]  ← 'a' twice: not sorted!

a, b = tee(iter([1, 2]))                     # two independent copies of one iterator
print(list(a), list(b))                      # [1, 2] [1, 2]
```

- `itertools.batched(it, n)` (3.12+): chunks of size n, good for bulk DB inserts.
- `zip`, `map`, `filter`, `enumerate`, `reversed` also return lazy iterators.

✓ KEY POINTS

- `for` calls `iter()` once, then `next()` until `StopIteration`.
- Generator body runs lazily, pauses at `yield`, keeps its local state.
- Iterators/generators are single-pass; lists are reusable.
- Use generator expressions for one-pass aggregation over big data.
- `async def` + `yield` gives an **async generator** (`async for`), used for streaming pages from an API.

❝ INTERVIEW LINE

*"A generator is a lazy iterator: calling it runs nothing, each `next()` runs the body until the next `yield`, and it keeps its state between calls. I use them to stream large data with constant memory, but they can only be consumed once."*

⚠ COMMON TRAP

looping over the same generator (or `map`/`zip`/file object) twice and getting nothing the second time, or expecting `print` inside a generator to run when you call the function.

↑ Back to top ☰ Contents

CHAPTER 1 · PYTHON

# Closures, Decorators & Context Managers

closure · nonlocal · free variable · decorator · functools.wraps · decorator with arguments · decorator factory · stacked decorators · context manager · with · __enter__ · __exit__ · contextlib.contextmanager · suppress exception · late binding · lambda in loop · functools.partial

SEE ALSO Python Gotchas (Predict the Output) · Generators & Iterators · Async / Await

ON THIS PAGE

1. Closures · p. 31
2. Late-binding closure trap · p. 32
3. Decorators · p. 32
4. Context managers · p. 34

**What:** a **closure** is a function that keeps access to variables from the scope where it was created. A **decorator** uses a closure to wrap a function with extra behavior. A **context manager** wraps a block of code with setup/cleanup (`with`).

> **i WHY IT MATTERS**
> decorators are everywhere in backend code (`@app.get`, `@retry`, `@cache`, auth checks), and context managers handle resources (files, DB transactions, locks). Interviewers ask you to write one, or predict the output.

## Closures

↑ TOP

PYTHON

```python
def make_counter():
    count = 0
    def inc():
        nonlocal count      # without this, count += 1 → UnboundLocalError
        count += 1
        return count
    return inc

c1, c2 = make_counter(), make_counter()
print(c1(), c1(), c2())     # 1 2 1  (each call to make_counter makes a new count)
print(c1.__closure__[0].cell_contents)   # 2
```

- The inner function keeps a reference to the **variable** (a "cell"), not a copy of its value.
- To **assign** to it you need `nonlocal` (or `global` for module level). Reading or mutating (`lst.append`) does not need it.

↑ TOP

## Late-binding closure trap

**Predict the output:**

PYTHON

```python
funcs = []
for i in range(3):
    funcs.append(lambda: i)
print([f() for f in funcs])        # [2, 2, 2]

funcs = [lambda i=i: i for i in range(3)]   # default arg is evaluated NOW
print([f() for f in funcs])        # [0, 1, 2]

from functools import partial
funcs = [partial(lambda x: x, i) for i in range(3)]
print([f() for f in funcs])        # [0, 1, 2]
```

Why: all lambdas share the same variable `i`, and look it up **when called**. By then the loop is done and `i == 2`. Default arguments are evaluated at definition time, so they capture the current value. Same with `def` inside a loop, not only lambdas.

## Decorators

↑ TOP

`@deco` above `def f` means `f = deco(f)`.

PYTHON

```python
import functools, time

def timer(func):
    @functools.wraps(func)                # keep func's __name__, __doc__
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.perf_counter() - start:.1f}s")
        return result
    return wrapper

@timer
def slow(x):
    time.sleep(0.1)
    return x * 2

print(slow(5))          # "slow took 0.1s", then 10
print(slow.__name__)    # slow  (without @wraps it would be "wrapper")
```

**Decorator with arguments** = a function that **returns** a decorator:

PYTHON

```python
import functools

def retry(times):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except ValueError:
                    print(f"attempt {attempt} failed")
            raise RuntimeError("all retries failed")
        return wrapper
    return decorator

calls = 0

@retry(times=3)          # = retry(3)(flaky)
def flaky():
    global calls
    calls += 1
    if calls < 3:
        raise ValueError
    return "ok"

print(flaky())   # attempt 1 failed, attempt 2 failed, ok
```

**Predict the output: when does decorator code run? And stacking order?**

PYTHON

```python
def deco(name):
    print(f"decorating with {name}")         # runs at definition time
    def decorator(func):
        def wrapper():
            print(f"{name} before")
            func()
            print(f"{name} after")
        return wrapper
    return decorator

@deco("A")
@deco("B")
def hello():
    print("hello")

print("--- call ---")
hello()
```

Output:

```
decorating with A
decorating with B
--- call ---
A before
B before
hello
B after
A after
```

`@deco("A")` is evaluated first (so it prints first), but `B` is **applied** to `hello` first. The result is `A(B(hello))`, so on call A's wrapper is the outer layer: like an onion.

## Context managers

↑ TOP

`with cm as x:` does:

1. `x = cm.__enter__()`
2. run the block
3. `cm.__exit__(exc_type, exc, tb)` **always** (args are `None` if no exception). If it returns `True`, the exception is suppressed.

PYTHON

```python
class Transaction:
    def __enter__(self):
        print("BEGIN")
        return self
    def __exit__(self, exc_type, exc, tb):
        if exc_type:
            print(f"ROLLBACK ({exc})")
        else:
            print("COMMIT")
        return False             # don't swallow the exception

with Transaction():
    print("insert")              # BEGIN, insert, COMMIT

try:
    with Transaction():
        raise ValueError("bad row")
except ValueError:
    print("error reached caller")   # BEGIN, ROLLBACK (bad row), error reached caller
```

**Same thing with** `contextlib.contextmanager` (a generator: code before `yield` = enter, after = exit):

PYTHON

```python
from contextlib import contextmanager, suppress
import time

@contextmanager
def timed(label):
    start = time.perf_counter()
    try:
        yield "resource"          # value bound by "as"
    finally:                      # without finally, an exception would skip this
        print(f"{label}: {time.perf_counter() - start:.1f}s")

with timed("query") as r:
    print(r)                      # resource
    time.sleep(0.1)               # then: query: 0.1s

with suppress(KeyError):          # built-in "ignore this exception" context manager
    {}["missing"]
print("still running")            # still running
```

Common built-ins: `open()`, `threading.Lock()`, `ExitStack` (dynamic number of CMs), `async with` (`__aenter__` / `__aexit__`) for async DB sessions.

✓ KEY POINTS

- Closures capture variables, not values; `nonlocal` to reassign.
- Decorator body runs at import time; wrapper runs per call. Use `*args, **kwargs` and `functools.wraps`.
- Decorator with args needs one more level of nesting.
- `__exit__` always runs; return `True` only if you really want to hide the exception.
- With `@contextmanager`, put `yield` inside `try/finally` so cleanup runs on errors.

❝ INTERVIEW LINE

*"A decorator is just a function that takes a function and returns a new one, so `@d def f` is `f = d(f)`; I use `functools.wraps` to keep the metadata. A context manager guarantees cleanup because `__exit__` runs even when the block raises."*

⚠ COMMON TRAP

closures in a loop all see the final loop value (`[2, 2, 2]`), and forgetting `try/finally` around `yield` in `@contextmanager`, so cleanup is skipped on error.

↑ Back to top ☰ Contents

CHAPTER 1 · PYTHON

# Python Gotchas (Predict the Output)

is vs == small int cache interning mutable default argument shallow copy deep copy copy.deepcopy list multiplication aliasing late binding lambda tuple += augmented assignment dict ordering hash collision True 1 floor division modulo negative banker's rounding round half to even float precision try finally return string immutability

SEE ALSO Closures, Decorators & Context Managers · Class vs Instance Attributes & Methods · OOP Internals & Memory Model · Generators & Iterators

ON THIS PAGE

1. 1. is vs == (small int cache) · p. 36
2. 2. Mutable default argument · p. 37
3. 3. Shallow vs deep copy · p. 38
4. 4. List multiplication [[0]*3]*3 · p. 38
5. 5. Late binding lambdas · p. 38
6. 6. += on a tuple that contains a list · p. 39
7. 7. Dict ordering and key equality · p. 39
8. 8. Integer division, modulo, rounding · p. 39
9. 9. return in finally · p. 40
10. 10. String immutability · p. 40

**What:** short snippets where Python does something surprising. For each: the code, the answer, and one line of why.

> i WHY IT MATTERS
> "What does this print?" is the most common Python screening question. The trick is always one of the rules below.

## 1. `is` vs `==` (small int cache)

↑ TOP

PYTHON

```python
a = int("256"); b = int("256")
print(a == b, a is b)    # True True   (-5..256 are cached singletons)

a = int("257"); b = int("257")
print(a == b, a is b)    # True False  (two different objects)

x = None
print(x is None)         # True  (the right way to check None)
```

**Why:** `==` compares values (`__eq__`), `is` compares identity (`id()`). CPython caches small ints and some strings, so `is` sometimes looks right. Note: `a = 257; b = 257` written in one file can still give `True`, because the compiler reuses the same constant. Never rely on it.

↑ TOP

## 2. Mutable default argument

```python
def add(item, bucket=[]):
    bucket.append(item)
    return bucket

print(add(1))   # [1]
print(add(2))   # [1, 2]  ← same list as before!

def add_fixed(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket

print(add_fixed(1), add_fixed(2))   # [1] [2]
```

**Why:** default values are evaluated **once**, when `def` runs, and stored on the function. Every call that doesn't pass `bucket` gets **the same list object**, and `.append()` mutates it:

```python
def add(item, bucket=[]):
    bucket.append(item)
    return bucket

add(1); add(2)
print(add.__defaults__)   # ([1, 2],)  ← the shared list lives on the function
```

**Same bug as a mutable class attribute** (see Class vs Instance Attributes):

| | Mutable default | Mutable class attribute |
|---|---|---|
| Code | `def f(x=[])` | `class A: l = []` |
| Created | once, when the **function** is defined | once, when the **class** is defined |
| Shared by | every **call** | every **instance** |
| Fix | `x=None`, then `if x is None: x = []` | `self.l = []` in `__init__` |

- Use `if x is None`, not `x = x or []`. With `or`, an **empty list passed by the caller** is replaced by a new one, so the caller's list never gets the change.
- The same trap applies to `{}`, `set()`, and `datetime.now()` (`def log(ts=datetime.now())` stamps every call with the **definition** time).
- Dataclasses block the list case: `items: list = []` raises `ValueError`. Use `field(default_factory=list)`.

↑ TOP

## 3. Shallow vs deep copy

PYTHON

```python
import copy

a = [[1, 2], [3]]
b = a                   # same object
c = a.copy()            # shallow: new outer list, same inner lists (also a[:], list(a))
d = copy.deepcopy(a)    # everything copied

a[0].append(99)
a.append([7])
print(b)   # [[1, 2, 99], [3], [7]]
print(c)   # [[1, 2, 99], [3]]
print(d)   # [[1, 2], [3]]
```

**Why:** assignment never copies. A shallow copy copies only the top level; nested objects are shared.

## 4. List multiplication `[[0]*3]*3`

↑ TOP

PYTHON

```python
grid = [[0] * 3] * 3
grid[0][0] = 1
print(grid)    # [[1, 0, 0], [1, 0, 0], [1, 0, 0]]

grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 1
print(grid)    # [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

**Why:** `*` copies **references**. The outer list holds the same inner list 3 times. `[0] * 3` is fine because ints are immutable.

## 5. Late binding lambdas

↑ TOP

PYTHON

```python
fs = [lambda: i for i in range(3)]
print([f() for f in fs])      # [2, 2, 2]

fs = [lambda i=i: i for i in range(3)]
print([f() for f in fs])      # [0, 1, 2]
```

**Why:** the lambda looks up `i` when it is **called**, not when it is created. See Closures, Decorators & Context Managers.

↑ TOP

## 6. `+=` on a tuple that contains a list

```python
t = ([1, 2],)
try:
    t[0] += [3]
except TypeError as e:
    print("TypeError:", e)   # TypeError: 'tuple' object does not support item assignment
print(t)                     # ([1, 2, 3],)  ← changed anyway!
```

**Why:** `t[0] += [3]` is two steps: `t[0].__iadd__([3])` (extends the list in place: works), then `t[0] = result` (assigning into a tuple: fails). Use `t[0].append(3)` / `extend` instead.

↑ TOP

## 7. Dict ordering and key equality

```python
d = {"b": 1, "a": 2}
d["c"] = 3
print(list(d))                       # ['b', 'a', 'c']  (insertion order, guaranteed since 3.7)

d = {1: "int", True: "bool", 1.0: "float"}
print(d)                             # {1: 'float'}

print({3, 1, 2} == {1, 2, 3})        # True  (sets have no order guarantee; never rely on set order)
print({"a": 1, "b": 2} == {"b": 2, "a": 1})   # True  (dict == ignores order)
```

**Why:** dicts keep insertion order. `1 == True == 1.0` and they have the same hash, so they are the **same key**: the first key object is kept, the last value wins.

↑ TOP

## 8. Integer division, modulo, rounding

```python
print(7 // 2, -7 // 2)       # 3 -4   (floor: rounds toward -infinity)
print(-7 % 2, 7 % -2)        # 1 -1   (result has the sign of the divisor)
print(int(-3.7))             # -3     (int() truncates toward zero)
print(round(0.5), round(1.5), round(2.5))   # 0 2 2  (banker's rounding: half → even)
print(round(2.675, 2))       # 2.67   (2.675 is really 2.67499999... in binary)
print(0.1 + 0.2 == 0.3)      # False
print(10 / 2)                # 5.0    (/ always returns float)
```

**Why:** `//` is floor division (Go and C truncate: `-7/2 == -3` in Go). `round` uses round-half-to-even to avoid bias. Use `decimal.Decimal` for money, `math.isclose` for float compare.

↑ TOP

## 9. `return` in `finally`

```python
def f():
    try:
        return "try"
    finally:
        return "finally"

def g():
    try:
        raise ValueError("lost")
    finally:
        return "swallowed"

def h():
    x = 1
    try:
        return x        # return value (1) is already computed here
    finally:
        x = 2           # changing x does not change the result

print(f(), g(), h())    # finally swallowed 1
```

**Why:** `finally` always runs last; its `return` replaces any pending return **or exception**. Python 3.14 gives a `SyntaxWarning` for this (PEP 765). Don't `return` from `finally`.

## 10. String immutability

↑ TOP

```python
s = "hello"
s.upper()
print(s)            # hello  (methods return a NEW string; result was thrown away)

s = s.upper()
print(s)            # HELLO

try:
    s[0] = "J"
except TypeError as e:
    print(e)        # 'str' object does not support item assignment

parts = ["a", "b", "c"]
print("".join(parts))   # abc  (join is O(n); s += x in a loop may copy every time → O(n²))
```

**Why:** `str` (like `int`, `tuple`, `frozenset`, `bytes`) is immutable. Every "change" creates a new object.

✓ KEY POINTS

- Identity vs equality: `is` only for `None` (and sentinels).
- Anything created at definition time (default args, class attributes) is shared.
- Assignment, `*`, and shallow copies share nested objects.
- Closures look up variables at call time.
- Know Python's floor `//` and banker's `round` vs Go's truncation.

❝ INTERVIEW LINE

*"Most Python surprises come from one idea: names are references to objects. Default args and class attributes are created once and shared, `*` and shallow copies duplicate references, and closures read the variable when called, not when defined."*

⚠ COMMON TRAP

answering from intuition. Before answering, ask: is this object **mutable**, and how many **references** point to it?

↑ Back to top ☰ Contents

CHAPTER 1 · PYTHON

# OOP Internals & Memory Model

MRO C3 linearization super() diamond inheritance multiple inheritance dunder methods magic methods __repr__ __str__ __eq__ __hash__ unhashable __len__ __bool__ __iter__ __getitem__ property setter __slots__ __new__ __init__ singleton ABC abstractmethod dataclass frozen default_factory reference counting garbage collector gc reference cycle pass by object reference interface ABC abstractmethod Protocol typing.Protocol runtime_checkable mixin structural typing Django mixin order

SEE ALSO Class vs Instance Attributes & Methods · Python Gotchas (Predict the Output) · Generators & Iterators · Concurrency Models (GIL, Threads, Processes, asyncio)

ON THIS PAGE

1. MRO & super() with diamond inheritance · p. 42
2. Dunder methods · p. 44
3. @property · p. 45
4. slots · p. 45
5. new vs init · p. 47
6. Interfaces: ABC vs Protocol vs mixin · p. 47
7. Dataclasses (briefly) · p. 50
8. Memory model · p. 50

**What:** how Python classes work under the hood: method lookup (MRO), special "dunder" methods, attribute control (`property`, `__slots__`), object creation (`__new__`), and how memory is managed.

> **i WHY IT MATTERS**
> senior Python interviews go past syntax: "what does `super()` call in a diamond?", "why can't I put my object in a set?", "how does Python free memory?". For basic attribute lookup see Class vs Instance Attributes.

## MRO & super() with diamond inheritance

↑ TOP

**Predict the output:**

```python
class A:
    def hi(self): print("A")

class B(A):
    def hi(self):
        print("B")
        super().hi()

class C(A):
    def hi(self):
        print("C")
        super().hi()

class D(B, C):
    def hi(self):
        print("D")
        super().hi()

D().hi()                                    # D B C A  (one per line)
print([k.__name__ for k in D.__mro__])      # ['D', 'B', 'C', 'A', 'object']
```

- **MRO** (Method Resolution Order) is computed by **C3 linearization**: children before parents, left-to-right order of bases kept, each class once.
- In `B`, `super()` goes to **C**, not A, because the object is a `D` and C is next in `D.__mro__`. This is why A runs only once.
- For cooperative classes, every class should call `super()` and accept `**kwargs`.
- With a plain `B()` object, `B`'s `super()` goes to A: `super()` depends on the **object's** class, not on where the method is written.

**When no MRO is possible:**

```python
class X: pass
class Y(X): pass
try:
    class Z(X, Y): pass      # asks X before Y, but Y is a child of X → must come before X
except TypeError as e:
    print("TypeError")       # Cannot create a consistent method resolution order (MRO)
```

**In Django: mixins go on the left.** `class MyView(LoginRequiredMixin, ListView)` gives the MRO `MyView → LoginRequiredMixin → ListView → ...`. The mixin's `dispatch()` runs first (it checks the login), then its `super().dispatch()` passes control to `ListView`. With the mixin on the right, `ListView.dispatch()` would run first and the login check would be skipped. The same rule applies in DRF: `class V(mixins.ListModelMixin, viewsets.GenericViewSet)`.

↑ TOP

## Dunder methods

| Method | Used by | Note |
|---|---|---|
| `__repr__` | `repr(x)`, the REPL, inside lists | For developers; should be unambiguous. Fallback for `str()`. |
| `__str__` | `print(x)`, `str(x)`, f-strings | For users. |
| `__eq__` | `==` | Defining it sets `__hash__ = None`. |
| `__hash__` | `set`, `dict` keys | Must be consistent with `__eq__`; base it on immutable fields. |
| `__len__` | `len(x)`, truthiness | `bool(x)` uses `__bool__`, then `__len__ ≠ 0`. |
| `__iter__` | `for`, `list(x)`, `in` | Return an iterator (often a generator). |
| `__getitem__` | `x[i]`, slicing | Also makes `for` work as a fallback. |
| `__lt__` etc. | `<`, `sorted` | `functools.total_ordering` fills the rest. |

PYTHON

```python
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __repr__(self):
        return f"Point({self.x}, {self.y})"
    def __eq__(self, other):
        return isinstance(other, Point) and (self.x, self.y) == (other.x, other.y)

p = Point(1, 2)
print([p], p == Point(1, 2))   # [Point(1, 2)] True
try:
    {p}
except TypeError as e:
    print(e)                   # unhashable type: 'Point'

class HPoint(Point):
    def __hash__(self):
        return hash((self.x, self.y))   # equal objects → equal hashes

print(len({HPoint(1, 2), HPoint(1, 2)}))   # 1

class Bag:
    def __init__(self, items): self.items = items
    def __len__(self): return len(self.items)
    def __iter__(self): return iter(self.items)

b = Bag([])
print(len(b), bool(b), list(Bag([1, 2])))   # 0 False [1, 2]
```

**Rule:** if `a == b` then `hash(a) == hash(b)`. Don't hash mutable fields: if the field changes, the object gets "lost" inside the set/dict.

## @property

↑ TOP

Use a method like an attribute; add validation without changing the public API.

PYTHON

```python
class Account:
    def __init__(self, balance):
        self.balance = balance          # goes through the setter below

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("negative balance")
        self._balance = value

a = Account(10)
a.balance = 5
print(a.balance)        # 5
try:
    a.balance = -1
except ValueError as e:
    print(e)            # negative balance
```

## slots

↑ TOP

PYTHON

```python
class Slotted:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x, self.y = x, y

s = Slotted(1, 2)
print(hasattr(s, "__dict__"))   # False
try:
    s.z = 3
except AttributeError as e:
    print(e)                    # 'Slotted' object has no attribute 'z' ...
```

**How it works:** normally every object carries its own `__dict__` (a hash table) to store attributes, so you can add any attribute at any time. `__slots__` tells Python the **fixed list of attributes** up front. Python then stores them in fixed positions inside the object (like a C struct), with **no** `__dict__`.

PYTHON

```python
import tracemalloc

class Plain:
    def __init__(self, x, y):
        self.x, self.y = x, y

class Slim:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x, self.y = x, y

def measure(cls):
    tracemalloc.start()
    objs = [cls(i, i) for i in range(100_000)]
    size = tracemalloc.get_traced_memory()[0]
    tracemalloc.stop()
    return size

print(measure(Slim) < measure(Plain))   # True: no per-object dict
```

| | Normal class | With `__slots__` |
|---|---|---|
| Storage | `__dict__` per object | fixed slots, no `__dict__` |
| Memory | bigger (a dict per object) | smaller: ~30% less on Python 3.11+ (which already shrinks object dicts), more on older versions |
| Attribute access | dict lookup | slightly faster (direct slot) |
| Add a new attribute later | yes | `AttributeError` |
| `weakref` | works | only if `"__weakref__"` is in `__slots__` |

**Rules and traps:**

- **Inheritance:** if a parent class has no `__slots__`, children still get a `__dict__`, so the saving is lost. **Every class in the chain** must define `__slots__`, listing only its own new names.
- You **can't** give a class-level default to a name that is also in `__slots__` (`x = 0` together with `__slots__ = ("x",)` raises `ValueError`). Set defaults in `__init__`.
- `@dataclass(slots=True)` (3.10+) generates the slots for you.
- **When to use it:** millions of small objects (data points, events, tree nodes, ORM-like rows in memory). Not worth it for normal classes: the gain is small, and you lose flexibility.

**“ INTERVIEW LINE**

*"`__slots__` replaces the per-instance `__dict__` with a fixed set of attribute slots. It saves memory and makes attribute access a bit faster, but you can't add new attributes, and every class in the inheritance chain needs it. I'd use it for very many small objects, often via `@dataclass(slots=True)`."*

↑ TOP

## new vs init

PYTHON

```python
class Singleton:
    _instance = None
    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            print("creating")
            cls._instance = super().__new__(cls)
        return cls._instance
    def __init__(self, value):
        print("init", value)           # runs on EVERY call, even for the cached object
        self.value = value

a = Singleton(1)    # creating, init 1
b = Singleton(2)    # init 2
print(a is b, a.value)   # True 2

class Upper(str):
    def __new__(cls, value):
        return super().__new__(cls, value.upper())   # str is immutable: must change it in
__new__

print(Upper("hi"))   # HI
```

| | `__new__(cls, ...)` | `__init__(self, ...)` |
|---|---|---|
| Job | Create and **return** the object | Initialize an existing object, returns `None` |
| Runs | First | After, only if `__new__` returned an instance of `cls` |
| Override for | Singletons, caching, subclassing immutables (`str`, `int`, `tuple`) | Normal setup (almost always) |

## Interfaces: ABC vs Protocol vs mixin

↑ TOP

Python has no `interface` keyword. There are three tools, each for a different job:

| | **ABC (`abc.ABC`)** | **Protocol (`typing.Protocol`)** | **Mixin** |
|---|---|---|---|
| What it is | **explicit interface**: a base class with `@abstractmethod`s | **implicit interface** (structural, "duck typing" checked by mypy) | a small class that **adds ready-made behavior** |
| Implementer must | **inherit** it and implement every abstract method | just have matching methods; no inheritance | inherit it (usually listed first) |

| | ABC ( `abc.ABC` ) | Protocol ( `typing.Protocol` ) | Mixin |
|---|---|---|---|
| Enforced when | at **runtime**: instantiating an incomplete subclass raises `TypeError` | by the **type checker** only (runtime only with `@runtime_checkable` + `isinstance`) | not enforced: it's code reuse, not a contract |
| Gives you code? | can (concrete helper methods too) | no, just the shape | **yes**, that's its purpose |
| Closest in Go/Java | Java `abstract class` / `interface` | **Go interface** (satisfied implicitly) | Java default methods / traits |
| Use for | plugin-style hierarchies you own ( `Storage` → `S3`, `Disk` ) | accepting any object that "has `.read()`", including third-party classes | sharing behavior across unrelated classes ( `TimestampMixin`, Django `LoginRequiredMixin` ) |

**So the interface is ABC or Protocol. A mixin is not an interface; it's reusable implementation.**

## ABC: explicit, checked at runtime

PYTHON

```python
from abc import ABC, abstractmethod

class Storage(ABC):
    @abstractmethod
    def save(self, data): ...

class S3(Storage):
    def save(self, data):
        return f"saved {data}"

try:
    Storage()
except TypeError as e:
    print("TypeError")      # TypeError: can't instantiate abstract class
print(S3().save("x"))       # saved x
```

### Protocol: implicit, like a Go interface

PYTHON

```python
from typing import Protocol, runtime_checkable

@runtime_checkable
class Reader(Protocol):
    def read(self) -> str: ...

class File:                     # does NOT inherit Reader
    def read(self) -> str:
        return "data"

def load(src: Reader) -> str:  # mypy accepts File: it has the right shape
    return src.read()

print(load(File()))                  # data
print(isinstance(File(), Reader))    # True (runtime_checkable only checks method names)
```

### Mixin: shared behavior, not a contract

PYTHON

```python
import json

class JsonMixin:
    def to_json(self):
        return json.dumps(self.__dict__)

class User(JsonMixin):
    def __init__(self, name):
        self.name = name

print(User("ali").to_json())   # {"name": "ali"}
```

Rules for mixins: keep them small, give them no `__init__` (or a cooperative one calling `super().__init__(**kwargs)`), and put them **left** of the main base class so their methods win in the MRO.

> **“ INTERVIEW LINE**
>
> *"Python has no interface keyword. For an explicit contract enforced at runtime I'd use an ABC. For Go-style implicit interfaces checked by mypy, a Protocol. A mixin isn't an interface; it's a way to reuse behavior, and it goes to the left of the base class because of the MRO."*

↑ TOP

## Dataclasses (briefly)

PYTHON

```python
from dataclasses import dataclass, field

@dataclass(frozen=True)
class User:
    name: str
    tags: list = field(default_factory=list)   # not tags=[] (shared default → ValueError)

u = User("ali")
print(u)                       # User(name='ali', tags=[])
print(u == User("ali"))        # True  (generated __eq__)
try:
    u.name = "x"
except Exception as e:
    print(type(e).__name__)    # FrozenInstanceError
```

Generates `__init__`, `__repr__`, `__eq__`. `frozen=True` also makes it hashable (if fields are hashable); `slots=True` (3.10+) adds `__slots__`.

↑ TOP

## Memory model

**Names are references to objects.** Assignment binds a name; it never copies.

PYTHON

```python
def f(lst):
    lst.append(1)     # mutates the caller's object
    lst = [9]         # rebinds the local name only

x = []
f(x)
print(x)              # [1]
```

**Reference counting:** every object counts how many references point to it. When the count reaches 0, it is freed **immediately** (that's why files closed by refcount close quickly in CPython).

PYTHON

```python
import sys
a = []
b = a
print(sys.getrefcount(a))   # 3 on 3.13 (a, b, and the temporary argument); exact count
varies by version
```

**Cyclic garbage collector:** refcount can't free cycles, so the `gc` module periodically finds unreachable groups (generational: young objects are checked more often).

PYTHON

```python
import gc

class Node:
    pass

a, b = Node(), Node()
a.other, b.other = b, a     # cycle: each refcount stays ⩾ 1
del a, b                    # unreachable, but not freed by refcount
print(gc.collect() ⩾ 2)     # True  (the GC found and freed them)
```

- `weakref` lets you reference an object without increasing its refcount (caches, parent pointers).
- Memory leaks in Python usually mean something still **holds a reference**: a global list, a cache, a closure.

✓ KEY POINTS

- `super()` follows the instance's MRO (C3); in a diamond each class runs once.
- `__eq__` without `__hash__` → unhashable. Hash only immutable fields.
- `property` = controlled attribute access; `__slots__` = memory savings, no new attrs.
- `__new__` creates, `__init__` initializes; `__init__` runs again even if `__new__` returns a cached object.
- CPython frees memory by refcount at once; the GC is only for cycles.

❝ INTERVIEW LINE

*"In multiple inheritance `super()` doesn't call the parent; it calls the next class in the instance's MRO, which C3 linearization computes so each class runs once. For memory, CPython uses reference counting to free objects immediately and a generational cyclic GC to clean up reference cycles."*

⚠ COMMON TRAP

thinking `super()` in `B` calls `A` (it calls `C` in the diamond), and defining `__eq__` then being surprised that the object can't go into a `set` or be a `dict` key.

↑ Back to top ☰ Contents

# 02
# Data Structures

1 pages. Each card says what the page covers and its key lines; open the page for the full depth.

### Python Data Structures Map · p. 53

each classic data structure and what to use for it in Python. Most of these are standard library. Click a name to jump to its example.

- Queue/deque: `collections.deque`, with `popleft()` O(1). Never `list.pop(0)` (O(n)).
- Min-heap: `heapq` on a list, min at `h[0]`; max-heap: push `-x`; top-k: heap of size k.
- "Next free / earliest / k smallest" → heap of `(time, id)` tuples (car wash).
- Counting: `Counter`; grouping: `defaultdict(list)`; LRU: `OrderedDict.move_to_end` + `popitem(last=False)`.
- Sorted search: `bisect`; no TreeMap in stdlib (say balanced BST / `sortedcontainers`).
- Memoize: `@functools.cache`; records: `@dataclass` with `field(default_factory=list)`.

Read the full page →

CHAPTER 2 · DATA STRUCTURES

# Python Data Structures Map

list stack queue deque collections.deque dict set frozenset Counter defaultdict OrderedDict LRU cache heapq heap min-heap max-heap priority queue top-k car wash LeetCode 1882 Dijkstra bisect queue.Queue asyncio.Queue sortedcontainers linked list union-find lru_cache namedtuple dataclass

SEE ALSO Two Sum / Complement Lookup · Async / Await · Big-O Cheat Sheet · Pattern Recognition Map

ON THIS PAGE

1. Dynamic array · p. 55
2. Stack (LIFO) · p. 55
3. Queue (FIFO) · p. 56
4. Deque · p. 56
5. Hash map · p. 57
6. Default map · p. 57
7. Counter · p. 57
8. Hash set · p. 58
9. Ordered map (LRU) · p. 58
10. Min-heap · p. 58
11. Thread-safe queue · p. 60
12. Async queue · p. 61
13. Binary search · p. 61
14. Sorted map · p. 62
15. Linked list · p. 62
16. Tree / Graph · p. 63
17. Union-Find · p. 64
18. Records · p. 64
19. Memoization · p. 65

**What:** each classic data structure and what to use for it in Python. Most of these are standard library. Click a name to jump to its example.

## Map

↑ TOP

| Data structure | Python | Key ops & cost | Notes |
|---|---|---|---|
| Dynamic array | `list` | index O(1), `append` / `pop()` O(1), `insert(0)` / `pop(0)` **O(n)** | |
| Stack (LIFO) | `list` | `append`, `pop()` | |
| Queue (FIFO) | `collections.deque` | `append`, `popleft()` O(1) | never use `list.pop(0)` |
| **Deque** (double-ended) | `collections.deque` | `append/appendleft`, `pop/popleft` O(1); `deque(maxlen=k)` | sliding window, BFS; indexing the middle is O(n) |
| Hash map | `dict` | get/set/del O(1) avg | insertion-ordered (3.7+) |
| Default map | `collections.defaultdict(list)` | auto-creates missing keys | grouping, adjacency lists |
| Counter / multiset | `collections.Counter` | counts; `.most_common(k)` | frequency problems, anagrams |

| Data structure | Python | Key ops & cost | Notes |
|---|---|---|---|
| Hash set | `set` / `frozenset` | add/in/remove O(1) avg | `frozenset` is hashable, so it can be a dict key |
| Ordered map (LRU) | `collections.OrderedDict` | `move_to_end`, `popitem(last=False)` O(1) | LRU cache |
| **Min-heap / priority queue** | `heapq` (on a list) | push/pop O(log n), peek `h[0]` O(1), `heapify` O(n) | max-heap: push `-x` |
| Thread-safe queue | `queue.Queue`, `queue.PriorityQueue` | blocking `get/put` | producer/consumer with threads |
| Async queue | `asyncio.Queue` | `await q.get()` | producer/consumer in asyncio |
| Binary search | `bisect` | `bisect_left/right` O(log n), `insort` O(n) | on an already-sorted list |
| Sorted map / set (BST) | none in stdlib; `sortedcontainers.SortedList` (3rd party) | O(log n) | in interviews, say "a balanced BST / TreeMap", or use bisect + list |
| Linked list | none, write a `Node` class | | `deque` covers most real uses |
| Tree / Graph | `dict` of lists (`defaultdict(list)`), or `Node` class | | |
| Union-Find | write it (≈10 lines) | ~O(1) amortized | |
| Fixed-size tuple / record | `tuple`, `collections.namedtuple`, `@dataclass` | | |
| Memoization | `functools.lru_cache` / `functools.cache` | | DP |

⚠ **COMMON TRAP**

using `list.pop(0)` as a queue, which is O(n) per pop. Use a `deque`.

↑ TOP

## Examples

### Dynamic array

```python
nums = [3, 1, 4]
nums.append(1)          # O(1) amortized → [3, 1, 4, 1]
nums[2]                 # 4, O(1) index
nums.insert(0, 9)       # O(n): shifts every element
nums.sort()             # in place, O(n log n) → [1, 1, 3, 4, 9]
nums[-2:]               # [4, 9] slicing copies
```

↑ back to table

### Stack (LIFO)

```python
def is_balanced(s):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)                    # push
        elif not stack or stack.pop() ≠ pairs[ch]:   # pop
            return False
    return not stack

is_balanced("{[()]}")   # True
```

↑ back to table

## Queue (FIFO)

PYTHON

```python
from collections import deque

def bfs(graph, start):
    seen, order = {start}, []
    q = deque([start])
    while q:
        node = q.popleft()                    # O(1), unlike list.pop(0)
        order.append(node)
        for nxt in graph[node]:
            if nxt not in seen:
                seen.add(nxt)
                q.append(nxt)
    return order

bfs({"a": ["b", "c"], "b": ["d"], "c": [], "d": []}, "a")   # ['a', 'b', 'c', 'd']
```

↑ back to table

## Deque

PYTHON

```python
from collections import deque

d = deque([2, 3])
d.appendleft(1)         # [1, 2, 3]
d.append(4)             # [1, 2, 3, 4]
d.pop(), d.popleft()    # (4, 1), both O(1)
d.rotate(1)             # [3, 2]

last3 = deque(maxlen=3) # keeps only the last 3; old items fall off the left
for x in range(5):
    last3.append(x)     # → deque([2, 3, 4])

# Sliding window maximum: deque holds indices with decreasing values
def window_max(nums, k):
    dq, out = deque(), []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] ⩽ x:
            dq.pop()
        dq.append(i)
        if dq[0] ⩽ i - k:
            dq.popleft()
        if i ⩾ k - 1:
            out.append(nums[dq[0]])
    return out

window_max([1, 3, -1, -3, 5, 3], 3)   # [3, 3, 5, 5]
```

↑ back to table

### Hash map

```python
ages = {"ali": 30}
ages["sara"] = 25                # set
ages.get("reza", 0)              # 0, no KeyError
"ali" in ages                    # True, O(1) avg
for name, age in ages.items():   # insertion order
    print(name, age)
ages.pop("ali")
```

↑ back to table

### Default map

```python
from collections import defaultdict

words = ["eat", "tea", "tan", "ate", "nat"]
groups = defaultdict(list)       # missing key → new []
for w in words:
    groups["".join(sorted(w))].append(w)
list(groups.values())            # [['eat', 'tea', 'ate'], ['tan', 'nat']]
```

↑ back to table

### Counter

```python
from collections import Counter

c = Counter("mississippi")
c["s"]                           # 4
c.most_common(2)                 # [('i', 4), ('s', 4)]
Counter("listen") == Counter("silent")   # True: anagram check
c["z"]                           # 0, missing keys count as 0
```

↑ back to table

### Hash set

```python
seen = set()
for x in [3, 1, 3, 2, 1]:
    if x in seen:                 # O(1) avg
        print("duplicate", x)
    seen.add(x)

{1, 2, 3} & {2, 3, 4}             # {2, 3}  intersection
{1, 2} | {3}                      # {1, 2, 3} union
visited = {frozenset({1, 2}): True}   # frozenset is hashable → usable as a key
```

↑ back to table

### Ordered map (LRU)

```python
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cap, self.data = capacity, OrderedDict()

    def get(self, key):
        if key not in self.data:
            return -1
        self.data.move_to_end(key)            # mark as most recently used
        return self.data[key]

    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.cap:
            self.data.popitem(last=False)     # evict least recently used

cache = LRUCache(2)
cache.put(1, "a"); cache.put(2, "b"); cache.get(1); cache.put(3, "c")
list(cache.data)                              # [1, 3]: key 2 was evicted
```

↑ back to table

### Min-heap

A binary tree stored in a list where each parent ≤ its children, so **the smallest item is always `h[0]`**. The rest is *not* sorted. Use it whenever you repeatedly need the smallest/earliest/next item of a changing collection: scheduling, top-k, merging k sorted lists, Dijkstra, the median of a stream.

PYTHON

```python
import heapq

h = []
heapq.heappush(h, 5); heapq.heappush(h, 1)
h[0]                                # 1 → peek min, O(1)
heapq.heappop(h)                    # 1 → remove min, O(log n)
nums = [5, 1, 9, 3, 7]
heapq.heapify(nums)                 # list → heap in place, O(n)
heapq.nlargest(2, [5, 1, 9, 3, 7]) # [9, 7]

# Max-heap: negate the values
mx = []
heapq.heappush(mx, -9); -heapq.heappop(mx)   # 9

# Priority + payload: tuples compare left to right (add a tie-breaker if the payload isn't
comparable)
tasks = [(2, "write"), (1, "fix bug"), (3, "deploy")]
heapq.heapify(tasks)
heapq.heappop(tasks)                # (1, 'fix bug')

# Top-k largest in O(n log k): keep a min-heap of size k
def top_k(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)        # drop the smallest
    return sorted(h, reverse=True)

top_k([5, 1, 9, 3, 7], 2)           # [9, 7]
```

**Car wash (assign each car to the worker who is free soonest):**

PYTHON

```python
import heapq

def car_wash(cars, n_workers):
    """cars: list of (arrival, duration). Returns (car_idx, worker, start, end)."""
    free = [(0, w) for w in range(n_workers)]   # (time worker becomes free, worker_id)
    heapq.heapify(free)
    schedule = []
    for i, (arrival, duration) in sorted(enumerate(cars), key=lambda c: c[1][0]):
        free_at, w = heapq.heappop(free)        # earliest-free worker, O(log k)
        start = max(arrival, free_at)           # wait if that worker is still busy
        end = start + duration
        schedule.append((i, w, start, end))
        heapq.heappush(free, (end, w))          # worker is busy until `end`
    return schedule

car_wash([(0, 5), (1, 3), (2, 2)], 2)   # [(0, 0, 0, 5), (1, 1, 1, 4), (2, 1, 4, 6)]
```

O(n log n) to sort the cars + O(n log k) for heap operations with k workers. Scanning for the free worker instead costs O(n·k). If several workers are already free and you must pick the **lowest id** (LeetCode 1882), use **two heaps**: `idle` keyed by `id`, and `busy` keyed by `(free_at, id)`. On each arrival, move every worker with `free_at ⩽ arrival` from busy to idle.

> ⚠ COMMON TRAP
> sorting again or scanning for the min on every step. If you keep needing the min of a changing collection, use a heap.

↑ back to table

### Thread-safe queue

PYTHON

```python
import queue, threading

q = queue.Queue(maxsize=10)        # put() blocks when full → backpressure

def worker():
    while (item := q.get()) is not None:
        print("processed", item)
        q.task_done()
    q.task_done()

t = threading.Thread(target=worker)
t.start()
for i in range(3):
    q.put(i)
q.put(None)                        # sentinel: tell the worker to stop
q.join(); t.join()
```

↑ back to table

### Async queue

```python
import asyncio

async def producer(q):
    for i in range(3):
        await q.put(i)
    await q.put(None)              # sentinel

async def consumer(q):
    while (item := await q.get()) is not None:
        print("got", item)

async def main():
    q = asyncio.Queue()
    await asyncio.gather(producer(q), consumer(q))

asyncio.run(main())
```

↑ back to table

### Binary search

```python
import bisect

scores = [10, 20, 20, 30]          # must already be sorted
bisect.bisect_left(scores, 20)     # 1: first index where 20 could go
bisect.bisect_right(scores, 20)    # 3: after the last 20
bisect.insort(scores, 25)          # [10, 20, 20, 25, 30], O(n) insert

# Grade lookup by thresholds
def grade(score, cuts=[60, 70, 80, 90], letters="FDCBA"):
    return letters[bisect.bisect(cuts, score)]

grade(85)                          # 'B'
```

↑ back to table

### Sorted map

```python
# Third party: pip install sortedcontainers
from sortedcontainers import SortedList, SortedDict

prices = SortedList([30, 10, 20])
prices.add(15)                      # O(log n) → [10, 15, 20, 30]
prices.bisect_left(20)              # 2
prices[0], prices[-1]               # min / max: (10, 30)

book = SortedDict({105: "ask", 101: "bid"})
book.peekitem(0)                    # (101, 'bid'): smallest key
```

In an interview without the library, say "a balanced BST (like Java's `TreeMap`)" and use `bisect` + a list, or a heap if you only need the min.

↑ back to table

### Linked list

```python
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next

def reverse(head):
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev

head = Node(1, Node(2, Node(3)))
r = reverse(head)
[r.val, r.next.val, r.next.next.val]   # [3, 2, 1]
```

↑ back to table

## Tree / Graph

PYTHON

```python
from collections import defaultdict

# Graph: adjacency list
edges = [("a", "b"), ("a", "c"), ("b", "d")]
graph = defaultdict(list)
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)             # drop this line for a directed graph

def dfs(node, seen=None):
    seen = seen if seen is not None else set()
    seen.add(node)
    for nxt in graph[node]:
        if nxt not in seen:
            dfs(nxt, seen)
    return seen

# Tree: Node class
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def depth(root):
    return 0 if root is None else 1 + max(depth(root.left), depth(root.right))

depth(TreeNode(1, TreeNode(2, TreeNode(3))))   # 3
```

↑ back to table

## Union-Find

PYTHON

```python
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]   # path compression
            x = self.parent[x]
        return x

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False               # already connected → a cycle in graph problems
        self.parent[ra] = rb
        return True

uf = UnionFind(4)
uf.union(0, 1); uf.union(2, 3)
uf.find(0) == uf.find(1), uf.find(1) == uf.find(2)   # (True, False)
```

↑ back to table

## Records

PYTHON

```python
from collections import namedtuple
from dataclasses import dataclass, field

point = (3, 4)                         # plain tuple: immutable, hashable
x, y = point

Point = namedtuple("Point", "x y")     # tuple with names
p = Point(3, 4); p.x                   # 3

@dataclass
class Order:                           # mutable, auto __init__/__repr__/__eq__
    id: int
    items: list = field(default_factory=list)   # a fresh list per instance, not shared

@dataclass(frozen=True)
class Key:                             # immutable + hashable → can be a dict key
    user: int
    day: str
```

↑ back to table

### Memoization

PYTHON

```python
from functools import cache

@cache                                  # same as lru_cache(maxsize=None)
def ways(n):                            # climbing stairs, 1 or 2 steps at a time
    if n <= 1:
        return 1
    return ways(n - 1) + ways(n - 2)

ways(50)                                # 20365011074, instant instead of exponential
```

↑ back to table

↑ Back to top ☰ Contents

# 03

# Algorithm Patterns

4 pages. Each card says what the page covers and its key lines; open the page for the full depth.

### Pattern Recognition Map · p. 68

a lookup table from the words in a problem to the pattern that solves it. Read the problem, find the signal words, pick the pattern.

- "Have I seen X before?" / pair with sum → **hash set/dict**, O(n).
- Contiguous subarray/substring + "longest/shortest/at most k" → **sliding window**.
- "Next free / earliest / k largest" → **heap** (`heapq`); shortest path, unweighted → **BFS** with `deque`.
- Sorted input or "minimum X such that..." → **binary search** (also on the answer).
- "All subsets/permutations" → **backtracking**; "count ways / min cost" with overlapping subproblems → **DP**.
- Dependencies / order of tasks → **topological sort** (Kahn's); "connected? merge groups" → **union-find**.

Read the full page →

### Big-O Cheat Sheet · p. 80

how running time and memory grow with input size `n`, and what Python's built-in operations cost.

- ~$10^8$ simple ops/s in C, ~$10^7$ in Python → $n \le 10^5$ needs O(n log n) or better.
- `x in list` is O(n); `x in set/dict` is O(1) avg. `list.pop(0)` / `insert(0)` are O(n) → use `deque`.
- Slicing copies: `a[i:j]` is O(j − i). `s += ch` in a loop can be $O(n^2)$ → `"".join(parts)`.
- `sort` is O(n log n); `heapify` O(n); `heappush/heappop` O(log n).
- Recursion depth counts as space: DFS on a path of n nodes is O(n) space (limit ~1000 in Python).

Read the full page →

### Two Sum / Complement Lookup · p. 84

find two elements with `a + b == target` in an unsorted array.

- Give the answers in order: brute force $O(n^2)$ → sort + two pointers O(n log n) → **hash set O(n)**.
- For each `x`, check whether `target - x` is in `seen`, then add `x`. Use a dict `value → index` if indices are needed.
- Already sorted input: two pointers is O(n) with O(1) space.
- Pattern: "have I seen X before?" → a hash set replaces the inner loop.

Read the full page →

## How to Solve a Coding Question · p. 86

a fixed routine for any coding question, plus what to say out loud at each step.

- Routine: **clarify → examples → brute force + Big-O → optimise (pattern map) → code → test → complexity**.
- Always ask: input size? duplicates? negatives? empty? sorted? return value or index?
- Say the brute force and its complexity **before** optimising. It is never wasted.
- Stuck? "Can I trade memory for time?" → hash set/dict. "Is there a sorted order I can use?" → two pointers / binary search.
- Test by walking through a small example line by line, then an edge case.

Read the full page →

CHAPTER 3 · ALGORITHM PATTERNS

# Pattern Recognition Map

pattern recognition · hash map · hash set · two pointers · sliding window · prefix sum · binary search · binary search on the answer · BFS · DFS · backtracking · heap · top-k · scheduling · intervals · merge intervals · monotonic stack · next greater element · union-find · dynamic programming · DP · memoization · greedy · topological sort · Kahn's algorithm

SEE ALSO Two Sum / Complement Lookup · Python Data Structures Map · Big-O Cheat Sheet · How to Solve a Coding Question

ON THIS PAGE

1. Map · p. 68
2. Hash map and set · p. 69
3. Two pointers · p. 70
4. Sliding window · p. 70
5. Prefix sums · p. 71
6. Binary search · p. 72
7. BFS · p. 72
8. DFS and backtracking · p. 73
9. Heap (top-k, scheduling) · p. 74
10. Intervals · p. 75
11. Monotonic stack · p. 75
12. Union-find · p. 76
13. Dynamic programming · p. 77
14. Greedy · p. 78
15. Topological sort · p. 78

**What:** a lookup table from the words in a problem to the pattern that solves it. Read the problem, find the signal words, pick the pattern.

> **i WHY IT MATTERS**
> most interview questions are one of ~15 patterns. If you recognise the pattern, the solution is mostly a template. If you don't, you get stuck on brute force.

## Map

↑ TOP

| If the problem says... | Pattern | Typical complexity |
|---|---|---|
| "pair/complement with sum k", "seen before?", "duplicates", "count frequency" | Hash map / set | O(n) time, O(n) space |
| sorted array, "pair in sorted", "palindrome", "remove duplicates in place" | Two pointers | O(n), O(1) space |
| "contiguous subarray/substring", "longest/shortest window", "at most k distinct" | Sliding window | O(n) |
| "sum of range i..j", "subarray sum equals k" | Prefix sums | O(n) build, O(1) query |
| sorted input, "find first/last", "minimum capacity/speed such that..." | Binary search | O(log n) or O(n log range) |

| If the problem says... | Pattern | Typical complexity |
|---|---|---|
| "shortest path", "minimum steps", grid, unweighted graph, "level by level" | BFS | O(V + E) |
| "all subsets / permutations / combinations", "generate all" | DFS / backtracking | $O(2^n)$ or O(n!) |
| "k largest/smallest", "next free worker", "merge k sorted", "earliest" | Heap | O(n log k) |
| "overlapping intervals", "meeting rooms", "merge ranges" | Intervals | O(n log n) |
| "next greater/smaller element", "daily temperatures", "histogram" | Monotonic stack | O(n) |
| "connected components", "are a and b connected?", "redundant edge" | Union-find | ~O(1) per op |
| "number of ways", "min/max cost", "can you reach", choices that repeat | Dynamic programming | O(n) or O(n·m) |
| "maximum number of non-overlapping", "minimum coins (canonical)", local best choice works | Greedy | O(n log n) usually |
| "prerequisites", "build order", "dependencies", "detect cycle in directed graph" | Topological sort | O(V + E) |

> ⚠ COMMON TRAP
> jumping to code before matching the problem to a row. Read the table first, then pick.

## Hash map and set

↑ TOP

**When:** "have I seen this before?", complement lookup, counting, grouping. A set replaces an inner loop. Full page: Two Sum.

PYTHON

```python
def has_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:          # O(1) average
            return True
        seen.add(x)
    return False

print(has_duplicate([3, 1, 4, 1]))   # True
```

**Complexity:** O(n) time, O(n) space.

↑ back to map

↑ TOP

## Two pointers

**When:** the input is **sorted** (or you can sort it), or you compare both ends (palindrome). One pointer at each end, move toward the middle.

PYTHON

```python
def pair_sum_sorted(nums, target):
    l, r = 0, len(nums) - 1
    while l < r:
        s = nums[l] + nums[r]
        if s == target:
            return l, r
        if s < target:
            l += 1              # need bigger sum
        else:
            r -= 1              # need smaller sum
    return None

print(pair_sum_sorted([1, 2, 4, 7, 11], 9))   # (1, 3)
```

**Complexity:** O(n) time, O(1) space.

↑ back to map

## Sliding window

↑ TOP

**When: contiguous** subarray/substring, "longest / shortest / max sum of size k". Two kinds:

- **Fixed size k:** add the new right item, remove the item that leaves on the left.
- **Variable size:** grow `right` every step; shrink `left` while the window is invalid.

PYTHON

```python
def max_sum_k(nums, k):                      # fixed window
    s = best = sum(nums[:k])
    for i in range(k, len(nums)):
        s += nums[i] - nums[i - k]           # add new, drop old
        best = max(best, s)
    return best

def longest_unique(s):                       # variable window
    last, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left:
            left = last[ch] + 1              # shrink: jump past the duplicate
        last[ch] = right
        best = max(best, right - left + 1)
    return best

print(max_sum_k([2, 1, 5, 1, 3, 2], 3))    # 9
print(longest_unique("abcabcbb"))           # 3
```

**Complexity:** O(n): each item enters and leaves the window at most once. For "max of each window" use a `deque`, see Deque.

↑ back to map

## Prefix sums

↑ TOP

**When:** many "sum of range i..j" queries, or "number of subarrays with sum k". `prefix[i]` = sum of the first `i` items, so `sum(i..j) = prefix[j+1] - prefix[i]`.

PYTHON

```python
from collections import defaultdict

def subarray_sum_k(nums, k):
    count, total = 0, 0
    seen = defaultdict(int)
    seen[0] = 1                     # empty prefix
    for x in nums:
        total += x
        count += seen[total - k]    # earlier prefix that makes a sum of k
        seen[total] += 1
    return count

print(subarray_sum_k([1, 1, 1], 2))      # 2
print(subarray_sum_k([3, 4, -7, 7], 7))  # 3
```

**Complexity:** O(n) time, O(n) space. Works with negative numbers (sliding window does not).

↑ back to map

↑ TOP

## Binary search

**When:** sorted input, or **binary search on the answer**: "find the minimum X such that `ok(X)` is true", where `ok` is monotonic (false, false, ..., true, true).

PYTHON

```python
def first_true(lo, hi, ok):
    """Smallest x in [lo, hi] with ok(x) True (ok must be monotonic)."""
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid                # mid might be the answer
        else:
            lo = mid + 1
    return lo

# Classic: lowest index with nums[i] ⩾ target (same as bisect_left)
nums = [1, 3, 5, 7]
print(first_true(0, len(nums), lambda i: i == len(nums) or nums[i] ⩾ 5))   # 2

# On the answer: minimum ship capacity to ship all weights within `days`
def ship_within_days(weights, days):
    def ok(cap):
        need, load = 1, 0
        for w in weights:
            if load + w > cap:
                need, load = need + 1, 0
            load += w
        return need ⩽ days
    return first_true(max(weights), sum(weights), ok)

print(ship_within_days([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5))   # 15
```

**Complexity:** O(log n) on an array; O(n · log(range)) on the answer.

↑ back to map

## BFS

↑ TOP

**When: shortest path in an unweighted** graph or grid, "minimum steps", level-by-level. Use `collections.deque` (never `list.pop(0)`).

PYTHON

```python
from collections import deque

def shortest_path(grid):
    """Steps from top-left to bottom-right; 0 = open, 1 = wall."""
    rows, cols = len(grid), len(grid[0])
    q = deque([(0, 0, 0)])                # (row, col, distance)
    seen = {(0, 0)}
    while q:
        r, c, d = q.popleft()
        if (r, c) == (rows - 1, cols - 1):
            return d
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 0 and (nr, nc) not in
seen:
                seen.add((nr, nc))        # mark when adding, not when popping
                q.append((nr, nc, d + 1))
    return -1

print(shortest_path([[0, 0, 0],
                     [1, 1, 0],
                     [0, 0, 0]]))          # 4
```

**Complexity:** O(V + E); for a grid O(rows · cols). Weighted edges → Dijkstra (heap).

↑ back to map

## DFS and backtracking

↑ TOP

**When:** "generate all" subsets, permutations, combinations; explore every path. Template: **choose → recurse → un-choose**.

PYTHON

```python
def subsets(nums):
    out, path = [], []
    def dfs(i):
        if i == len(nums):
            out.append(path[:])            # copy!
            return
        path.append(nums[i]); dfs(i + 1); path.pop()   # take nums[i]
        dfs(i + 1)                                     # skip nums[i]
    dfs(0)
    return out

def permutations(nums):
    out, path, used = [], [], [False] * len(nums)
    def dfs():
        if len(path) == len(nums):
            out.append(path[:])
            return
        for i, x in enumerate(nums):
            if not used[i]:
                used[i] = True; path.append(x)
                dfs()
                used[i] = False; path.pop()   # undo the choice
    dfs()
    return out

print(subsets([1, 2]))            # [[1, 2], [1], [2], []]
print(len(permutations([1, 2, 3])))   # 6
```

**Complexity:** subsets $O(n \cdot 2^n)$, permutations $O(n \cdot n!)$. Recursion depth $O(n)$.

↑ back to map

## Heap (top-k, scheduling)

↑ TOP

**When:** you **repeatedly need the smallest/largest** of a changing collection: top-k, "next free worker", merge k sorted lists, Dijkstra. Details and the car-wash example: Min-heap.

```python
import heapq

def min_rooms(meetings):
    """Meeting rooms II: heap holds end times of busy rooms."""
    ends = []
    for start, end in sorted(meetings):
        if ends and ends[0] <= start:      # earliest-free room is free
            heapq.heapreplace(ends, end)   # reuse it
        else:
            heapq.heappush(ends, end)      # need a new room
    return len(ends)

print(min_rooms([(0, 30), (5, 10), (15, 20)]))   # 2
print(heapq.nlargest(2, [5, 1, 9, 3]))           # [9, 5]
```

**Complexity:** push/pop O(log k); top-k of n items O(n log k).

↑ back to map

## Intervals

↑ TOP

**When:** ranges that may overlap: merge, insert, meeting rooms. Almost always: **sort by start**, then one pass.

```python
def merge(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:            # overlaps the last one
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out

print(merge([[1, 3], [8, 10], [2, 6], [15, 18]]))   # [[1, 6], [8, 10], [15, 18]]
```

**Complexity:** O(n log n) for the sort, O(n) for the pass.

↑ back to map

## Monotonic stack

↑ TOP

**When:** "next greater / next smaller element", "how many days until warmer", largest rectangle in histogram. The stack keeps indices whose values are in decreasing order; a bigger value pops them and is their answer.

```python
def next_greater(nums):
    ans = [-1] * len(nums)
    stack = []                              # indices, values decreasing
    for i, x in enumerate(nums):
        while stack and nums[stack[-1]] < x:
            ans[stack.pop()] = x            # x is the next greater for that index
        stack.append(i)
    return ans

print(next_greater([2, 1, 2, 4, 3]))   # [4, 2, 4, -1, -1]
```

**Complexity:** O(n): each index is pushed and popped once.

↑ back to map

## Union-find

↑ TOP

**When:** groups that **merge** over time: connected components, "are a and b connected?", detect a cycle in an undirected graph. Class code: Union-Find.

```python
def count_components(n, edges):
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]   # path compression
            x = parent[x]
        return x
    groups = n
    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb
            groups -= 1
    return groups

print(count_components(5, [(0, 1), (1, 2), (3, 4)]))   # 2
```

**Complexity:** ~O(1) amortized per operation (with path compression + union by rank). BFS/DFS also works for static graphs; union-find shines when edges arrive one by one.

↑ back to map

↑ TOP

## Dynamic programming

**When:** "number of ways", "min/max cost", "is it possible", and the same subproblem appears many times. Steps: define `dp[i]` in words → write the recurrence → base cases → order.

- **Top-down:** recursion + `@cache`. Easy to write from the recurrence.
- **Bottom-up:** fill a table in order. No recursion limit; often O(1) space.

PYTHON

```python
from functools import cache

# 1D: climbing stairs (1 or 2 steps), top-down
@cache
def climb(n):
    return 1 if n <= 1 else climb(n - 1) + climb(n - 2)

# 1D: house robber, bottom-up with two variables
def rob(houses):
    prev, cur = 0, 0                # best up to i-2, best up to i-1
    for h in houses:
        prev, cur = cur, max(cur, prev + h)   # skip h, or take h
    return cur

# 2D: longest common subsequence
def lcs(a, b):
    dp = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[-1][-1]

# 2D: unique paths in an m x n grid (move right or down)
def grid_paths(m, n):
    row = [1] * n
    for _ in range(1, m):
        for j in range(1, n):
            row[j] += row[j - 1]     # from above + from left
    return row[-1]

print(climb(5))                 # 8
print(rob([2, 7, 9, 3, 1]))     # 12
print(lcs("abcde", "ace"))      # 3
print(grid_paths(3, 7))         # 28
```

**Complexity:** (number of states) × (work per state): 1D O(n), 2D O(n·m).

↑ back to map

↑ TOP

## Greedy

**When:** a local best choice is always safe: "maximum number of non-overlapping intervals", "minimum arrows/platforms", jump game. Usually **sort, then pick**. If you can't argue why greedy is safe, try DP.

PYTHON

```python
def max_non_overlapping(intervals):
    count, end = 0, float("-inf")
    for s, e in sorted(intervals, key=lambda x: x[1]):   # earliest end first
        if s ⩾ end:
            count += 1
            end = e
    return count

print(max_non_overlapping([(1, 2), (2, 3), (3, 4), (1, 3)]))   # 3
```

**Complexity:** O(n log n) for the sort.

↑ back to map

## Topological sort

↑ TOP

**When:** "prerequisites", "build/install order", "task dependencies", "is there a cycle in a directed graph?". **Kahn's algorithm:** start with nodes that have no incoming edges; remove them and update in-degrees.

PYTHON

```python
from collections import deque, defaultdict

def course_order(n, prereqs):
    """prereqs: (course, needs) pairs. Returns an order or [] if there is a cycle."""
    graph, indeg = defaultdict(list), [0] * n
    for course, need in prereqs:
        graph[need].append(course)
        indeg[course] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for nxt in graph[node]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                q.append(nxt)
    return order if len(order) == n else []    # leftover nodes → cycle

print(course_order(4, [(1, 0), (2, 0), (3, 1), (3, 2)]))   # [0, 1, 2, 3]
print(course_order(2, [(0, 1), (1, 0)]))                   # []
```

**Complexity:** O(V + E).

↑ back to map

✓ KEY POINTS

- Look for signal words first: *contiguous*, *sorted*, *shortest*, *all*, *k-th*, *next greater*, *dependencies*.
- "Can I trade memory for time?" usually means a hash map/set or prefix sums.
- Many problems combine two patterns: sort + two pointers, BFS + set, heap + greedy.

⚠ COMMON TRAP

using a sliding window when numbers can be negative (the window sum is not monotonic). Use prefix sums + hash map instead.

↑ Back to top ☰ Contents

CHAPTER 3 · ALGORITHM PATTERNS

# Big-O Cheat Sheet

big-o time complexity space complexity amortized analysis recursion stack input size limits
list operations cost dict cost set cost deque cost heapq cost sort complexity string concatenation

SEE ALSO Pattern Recognition Map · Python Data Structures Map · How to Solve a Coding Question · V2·10 Real-time Gaming Leaderboard · V2·13 Stock Exchange

ON THIS PAGE

1. Complexity classes · p. 80
2. What n is allowed in ~1 second · p. 80
3. Python built-in costs · p. 81
4. Amortized analysis · p. 82
5. Space complexity · p. 82
6. How to state complexity in an interview · p. 82

**What:** how running time and memory grow with input size `n`, and what Python's built-in operations cost.

> **i WHY IT MATTERS**
> the interviewer always asks "what is the complexity?". The input size also tells you which complexity they expect.

## Complexity classes

↑ TOP

| Class | Name | Example |
|---|---|---|
| O(1) | constant | dict lookup, `list.append`, `h[0]` |
| O(log n) | logarithmic | binary search, heap push/pop |
| O(n) | linear | one pass, hash-set Two Sum, BFS on a list |
| O(n log n) | linearithmic | sort, n heap operations |
| $O(n^2)$ | quadratic | all pairs, nested loops |
| $O(2^n)$ | exponential | all subsets, naive Fibonacci |
| O(n!) | factorial | all permutations |

## What n is allowed in ~1 second

↑ TOP

Python does roughly $10^7$ simple operations per second. Use the input size to guess the target:

| n up to | Target complexity | Typical pattern |
|---|---|---|
| ~10 | O(n!) | permutations |
| ~20 | $O(2^n)$ | subsets, bitmask |
| ~500 | $O(n^3)$ | 3 nested loops, small 2D DP + loop |
| ~5,000 | $O(n^2)$ | all pairs, 2D DP |
| ~$10^6$ | O(n log n) | sort, heap, binary search |
| ~$10^7$+ | O(n) or O(log n) | one pass, hash map, math |

## Python built-in costs

↑ TOP

| Operation | Cost | Note |
|---|---|---|
| `list[i]`, `list.append(x)`, `list.pop()` | O(1) | append is amortized O(1) |
| `list.insert(0, x)`, `list.pop(0)` | **O(n)** | shifts every element → use `deque` |
| `x in list`, `list.index(x)`, `list.remove(x)` | **O(n)** | linear scan |
| `a[i:j]` slicing, `list(a)`, `a.copy()` | O(j − i) / O(n) | makes a copy |
| `a + b` (lists) | O(len(a) + len(b)) | new list |
| `dict[k]`, `k in dict`, `dict[k] = v`, `del dict[k]` | O(1) avg | O(n) worst case (hash collisions) |
| `x in set`, `set.add`, `set.remove` | O(1) avg | |
| `s1 & s2` | O(min(len(s1), len(s2))) | |
| `deque.append/appendleft/pop/popleft` | O(1) | `deque[i]` in the middle is O(n) |
| `heapq.heappush/heappop` | O(log n) | `h[0]` peek is O(1) |
| `heapq.heapify(list)` | O(n) | not O(n log n) |
| `heapq.nlargest(k, a)` | O(n log k) | |
| `sorted(a)`, `a.sort()` | O(n log n) | Timsort; O(n) if already sorted |
| `bisect.bisect_left` | O(log n) | `bisect.insort` is O(n) (list insert) |
| `min(a)`, `max(a)`, `sum(a)`, `len(a)` | O(n), O(n), O(n), **O(1)** | |
| `s += ch` in a loop | up to **$O(n^2)$** total | each `+=` may copy the string → use `"".join(parts)` |
| `"".join(parts)` | O(total length) | |

```python
from collections import deque

parts = []
for i in range(5):
    parts.append(str(i))       # O(1) amortized each
print("".join(parts))          # 01234, O(n) total

q = deque([1, 2, 3])
print(q.popleft())             # 1, O(1); list.pop(0) would be O(n)

nums = list(range(10))
print(9 in nums, 9 in set(nums))   # True True: O(n) scan vs O(1) avg (building the set is O(n))
```

## Amortized analysis

↑ TOP

Some operations are usually cheap but sometimes expensive. **Amortized** cost is the average over a long sequence. Example: `list.append` is O(1) almost always, but when the array is full Python copies it into a bigger one (O(n)). Because the size grows by a constant factor, these copies are rare: n appends cost O(n) in total, so each append is **O(1) amortized**. Same idea for dict/set resizing and union-find with path compression. Say "amortized" when the worst single call is slow but the total is fast.

## Space complexity

↑ TOP

- Count **extra** memory you create: sets, dicts, new lists, the output (say whether you include it).
- **The recursion stack counts.** Each call frame is memory. DFS on a tree is O(h) (h = height, O(n) when the tree is a line). `@cache` on `f(n)` stores n results **and** goes n calls deep.
- Python's default recursion limit is about 1000. For deep inputs, use an explicit stack or bottom-up DP.

```python
import sys

def depth(n):
    return 0 if n == 0 else 1 + depth(n - 1)   # O(n) stack space

print(sys.getrecursionlimit())   # 1000 (default)
print(depth(500))                # 500
```

## How to state complexity in an interview

↑ TOP

1. Name the variables: "n is the length of the array, k is the number of workers".

2. Time: go through the code top to bottom. Sort = n log n, loop = n, inside the loop a heap op = log k → "O(n log n + n log k)".
3. Keep only the biggest term, but keep separate variables: O(n + m), not O(n).
4. Space: "O(n) for the set; O(1) extra if we don't count the output".
5. Mention average vs worst case for hash maps if asked: "O(1) average, O(n) worst".

✓ KEY POINTS

- Input size hints at the expected complexity: n = $10^5$ means $O(n^2)$ is too slow.
- The hidden O(n) costs in Python: `in list`, `pop(0)`, `insert(0)`, slicing, string `+=`.
- Recursion uses stack space; say it.

❝ INTERVIEW LINE

*"Time is O(n log n) because of the sort; the loop after it is O(n). Space is O(n) for the hash map."*

⚠ COMMON TRAP

calling a loop "O(n)" when its body does `x in some_list` or `list.pop(0)`. That makes it $O(n^2)$.

↑ Back to top ☰ Contents

CHAPTER 3 · ALGORITHM PATTERNS

# Two Sum / Complement Lookup

two sum  hash set  complement lookup  two pointers  3Sum  time-space tradeoff

SEE ALSO  Python Data Structures Map · How to Solve a Coding Question · Pattern Recognition Map

**What:** find two elements with `a + b == target` in an unsorted array.

**The three levels (say them in this order):**

| Approach | Time | Space | Idea |
|---|---|---|---|
| Brute force | $O(n^2)$ | O(1) | check every pair |
| Sort + two pointers | O(n log n) | O(1)–O(n) | `l` at the start, `r` at the end. If the sum is too small, move `l` right; if too big, move `r` left. |
| **Hash set** | **O(n)** | O(n) | for each `x`, ask "have I already seen `target - x`?" |

PYTHON

```python
def two_sum(nums, target=7):
    seen = set()
    for x in nums:
        if target - x in seen:      # O(1) average lookup
            return (target - x, x)
        seen.add(x)                 # add AFTER checking, so x isn't paired with itself
    return None

# Need indices? Use a dict: value → index
def two_sum_idx(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return seen[target - x], i
        seen[x] = i
```

**The general pattern:** "I need something I've already seen, quickly" means a **hash set/dict**. It turns an inner loop (O(n)) into a lookup (O(1)). You trade memory for time.

✓ KEY POINTS

- One pass. Each element only looks *backwards* at what's been seen.
- Two pointers only work on **sorted** input. If the array is already sorted, that's O(n) with O(1) space, which beats the hash set.
- Follow-ups to expect: return all pairs, handle duplicates, 3Sum (sort, then two pointers inside a loop, $O(n^2)$), or a data stream (keep the set).

> ⚠ COMMON TRAP
> stopping at sorting. When you need O(n) and the input is unsorted, ask yourself: "can a hash set remove the inner loop?"

↑ Back to top ☰ Contents

CHAPTER 3 · ALGORITHM PATTERNS

# How to Solve a Coding Question

coding interview · problem solving · clarifying questions · edge cases · brute force · optimisation · testing · think out loud · asking for a hint · stuck

SEE ALSO Pattern Recognition Map · Big-O Cheat Sheet · Two Sum / Complement Lookup

ON THIS PAGE


1. The routine · p. 86
2. 1. Restate and clarify · p. 86
3. 2. Examples and edge cases · p. 87
4. 3. Brute force first · p. 87
5. 4. Optimise · p. 87
6. 5. Code cleanly · p. 87
7. 6. Test by walking through · p. 88
8. 7. State the complexity · p. 88
9. When you are stuck · p. 88


**What:** a fixed routine for any coding question, plus what to say out loud at each step.

> **i WHY IT MATTERS**
> interviewers grade the process, not only the final code. A clear routine also stops you from freezing: you always know the next step.

## The routine

↑ TOP

| Step | What you do | Time (45-min round) |
|---|---|---|
| 1. Restate + clarify | Say the problem in your words, ask questions | 2–3 min |
| 2. Examples + edge cases | Write 1 normal example and 2–3 edge cases | 2–3 min |
| 3. Brute force | Describe it and give its Big-O | 2 min |
| 4. Optimise | Use the Pattern Map; agree on the approach | 5–10 min |
| 5. Code | Clean, small functions, good names | 15 min |
| 6. Test | Walk through your example, then edge cases | 5 min |
| 7. Complexity | Time and space, with variable names | 1 min |

## 1. Restate and clarify

↑ TOP

Ask before you design. The answers often decide the pattern:

- **Input size?** $n = 10^5$ → you need O(n log n) or better (see Big-O).
- **Duplicates?** Changes sets vs counts, and "all pairs" answers.
- **Negative numbers / zero?** Negatives break sliding window → prefix sums.

- **Empty input / one element?** What should I return?
- **Sorted?** Sorted → two pointers or binary search.
- **Output:** values or indices? Any order? One answer or all?
- **Memory limits?** In-place required?

Say: *"Just to confirm: I get an unsorted list of integers, can include negatives and duplicates, and I return the indices of one pair. Is that right?"*

## 2. Examples and edge cases

↑ TOP

Write a small normal example and solve it by hand. Then list edge cases: empty, one element, all the same, negatives, no answer, very large n.

Say: *"Let me write a quick example to make sure I understand:* `[2, 7, 11, 15]`*, target 9 → indices 0 and 1."*

## 3. Brute force first

↑ TOP

State the simplest correct solution and its cost, even if it is slow. It shows you understand the problem and gives you a fallback.

Say: *"The brute force is to check every pair. That's $O(n^2)$ time and $O(1)$ space. With n up to $10^5$ that's too slow, so let me improve it."*

## 4. Optimise

↑ TOP

Find the **bottleneck** in the brute force (usually the inner loop), then match the problem to the Pattern Map. Questions to ask yourself:

- "Can I trade memory for time?" → hash set/dict removes the inner loop (Two Sum).
- "Would sorting help?" → two pointers, intervals, greedy.
- "Is the input contiguous?" → sliding window / prefix sums.
- "Do I keep needing the min/max of a changing set?" → heap.
- "Am I solving the same subproblem again?" → DP with `@cache`.
- "Is it a graph in disguise?" (grid, dependencies, states) → BFS / DFS / topological sort.

Say: *"The inner loop is looking for* `target - x`*. If I keep the values I've seen in a hash set, that lookup becomes O(1), so the whole thing is O(n) time and O(n) space. Does that sound good before I code it?"*

Get a "yes" before coding.

## 5. Code cleanly

↑ TOP

- Good names (`seen`, `left`, `free_at`), not `a`, `b`, `tmp2`.

- Small helper functions for repeated logic.
- Handle edge cases at the top (`if not nums: return []`).
- Keep talking, but briefly: *"Now I add x after the check, so x doesn't pair with itself."*

## 6. Test by walking through

↑ TOP

Don't just say "looks right". Trace your example through the code, writing variable values as you go. Then run one edge case (empty, duplicates). If you find a bug, say so calmly and fix it; that is a good signal, not a bad one.

PYTHON

```python
def two_sum(nums, target):
    seen = {}                           # value → index
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []

# walk-through: i=0 x=2 → 7 not in {} → seen={2:0}; i=1 x=7 → 2 in seen → [0, 1]
print(two_sum([2, 7, 11, 15], 9))   # [0, 1]
print(two_sum([3, 3], 6))           # [0, 1]  duplicates
print(two_sum([], 5))               # []      empty
```

## 7. State the complexity

↑ TOP

Say: *"Time is O(n): one pass with O(1) average dict lookups. Space is O(n) for the dict."* See How to state complexity.

## When you are stuck

↑ TOP

Silence is the worst option. Think out loud and use these moves in order:

1. **Go back to brute force** and find the slow part: *"The slow part is this inner loop. What is it searching for?"*
2. **Ask the pattern questions** from step 4, out loud.
3. **Try a smaller or special case:** sorted input, n = 3, all positive.
4. **Ask for a hint gracefully** (after 3–5 minutes stuck). Show what you tried: - *"I have an O($n^2$) solution. I think I could trade memory for time, but I'm not sure what to store. Am I on the right track?"* - *"I'm thinking of either a heap or sorting here. Which direction would you look at?"*
5. When you get a hint, **use it right away** and say how it changes the plan. Taking a hint well is a positive signal.

✓ KEY POINTS

- Clarify first; the answers (sorted? negatives? size?) often choose the pattern for you.
- Brute force + complexity always comes before the optimal solution.
- Agree on the approach before writing code.
- Test by tracing, not by saying "I think it works".

❝ INTERVIEW LINE

*"Let me start with the brute force so we have something correct, then look at what makes it slow."*

⚠ COMMON TRAP

going quiet and coding straight away. If you pick the wrong approach, you lose 15 minutes and the interviewer can't help you because they don't know what you are thinking.

↑ Back to top ☰ Contents

# 04

# System Design

5 pages. Each card says what the page covers and its key lines; open the page for the full depth.

### How to Run a System Design Interview · p. 92

a fixed path to follow in **every** system design interview (45–60 min), so you never freeze or jump into boxes too early. The interviewer grades **how you think**: clarifying, prioritizing, justifying trade-offs. A "perfect architecture" isn't the goal.

- **Drive it in order:** requirements → estimates → API → data model → high-level design → deep dives → wrap-up.
- Spend the first ~5–8 min on **questions**, not boxes. Write down the functional + non-functional requirements and agree on them.
- Estimates: **1M requests/day ≈ 12 QPS**, peak ≈ 2–3× average; storage = items/day × size × retention.
- Start with the **simplest design that works**, then scale only the bottleneck the numbers point to.
- Every choice gets a **trade-off sentence**: "I pick X because Y; the cost is Z."

Read the full page →

### Kafka vs RabbitMQ · p. 96

both move messages between services asynchronously, but they're different models.

- RabbitMQ = **queue** (routed, deleted on ack). Kafka = **log** (retained, replayable, consumers track offsets).
- Task/job queue, complex routing, per-message retry/TTL/priority → **RabbitMQ**.
- Event stream, many independent consumers, replay, high throughput → **Kafka**.
- Kafka orders only **per partition** (pick the key); parallelism ≤ partition count.
- Both are at-least-once → idempotent consumers; poison messages → DLQ (DLX in RabbitMQ / retry + DLQ topics in Kafka).

Read the full page →

## The 4-Step Framework (Alex Xu, with a News Feed Example) · p. 100

the framework from *System Design Interview* (Alex Xu), chapter 3. It has 4 steps. It is the same idea as the 7-step path in coarser steps: there, steps 1–2 are Xu's step 1, steps 3–5 are his step 2, step 6 is his step 3, and step 7 is his step 4. Each step below has a short version and a worked **news feed** example.

- **4 steps in 45 min:** scope (3–10) → high-level design + buy-in (10–15) → deep dive (10–25) → wrap-up (3–5).
- **Don't be "Jimmy":** ask about features, users, growth and tech stack first; write the assumptions down.
- Get **buy-in** on the box diagram before going deep; walk through 1–2 **concrete use cases** to find edge cases.
- Deep-dive what the **interviewer** cares about; skip trivia (e.g. EdgeRank ranking details).
- Wrap-up: bottlenecks, recap, failures, ops, the next 10×. **Never say the design is perfect.**

Read the full page →

## Redis Pub/Sub vs RabbitMQ vs Kafka · p. 105

`PUBLISH channel msg` pushes the message to every client **subscribed right now**, then Redis forgets it.

- Redis Pub/Sub = **live broadcast, nothing stored**: offline or slow subscribers miss messages (at-most-once, no ack).
- Every subscriber gets every message, so it's **not a work queue**.
- Use it for signals that are fine to lose: WebSocket fan-out across servers, typing/online indicators, cache invalidation, config reload.
- Need durability in Redis? **Streams** (`XADD` / `XREADGROUP` / `XACK`): log + consumer groups + pending list, but memory-bound.

Read the full page →

## Celery with Redis as Broker · p. 107

Celery talks to brokers through **Kombu**, which **emulates RabbitMQ (AMQP) concepts on top of Redis**: queues become Redis **Lists**, bindings are stored in Redis sets, and fanout exchanges become `PUBLISH`.

- Celery + Redis uses **Lists** for tasks (`LPUSH` to send, `BRPOP` to receive → one worker per task), **not** Pub/Sub.
- Pub/Sub is used only for broadcast control (`inspect` / `revoke`), monitoring events, and result notifications.
- No native acks → Kombu uses an `unacked` hash + **`visibility_timeout` (1h)**: a crashed worker's tasks come back only after ~1h.
- Long ETA/countdown tasks (longer than `visibility_timeout`) can **run twice**.
- Default acks **early** (at-most-once). `task_acks_late=True` → at-least-once → **idempotent tasks**. Long tasks → `worker_prefetch_multiplier=1`.

Read the full page →

CHAPTER 4 · SYSTEM DESIGN

# How to Run a System Design Interview

system design framework · requirements · functional requirements · non-functional requirements · back-of-the-envelope · capacity estimation · QPS · API design · data model · high-level design · deep dive · bottleneck · trade-offs · latency numbers · CAP

SEE ALSO Kafka vs RabbitMQ · Redis Pub/Sub vs RabbitMQ vs Kafka · Celery with Redis as Broker · The 4-Step Framework (Alex Xu, with a News Feed Example) · SQL vs NoSQL & Scaling Databases

ON THIS PAGE

1. 1. Requirements: ask before drawing · p. 92
2. 2. Estimates: rough, fast, out loud · p. 93
3. 3. API · p. 93
4. 4. Data model · p. 94
5. 5. High-level design · p. 94
6. 6. Deep dives: where you get the most credit · p. 94
7. 7. Wrap-up · p. 95
8. Communication habits · p. 95

**What:** a fixed path to follow in **every** system design interview (45–60 min), so you never freeze or jump into boxes too early. The interviewer grades **how you think**: clarifying, prioritizing, justifying trade-offs. A "perfect architecture" isn't the goal.

**Time budget (45 min; scale it for 60):**

| # | Step | Time | Output on the board |
|---|---|---|---|
| 1 | Requirements | 5–8 min | functional list, non-functional list, out-of-scope |
| 2 | Estimates | 3–5 min | QPS (avg/peak), read:write ratio, storage, bandwidth |
| 3 | API | 3–5 min | 3–5 main endpoints/events |
| 4 | Data model | 3–5 min | entities, keys, DB choice |
| 5 | High-level design | 10 min | boxes and arrows for the main flows |
| 6 | Deep dives | 10–15 min | 2–3 bottlenecks solved in detail |
| 7 | Wrap-up | 2–3 min | trade-offs, failures, monitoring, what's next |

## 1. Requirements: ask before drawing

↑ TOP

**Functional (what it does):** list the 3–5 core features and agree on them. Explicitly park the rest: "I'll leave search and admin out of scope unless you want them."

**Non-functional (how well):** go through this checklist out loud:

- **Scale:** how many users / DAU, requests per second, data size, growth?
- **Read vs write heavy?** (This decides caching and replicas.)
- **Latency:** what has to be fast (e.g. p99 < 200 ms for the feed)?

- **Consistency vs availability:** is stale data OK (a feed, a like count) or not (payments, inventory)?
- **Durability:** can we ever lose data?
- **Special:** real-time? ordering? global/multi-region? security/privacy?

**Questions to always ask:** "Who are the users and how many?", "What's the most important flow?", "Is it read-heavy or write-heavy?", "Is eventual consistency acceptable here?"

## 2. Estimates: rough, fast, out loud

↑ TOP

Round aggressively; the goal is the **order of magnitude**, which drives decisions.

- **1 day ≈ 86,400 s ≈ $10^5$ s**, so **1M requests/day ≈ 12 QPS**, and 100M/day ≈ 1,200 QPS.
- **Peak ≈ 2–3× average.**
- **Storage** = new items/day × item size × retention. Example: 10M posts/day × 1 KB × 365 days × 5 years ≈ **18 TB**.
- **Bandwidth** = QPS × response size.
- Size units: 1 KB text row, ~100 KB–1 MB image, ~MBs/s video.

**Latency numbers to know (rough):**

| Operation | Time |
|---|---|
| Memory read | ~100 ns |
| SSD random read | ~100 µs |
| Round trip within a datacenter | ~0.5 ms |
| HDD seek | ~10 ms |
| Round trip across continents | ~150 ms |

**Rough single-node capacity:** a Redis node handles ~100k simple ops/s; a well-indexed Postgres/MySQL node handles thousands to ~10k simple queries/s; a Kafka broker handles hundreds of MB/s. Then say what that implies: "1,200 write QPS fits one primary DB; 50k read QPS needs cache + replicas."

## 3. API

↑ TOP

Write only the core calls, with their key params and responses:

```
POST /orders            {user_id, items}       → {order_id, status}
GET  /orders/{id}                              → order
GET  /feed?user_id=&cursor=&limit=20           → {items, next_cursor}
```

Mention **pagination (cursor)**, **idempotency keys** for writes that must not repeat (payments), auth, and rate limiting. For async flows, name the **events** (`order_created`) instead of endpoints.

## 4. Data model

↑ TOP

- List the entities and their main fields, plus **primary keys and access patterns** ("fetch orders by user, newest first" → index `(user_id, created_at)`).
- **Pick the DB from the access patterns**:
- SQL for relations, transactions and consistency (orders, payments).
- Key-value or wide-column for huge, simple lookups.
- Document DB for flexible schemas.
- A search engine for full-text.
- Object storage (S3) for files.
- Say the **partition / shard key** if the scale requires it, and why (even distribution, queries hit one shard).

## 5. High-level design

↑ TOP

Draw the **main request flow** end to end, simplest version first:

```
client → CDN → load balancer → stateless app servers → cache → DB
                                                    └→ queue → workers (async jobs)
                                                    └→ object storage (files)
```

Walk through 1–2 core use cases on the diagram ("when a user posts..."). Keep app servers **stateless** so they scale horizontally.

## 6. Deep dives: where you get the most credit

↑ TOP

Pick the **2–3 hardest parts** (or ask: "which part would you like me to go deeper on?"). The usual toolkit:

| Problem | Tools |
|---|---|
| Read-heavy / slow reads | cache (Redis) with cache-aside + TTL, read replicas, CDN, denormalization |
| Write-heavy | sharding, a write queue/buffer, batching, an LSM-based store (Cassandra) |
| Big data / hot keys | sharding + consistent hashing, splitting hot keys, rate limiting |
| Slow or long work | async via queue + workers (RabbitMQ / Kafka) |
| Real-time push | WebSockets/SSE + Redis Pub/Sub fan-out across servers |
| Reliability | replication, retries with backoff, idempotency, DLQs, circuit breakers |

| Problem | Tools |
|---|---|
| Consistency | transactions, the outbox pattern, sagas, optimistic locking (version column) |
| Feeds / timelines | fan-out on write vs on read (hybrid for celebrities) |
| Unique IDs | UUID, Snowflake-style IDs (time-ordered) |

For each deep dive, cover: **the problem (with a number) → options → your choice → its trade-off.**

## 7. Wrap-up

↑ TOP

- **Bottlenecks and single points of failure:** what breaks first at 10× load?
- **Failure handling:** what happens if the cache, the DB primary, or a worker dies?
- **Monitoring:** QPS, p99 latency, error rate, queue/consumer lag, alerts.
- **What you'd do next** with more time.

## Communication habits

↑ TOP

- **Think out loud** and **check in** often: "Does this match what you had in mind?"
- **Trade-off sentence for every choice:** "I'll use X because Y; the cost is Z, and that's acceptable because..."
- If the interviewer pushes on something, **follow them**: it's a hint about what they want to see.
- Name what you're **not** doing and why. Silence reads as not knowing.
- Keep the board clean: requirements and numbers on one side, the diagram in the middle.

**Practice:** write each problem up in `system-design/<problem>.md` using the same steps (template in `system-design/README.md`), then add a "My mistakes / what I missed" section.

> ⚠ **COMMON TRAPS**
>
> - Drawing boxes in minute 1 without requirements.
> - Over-engineering: Kafka, microservices and sharding for 10 QPS.
> - Naming tech without justification ("use Cassandra" without saying why).
> - Skipping numbers, then making scaling choices that don't follow from anything.
> - Going deep on one component and running out of time before the end-to-end flow exists.

↑ Back to top ☰ Contents

CHAPTER 4 · SYSTEM DESIGN

# Kafka vs RabbitMQ

Kafka RabbitMQ competing consumers direct exchange topic exchange fanout exchange TTL priority queue request/reply RPC correlation_id reply_to message broker message queue event streaming pub/sub partition consumer group offset exchange routing key ack dead letter queue DLQ replay ordering backpressure exactly-once KRaft quorum queue RabbitMQ streams

SEE ALSO Redis Pub/Sub vs RabbitMQ vs Kafka · Celery with Redis as Broker · How to Run a System Design Interview · SQL vs NoSQL & Scaling Databases · V1·10 Notification System · V1·14 YouTube · V2·3 Google Maps · V2·4 Distributed Message Queue · V2·5 Metrics Monitoring and Alerting · V2·6 Ad Click Event Aggregation · V2·11 Payment System

**What:** both move messages between services asynchronously, but they're different models.

- **RabbitMQ is a message broker (smart broker, dumb consumer).** It routes each message to queues. A consumer takes a message, acks it, and the message is **deleted**.
- **Kafka is a distributed append-only log (dumb broker, smart consumer).** Messages are written to partitions and **kept for a retention period**. Each consumer tracks its own **offset** (position) and can re-read.

> **i WHY IT MATTERS**
> "which queue would you use and why?" comes up in almost every system design round. The answer should follow from requirements (replay? ordering? throughput? routing?), not habit.

**Mental model:**

```
RabbitMQ:  producer → exchange ──(routing key / bindings)──▶ queue → consumer (ack → deleted)
Kafka:     producer → topic [partition 0 | partition 1 | ...] ← consumer group (each reads its offset)
```

**Comparison:**

| | RabbitMQ | Kafka |
|---|---|---|
| Model | queue: work is distributed and then gone | log: events are stored and re-readable |
| After consume | deleted on ack | kept until retention (time/size) or compaction |
| Replay | no (classic queues); yes with **RabbitMQ Streams** | yes: reset the offset and re-read history |
| Routing | rich: exchanges (direct, topic, fanout, headers) | simple: topic + partition key; consumers filter |

| | RabbitMQ | Kafka |
|---|---|---|
| Ordering | per queue, but lost with multiple competing consumers or requeues | **per partition** (same key → same partition → ordered) |
| Scaling consumers | add consumers to a queue (competing consumers) | parallelism = number of partitions; one partition → one consumer in a group |
| Throughput | ~tens of thousands of msgs/s per queue | ~millions of msgs/s; scales out by adding partitions and brokers |
| Delivery | push, with `prefetch` limiting in-flight messages | pull, so consumers control their pace (natural backpressure) |
| Per-message features | ack/nack, requeue, TTL, priority queues, dead-letter exchanges, delayed messages (plugin) | none per message; retries/DLQ are built by the app (retry topics) |
| Many independent readers of the same data | needs a fanout exchange plus one queue per reader | free: each consumer group reads the same log independently |
| Exactly-once | at-least-once + idempotent consumers | exactly-once **within Kafka** (idempotent producer + transactions); external effects still need idempotency |
| Ops | single node is simple; quorum queues for HA | heavier: brokers, partitions, replication, rebalancing (KRaft replaced ZooKeeper in Kafka 4.0) |

**Pick RabbitMQ when:**

- It's a **task/job queue**: send email, resize image, process payment. Each job is handled once by one worker.
- You need **complex routing** (route by type, region, priority).
- You need **per-message control**: retries with requeue, TTL, priority, delayed jobs, dead-lettering.
- The volume is moderate and you want low latency and simple ops.
- Request/reply (RPC-style) messaging.

**Pick Kafka when:**

- It's an **event stream / source of truth**: user activity, clicks, logs, metrics, CDC (database change capture).
- **Several consumers** need the same events: analytics, search indexing, notifications, each at its own pace.
- You need **replay**: reprocess history after a bug, or rebuild a new service's state from the beginning.
- The throughput is **high** and you need **ordering per key** (all events of `user_id=42` in order).
- Stream processing (Kafka Streams, Flink) and event sourcing.

**Why RabbitMQ fits those cases (in depth):**

1. **Task/job queue: each job goes to one worker.** With many workers on one queue (*competing consumers*), RabbitMQ hands each message to exactly one. The worker acks when it's done, and the message is deleted. If the worker crashes before acking, the message is **redelivered to another worker**. Scale by adding workers. *Kafka here:* parallelism is capped by partitions (3 partitions → at most 3 workers), and one slow job (a 10-minute encode) **blocks everything behind it** in that partition.
2. **Complex routing.** Producers publish to an **exchange**, and bindings decide which queues get a copy:

   | Exchange | Rule | Example |
   |---|---|---|
   | direct | exact key | `pdf` → `pdf_queue` |
   | topic | wildcards | `order.eu.*` → EU team, `order.*.refund` → refunds, `order.#` → audit |
   | fanout | every bound queue | `user_signed_up` → email + analytics + CRM |
   | headers | header match | `region=eu AND priority=high` |

   The producer doesn't know who receives it. *Kafka here:* consumers read the whole topic and filter themselves, or you add more topics.
3. **Per-message control.** Each message is managed individually:
   - nack/requeue to retry
   - **TTL**: drop it if it isn't processed in 30s (e.g. an OTP)
   - **priority queues**: VIP orders first
   - **delayed delivery**: a plugin, or TTL + DLX
   - **dead-lettering**: after N failures, or on TTL expiry, the message goes to a DLQ to inspect later

   *Kafka here:* nothing per message, so you build retry and DLQ topics yourself.
4. **Moderate volume, low latency, simple ops.** Thousands to tens of thousands of msgs/s covers most business apps. RabbitMQ **pushes** each message to a waiting consumer (Kafka consumers poll in batches). One node or a small cluster with quorum queues is easy to run; Kafka needs partitions, replication, retention, rebalancing and lag monitoring.
5. **Request/reply (RPC).** The request carries `reply_to` (the queue to answer on) and `correlation_id` (to match the reply to its request). RabbitMQ supports this natively, including the built-in "direct reply-to". With Kafka you'd build it yourself on topics.

> **RabbitMQ = a smart post office:** it routes each letter to the right mailbox, one person handles it, it's retried or set aside on failure, then thrown away. **Kafka = a recorded journal:** everything is written in order and kept, and anyone can read it from any point, any number of times.

**Interview answer template:**

> "If this is a work queue where each task is processed once and I need retries and routing, I'd use RabbitMQ. If it's a stream of events that several services consume independently, or that I may need to replay, with high throughput and per-key ordering, I'd use Kafka. Either way, consumers are idempotent because delivery is at-least-once."

✓ KEY POINTS

- Kafka ordering is only **per partition**. Choose the partition key (e.g. `order_id`) so related events land together.
- Kafka parallelism is capped by the partition count: having more consumers than partitions leaves some idle.
- A slow Kafka consumer doesn't hurt others: it just lags (monitor **consumer lag**). In RabbitMQ, slow consumers mean **queues grow in broker memory/disk**.
- Both are at-least-once by default, so make **consumers idempotent** (dedupe by message id, upserts).
- Poison messages: in RabbitMQ, use a **dead-letter exchange**. In Kafka, use a retry topic and then a DLQ topic, so one bad message doesn't block the partition.
- The two are converging. RabbitMQ **Streams** add a replayable log, and Kafka is adding queue-style **share groups** (KIP-932). The default trade-off above still holds.

⚠ COMMON TRAP

"Kafka is just a faster RabbitMQ." No: it's a different model (a log vs a queue). Picking Kafka for a simple job queue gives you no per-message retry/priority/TTL and more ops work. Picking RabbitMQ for event streaming loses replay and multi-consumer fan-out.

↑ Back to top ☰ Contents

CHAPTER 4 · SYSTEM DESIGN

# The 4-Step Framework (Alex Xu, with a News Feed Example)

system design framework · Alex Xu · design scope · high-level design · buy-in · deep dive · wrap up · use case · news feed · fanout · time allocation

SEE ALSO How to Run a System Design Interview · V1·11 News Feed System

ON THIS PAGE

1. Step 1 — Understand the problem and establish design scope · p. 100
2. Step 2 — Propose a high-level design and get buy-in · p. 101
3. Step 3 — Design deep dive · p. 102
4. Step 4 — Wrap up · p. 103
5. Dos and Don'ts · p. 104

**What:** the framework from *System Design Interview* (Alex Xu), chapter 3. It has 4 steps. It is the same idea as the 7-step path in coarser steps: there, steps 1–2 are Xu's step 1, steps 3–5 are his step 2, step 6 is his step 3, and step 7 is his step 4. Each step below has a short version and a worked **news feed** example.

**What the interviewer grades:** collaboration, working under pressure, resolving ambiguity, and asking good questions. The final design matters less than how you got there. **Red flags:** over-engineering, narrow-mindedness, stubbornness.

| # | Step | Time (45 min) |
|---|---|---|
| 1 | Understand the problem and establish design scope | 3–10 min |
| 2 | Propose a high-level design and get buy-in | 10–15 min |
| 3 | Design deep dive | 10–25 min |
| 4 | Wrap up | 3–5 min |

## Step 1 — Understand the problem and establish design scope

↑ TOP

- **Don't answer fast** (the book's "Jimmy" story). Quick answers earn no points. Answering without understanding the requirements is a red flag.
- **Ask** about the exact requirements:
- What features are we building?
- How many users are there?
- How fast will it grow? What scale do we expect in 3 months, 6 months and a year?
- What is the tech stack? Which existing services can we reuse?

- If the interviewer says "you decide", **write your assumptions on the board**. You will need them later.

NEWS FEED EXAMPLE

The candidate asks, and the interviewer answers:

| Question | Answer |
|---|---|
| Mobile app, web app, or both? | Both |
| Most important features? | Make a post, and see friends' news feed |
| Feed order: reverse chronological or ranked? | Reverse chronological (keep it simple) |
| How many friends can a user have? | 5,000 |
| Traffic? | 10M DAU |
| Text only, or media too? | Images and videos too |

**Scope you write down:** post + view feed, newest first, ≤ 5,000 friends, 10M DAU, media allowed. Every later choice should trace back to one of these lines.

## Step 2 — Propose a high-level design and get buy-in

↑ TOP

- Sketch an **initial blueprint** and ask for feedback. Treat the interviewer as a **teammate**.
- Draw **box diagrams**: clients (mobile/web), APIs, web servers, data stores, cache, CDN, message queue.
- Do **back-of-the-envelope** math to check the blueprint fits the scale. Think out loud, and ask whether the math is needed first.
- **Walk through 1–2 concrete use cases**: follow one real request through the boxes. This frames the design and exposes edge cases.
- API endpoints and DB schema: include them only if the problem is small enough. Yes for a poker game backend; too low-level for "design Google search". Ask the interviewer.

NEWS FEED EXAMPLE

Split the design into **two flows**:

**Feed publishing** (Figure 3-1): the user writes a post. It is saved to cache/DB and pushed into friends' feeds.

```
client ──POST /v1/me/feed?content=Hello──► load balancer ► web servers
   web servers ─► Post Service ─► post cache ─► post DB
               ─► Fanout Service ─► news feed cache
               ─► Notification Service
```

**News feed building** (Figure 3-2): the feed is the friends' posts, newest first.

```
client ──GET /v1/me/feed──► load balancer ► web servers ► News Feed Service ► news
feed cache
```

**Use-case walkthrough:**

1. **Alice posts "Hello":** the Post Service stores it, the Fanout Service adds it to every friend's feed cache, and the Notification Service tells the friends.
2. **Bob opens his feed:** the News Feed Service reads Bob's precomputed feed from the cache.

**Edge cases this walkthrough exposes:**

- Alice has 5,000 friends, so one post means 5,000 cache writes. Do them synchronously or async? → queue + workers (step 3).
- The feed cache holds IDs, so where do the names and post text come from? → hydration from user/post caches (step 3).
- Videos should not go through the web servers. → CDN.
- Alice deletes the post, or Bob unfriends her. → store only IDs in feeds and **filter at read time** (skip deleted posts and non-friends). Cleanup in the background is optional.

## Step 3 — Design deep dive

↑ TOP

By now you should have: agreed goals and scope, a high-level blueprint, feedback on it, and an idea of which areas to focus on.

- **Prioritize components with the interviewer.** Every interview differs: some want high level only. Senior roles often focus on performance, bottlenecks and resource estimates. Most want one or two components in detail.
- URL shortener → the hash function that turns a long URL into a short one.
- Chat system → lowering latency, and online/offline status.
- **Time management:** don't get lost in details that show no design skill. For example, Facebook's EdgeRank ranking algorithm takes time and proves nothing about scaling.

NEWS FEED EXAMPLE

Deep-dive the two most important use cases. Full details are in Xu's chapter 11.

**1. Feed publishing** (Figure 3-3)

- Web servers also do **authentication** and **rate limiting** (to stop spam posts).
- The Fanout Service: 1. gets the friend IDs from a **graph DB**; 2. gets the friends' data from the **user cache** (backed by the user DB); 3. puts a fanout job on a **message queue**; 4. **fanout workers** consume the queue; 5. the workers write into the **news feed cache**.
- Why the queue: the post request returns fast, and the 5,000 writes happen async and can scale with more workers.

**2. News feed retrieval** (Figure 3-4)

1. The client sends `GET /v1/me/feed`.
2. The load balancer routes it to the web servers (auth + rate limiting).
3. The web servers call the **News Feed Service**.
4. It reads the **post IDs** from the news feed cache.
5. It **hydrates** them: user data from the user cache/DB and post content from the post cache/DB.
6. It returns the full feed as JSON. Images and videos come from the **CDN**.

## Step 4 — Wrap up

↑ TOP

- **Bottlenecks and improvements.** Never say the design is perfect; there is always something to improve. Use this to show critical thinking.
- **Recap** the design, especially if you discussed several options.
- **Error cases:** server failure, network loss.
- **Operations:** monitoring metrics and error logs, and how to roll the system out.
- **The next scale curve:** what changes from 1M to 10M users?
- **Refinements** you would add with more time.

NEWS FEED EXAMPLE

A 3–5 minute wrap-up might sound like this:

- **Bottleneck:** fanout for users with huge follower counts. Use a hybrid: push to normal friends' feeds on write, and pull celebrities' posts at read time.
- **Recap:** "Two flows: publishing goes through a queue and fanout workers into feed caches; retrieval reads post IDs from the cache and hydrates them from the user and post caches. Media is on the CDN."
- **Errors:** if the cache dies, rebuild the feed from the DB. If a worker crashes, the job stays in the queue and is retried.
- **Ops:** watch queue lag, p99 feed latency, cache hit rate and error rate. Roll out behind gradual deploys.

- **Next 10×:** shard the caches and DBs by user ID, add more fanout workers, and add read replicas.

## Dos and Don'ts

↑ TOP

| Do | Don't |
|---|---|
| Ask for clarification; don't trust your own assumptions | Be unprepared for the typical questions |
| Understand the requirements; there is no single right answer | Jump into a solution before clarifying the requirements |
| Think out loud and tell the interviewer what you're thinking | Go deep on one component before the high-level design exists |
| Suggest multiple approaches | Stay stuck silently; ask for hints |
| After the blueprint is agreed, detail the most critical components first | Think in silence |
| Bounce ideas off the interviewer as a teammate; never give up | Assume you're done when the design is drawn. You're done when the interviewer says so; ask for feedback early and often |

⚠ COMMON TRAP

drawing the "final" architecture in minute 2. Scope first, then get buy-in, then go deep.

↑ Back to top ☰ Contents

CHAPTER 4 · SYSTEM DESIGN

# Redis Pub/Sub vs RabbitMQ vs Kafka

Redis Redis Pub/Sub PUBLISH SUBSCRIBE PSUBSCRIBE fire-and-forget at-most-once Redis Streams XADD XREADGROUP XACK XAUTOCLAIM Redis List queue BRPOP BLMOVE WebSocket fan-out cache invalidation

SEE ALSO Kafka vs RabbitMQ · Celery with Redis as Broker · How to Run a System Design Interview · Web & Networking Fundamentals · V1·12 Chat System · V2·2 Nearby Friends

**What:** `PUBLISH channel msg` pushes the message to every client **subscribed right now**, then Redis forgets it.

```
publisher ──PUBLISH chat:room1 "hi"──► Redis ──► every subscriber connected *right now*
```

- **No storage:** if a subscriber is offline, reconnecting, or not subscribed yet, it **never** gets the message.
- **No ack / retry:** if a subscriber crashes while handling the message, it's gone. This is **at-most-once** delivery.
- **Broadcast only:** every subscriber gets every message; there's no "one job → one worker".
- **Slow subscriber:** its output buffer fills up and Redis **disconnects** it, so those messages are lost.
- Very fast (in-memory, sub-ms) and trivial to use: `SUBSCRIBE`, `PSUBSCRIBE news.*`.

**Comparison:**

| | Redis Pub/Sub | RabbitMQ | Kafka |
|---|---|---|---|
| Model | live broadcast | queue (routed, deleted on ack) | log (retained) |
| Stored? | **no** | until acked | until retention expires |
| Offline consumer | **misses messages** | gets them when back | reads from its offset when back |
| Delivery | at-most-once | at-least-once | at-least-once (exactly-once within Kafka) |
| Ack / retry / DLQ | none | yes | app-built |
| One job → one worker | no | yes | yes (per partition, in a group) |
| Replay | no | no (except Streams) | yes |
| Latency | lowest | low | low (batching) |
| Ops | trivial if you already run Redis | moderate | heavy |

**Use Redis Pub/Sub when a lost message doesn't matter**, because it's only useful right now or the real data is stored elsewhere:

- **WebSocket fan-out across app servers:** users of one chat room are connected to different instances. Each instance subscribes to `chat:room1`, so one publish reaches all of them. The message itself is saved in the DB.
- **Live UI signals:** typing indicators, online status, live prices or scores, where a newer value replaces the old one.
- **Cache invalidation broadcast:** "key X changed, drop it". If an instance misses one, the TTL covers it.
- **Config reload** signals.

**Never** use it for orders, payments, emails or anything that must happen: use RabbitMQ (jobs) or Kafka (events).

**Durable options inside Redis:**

| | How | Gives you | Limits |
|---|---|---|---|
| **Streams** | `XADD` / `XREADGROUP` / `XACK` | append-only log with IDs (replay), consumer groups (one entry → one consumer), a pending list + `XAUTOCLAIM` to take over a dead consumer's entries | memory-bound (trim with `MAXLEN`); scales far less than Kafka |
| **List queue** | `LPUSH` + `BRPOP` (or `BLMOVE` for a safer "move to processing list") | simple job queue, one job → one worker | no real acks or consumer groups (this is what Celery uses, see Celery with Redis) |

> **❝ INTERVIEW LINE**
>
> *"Redis Pub/Sub is fire-and-forget: no persistence or acks, and offline subscribers miss messages. I'd use it for real-time signals like WebSocket fan-out or cache invalidation. For durable jobs I'd use RabbitMQ (or Redis Streams if it's small and we already run Redis), and for durable, replayable event streams at scale, Kafka."*

> **⚠ COMMON TRAP**
>
> "We already have Redis, so let's use Pub/Sub as our queue." Messages get silently lost on any disconnect or deploy.

↑ Back to top ☰ Contents

CHAPTER 4 · SYSTEM DESIGN

# Celery with Redis as Broker

Celery Kombu Redis broker LPUSH BRPOP result backend visibility_timeout unacked task_acks_late acks early worker_prefetch_multiplier ETA countdown idempotent tasks task priority

SEE ALSO Redis Pub/Sub vs RabbitMQ vs Kafka · Kafka vs RabbitMQ · Async / Await · How to Run a System Design Interview · Locking & Concurrency Control · Django in Production: Performance, Security, Async · Transactions & Migrations · V1·10 Notification System

**What:** Celery talks to brokers through **Kombu**, which **emulates RabbitMQ (AMQP) concepts on top of Redis**: queues become Redis **Lists**, bindings are stored in Redis sets, and fanout exchanges become `PUBLISH`.




1. **Send:** `add.delay(2, 3)` serializes the task name, args and id into a message and runs `LPUSH celery <msg>`. The list name is the queue name (default `celery`).
2. **Receive:** workers block on `BRPOP`, and Redis gives each message to **exactly one** worker (competing consumers).
3. **Result (optional):** the worker stores the result under `celery-task-meta-<task_id>`, which expires after 1 day by default. `AsyncResult.get()` reads it and gets notified via Pub/Sub.

**Where Pub/Sub is used** (all fine to lose occasionally):

- **Broadcast/control:** `celery inspect active`, `celery control revoke`, and shutdown go to *all* workers.
- **Monitoring events:** `task-started` / `task-succeeded` for Flower and `celery events`.
- **Result notifications** to code waiting on `.get()`.

**Fake acks: the tricky part.** Redis has no acks, so when a worker takes a task, Kombu also copies it into an `unacked` hash, with a timestamp in a sorted set called `unacked_index`. The ack removes it. If the worker **dies**, the message is restored to the queue only after **`visibility_timeout` (default 1 hour)**.

| Problem | Why | Fix |
|---|---|---|
| A crashed worker's tasks reappear only after ~1h | the visibility timeout is the only failure detection | lower `visibility_timeout` (but keep it longer than your longest task) |

| Problem | Why | Fix |
|---|---|---|
| **ETA/countdown tasks run multiple times** | `apply_async(countdown=7200)`: the worker fetches the task now and holds it in memory until the ETA, so it stays unacked > 1h and gets restored to another worker | `visibility_timeout` > the longest ETA, or avoid long countdowns (use a scheduler or DB) |

With **RabbitMQ** as the broker, neither happens: acks are tied to the worker's connection, so if a worker dies the connection drops and RabbitMQ **redelivers immediately**.

**When the ack happens (any broker):**

- **Default `task_acks_late=False`:** acked **when the worker receives the task, before running it.** A crash mid-task means the task is **lost** (at-most-once).
- **`task_acks_late=True`:** acked **after the task finishes.** A crash means it runs again (at-least-once), so tasks **must be idempotent** (e.g. check "already charged?" before charging).
- `worker_prefetch_multiplier` (default 4): each worker process reserves 4 tasks ahead. For long tasks, set it to **1** so one worker doesn't hoard jobs while others are idle.

**Other Redis-broker gotchas:**

- **Durability:** queued tasks live in Redis memory. If Redis restarts without AOF/RDB persistence, the **queue is lost**. RabbitMQ has durable queues on disk.
- **Priorities are emulated:** Kombu uses a separate list per priority level (default `0, 3, 6, 9`), and on Redis **0 is the highest priority**, the opposite of RabbitMQ.
- **Memory:** a big backlog uses RAM. Pass IDs, not big objects, in task args.

> **❝ INTERVIEW LINE**
> *"In Celery, Redis acts as the broker through Lists: `LPUSH` to send, `BRPOP` to receive, so each task goes to one worker. Pub/Sub is only used for broadcast commands, events and result notifications. Redis has no native acks, so Kombu emulates them with an unacked set and a visibility timeout (1h by default). That's why crashed tasks come back late and long ETA tasks can run twice. RabbitMQ handles acks natively, so it's the more robust broker. Redis is simpler if you already run it. Either way, use `acks_late` with idempotent tasks for anything important."*

> **⚠ COMMON TRAP**
> assuming Celery with defaults never loses tasks. With early acks, a worker crash (OOM kill, deploy) silently drops the task it was running.

↑ Back to top ☰ Contents

# 05
# Databases

4 pages. Each card says what the page covers and its key lines; open the page for the full depth.

## Indexes · p. 111

an index is a separate data structure that maps column values to row locations, so Postgres can find rows without reading the whole table. The default type is a **B-tree**: a balanced, sorted tree. Lookup cost is O(log n) instead of O(n).

- B-tree = sorted tree, O(log n): serves `=`, ranges, `ORDER BY`, prefix `LIKE 'abc%'`.
- Composite `(a, b, c)` works for `a`, `a,b`, `a,b,c` (leftmost prefix). Put **equality columns first, range column last**.
- Index is skipped when you wrap the column in a function, use `LIKE '%x'`, mismatch types, or the filter matches a big part of the table.
- Partial index (`WHERE status = 'pending'`) = small and fast; `INCLUDE (...)` = covering → Index Only Scan.
- Every index slows writes. Production: `CREATE INDEX CONCURRENTLY`.
- `EXPLAIN ANALYZE`: compare estimated `rows` vs actual rows; a big gap means stale stats (`ANALYZE`).

Read the full page →

## Transactions & Isolation Levels · p. 116

a transaction groups several statements into one unit: either all of them take effect, or none do. The **isolation level** decides how much concurrent transactions can see of each other.

- ACID: all-or-nothing, rules kept, concurrent txns don't see each other's mess, committed = survives a crash (WAL).
- Postgres default = **Read Committed**: each *statement* sees a fresh snapshot.
- Postgres **Repeatable Read** = snapshot isolation: no phantoms either, but **write skew** is still possible.
- Only **Serializable** stops write skew; you must **retry** on SQLSTATE `40001`.
- MVCC: readers never block writers, writers never block readers; only writer vs writer on the same row waits.
- Lost update in Read Committed: read in app → compute → write. Fix with atomic `UPDATE`, `FOR UPDATE`, or a version column.

Read the full page →

## Locking & Concurrency Control · p. 121

ways to stop concurrent transactions from breaking each other's data. Three families: **pessimistic** (lock first, then work), **optimistic** (work, then check nobody changed it), and **atomic** single statements (let the database do read+write in one step).

- Pessimistic: `SELECT ... FOR UPDATE` locks the row until commit; others wait. Good under **high contention**.
- Optimistic: `UPDATE ... SET version = version + 1 WHERE id = ? AND version = ?`; 0 rows updated → conflict, retry. Good under **low contention**.
- Simplest and best for counters/balances: one atomic `UPDATE ... SET balance = balance - 10 WHERE id = 1 AND balance ⩾ 10 RETURNING balance`.
- Job queue in Postgres: `FOR UPDATE SKIP LOCKED LIMIT n`.
- Deadlock = two txns lock rows in opposite order. Fix: always lock in the **same order** (e.g. by id).
- Advisory lock = app-defined lock on a number: `pg_advisory_xact_lock(key)`.

Read the full page →

## SQL vs NoSQL & Scaling Databases · p. 126

how to choose a database type, and how to scale one when a single server is not enough: replication for reads, partitioning/sharding for size and writes, pooling for connections, and fixing ORM query patterns.

- Default to **Postgres**; pick NoSQL for a specific need (huge write volume, flexible schema, simple key lookups, graph traversal).
- Scale reads with **replicas**; replicas are usually **async**, so expect **replication lag** → read-your-writes from the primary.
- **Partitioning** = split a table inside one server; **sharding** = split data across servers.
- Shard key: high cardinality, even spread, and present in most queries (e.g. `user_id`). Never a timestamp.
- Postgres = one process per connection → use **PgBouncer** (transaction mode).
- N+1: 1 query for the list + 1 per row. Fix: `select_related` / `prefetch_related` (Django), `joinedload` / `selectinload` (SQLAlchemy).

Read the full page →

CHAPTER 5 · DATABASES

# Indexes

index B-tree composite index multicolumn index leftmost prefix covering index INCLUDE index only scan partial index expression index functional index hash index GIN GiST BRIN pg_trgm selectivity cardinality EXPLAIN EXPLAIN ANALYZE Seq Scan Index Scan Bitmap Heap Scan CREATE INDEX CONCURRENTLY write amplification HOT update visibility map

SEE ALSO Transactions & Isolation Levels · Locking & Concurrency Control · SQL vs NoSQL & Scaling Databases · Django ORM in Depth · V2·1 Proximity Service · V2·9 S3-like Object Storage

ON THIS PAGE

1. What a B-tree can do · p. 111
2. Composite index and the leftmost-prefix rule · p. 112
3. Covering index (INCLUDE) and Index Only Scan · p. 112
4. Partial and expression indexes · p. 113
5. Other index types (brief) · p. 113
6. When the index is NOT used · p. 113
7. Write cost · p. 114
8. Reading EXPLAIN · p. 114

**What:** an index is a separate data structure that maps column values to row locations, so Postgres can find rows without reading the whole table. The default type is a **B-tree**: a balanced, sorted tree. Lookup cost is O(log n) instead of O(n).

> **i WHY IT MATTERS**
> "this query is slow, what do you do?" is the most common database question. The answer is almost always: read `EXPLAIN ANALYZE`, then add or fix an index.

## What a B-tree can do

↑ TOP

Because the keys are **sorted**, one B-tree serves many query shapes:

```sql
CREATE INDEX idx_orders_created ON orders (created_at);

SELECT * FROM orders WHERE created_at = '2026-01-01';                -- equality
SELECT * FROM orders WHERE created_at BETWEEN '2026-01-01' AND '2026-02-01';  -- range
SELECT * FROM orders ORDER BY created_at DESC LIMIT 20;              -- sort without
sorting
SELECT max(created_at) FROM orders;                                  -- one step to the end
```

`LIKE 'abc%'` (prefix) can also use a B-tree, but only with the `C` collation or a `text_pattern_ops` index.

↑ TOP

## Composite index and the leftmost-prefix rule

An index on `(a, b, c)` is sorted by `a`, then by `b` inside each `a`, then by `c`. Like a phone book sorted by last name, then first name.

```sql
CREATE INDEX idx_tx ON transactions (user_id, status, created_at);
```

| Query filter | Uses the index well? |
|---|---|
| `user_id = 5` | yes |
| `user_id = 5 AND status = 'done'` | yes |
| `user_id = 5 AND status = 'done' AND created_at > now() - interval '7 days'` | yes, best case |
| `user_id = 5 AND created_at > ...` | partly: finds `user_id`, then filters `created_at` inside it |
| `status = 'done'` | usually no (not the leftmost column) |

Rule: **equality columns first, then the range or sort column.** `WHERE user_id = ? ORDER BY created_at DESC` is perfect for `(user_id, created_at)`.

Note: Postgres 18 added **skip scan**, so a query without the first column can sometimes use the index if the first column has few distinct values. Don't rely on it; design for the leftmost prefix.

↑ TOP

## Covering index (INCLUDE) and Index Only Scan

If the index holds every column the query needs, Postgres does not visit the table at all: an **Index Only Scan**.

```sql
CREATE INDEX idx_users_email ON users (email) INCLUDE (id, name);

SELECT id, name FROM users WHERE email = 'a@b.com';   -- Index Only Scan
```

`INCLUDE` columns are stored in the index leaves but are not part of the sort key. Index Only Scan works well only if the table is recently **VACUUMed** (Postgres checks the *visibility map* to know a row is visible without reading the table).

## Partial and expression indexes

↑ TOP

SQL

```sql
-- Partial: index only the rows you query. Small and fast.
CREATE INDEX idx_jobs_pending ON jobs (created_at) WHERE status = 'pending';
SELECT * FROM jobs WHERE status = 'pending' ORDER BY created_at LIMIT 10;

-- Partial unique: "one active wallet per user"
CREATE UNIQUE INDEX uq_active_wallet ON wallets (user_id) WHERE is_active;

-- Expression: index the result of a function
CREATE INDEX idx_users_lower_email ON users (lower(email));
SELECT * FROM users WHERE lower(email) = 'a@b.com';   -- must match the exact expression
```

## Other index types (brief)

↑ TOP

| Type | Good for | Example |
|---|---|---|
| B-tree | default: `=`, `<`, `>`, `BETWEEN`, `ORDER BY` | almost everything |
| Hash | only `=`; rarely better than B-tree | long keys, equality only |
| GIN | "contains" queries on many values per row | `jsonb @>`, arrays, full-text search, `pg_trgm` for `LIKE '%abc%'` |
| GiST | geometric / range / nearest neighbor | PostGIS, `tstzrange` overlap, exclusion constraints |
| BRIN | huge tables where values follow physical order | append-only logs by `created_at`; tiny index |

SQL

```sql
CREATE INDEX idx_meta ON events USING gin (metadata);       -- WHERE metadata @>
'{"type":"login"}'
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_name_trgm ON users USING gin (name gin_trgm_ops);  -- WHERE name ILIKE
'%reza%'
```

## When the index is NOT used

↑ TOP

| Cause | Bad | Fix |
|---|---|---|
| Function on the column | `WHERE lower(email) = 'x'` with index on `email` | expression index on `lower(email)` |
| Math on the column | `WHERE created_at + interval '1 day' > now()` | move math to the other side: `created_at > now() - interval '1 day'` |

| Cause | Bad | Fix |
|---|---|---|
| Leading wildcard | `WHERE name LIKE '%reza'` | GIN + `pg_trgm`, or full-text search |
| Low selectivity | `WHERE is_active = true` (95% of rows) | planner correctly picks Seq Scan; use a partial index on the rare value |
| Type mismatch | `bigint` column compared to a `numeric` param (`WHERE id = 5.0`) | pass the right type (watch ORM/driver params) |
| Not leftmost column | `WHERE status = ?` on `(user_id, status)` | add an index that starts with `status`, or reorder |
| Stale statistics | planner thinks 10 rows, really 1M | `ANALYZE table;` (autovacuum usually does it) |
| Tiny table | a few pages | Seq Scan is cheaper; fine |

## Write cost

↑ TOP

- Every `INSERT` / `DELETE` (and every `UPDATE` that can't be a **HOT update**) must update **every index**. 10 indexes = much slower writes.
- Updating an indexed column prevents HOT (heap-only tuple) updates, so more bloat.
- Indexes use disk and RAM (cache).
- Drop unused indexes: check `pg_stat_user_indexes` where `idx_scan = 0`.
- On a live table use `CREATE INDEX CONCURRENTLY` (no long write lock; slower; can't run inside a transaction; if it fails it leaves an `INVALID` index to drop).

## Reading EXPLAIN

↑ TOP

`EXPLAIN` shows the plan with **estimates**. `EXPLAIN ANALYZE` actually **runs** the query and adds real times and row counts (careful with `UPDATE` / `DELETE`: wrap in `BEGIN ... ROLLBACK`).

SQL

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE user_id = 42 ORDER BY created_at DESC LIMIT 20;
```

```
Limit  (cost=0.43..8.95 rows=20 width=64) (actual time=0.03..0.09 rows=20 loops=1)
  →  Index Scan Backward using idx_orders_user_created on orders
       (cost=0.43..512.10 rows=1200 width=64) (actual time=0.03..0.08 rows=20 loops=1)
       Index Cond: (user_id = 42)
       Buffers: shared hit=24
Planning Time: 0.1 ms
Execution Time: 0.12 ms
```

- `cost=start..total`: planner units, not ms. `rows` = estimated rows. `actual time` is in ms, **per loop**.

- **Estimated rows vs actual rows**: a 10x+ gap means bad statistics or correlated columns, and that causes bad plans.
- `Buffers: shared hit` = pages from cache, `read` = from disk.

| Node | Meaning | When |
|---|---|---|
| Seq Scan | read the whole table | no useful index, or the filter matches many rows |
| Index Scan | walk the index, then fetch each row from the table | few rows, or ordered output needed |
| Index Only Scan | answer from the index alone | all needed columns are in the index, table is vacuumed |
| Bitmap Index Scan + Bitmap Heap Scan | collect matching row locations first, then read table pages in physical order | medium number of rows; can combine indexes (`BitmapAnd` / `BitmapOr`) |

Also watch for: `Sort` with `external merge Disk` (raise `work_mem` or add an index for the order), `Nested Loop` with a huge `loops=` count, and `Rows Removed by Filter` being large (index not selective enough).

✓ KEY POINTS

- B-tree is sorted, so it helps `=`, ranges, `ORDER BY` and `LIMIT`.
- Composite index: leftmost prefix; equality columns first, range/sort column last.
- `INCLUDE` makes covering indexes → Index Only Scan, no table access.
- Partial indexes for hot subsets (`pending` jobs); expression indexes for `lower(email)`.
- GIN for jsonb/arrays/full-text/trigram, GiST for geo/ranges, BRIN for huge append-only tables.
- Indexes cost writes, disk and memory: add them for real queries, not "just in case".

❝ INTERVIEW LINE

*"I'd run `EXPLAIN ANALYZE`, check whether it's a Seq Scan and whether estimated rows match actual rows, then add a composite index matching the query: equality columns first, then the range or sort column, maybe with `INCLUDE` for an index-only scan. On production I'd create it `CONCURRENTLY`."*

⚠ COMMON TRAP

thinking "more indexes = faster". Reads may get faster, but every write pays for every index. Second trap: an index on `(a, b)` does not help a query that filters only on `b`.

↑ Back to top ☰ Contents

CHAPTER 5 · DATABASES

# Transactions & Isolation Levels

transaction ACID atomicity consistency isolation durability isolation level Read Uncommitted Read Committed Repeatable Read Serializable snapshot isolation SSI MVCC dirty read non-repeatable read phantom read lost update write skew serialization failure 40001 retry WAL VACUUM xmin xmax

SEE ALSO Locking & Concurrency Control · Indexes · SQL vs NoSQL & Scaling Databases · Transactions & Migrations · V1·15 Google Drive · V2·7 Hotel Reservation System · V2·11 Payment System · V2·12 Digital Wallet

ON THIS PAGE

1. ACID · p. 116
2. The anomalies (two-session timelines) · p. 117
3. Isolation levels: SQL standard vs Postgres · p. 118
4. MVCC in one paragraph · p. 119
5. Serialization failures → retry · p. 119

**What:** a transaction groups several statements into one unit: either all of them take effect, or none do. The **isolation level** decides how much concurrent transactions can see of each other.

> **i WHY IT MATTERS**
> in a wallet or exchange, two requests at the same time can double-spend money. Interviewers want you to name the anomaly and the right fix, not just say "use a transaction".

## ACID

↑ TOP

| Letter | Meaning | How Postgres does it |
|---|---|---|
| **A**tomicity | all statements commit, or all roll back | `BEGIN ... COMMIT` / `ROLLBACK`; an error aborts the whole txn |
| **C**onsistency | data moves from one valid state to another | constraints: `CHECK`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL` |
| **I**solation | concurrent txns don't corrupt each other | MVCC + isolation levels + locks |
| **D**urability | after `COMMIT`, data survives a crash | WAL (write-ahead log) is flushed to disk before commit returns |

SQL

```sql
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;   -- both or neither
```

↑ TOP

## The anomalies (two-session timelines)

Table used: `accounts(id, balance)`, account 1 starts with balance 100.

**1. Dirty read**: reading data another txn has **not committed** yet.

```
A: BEGIN; UPDATE accounts SET balance = 0 WHERE id = 1;   -- not committed
B: SELECT balance FROM accounts WHERE id = 1;  -- dirty read would see 0
A: ROLLBACK;                                    -- B used a value that never existed
```

Never happens in Postgres, at any level.

**2. Non-repeatable read**: the same row read twice in one txn gives different values.

```
A: BEGIN; SELECT balance FROM accounts WHERE id = 1;   -- 100
B: UPDATE accounts SET balance = 50 WHERE id = 1; COMMIT;
A: SELECT balance FROM accounts WHERE id = 1;          -- 50 in Read Committed
```

**3. Phantom read**: the same *query* (range/filter) returns a different set of rows.

```
A: BEGIN; SELECT count(*) FROM orders WHERE user_id = 7;   -- 3
B: INSERT INTO orders (user_id, ...) VALUES (7, ...); COMMIT;
A: SELECT count(*) FROM orders WHERE user_id = 7;          -- 4 in Read Committed
```

**4. Lost update**: two txns read the same value, both compute from it, and the second write **overwrites** the first.

```
A: SELECT balance FROM accounts WHERE id = 1;   -- 100 (app computes 100 - 30 = 70)
B: SELECT balance FROM accounts WHERE id = 1;   -- 100 (app computes 100 - 50 = 50)
A: UPDATE accounts SET balance = 70 WHERE id = 1; COMMIT;
B: UPDATE accounts SET balance = 50 WHERE id = 1; COMMIT;   -- A's -30 is lost
```

Final 50, should be 20. In Postgres Repeatable Read, B's `UPDATE` fails with *"could not serialize access due to concurrent update"*.

**5. Write skew**: two txns read the **same condition**, then each writes a **different row**. Each write is fine alone, together they break a rule.

Rule: at least one doctor must be on call. Alice and Bob are both on call.

```
A: BEGIN ISOLATION LEVEL REPEATABLE READ;
A: SELECT count(*) FROM doctors WHERE on_call;          -- 2, so I can leave
B: BEGIN ISOLATION LEVEL REPEATABLE READ;
B: SELECT count(*) FROM doctors WHERE on_call;          -- 2, so I can leave
A: UPDATE doctors SET on_call = false WHERE name = 'alice'; COMMIT;
B: UPDATE doctors SET on_call = false WHERE name = 'bob';   COMMIT;
-- 0 doctors on call. No row conflict, so Repeatable Read allows it.
```

Fintech version: two withdrawals from two different sub-accounts that share one total limit. Only **Serializable** (or an explicit lock on a shared row) prevents it.

## Isolation levels: SQL standard vs Postgres

↑ TOP

The standard only says what each level **must prevent at minimum**. Postgres is stricter.

| Level | Dirty read | Non-repeatable read | Phantom | Lost update | Write skew |
|---|---|---|---|---|---|
| Read Uncommitted (standard) | possible | possible | possible | possible | possible |
| Read Uncommitted (**Postgres**) | **no** (behaves as Read Committed) | possible | possible | possible | possible |
| Read Committed (standard and **Postgres default**) | no | possible | possible | possible | possible |
| Repeatable Read (standard) | no | no | possible | – | – |
| Repeatable Read (**Postgres**, snapshot isolation) | no | no | **no** | **no** (error, retry) | **possible** |
| Serializable (**Postgres**, SSI) | no | no | no | no | **no** (error, retry) |

SQL

```sql
BEGIN ISOLATION LEVEL SERIALIZABLE;
...
COMMIT;

SHOW default_transaction_isolation;   -- read committed
```

Note: MySQL InnoDB's default is Repeatable Read, and it behaves differently (locking reads see the latest data). Don't mix the two in an interview.

↑ TOP

## MVCC in one paragraph

Postgres never overwrites a row in place. An `UPDATE` writes a **new row version** and marks the old one as expired (each version has `xmin` = txn that created it, `xmax` = txn that deleted it). Each statement or transaction reads from a **snapshot**: "which txns were committed when I started". So **readers never block writers and writers never block readers**; only two writers on the same row wait for each other. Read Committed takes a new snapshot for **each statement**; Repeatable Read and Serializable take one snapshot at the **first statement** of the txn and keep it. Old versions become garbage that **VACUUM** cleans up, which is why long-running transactions cause table bloat.

↑ TOP

## Serialization failures → retry

At Repeatable Read and Serializable, Postgres prefers to **abort** a txn rather than allow an anomaly. You get SQLSTATE `40001` (`serialization_failure`); deadlocks give `40P01`. The app must **retry the whole transaction** from the start (re-read the data, not just repeat the last statement).

PYTHON

```python
import psycopg
from psycopg import errors

def transfer(conn, src, dst, amount, attempts=5):
    for _ in range(attempts):
        try:
            with conn.transaction():
                conn.execute("SET TRANSACTION ISOLATION LEVEL SERIALIZABLE")
                ...  # all reads and writes here
            return
        except (errors.SerializationFailure, errors.DeadlockDetected):
            continue   # optionally sleep with jitter
    raise RuntimeError("too much contention")
```

Keep these txns short, and make side effects (HTTP calls, Kafka publishes) happen **after** commit, because a retried txn runs twice.

✓ KEY POINTS

- Postgres default is **Read Committed**; dirty reads never happen in Postgres.
- Postgres Repeatable Read is **snapshot isolation**: no phantoms, lost updates raise an error, but write skew can still happen.
- **Serializable (SSI)** is the only level that prevents write skew, at the cost of retries.
- In Read Committed, `UPDATE ... SET balance = balance - 10` is safe (it re-reads the latest row), but "SELECT in app, then UPDATE with the computed value" is a lost update.
- Higher isolation → more aborts → you **must** write retry logic.

**❝ INTERVIEW LINE**

*"Postgres defaults to Read Committed, which still allows lost updates and write skew. For a balance change I'd use an atomic* `UPDATE` *or* `SELECT ... FOR UPDATE`*; for rules that span several rows, like a shared limit, I'd use Serializable and retry on* `40001`*."*

**⚠ COMMON TRAP**

saying "Repeatable Read prevents all problems" or "Postgres Repeatable Read allows phantoms". In Postgres it does **not** allow phantoms, but it **does** allow write skew.

↑ Back to top ☰ Contents

CHAPTER 5 · DATABASES

# Locking & Concurrency Control

locking · pessimistic locking · optimistic locking · SELECT FOR UPDATE · FOR NO KEY UPDATE · FOR SHARE · SKIP LOCKED · NOWAIT · lock_timeout · version column · compare-and-swap · atomic update · RETURNING · deadlock · lock ordering · deadlock_timeout · advisory lock · pg_advisory_xact_lock · pg_try_advisory_lock · job queue · double spend · race condition · wallet · ledger · idempotency key

SEE ALSO Transactions & Isolation Levels · Indexes · Celery with Redis as Broker · SQL vs NoSQL & Scaling Databases · Django ORM in Depth · Transactions & Migrations · V1·4 Rate Limiter · V2·7 Hotel Reservation System · V2·12 Digital Wallet

ON THIS PAGE

1. Pessimistic locking: SELECT ... FOR UPDATE · p. 121
2. Optimistic locking: version column · p. 122
3. Atomic update: let the database do it · p. 123
4. Deadlocks and lock ordering · p. 123
5. Advisory locks · p. 124
6. Wallet withdrawal, three ways · p. 124

**What:** ways to stop concurrent transactions from breaking each other's data. Three families: **pessimistic** (lock first, then work), **optimistic** (work, then check nobody changed it), and **atomic** single statements (let the database do read+write in one step).

> **i WHY IT MATTERS**
> for a wallet or exchange, the classic question is "two withdrawal requests arrive at the same time, how do you prevent double spending?". Isolation levels alone (default Read Committed) do not solve it. See Transactions & Isolation.

## Pessimistic locking: `SELECT ... FOR UPDATE`

↑ TOP

Locks the selected rows until the txn ends. Other txns that try to `UPDATE`, `DELETE` or `SELECT ... FOR UPDATE` the same rows **wait**. Plain `SELECT` is never blocked (MVCC).

SQL

```sql
BEGIN;
SELECT balance FROM wallets WHERE id = 1 FOR UPDATE;   -- other writers wait here
-- app checks balance, computes...
UPDATE wallets SET balance = balance - 30 WHERE id = 1;
COMMIT;                                                -- lock released
```

| Lock mode | Use |
|---|---|
| `FOR UPDATE` | you will update/delete the row (strongest row lock) |
| `FOR NO KEY UPDATE` | you update non-key columns; does not block FK checks from child inserts (this is what a plain `UPDATE` takes) |

| Lock mode | Use |
|---|---|
| `FOR SHARE` | read and make sure nobody changes it until commit; other `FOR SHARE` allowed |

**Options for waiting:**

```sql
-- Fail immediately instead of waiting
SELECT * FROM wallets WHERE id = 1 FOR UPDATE NOWAIT;    -- ERROR 55P03 if locked

-- Wait at most 2 seconds
SET LOCAL lock_timeout = '2s';

-- Skip rows that are locked: perfect for a job queue
BEGIN;
SELECT id, payload FROM jobs
WHERE status = 'pending'
ORDER BY created_at
LIMIT 10
FOR UPDATE SKIP LOCKED;          -- each worker gets different rows, no waiting
UPDATE jobs SET status = 'running' WHERE id = ANY(:ids);
COMMIT;
```

`SKIP LOCKED` gives a reliable job queue inside Postgres (no extra broker), as long as the volume is moderate. Pair it with a partial index `ON jobs (created_at) WHERE status = 'pending'`.

## Optimistic locking: version column

↑ TOP

No lock while the user/app thinks. At write time, update **only if nobody changed the row** since you read it.

```sql
ALTER TABLE wallets ADD COLUMN version int NOT NULL DEFAULT 0;

SELECT balance, version FROM wallets WHERE id = 1;     -- 100, version 7

UPDATE wallets
SET balance = 70, version = version + 1
WHERE id = 1 AND version = 7;    -- 1 row → success; 0 rows → someone else won, retry
```

- Always check the **affected row count**. 0 rows = conflict.
- Good when conflicts are **rare** (editing a profile, admin forms, long "read → user edits → save" flows).
- Bad when conflicts are **frequent** (hot wallet): many retries waste work.
- Django: `Wallet.objects.filter(id=1, version=7).update(balance=70, version=F("version") + 1)` returns the row count.

↑ TOP

## Atomic update: let the database do it

For counters and balances, the best answer is often one statement, no explicit lock, no retry:

```sql
UPDATE wallets
SET balance = balance - 30
WHERE id = 1 AND balance ⩾ 30
RETURNING balance;      -- 0 rows → insufficient funds
```

Why it's safe even in Read Committed: the `UPDATE` takes a row lock, and if another txn changed the row first, Postgres **waits, then re-checks the `WHERE` on the newest version**. So two concurrent withdrawals can never both pass `balance ⩾ 30` on stale data. Add `CHECK (balance ⩾ 0)` as a final safety net.

The trap is the read-modify-write in app code:

```python
w = Wallet.objects.get(id=1)        # ❌ lost update
w.balance -= 30
w.save()

Wallet.objects.filter(id=1, balance__gte=30).update(balance=F("balance") - 30)   # ✅ atomic
```

## Deadlocks and lock ordering

↑ TOP

Two txns each hold a lock the other needs:

```
A: UPDATE wallets SET balance = balance - 10 WHERE id = 1;   -- locks 1
B: UPDATE wallets SET balance = balance - 10 WHERE id = 2;   -- locks 2
A: UPDATE wallets SET balance = balance + 10 WHERE id = 2;   -- waits for B
B: UPDATE wallets SET balance = balance + 10 WHERE id = 1;   -- waits for A → deadlock
```

Postgres checks for deadlocks after `deadlock_timeout` (default 1s) and **aborts one txn** with SQLSTATE `40P01`. The app should retry it.

Prevent it:

- **Lock in a fixed order**, e.g. by id: `SELECT ... FROM wallets WHERE id IN (1, 2) ORDER BY id FOR UPDATE;` then do the updates.
- Keep transactions **short**; no network calls while holding locks.
- Use `lock_timeout` so a stuck txn fails fast instead of piling up waiters.

↑ TOP

## Advisory locks

A lock on an **app-chosen number**, not on a row. Postgres doesn't know what it means; your code does.

SQL

```sql
-- Transaction-level: released automatically at COMMIT/ROLLBACK (preferred)
SELECT pg_advisory_xact_lock(hashtext('payout:user:42'));

-- Try without waiting: returns true/false
SELECT pg_try_advisory_lock(42);      -- session-level: you must pg_advisory_unlock(42)
```

Use for: "only one cron instance runs this job", "only one payout process per user at a time", serializing work for a key that has no row yet. Caution: **session-level** advisory locks break with PgBouncer in transaction mode (the next txn may run on a different server connection); use the `xact` versions there.

## Wallet withdrawal, three ways

↑ TOP

Requirement: withdraw 30 from wallet 1, never go below 0, and record it in a ledger.

**1. Atomic update (best default):**

SQL

```sql
BEGIN;
UPDATE wallets SET balance = balance - 30
WHERE id = 1 AND balance ⩾ 30
RETURNING balance;                -- 0 rows → ROLLBACK, "insufficient funds"
INSERT INTO ledger (wallet_id, amount, kind) VALUES (1, -30, 'withdraw');
COMMIT;
```

**2. Pessimistic (when you need complex checks in app code, e.g. daily limit, KYC level):**

SQL

```sql
BEGIN;
SELECT balance, daily_withdrawn FROM wallets WHERE id = 1 FOR UPDATE;
-- app: check balance ⩾ 30, check daily limit, fraud rules...
UPDATE wallets SET balance = balance - 30, daily_withdrawn = daily_withdrawn + 30 WHERE id
= 1;
INSERT INTO ledger (wallet_id, amount, kind) VALUES (1, -30, 'withdraw');
COMMIT;
```

**3. Optimistic (low contention, or no long-held locks allowed):**

```sql
SELECT balance, version FROM wallets WHERE id = 1;      -- 100, v7
-- app: check 100 ⩾ 30
BEGIN;
UPDATE wallets SET balance = 70, version = 8 WHERE id = 1 AND version = 7;
-- 0 rows → ROLLBACK and retry from the SELECT
INSERT INTO ledger (wallet_id, amount, kind) VALUES (1, -30, 'withdraw');
COMMIT;
```

| | Atomic UPDATE | Pessimistic `FOR UPDATE` | Optimistic version |
|---|---|---|---|
| Locks held | very short | whole txn | very short |
| Retries | none | none (but deadlock possible) | yes, on conflict |
| Complex app checks | hard (only what fits in `WHERE`) | easy | easy |
| Best for | balances, counters, stock | hot rows + complex rules | rare conflicts, long user flows |

In all three, also use an **idempotency key** (unique constraint on `request_id` in the ledger) so a client retry does not withdraw twice.

✓ KEY POINTS

- Read Committed + read-modify-write in the app = lost update. Use one of the three patterns.
- `FOR UPDATE` blocks other writers on those rows, not plain readers.
- `SKIP LOCKED` = job queue; `NOWAIT` / `lock_timeout` = fail fast.
- Deadlocks: fixed lock order, short txns, retry on `40P01`.
- Advisory locks for app-level mutexes; prefer the `xact` version.

❝ INTERVIEW LINE

*"For a withdrawal I'd do a single conditional* `UPDATE ... WHERE balance ⩾ amount RETURNING`*, so the check and the write are atomic, plus a unique idempotency key in the ledger. If the rules are too complex for a* `WHERE`*, I'd lock the row with* `SELECT ... FOR UPDATE`*, and I'd always lock multiple rows in id order to avoid deadlocks."*

⚠ COMMON TRAP

thinking "it's inside a transaction, so it's safe". A transaction at Read Committed does **not** stop two requests from both reading balance 100 and both withdrawing. You need a lock, an atomic update, or a version check.

↑ Back to top ☰ Contents

CHAPTER 5 · DATABASES

# SQL vs NoSQL & Scaling Databases

SQL NoSQL relational document store MongoDB key-value Redis wide-column Cassandra ScyllaDB graph database Neo4j normalization denormalization replication primary replica read replica synchronous replication asynchronous replication replication lag read-your-writes failover partitioning sharding shard key hot partition resharding consistent hashing Citus connection pooling PgBouncer N+1 query select_related prefetch_related joinedload selectinload ORM

SEE ALSO Indexes · Transactions & Isolation Levels · Locking & Concurrency Control · How to Run a System Design Interview · Kafka vs RabbitMQ · V1·5 Consistent Hashing · V1·6 Key-Value Store · V1·7 Unique ID Generator · V1·8 URL Shortener · V2·1 Proximity Service · V2·3 Google Maps · V2·8 Distributed Email Service · V2·9 S3-like Object Storage · V2·10 Real-time Gaming Leaderboard

ON THIS PAGE

1. SQL vs NoSQL · p. 126
2. Normalization vs denormalization · p. 127
3. Replication · p. 127
4. Partitioning vs sharding · p. 128
5. Connection pooling (PgBouncer) · p. 129
6. The N+1 query problem · p. 129

**What:** how to choose a database type, and how to scale one when a single server is not enough: replication for reads, partitioning/sharding for size and writes, pooling for connections, and fixing ORM query patterns.

> **i WHY IT MATTERS**
> every system design round has a "data layer" part. A good answer starts with one Postgres, explains the bottleneck, and adds the **smallest** step that fixes it. See the SD framework.

## SQL vs NoSQL

↑ TOP

| Type | Examples | Data model | Good for | Weak at |
|---|---|---|---|---|
| Relational (SQL) | Postgres, MySQL | tables, joins, schema | money, orders, users: anything needing transactions and constraints | horizontal write scaling (needs work) |
| Document | MongoDB | JSON documents | flexible / nested data read as one unit (product catalog, CMS) | joins, multi-document transactions (possible but costly) |
| Key-value | Redis, DynamoDB | key → value | cache, sessions, rate limits, very fast lookups by key | queries by anything other than the key |
| Wide-column | Cassandra, ScyllaDB | partition key → sorted rows | huge write volume, time series, messages, logs | ad-hoc queries, joins, strong consistency (tunable) |

| Type | Examples | Data model | Good for | Weak at |
|---|---|---|---|---|
| Graph | Neo4j | nodes + edges | relationships many hops deep (fraud rings, recommendations) | bulk aggregations |

Rule of thumb: **Postgres by default** (it also has `jsonb`, full-text search, partitioning). Add a specialized store when a clear requirement asks for it, often **next to** Postgres (Redis cache, Elasticsearch search, ClickHouse analytics).

For a crypto exchange: balances, orders, ledger → Postgres (ACID). Order book in memory / Redis. Trade history and market data at high volume → time-series or columnar store.

## Normalization vs denormalization

↑ TOP

- **Normalization**: each fact stored once (3NF). `orders.user_id` points to `users`; the user's name lives only in `users`. No update anomalies, smaller writes, but reads need joins.
- **Denormalization**: copy data to avoid joins or recomputation. Faster reads, but you must keep copies in sync.

```sql
-- Normalized: compute on read
SELECT u.id, count(o.id) FROM users u LEFT JOIN orders o ON o.user_id = u.id GROUP BY u.id;

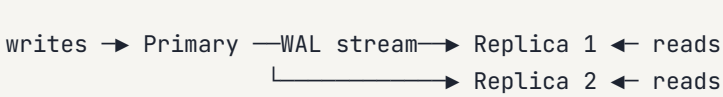
-- Denormalized: keep a counter, update it in the same txn as the insert
UPDATE users SET orders_count = orders_count + 1 WHERE id = 7;
```

Start normalized. Denormalize **specific** hot read paths (counters, cached totals, materialized views, read models built from events), and know who updates the copy.

## Replication

↑ TOP

The **primary** takes all writes; **replicas** receive the changes (Postgres streams the WAL) and serve reads.




| | Async (default) | Sync |
|---|---|---|
| Commit returns when | primary has written WAL locally | at least one replica has also confirmed |
| Write latency | low | higher (network round trip) |
| Data loss on primary crash | last few commits may be lost | none (for confirmed standbys) |

| | Async (default) | Sync |
|---|---|---|
| Postgres setting | default | `synchronous_standby_names` + `synchronous_commit = on` (or `remote_apply` to make the change visible on the replica too) |

**Replication lag**: an async replica can be milliseconds to seconds behind. Problem: user updates their profile, the next page reads from a replica, and shows the **old** data.

Fixes for **read-your-writes**:

- Read from the primary for that user for a few seconds after they write (sticky flag in session/cookie).
- Always read critical data (balances, order status) from the primary.
- Track the WAL position: after a write, remember `pg_current_wal_lsn()`; only use a replica whose `pg_last_wal_replay_lsn()` has passed it.

Also: replicas scale **reads only**; writes still hit one primary. Failover (Patroni, cloud managed) promotes a replica when the primary dies.

Logical replication (row changes per table) is a different tool, used for migrations, partial copies and CDC (e.g. Debezium → Kafka).

## Partitioning vs sharding

↑ TOP

| | Partitioning | Sharding |
|---|---|---|
| Where | one Postgres server, one table split into child tables | data split across **many servers** |
| Goal | manage huge tables: faster pruning, cheap deletes of old data | scale **writes** and storage beyond one machine |
| Transparent to app | yes | no (or via Citus / a proxy) |
| Cross-part queries / txns | normal SQL | expensive or not supported |

SQL

```sql
CREATE TABLE trades (
    id bigint, symbol text, created_at timestamptz NOT NULL, ...
) PARTITION BY RANGE (created_at);

CREATE TABLE trades_2026_10 PARTITION OF trades
    FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');

-- WHERE created_at ⩾ '2026-10-05' scans only trades_2026_10 (partition pruning)
-- Retention: DROP TABLE trades_2025_01;   instant, no huge DELETE
```

Postgres supports `RANGE`, `LIST` and `HASH` partitioning. Note: a unique key or primary key must include the partition key.

**Choosing a shard key:**

- **High cardinality** and **even distribution**: `user_id`, `account_id`.
- **Matches the main queries**: most queries should hit one shard (`WHERE user_id = ?`).
- Avoid **hot keys**: `created_at` sends all new writes to the last shard; one huge market maker on `account_id` may need special handling.
- Related data lives together: shard `orders`, `wallets`, `ledger` all by `user_id` so a user's txn stays on one shard.

**Resharding:** with `shard = hash(key) % N`, changing N moves almost every key. Common fixes:

- Create **many logical shards** (e.g. 1024) up front and map them to a few physical servers; to grow, move whole logical shards.
- **Consistent hashing**: keys and nodes on a ring; adding a node moves only about 1/N of the keys (used by Cassandra, DynamoDB, cache clusters).

Before sharding, try: indexes, query fixes, caching, read replicas, partitioning, a bigger machine. Sharding adds lasting complexity (cross-shard joins, distributed transactions, rebalancing).

## Connection pooling (PgBouncer)

↑ TOP

Postgres starts **one OS process per connection** (several MB each). 50 app pods × 20 connections = 1000 connections, which hurts Postgres badly. A pooler keeps a small number of real server connections and shares them.

| PgBouncer mode | A server connection is given to a client for | Notes |
|---|---|---|
| session | the whole client session | safe, but little sharing |
| **transaction** | one transaction | most common; breaks session state: `SET` (use `SET LOCAL`), session advisory locks, `LISTEN`, temp tables; prepared statements need PgBouncer 1.21+ with `max_prepared_statements` |
| statement | one statement | no multi-statement txns; rare |

Also keep app-side pools small (SQLAlchemy `pool_size`, Django `CONN_MAX_AGE`, Go `db.SetMaxOpenConns`). More connections than CPU cores rarely makes Postgres faster.

## The N+1 query problem

↑ TOP

The ORM loads a list with **1 query**, then runs **1 more query per row** when you touch a relation. 100 orders → 101 queries.

PYTHON

```python
# Django: N+1
for order in Order.objects.all()[:100]:
    print(order.user.email)            # 1 query per order

# Fix: FK / one-to-one → JOIN in the same query
for order in Order.objects.select_related("user")[:100]:
    print(order.user.email)            # 1 query total

# Reverse FK / many-to-many → 2 queries: orders, then items WHERE order_id IN (...)
Order.objects.prefetch_related("items")
```

PYTHON

```python
# SQLAlchemy 2.0
from sqlalchemy.orm import joinedload, selectinload

stmt = select(Order).options(joinedload(Order.user))      # many-to-one: JOIN
stmt = select(Order).options(selectinload(Order.items))   # one-to-many: second query with
IN (...)
orders = session.scalars(stmt).unique().all()
```

Detect it: Django Debug Toolbar / `nplusone`, SQLAlchemy `echo=True`, or the same query repeated many times in `pg_stat_statements`. In Go (no ORM magic) the same bug is a query inside a loop; fix with one `WHERE id = ANY($1)` query and a map.

✓ KEY POINTS

- Start with one Postgres; add replicas, partitioning, caches, then sharding, only when a bottleneck shows.
- Async replicas lag: critical reads and read-your-writes go to the primary.
- Partitioning is inside one server; sharding is across servers and changes your app.
- Shard key = even, high cardinality, in most queries; plan for resharding (logical shards / consistent hashing).
- PgBouncer in transaction mode; avoid session-level features.
- N+1: `select_related` / `joinedload` for to-one, `prefetch_related` / `selectinload` for to-many.

❝ INTERVIEW LINE

*"I'd keep money data in Postgres for ACID. To scale reads I'd add async replicas but send read-your-writes and balance reads to the primary, put PgBouncer in front, partition big time-based tables, and shard by* `user_id` *only when one primary can't handle the writes."*

⚠ COMMON TRAP

"NoSQL scales, SQL doesn't." Postgres handles a lot on one machine with good indexes, replicas and partitioning, and NoSQL doesn't remove the need to pick a good partition key. Second trap: adding read replicas and forgetting replication lag.

↑ Back to top ☰ Contents

# 06
# Networking

2 pages. Each card says what the page covers and its key lines; open the page for the full depth.

### HTTP/1.1 vs HTTP/2 vs HTTP/3 · p. 133

three versions of the same protocol. The **semantics** are identical (GET/POST, status codes, headers). What changes is **how bytes travel**: text vs binary, how many requests share a connection, and TCP vs UDP.

- HTTP/1.1: keep-alive, but **one request at a time per connection** → browsers open ~6 connections per host.
- HTTP/2: binary frames, **many streams on one TCP connection**, HPACK headers. Still **TCP head-of-line blocking** on packet loss.
- HTTP/3: HTTP over **QUIC (UDP)**: per-stream loss recovery, TLS 1.3 built in (1-RTT, 0-RTT resume), **connection migration**.
- Same semantics (methods, status codes, headers) in all versions; only the **transport/wire format** changes.
- gRPC **needs HTTP/2** (streams + trailers) end to end → the load balancer must speak HTTP/2 to backends.

Read the full page →

### Web & Networking Fundamentals · p. 137

the basic building blocks under every web backend: transport (TCP/UDP), security (TLS), naming (DNS), and the HTTP layer (methods, status codes, API styles, auth, CORS).

- TCP = reliable, ordered, connection (3-way handshake). UDP = no handshake, no order, no retransmit (DNS, video, QUIC).
- URL → DNS → TCP handshake → TLS → HTTP request → server/app/DB → response → render.
- Safe = no side effects (GET, HEAD, OPTIONS). Idempotent = same result if repeated (safe + PUT, DELETE). **POST and PATCH are not.**
- 401 = not authenticated, 403 = authenticated but not allowed. 502 = bad upstream answer, 504 = upstream timeout.
- Server → client push: **SSE** (one-way, simple), **WebSocket** (two-way), long polling (fallback).
- CORS is enforced by the **browser**, not the server; it doesn't protect your API from curl.

Read the full page →

CHAPTER 6 · NETWORKING

# HTTP/1.1 vs HTTP/2 vs HTTP/3

HTTP/1.0 HTTP/1.1 HTTP/2 HTTP/3 QUIC keep-alive persistent connection pipelining head-of-line blocking HOL blocking multiplexing stream binary framing HPACK QPACK server push 103 Early Hints ALPN Alt-Svc 0-RTT 1-RTT TLS 1.3 connection migration connection ID UDP gRPC TLS termination load balancer h2c

SEE ALSO Web & Networking Fundamentals · Go Essentials & Gotchas · Django REST Framework

ON THIS PAGE

1. HTTP/1.0 → HTTP/1.1 · p. 133
2. HTTP/2 (2015) · p. 134
3. HTTP/3 (2022, RFC 9114) · p. 134
4. Comparison · p. 135
5. What this means for a backend engineer · p. 135

**What:** three versions of the same protocol. The **semantics** are identical (GET/POST, status codes, headers). What changes is **how bytes travel**: text vs binary, how many requests share a connection, and TCP vs UDP.

> **i WHY IT MATTERS**
> it explains real performance questions ("why is my page slow on mobile?", "why does gRPC break behind this proxy?") and is a common "what's the difference" interview question.

## HTTP/1.0 → HTTP/1.1

↑ TOP

- **HTTP/1.0:** a new TCP connection for **every** request. Each one pays the TCP (and TLS) handshake.
- **HTTP/1.1 (1997):**
  - **Keep-alive (persistent connections)** by default: reuse one TCP connection for many requests.
  - `Host` **header** required → many sites on one IP (virtual hosting).
  - **Chunked transfer encoding**: stream a body without knowing its length.
  - **Pipelining**: send several requests without waiting, but responses must come back **in order**. One slow response blocks all others behind it (**head-of-line blocking**). Buggy proxies → browsers never enabled it.
- Result: per connection, effectively **one request in flight at a time**. Browsers work around it by opening **~6 parallel connections per host**. Old hacks: domain sharding, sprites, bundling files.

```
HTTP/1.1, one connection:
  req A ──► ... resp A ◄──  req B ──► ... resp B ◄──     (serial)
HTTP/1.1, pipelining:
  req A, req B ──►   resp A (slow) ... resp B waits behind A   (HOL blocking)
```

## HTTP/2 (2015)

↑ TOP

- **Binary framing:** messages are split into **frames** (HEADERS, DATA, ...), each tagged with a **stream ID**.
- **Multiplexing:** many **streams** (request/response pairs) are interleaved on **one TCP connection**. A slow response no longer blocks others **at the HTTP level**.
- **HPACK header compression:** static + dynamic tables, so repeated headers (cookies, user-agent) cost a few bytes.
- **Stream priority and flow control** per stream and per connection.
- **Server push:** server sends resources before they are asked for. **Deprecated in practice** (Chrome removed it in 2022); use `103 Early Hints` / `<link rel=preload>` instead.
- Negotiated via **ALPN** during the TLS handshake ( `h2` ). Browsers only use HTTP/2 over TLS; cleartext `h2c` exists for internal traffic.

```
One TCP connection:
  [S1 HEADERS][S3 HEADERS][S1 DATA][S3 DATA][S5 HEADERS][S1 DATA] ...
   stream 1, 3, 5 interleaved → no HTTP-level HOL blocking
```

**The remaining problem: TCP head-of-line blocking.** TCP delivers one ordered byte stream. If **one packet is lost**, TCP holds **all later bytes** until the retransmit arrives, even bytes belonging to other streams. On a lossy network, HTTP/2 with one connection can be **worse** than HTTP/1.1 with 6.

```
packet loss on stream 1:
  TCP:  [S1 ✗][S3 ✓][S5 ✓]  → S3 and S5 wait for S1's retransmit (kernel buffers them)
```

## HTTP/3 (2022, RFC 9114)

↑ TOP

HTTP/3 = HTTP semantics on **QUIC**, a transport built on **UDP** (QUIC runs in user space, not in the kernel).

- **Independent streams in the transport:** a lost packet only blocks **its own stream**. No TCP HOL blocking.
- **TLS 1.3 is built into QUIC:** transport + crypto handshake together → **1-RTT** for a new connection (TCP + TLS 1.3 needs 2 RTT). **0-RTT** on resumption (send data in the first packet; replayable, so only safe for idempotent requests).

- **Connection migration:** connections are identified by a **connection ID**, not the IP/port 4-tuple. Phone moves Wi-Fi → 4G and the connection survives.
- **QPACK** header compression (HPACK adapted for out-of-order streams).
- Almost everything is encrypted, even transport headers → middleboxes can't mess with it.
- Discovery: server advertises `Alt-Svc: h3=":443"` (or an HTTPS DNS record); the client tries QUIC and **falls back to TCP** if UDP is blocked.

```
Handshake round trips before the first request:
  HTTP/1.1 or 2 + TLS 1.3:  TCP SYN/ACK (1 RTT) + TLS (1 RTT) = 2 RTT
  HTTP/3 (QUIC):            QUIC+TLS together              = 1 RTT   (0 RTT on resume)
```

## Comparison

↑ TOP

| | HTTP/1.1 | HTTP/2 | HTTP/3 |
|---|---|---|---|
| Transport | TCP | TCP | QUIC over UDP |
| Format | text | binary frames | binary frames (QUIC streams) |
| Requests per connection | one at a time (pipelining unused) | many multiplexed streams | many multiplexed streams |
| Connections per host (browser) | ~6 | 1 | 1 |
| HOL blocking | HTTP level + TCP level | TCP level only | none across streams |
| Header compression | none | HPACK | QPACK |
| TLS | optional (separate) | in practice required (browsers) | always, TLS 1.3 built in |
| New-connection setup (with TLS 1.3) | 2 RTT | 2 RTT | 1 RTT (0-RTT resume) |
| Network change (Wi-Fi → 4G) | reconnect | reconnect | connection migrates |
| Server push | no | yes, deprecated | in spec, rarely used |

## What this means for a backend engineer

↑ TOP

- **gRPC needs HTTP/2** end to end: it uses streams for streaming RPCs and **trailers** for `grpc-status`. A proxy/load balancer that downgrades to HTTP/1.1 breaks it. Configure the LB for HTTP/2 to backends (e.g. Nginx `grpc_pass`, Envoy, AWS ALB with gRPC target group).
- **Load balancing gRPC/HTTP/2:** one long-lived connection carries all requests, so an **L4 (TCP) load balancer pins one client to one backend**. Use an **L7** LB (balances per request/stream) or client-side balancing.

- **TLS termination:** usually at the edge (CDN, LB, Nginx). The edge can speak HTTP/2 or HTTP/3 to clients and plain HTTP/1.1 to the app. Your Python/Go app often never sees HTTP/3.
- **When HTTP/3 helps:** mobile and **lossy/high-latency** networks, users switching networks, many small parallel requests. Inside a data center (low loss, low RTT) the gain is small.
- **HTTP/3 costs:** UDP may be blocked or rate-limited by firewalls, QUIC uses more CPU (user space, encrypted headers), and tooling/debugging is less mature.
- Old HTTP/1.1 hacks (domain sharding, bundling everything) **hurt** with HTTP/2 because they defeat the single multiplexed connection.

✓ KEY POINTS

- All versions keep the same methods, status codes and headers. Code at the handler level doesn't change.
- HTTP/2 fixed HOL blocking at the **HTTP** layer; HTTP/3 fixed it at the **transport** layer.
- QUIC's handshake merges transport + TLS → faster setup; 0-RTT data can be **replayed**, so only use it for idempotent requests.
- Version is chosen with **ALPN** (h2) and **Alt-Svc** / DNS HTTPS records (h3), with fallback.

❝ INTERVIEW LINE

*"HTTP/1.1 handles one request at a time per connection, so browsers open several connections. HTTP/2 multiplexes many streams over one TCP connection with binary frames and header compression, but a lost TCP packet still blocks all streams. HTTP/3 runs on QUIC over UDP, so loss only affects one stream, the TLS 1.3 handshake is built in, and connections survive network changes, which helps most on mobile."*

⚠ COMMON TRAP

saying "HTTP/2 removed head-of-line blocking". It removed it at the HTTP level only. TCP still delivers bytes in order, so one lost packet stalls every stream on the connection; that is exactly what HTTP/3/QUIC fixes.

↑ Back to top ☰ Contents

CHAPTER 6 · NETWORKING

# Web & Networking Fundamentals

TCP UDP three-way handshake SYN ACK FIN TIME_WAIT TLS 1.3 TLS handshake certificate SNI DNS recursive resolver root server TLD authoritative name server TTL A record CNAME what happens when you type a URL REST gRPC Protobuf WebSocket SSE Server-Sent Events long polling HTTP status codes idempotent safe methods PUT vs PATCH cookies JWT session CORS preflight OPTIONS same-origin policy

SEE ALSO HTTP/1.1 vs HTTP/2 vs HTTP/3 · Redis Pub/Sub vs RabbitMQ vs Kafka · Request Lifecycle, Middleware & Signals · Django REST Framework · V1·8 URL Shortener · V1·9 Web Crawler · V1·12 Chat System · V1·15 Google Drive

ON THIS PAGE

1. TCP vs UDP · p. 137
2. DNS resolution · p. 138
3. What happens when you type a URL · p. 138
4. API styles · p. 139
5. HTTP methods · p. 139
6. Status codes to know · p. 140
7. Cookies vs tokens (JWT) · p. 140
8. CORS · p. 141

**What:** the basic building blocks under every web backend: transport (TCP/UDP), security (TLS), naming (DNS), and the HTTP layer (methods, status codes, API styles, auth, CORS).

> **i WHY IT MATTERS**
> "what happens when you type a URL?" and "REST vs gRPC vs WebSocket?" are classic warm-up questions. Wrong status codes or non-idempotent retries are common real bugs.

## TCP vs UDP

↑ TOP

| | TCP | UDP |
|---|---|---|
| Connection | yes (handshake) | no |
| Reliability | retransmits lost packets | none (app handles it) |
| Ordering | ordered byte stream | datagrams, any order |
| Flow/congestion control | yes | no (app may add it) |
| Header | 20+ bytes | 8 bytes |
| Used by | HTTP/1.1, HTTP/2, DB connections, SSH | DNS, video/voice, games, QUIC (HTTP/3) |

**TCP 3-way handshake (1 RTT before data):**

```
client                      server
  | ── SYN (seq=x) ──────────► |
  | ◄── SYN-ACK (seq=y,ack=x+1)|
  | ── ACK (ack=y+1) ─────────► |   connection open; data can go with this ACK
```

Close: FIN/ACK in each direction (4 steps). The side that closes first waits in **TIME_WAIT** (many short connections → port exhaustion; use keep-alive / connection pools).

**TLS 1.3 handshake (1 RTT, on top of TCP):**

```
ClientHello (ciphers, key share, SNI) ──►
◄── ServerHello (key share) + {certificate, signature, Finished}   (already encrypted)
Finished + first HTTP request ──►
```

- Both sides compute the shared key from the key shares (ephemeral Diffie-Hellman → forward secrecy).
- Client checks the certificate chain against trusted CAs and the hostname.
- **SNI** tells the server which hostname you want (one IP, many certificates).
- Resumption can use **0-RTT** (replayable → only for idempotent requests). TLS 1.2 needed 2 RTT.

## DNS resolution

↑ TOP

1. Browser cache → OS cache (`/etc/hosts`, stub resolver).
2. Ask the **recursive resolver** (ISP, 8.8.8.8, 1.1.1.1). If cached, done.
3. Resolver asks a **root server** → "ask the `.com` TLD servers".
4. Asks the **TLD server** → "ask `example.com`'s authoritative servers".
5. Asks the **authoritative server** → `A 93.184.216.34` (or `CNAME` → repeat for the target).
6. Result is cached at every level for its **TTL**.

Records: `A` (IPv4), `AAAA` (IPv6), `CNAME` (alias), `MX` (mail), `TXT`, `NS`. Mostly UDP port 53 (TCP for large answers). Low TTL = faster failover, more queries.

## What happens when you type a URL

↑ TOP

1. **Parse** the URL (scheme, host, port, path). Check HSTS → force HTTPS.
2. **DNS** lookup → IP (steps above).
3. **TCP handshake** to IP:443 (or QUIC for HTTP/3).
4. **TLS handshake**: certificate check, keys, ALPN picks `h2` / `http/1.1`.
5. **HTTP request**: `GET /path` with headers (Host, cookies, Accept, ...).
6. **Server side**: CDN/load balancer → reverse proxy (Nginx) → app server (Gunicorn/uvicorn, Go `net/http`) → cache/DB → build response.
7. **Response**: status, headers (`Cache-Control`, `Set-Cookie`), body.

8. **Browser renders**: parse HTML → fetch CSS/JS/images (more requests, reuse the connection) → build DOM/CSSOM → layout → paint. JS runs.

## API styles

↑ TOP

| | REST | gRPC | WebSocket | SSE | Long polling |
|---|---|---|---|---|---|
| Direction | request/response | request/response + streaming (client, server, bidi) | full duplex | server → client only | server → client (emulated) |
| Transport | HTTP/1.1 or 2 | HTTP/2 | HTTP/1.1 Upgrade, then raw frames over TCP | plain HTTP, kept open | repeated HTTP requests |
| Format | JSON (usually) | Protobuf (binary, typed, generated code) | any (text/binary) | text `data:` events | JSON |
| Browser support | native | needs gRPC-Web proxy | native | native (`EventSource`, auto-reconnect) | native |
| Best for | public/CRUD APIs, caching | internal service-to-service, low latency, strict contracts | chat, games, collaborative editing | notifications, live feeds, LLM token streaming | fallback when nothing else works |
| Cost | simple, cacheable | tooling, harder to debug | stateful connections, harder to scale (sticky/pub-sub fan-out) | one-way only | high overhead, latency |

Scaling note: WebSocket/SSE connections are **long-lived and stateful**. With many servers, use a pub/sub layer (e.g. Redis Pub/Sub) so any server can push to any user.

## HTTP methods

↑ TOP

| Method | Safe | Idempotent | Note |
|---|---|---|---|
| GET | yes | yes | read; cacheable |
| HEAD | yes | yes | headers only |
| OPTIONS | yes | yes | CORS preflight |
| PUT | no | yes | **replace** the whole resource |
| DELETE | no | yes | second call may return 404, but the state is the same |

| Method | Safe | Idempotent | Note |
|---|---|---|---|
| POST | no | **no** | create / action; use an **Idempotency-Key** header for safe retries |
| PATCH | no | **no** (not guaranteed) | partial update (`{"op": "increment"}` isn't idempotent) |

**Safe** = doesn't change server state. **Idempotent** = doing it N times has the same effect as once. Retries (clients, proxies, queues) are only automatic-safe for idempotent methods.

## Status codes to know

↑ TOP

| Code | Meaning |
|---|---|
| 200 OK / 201 Created / 204 No Content | success (201 + `Location` header for new resources) |
| 301 / 308 | permanent redirect (308 keeps the method) |
| 302 / 307 | temporary redirect (307 keeps the method) |
| 304 Not Modified | cache still valid (`ETag` / `If-None-Match`) |
| 400 Bad Request | malformed input |
| 401 Unauthorized | **not authenticated** (no/invalid credentials) |
| 403 Forbidden | authenticated, but **no permission** |
| 404 Not Found | no such resource |
| 409 Conflict | state conflict (duplicate, version mismatch) |
| 422 Unprocessable Content | valid syntax, failed validation (FastAPI uses it) |
| 429 Too Many Requests | rate limited (`Retry-After`) |
| 500 Internal Server Error | bug in your server |
| 502 Bad Gateway | proxy got an invalid response from upstream (app crashed/closed) |
| 503 Service Unavailable | overloaded / maintenance |
| 504 Gateway Timeout | proxy got **no response in time** from upstream |

## Cookies vs tokens (JWT)

↑ TOP

- **Session cookie:** server stores the session (DB/Redis), the browser sends an opaque session ID cookie automatically. Easy to revoke. Set `HttpOnly` (no JS access → XSS-safe), `Secure`, `SameSite` (CSRF protection).
- **JWT:** a signed token `header.payload.signature` with claims (`sub`, `exp`). The server verifies the signature, **no DB lookup** → stateless, good for microservices. Downsides: **hard to revoke**

before `exp` (use short-lived access tokens + refresh tokens, or a denylist); the payload is only base64, **not encrypted**.
- They're not opposites: a JWT can be stored in a cookie. Cookie = **transport**, JWT = **token format**.

## CORS

↑ TOP

Browsers apply the **same-origin policy**: JS on `app.com` can't read responses from `api.com` (different scheme, host or port). **CORS** lets the server opt in with headers such as `Access-Control-Allow-Origin: https://app.com`. For "non-simple" requests (e.g. `PUT`, JSON content type, custom headers like `Authorization`), the browser first sends a **preflight** `OPTIONS` request and checks the `Access-Control-Allow-*` headers. With credentials (cookies) you can't use `*` as the origin. CORS is **only enforced by browsers**; it isn't server-side security.

✓ KEY POINTS

- TCP handshake costs 1 RTT, TLS 1.3 another 1 RTT → reuse connections (keep-alive, pools).
- DNS is cached at every level by TTL; it's also a cheap way to do failover/geo-routing.
- Choose the API style by direction and client: REST for public, gRPC internal, SSE for server push, WebSocket for two-way.
- Make retried operations idempotent (PUT, or POST + Idempotency-Key).

❝ INTERVIEW LINE

*"When I type a URL, the browser resolves the name through DNS, opens a TCP connection, does a TLS handshake, sends the HTTP request, and the request passes through a load balancer and reverse proxy to the app, which may hit a cache or database before the response comes back and the browser renders it."*

⚠ COMMON TRAP

mixing up 401 and 403, or thinking CORS protects the API. 401 means "who are you?", 403 means "I know you, but no". CORS only stops a browser page from reading the response; any non-browser client ignores it, so you still need real auth.

↑ Back to top ☰ Contents

# 07

# Go

1 pages. Each card says what the page covers and its key lines; open the page for the full depth.

### Go Essentials & Gotchas · p. 143

the Go concurrency toolkit (goroutines, channels, `select`, `sync`, `context`) plus the classic "predict the output" gotchas.

- Goroutine = ~2 KB growable stack, scheduled by Go (GMP: many G on few OS threads M via P = GOMAXPROCS).
- Unbuffered send blocks until a receiver is ready; receive from a **closed** chan → zero value, `ok=false`; **nil** chan blocks forever.
- Only the **sender** closes a channel. Send on closed chan → panic.
- `append` may share the backing array → two slices from one base can overwrite each other.
- An interface holding a typed nil pointer is **not** `== nil`. Return a literal `nil` error.
- Concurrent map writes → **fatal error** (not recoverable). Use `sync.Mutex` or `sync.Map`; run tests with `-race`.

Read the full page →

CHAPTER 7 · GO

# Go Essentials & Gotchas

Go goroutine GMP scheduler channel buffered channel unbuffered channel close nil channel select time.After context context.WithTimeout context.WithCancel sync.WaitGroup sync.Mutex sync.RWMutex sync.Once errgroup slice append backing array loop variable capture Go 1.22 nil interface typed nil defer concurrent map writes sync.Map map iteration order goroutine leak race detector

SEE ALSO Async / Await · HTTP/1.1 vs HTTP/2 vs HTTP/3

ON THIS PAGE

1. Goroutines vs threads · p. 143
2. Channels · p. 144
3. select, timeouts, context · p. 144
4. sync package and errgroup · p. 145
5. Gotchas: predict the output · p. 146

**What:** the Go concurrency toolkit (goroutines, channels, `select`, `sync`, `context`) plus the classic "predict the output" gotchas.

> **i WHY IT MATTERS**
> Go interviews almost always include one concurrency question and one gotcha snippet. Compare with Python's model in Async / Await.

## Goroutines vs threads

↑ TOP

| | Goroutine | OS thread |
|---|---|---|
| Start cost | ~2 KB stack, grows as needed | ~1–8 MB fixed stack |
| Scheduled by | Go runtime (user space) | OS kernel |
| Switch cost | cheap | expensive (kernel) |
| How many | 100k+ is fine | thousands |

**GMP scheduler: G** = goroutine, **M** = OS thread, **P** = logical processor with a run queue (`GOMAXPROCS`, default = CPU cores). An M needs a P to run Gs; idle Ps **steal work** from others. A G blocked in a syscall releases its P so another M keeps running Gs. Unlike Python asyncio, goroutines run **in parallel** on multiple cores and are preempted (no `await` needed).

↑ TOP

## Channels

```go
ch := make(chan int, 3) // buffered: send blocks only when full
ch ← 1
ch ← 2
close(ch)            // only the sender closes
for v := range ch {  // range stops after close + drain
    fmt.Print(v, " ")
}
v, ok := ←ch
fmt.Println(v, ok)   // 1 2 0 false
```

| Operation | nil channel | open channel | closed channel |
|---|---|---|---|
| send | blocks forever | blocks until receiver (unbuffered) / buffer space | **panic** |
| receive | blocks forever | blocks until value | zero value, `ok=false` |
| close | panic | ok | **panic** |

- **Unbuffered** = synchronization point (sender and receiver meet). **Buffered** = queue of size N.
- Nil channel blocking is useful: set a channel var to `nil` in a `select` to disable that case.

## select, timeouts, context

↑ TOP

```go
select {
case r := ←res:
    fmt.Println(r)
case ←time.After(50 * time.Millisecond):
    fmt.Println("timeout")
}

ctx, cancel := context.WithTimeout(context.Background(), 50*time.Millisecond)
defer cancel() // always call cancel, or the timer leaks until it fires
select {
case ←time.After(time.Second):
    fmt.Println("finished")
case ←ctx.Done():
    fmt.Println(ctx.Err()) // context deadline exceeded
}
```

- `select` picks a **random** ready case; `default` makes it non-blocking.
- **context**: pass `ctx` as the first argument down the call chain; long work checks `ctx.Done()`. Cancelling a parent cancels all children. Used for request deadlines in HTTP handlers and DB calls.

GO

**Worker that stops on close or cancel:**

```go
func worker(ctx context.Context, jobs <-chan int, results chan<- int) {
    for {
        select {
        case j, ok := <-jobs:
            if !ok {
                return // jobs closed
            }
            results <- j * 2
        case <-ctx.Done():
            return
        }
    }
}
```

## sync package and errgroup

↑ TOP

| Tool | Use |
|---|---|
| `sync.WaitGroup` | wait for N goroutines: `Add(1)` **before** `go`, `defer Done()`, `Wait()` |
| `sync.Mutex` | exclusive lock around shared state |
| `sync.RWMutex` | many readers **or** one writer (read-heavy data) |
| `sync.Once` | run init exactly once (lazy singleton) |
| `errgroup.Group` | like WaitGroup + **first error** returned; `WithContext` cancels the others on error |

```go
var wg sync.WaitGroup
var mu sync.Mutex
count := 0
for i := 0; i < 100; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        mu.Lock()
        count++
        mu.Unlock()
    }()
}
wg.Wait()
fmt.Println(count) // 100 (without the mutex: a data race, often < 100)
```

```go
// import "golang.org/x/sync/errgroup"
func fetchAll(ctx context.Context, urls []string) error {
    g, ctx := errgroup.WithContext(ctx)
    for _, u := range urls {
        g.Go(func() error { return fetch(ctx, u) }) // first error cancels ctx
    }
    return g.Wait() // first non-nil error
}
```

## Gotchas: predict the output

↑ TOP

### 1. append aliasing

```go
a := make([]int, 3, 4) // len 3, cap 4
b := append(a, 1)      // fits in cap → shares a's array
c := append(a, 2)      // writes the same slot
fmt.Println(a, b, c)   // [0 0 0] [0 0 0 2] [0 0 0 2]

s := []int{1, 2, 3}
t := s[:2]
t = append(t, 99)      // overwrites s[2]
fmt.Println(s, t)      // [1 2 99] [1 2 99]
```

Why: a slice is (pointer, len, cap). `append` reuses the array while `len < cap`; it copies only when it must grow. Fix: `slices.Clone`, or a full slice expression `s[:2:2]` (cap = len, so append must copy).

### 2. Loop variable capture

```go
var fs []func()
for i := 0; i < 3; i++ {
    fs = append(fs, func() { fmt.Print(i, " ") })
}
for _, f := range fs {
    f()
}
// Go 1.22+: 0 1 2     before 1.22: 3 3 3
```

Before Go 1.22 the loop had **one** `i` shared by all closures/goroutines. Since 1.22, each iteration gets a new variable (only when `go.mod` says `go 1.22` or later). Old fix: `i := i` inside the loop.

### 3. Nil interface holding a nil pointer

```go
type MyErr struct{}

func (*MyErr) Error() string { return "boom" }

func find() error {
    var p *MyErr = nil
    return p // interface = (type=*MyErr, value=nil)
}

func main() {
    err := find()
    fmt.Println(err == nil, err) // false boom
}
```

An interface is `nil` only if **both type and value** are nil. Fix: `return nil` explicitly.

**4. defer order and argument evaluation**

```go
func loop() {
    for i := 0; i < 3; i++ {
        defer fmt.Print(i, " ")
    }
} // prints 2 1 0 (LIFO)

func args() {
    x := 1
    defer fmt.Println("deferred:", x) // x evaluated NOW → "deferred: 1"
    x = 2
}

func named() (n int) {
    defer func() { n++ }() // can change named results
    return 1
} // returns 2
```

Deferred calls run **LIFO** at function return; their **arguments are evaluated immediately**. A closure (`defer func(){...}()`) sees the variable's final value. `defer` in a loop runs only when the **function** returns, not each iteration (file-handle leak).

**5. Concurrent map writes**

```go
m := map[int]int{}
for i := 0; i < 100; i++ {
    go func() { m[i] = i }() // fatal error: concurrent map writes
}
```

Not a panic: a **fatal error**, `recover` can't catch it. Fix: `sync.Mutex` / `RWMutex` around the map, or `sync.Map` (good for mostly-read or disjoint-key workloads). Detect with `go test -race`.

**6. Map iteration order** is **randomized** on purpose. Never rely on it; sort the keys (`slices.Sorted(maps.Keys(m))`).

**7. Goroutine leak on a blocked channel**

```go
func first(urls []string) string {
    ch := make(chan string) // unbuffered
    for _, u := range urls {
        go func() { ch ← fetch(u) }()
    }
    return ←ch // the other goroutines block on send forever → leak
}
```

Fix: `make(chan string, len(urls))` so every send succeeds, or pass a `ctx` and `select` on `ctx.Done()` when sending. A leaked goroutine is never garbage-collected.

✓ KEY POINTS

- "Don't communicate by sharing memory; share memory by communicating": channels for hand-off, mutex for simple shared state.
- Every goroutine you start needs a clear way to **stop** (close, ctx cancel).
- Always `defer cancel()` after `context.WithTimeout/WithCancel`.
- Run tests with `-race` for any concurrent code.

❝ INTERVIEW LINE

*"Goroutines are cheap user-space threads multiplexed onto OS threads by the GMP scheduler, so they run in parallel. I coordinate them with channels or sync primitives, propagate cancellation with context, and use errgroup when I need the first error and cancellation of the rest."*

⚠ COMMON TRAP

returning a typed nil pointer as an `error` (it's `≠ nil`), and starting goroutines that block forever on an unbuffered channel nobody reads. Both compile fine and only fail at runtime.

↑ Back to top ☰ Contents

# 08
# Django

6 pages. Each card says what the page covers and its key lines; open the page for the full depth.

### Django Interview Questions · p. 152

quick-fire Django questions with short answers, grouped by topic. Each links to the deep page.

- Count queries: lazy QuerySet, cache per object, FK in a loop = N+1, `.filter()` on a prefetched manager = new query.
- `update()` / `bulk_*` skip `save()` and save-signals; `on_commit` for Celery; `F()` for counters.
- `select_related` = JOIN (FK), `prefetch_related` = extra `IN` query (M2M / reverse FK).
- DRF: scope in `get_queryset()`, object permissions don't run on list, cursor pagination for big lists.
- Async ORM = thread wrapper; no `atomic()` in async code.

Read the full page →

### Django ORM in Depth · p. 158

the ORM turns Python method chains into SQL. A `QuerySet` is a *description* of a query that runs only when you need the rows.

- A QuerySet is **lazy**: it hits the DB only when evaluated (iterate, `list()`, `len()`, `bool()`, step-slice, `repr`). Its results are cached on that object.
- `select_related` = SQL **JOIN** (FK / one-to-one, 1 query). `prefetch_related` = **separate query + Python join** (M2M / reverse FK, 1 + 1 per relation).
- N+1 shows up in loops and serializers. Catch it with `assertNumQueries` / debug-toolbar.
- `F()` = do the math **in the DB** (no lost update). `Q()` = OR / NOT / complex filters.
- `update()`, `bulk_create()`, `bulk_update()` skip `save()` and `pre/post_save` signals.
- `get_or_create` is only race-safe with a **unique constraint** in the DB.

Read the full page →

## Request Lifecycle, Middleware & Signals · p. 166

how an HTTP request travels through Django, and the hooks (middleware, signals) you can plug into that path.

- Flow: server (gunicorn/uvicorn) → WSGI/ASGI handler → **middleware (top→down)** → URL resolver → `process_view` → **view** → response → **middleware (bottom→up)**.
- Middleware is an **onion**: order in `MIDDLEWARE` matters (Session before Auth, Security first).
- CBV: `as_view()` creates a new instance per request, `dispatch()` picks the method handler.
- Signals are **synchronous**, run in the **same transaction**, and are **not fired** by `update()` / `bulk_*`.
- Prefer explicit service functions over signals for business logic; use `transaction.on_commit` for side effects.

Read the full page →

## Transactions & Migrations · p. 172

how Django manages DB transactions and row locks, and how schema changes (migrations) are created and deployed safely.

- Default is **autocommit**: every query commits alone. `atomic()` groups them; nested `atomic()` = **savepoint**.
- `select_for_update()` locks rows until the transaction ends; only works **inside** `atomic()`. `skip_locked=True` for job queues.
- Start Celery tasks with `transaction.on_commit(...)`, never directly inside a transaction.
- Migrations: Django diffs models vs the **state built from migration files**, not the DB. `django_migrations` records what ran.
- Zero-downtime = **expand / contract**: add nullable (or `db_default`), backfill, then constrain; rename/drop over several deploys; `AddIndexConcurrently` with `atomic = False`.

Read the full page →

## Django REST Framework · p. 178

DRF is the standard toolkit for JSON APIs on Django: serializers (validation + conversion), views/viewsets, routers, and plug-in policies for auth, permissions, throttling, pagination, filtering and versioning.

- Validation order: each field (`to_internal_value` + field validators) → `validate_<field>` → Meta/class `validators` → `validate(attrs)`.
- `ModelSerializer` does **not** write nested data; override `create()` / `update()`.
- `SerializerMethodField` that queries = **N+1**. Fix in `get_queryset()` with `select_related` / `prefetch_related` / `annotate`.
- Authentication = **who are you** (sets `request.user`). Permission = **are you allowed**. `has_object_permission` runs only via `get_object()`, **not on list**.
- Offset pagination = slow on deep pages + `COUNT(*)`; **cursor** pagination = stable and fast, no page numbers.
- Make POST safe to retry with an **Idempotency-Key** + unique constraint.

Read the full page →

## Django in Production: Performance, Security, Async · p. 185

the things that decide whether a Django app survives real traffic: caching, DB connections, process model, static files, security settings, async limits, background jobs, observability and tests.

- Cache layers: per-view (`cache_page`), template fragment (`{% cache %}`), low-level (`cache.get_or_set`). Hard part = **invalidation** (versioned keys, delete in `on_commit`).
- `CONN_MAX_AGE=0` (default) = new DB connection per request. Use `CONN_MAX_AGE=60` + `CONN_HEALTH_CHECKS`, or pgbouncer / Django 5.1 `"pool"`. Total connections = instances × workers × threads.
- Security: `DEBUG=False`, `ALLOWED_HOSTS`, secrets from env, HTTPS settings, never f-strings in `raw()`, `|safe` only on trusted HTML. Run `check --deploy`.
- Async ORM methods (`aget`, `async for`) still run the sync driver in a thread; `atomic()` **doesn't work in async code** → wrap in `sync_to_async`.
- `TestCase` = rollback per test (fast, `on_commit` never fires); `TransactionTestCase` = real commits (slow, needed for locking / `on_commit`).

Read the full page →

CHAPTER 8 · DJANGO

# Django Interview Questions

Django interview questions · Django quiz · how many queries · what's wrong with this code · ORM questions · DRF questions · middleware questions · transaction questions · Django security questions · Django async questions · Django deployment questions

SEE ALSO Django ORM in Depth · Request Lifecycle, Middleware & Signals · Transactions & Migrations · Django REST Framework · Django in Production: Performance, Security, Async

ON THIS PAGE

1. ORM · p. 152
2. Models and signals · p. 153
3. Request lifecycle and middleware · p. 154
4. Transactions · p. 154
5. Migrations · p. 155
6. DRF · p. 155
7. Performance and deployment · p. 156
8. Security · p. 156
9. Async and testing · p. 156

**What:** quick-fire Django questions with short answers, grouped by topic. Each links to the deep page.

> **i WHY IT MATTERS**
> Iranian backend interviews (Digikala, Snapp, Divar, Cafe Bazaar, fintechs) usually start with fast "do you know X?" questions before going deep. Answer in 1–3 sentences, then offer detail.

## ORM

↑ TOP

**1. When does a QuerySet hit the database?** Only when it's evaluated: iteration, `list()`, `len()`, `bool()`, step-slicing, `repr()`. Then the result is cached on that QuerySet object. `count()`, `exists()`, `first()` run their own queries. more

**2. `select_related` vs `prefetch_related`?** `select_related` follows FK/one-to-one with a SQL JOIN (one query). `prefetch_related` runs a separate query per relation (`WHERE id IN (...)`) and joins in Python; use it for M2M and reverse FK. more

**3. How many queries?**

```python
books = Book.objects.all()          # Book has FK author
for b in books:
    print(b.title, b.author.name)
```

**1 + N** (one for books, one per book for its author). With `.select_related("author")` → **1**.

**4. How many queries?**

```python
authors = Author.objects.prefetch_related("books")
for a in authors:
    print(a.books.filter(published=True).count())
```

**1 + 1 + N.** The prefetch runs (2 queries), but `.filter()` on the related manager ignores the prefetch cache and runs a new query per author. Fix: `Prefetch("books", queryset=Book.objects.filter(published=True), to_attr="pub")` then `len(a.pub)` → **2**, or `annotate(Count(...))` → **1**.

**5. How many queries?**

```python
qs = Order.objects.filter(status="paid")
if qs:
    total = len(qs)
    first = qs[0]
print(qs.count())
```

**1.** `if qs` evaluates and caches all rows; `len()`, `qs[0]` and `count()` use the cache. Correct but wasteful if you only need a check: `exists()` loads no rows.

**6. What are `F()` and `Q()`?** `F("field")` refers to a column inside SQL, so `update(stock=F("stock") - 1)` is atomic in the DB (no lost update). `Q()` builds OR/NOT/dynamic conditions: `Q(a=1) | ~Q(b=2)`. more

**7. `annotate` vs `aggregate`?** `aggregate` returns one dict for the whole queryset; `annotate` adds a computed value to each row (with `GROUP BY`). Two `Count`s over different joins multiply rows → use `distinct=True` or `Subquery`. more

**8. `only()` / `defer()` vs `values()`?** `only` / `defer` still return model instances but skip columns; touching a skipped field = an extra query per object. `values()` / `values_list()` return dicts/tuples, no model overhead.

**9. Is `get_or_create` safe under concurrency?** Only if there's a **unique constraint** on the lookup fields. It does SELECT → INSERT → on `IntegrityError`, SELECT again. Without the constraint, two requests can both insert duplicates. more

**10. How do you process 10 million rows?** `.iterator(chunk_size=...)` (no cache, server-side cursor on Postgres), or loop over PK ranges in batches with `update()` / `bulk_update()` per batch in short transactions. Never `list(Model.objects.all())`. more

## Models and signals

↑ TOP

**11. Does `QuerySet.update()` call `save()` or send `post_save`?** No. `update()`, `bulk_create()` and `bulk_update()` run SQL directly: no `save()`, no `pre/post_save`, no `auto_now` update. (`qs.delete()` *does* send `pre/post_delete`.) more

**12. Why can signals be dangerous?** They hide side effects, run synchronously in the same transaction (before commit), slow down every save, and are skipped by bulk operations. Prefer explicit service functions for business logic. more

**13. What's wrong with this code?**

PYTHON

```python
@receiver(post_save, sender=Order)
def notify(sender, instance, created, **kw):
    if created:
        send_invoice.delay(instance.id)
```

If the order is created inside `atomic()`, the task can start **before commit** → `DoesNotExist` in the worker, or run after a rollback. Use `transaction.on_commit(lambda: send_invoice.delay(instance.id))`.

## Request lifecycle and middleware

↑ TOP

**14. Describe the request lifecycle.** Web server (gunicorn/uvicorn) → WSGI/ASGI handler → middleware top-down → URL resolver → `process_view` → view → response → middleware bottom-up → `request_finished` (closes old DB connections). more

**15. Why does middleware order matter? Give an example.** It's an onion: each layer sees what outer layers already did. `AuthenticationMiddleware` needs `request.session`, so it must come after `SessionMiddleware`; `SecurityMiddleware` goes first to redirect to HTTPS early.

**16. WSGI vs ASGI?** WSGI is synchronous, one request per worker thread. ASGI is async and supports websockets and long-lived connections. Under ASGI, sync views run in a thread pool, so it only helps if your code is really async. more

## Transactions

↑ TOP

**17. What does nested `atomic()` do?** The outer block is a transaction; each inner block is a **savepoint**. An exception leaving the inner block rolls back only to that savepoint. Catch it *outside* the inner `atomic`, never inside. more

**18. When do you use `select_for_update`? What are `skip_locked` and `nowait`?** To lock rows you read and then change based on logic (balances, seats). Must be inside `atomic()`. `skip_locked` skips locked rows (job queues), `nowait` fails immediately instead of waiting. more

**19. What's wrong with this code?**

```python
def withdraw(account_id, amount):
    acc = Account.objects.get(pk=account_id)
    if acc.balance >= amount:
        acc.balance -= amount
        acc.save()
```

Race condition: two requests read the same balance and both withdraw (lost update, negative balance). Fix: `Account.objects.filter(pk=account_id, balance__gte=amount).update(balance=F("balance") - amount)` and check the returned row count, or `select_for_update()` inside `atomic()`.

**20. What is `ATOMIC_REQUESTS` and why do many teams avoid it?** It wraps every view in a transaction. Simple, but transactions last the whole request (including slow API calls), holding locks and connections longer. Explicit `atomic()` around the critical part is better.

## Migrations

↑ TOP

**21. How does `makemigrations` know what changed?** It replays all existing migration files to build an in-memory model state and diffs it against your current models. It never reads the real DB. `migrate` checks `django_migrations` for what's applied. more

**22. How do you add a NOT NULL column to a big table with zero downtime?** Add it nullable (or with `db_default` in Django 5.0+), deploy code that writes it, backfill in batches, then add NOT NULL (on Postgres via a `NOT VALID` check constraint + validate). Indexes with `AddIndexConcurrently` and `atomic = False`. more

**23. Why not `RenameField` in production?** During a rolling deploy, old code still uses the old column name and crashes. Use expand/contract: new column, dual-write, backfill, switch reads, drop the old one in a later deploy.

## DRF

↑ TOP

**24. What's the serializer validation order?** Each field's parsing and validators → `validate_<field>()` → serializer-level validators (e.g. unique together) → `validate(attrs)`. more

**25. What's wrong with this serializer?**

```python
class PostSerializer(serializers.ModelSerializer):
    comments_count = serializers.SerializerMethodField()
    def get_comments_count(self, obj):
        return obj.comments.count()
```

One `COUNT` query per post (N+1). Annotate in the view: `Post.objects.annotate(comments_count=Count("comments"))` and use `IntegerField(read_only=True)`.

**26. Authentication vs permission? 401 vs 403?** Authentication identifies the user (sets `request.user`); permissions decide access. 401 = not authenticated (with a `WWW-Authenticate` header, e.g. token/JWT), 403 = authenticated but not allowed (or session auth). more

**27. Why isn't `has_object_permission` enough to protect a list endpoint?** DRF calls it only from `get_object()` (detail routes). `list()` never calls it, so filter the queryset by user in `get_queryset()`.

**28. Offset vs cursor pagination?** Offset needs `COUNT(*)` and scans skipped rows, so deep pages are slow and inserts cause duplicates/skips. Cursor uses `WHERE key < cursor` on an indexed, unique ordering: fast and stable, but no total or jump-to-page. more

**29. How do you make a "create payment" POST safe to retry?** Client sends an `Idempotency-Key`; store it with a unique constraint and the first response; on a repeat, return the stored response instead of creating again. more

## Performance and deployment

↑ TOP

**30. Your endpoint is slow. What do you check first?** Query count and time (debug-toolbar / Sentry / `assertNumQueries`) → N+1 and missing indexes (`EXPLAIN ANALYZE`) → payload size and serializer cost → caching → external calls that should be async (Celery). more

**31. What do `CONN_MAX_AGE` and pgbouncer solve?** `CONN_MAX_AGE=0` opens a new DB connection per request. A positive value reuses connections per thread (add `CONN_HEALTH_CHECKS`). Pgbouncer (transaction mode) or Django 5.1's `"pool"` option limits real Postgres connections when you have many workers; with pgbouncer set `DISABLE_SERVER_SIDE_CURSORS`. more

## Security

↑ TOP

**32. How does Django prevent CSRF, XSS and SQL injection?** CSRF: middleware checks a secret token on unsafe methods for cookie-based auth. XSS: templates auto-escape (broken by `mark_safe` / `safe`). SQL injection: ORM and `params` are parameterized (broken by f-strings in `raw()` / `extra()` / `cursor.execute()`). more

**33. Production settings checklist?** `DEBUG=False`, real `ALLOWED_HOSTS`, `SECRET_KEY` from env, `SECURE_SSL_REDIRECT`, secure cookies, HSTS, `SECURE_PROXY_SSL_HEADER` behind a proxy, and `manage.py check --deploy` in CI.

## Async and testing

↑ TOP

**34. Can you use the ORM in an async view? Transactions?** Yes via the async API (`aget`, `acreate`, `async for`), but it still runs the sync driver in a thread. `atomic()` doesn't work in async code: put that logic in a sync function and call it with `sync_to_async`. Calling the sync ORM directly raises `SynchronousOnlyOperation`. more

**35.** `TestCase` **vs** `TransactionTestCase`**?** `TestCase` wraps each test in a transaction and rolls back (fast), so `on_commit` callbacks never run (use `captureOnCommitCallbacks`). `TransactionTestCase` really commits and truncates tables (slow); use it for locking, `on_commit` and multi-connection tests.

✓ KEY POINTS

- Lead with the one-line answer, then the "why", then offer a code example.
- For "how many queries" questions: find every place a lazy relation is touched in a loop.
- For "what's wrong" questions: look for N+1, read-modify-write races, side effects before commit, and missing user scoping.

❝ INTERVIEW LINE

*"Most Django problems I've fixed in production were N+1 queries, races from read-modify-write, and Celery tasks started before commit, so I check for exactly those three in every code review."*

⚠ COMMON TRAP

answering only with API names ("use `select_related`"). Interviewers want the mechanism: what SQL runs, how many queries, and what happens under concurrency.

↑ Back to top ☰ Contents

CHAPTER 8 · DJANGO

# Django ORM in Depth

QuerySet lazy evaluation queryset cache select_related prefetch_related Prefetch to_attr N+1 assertNumQueries django-debug-toolbar F expression Q object annotate aggregate Subquery OuterRef Exists values values_list only defer bulk_create bulk_update update vs save iterator chunk_size get_or_create update_or_create Meta indexes UniqueConstraint raw SQL RawSQL connection.cursor

SEE ALSO Transactions & Migrations · Django REST Framework · Django in Production: Performance, Security, Async · Indexes · Locking & Concurrency Control · Django Interview Questions · Request Lifecycle, Middleware & Signals

ON THIS PAGE

1. Laziness and evaluation · p. 158
2. select_related vs prefetch_related · p. 159
3. Detecting N+1 · p. 160
4. F() and Q() · p. 160
5. annotate, aggregate, Subquery, Exists · p. 161
6. Loading less: values, values_list, only, defer · p. 162
7. Bulk writes and update() vs save() · p. 162
8. Big tables: iterator() · p. 163
9. get_or_create / update_or_create races · p. 163
10. Indexes and constraints in Meta · p. 164
11. Raw SQL when the ORM is not enough · p. 164

**What:** the ORM turns Python method chains into SQL. A `QuerySet` is a *description* of a query that runs only when you need the rows.

> **i WHY IT MATTERS**
> most Django performance bugs are ORM bugs: N+1 queries, loading whole rows you don't need, read-modify-write races. Interviewers love "how many queries does this run?".

## Laziness and evaluation

↑ TOP

PYTHON

```python
qs = Book.objects.filter(published=True)   # 0 queries: nothing runs
qs = qs.exclude(price=0).order_by("-id")   # 0 queries: still building SQL
for b in qs: ...                           # 1 query: evaluated, results cached in qs
for b in qs: ...                           # 0 queries: uses qs's cache
```

**What evaluates a QuerySet** (runs the query and fills the cache): iteration, `list(qs)`, `len(qs)`, `bool(qs)` / `if qs:`, slicing with a step (`qs[::2]`), `repr(qs)` (shell), pickling.

**What does NOT fill the cache** (runs its own query every time):

| Call | SQL | Note |
|---|---|---|
| `qs.count()` | `SELECT COUNT(*)` | uses cache if `qs` already evaluated |
| `qs.exists()` | `SELECT 1 ... LIMIT 1` | cheapest "is there any?" |

| Call | SQL | Note |
|---|---|---|
| `qs[0]`, `qs.first()` | `LIMIT 1` | on an unevaluated qs, a new query every call |
| `qs[10:20]` | `LIMIT 10 OFFSET 10` | returns a new lazy QuerySet |
| `qs.get(...)` | `WHERE ...` | raises `DoesNotExist` / `MultipleObjectsReturned` |

PYTHON

```python
# How many queries?
qs = Book.objects.all()
if qs:                  # 1: loads ALL rows into memory just to check
    print(qs[0])        # 0: qs is already evaluated, uses cache
# Better: if qs.exists() when you don't need the rows.

print(Book.objects.all()[0])  # 1
print(Book.objects.all()[0])  # 1 again: a new QuerySet every time → 2 total
```

Calling `.filter()` / `.all()` on a QuerySet returns a **new** QuerySet without the cache.

## select_related vs prefetch_related

↑ TOP

| | `select_related` | `prefetch_related` |
|---|---|---|
| Relation | ForeignKey, OneToOne (forward; reverse one-to-one too) | ManyToMany, reverse FK (`author.book_set`), also FK |
| SQL | one query with `JOIN` | one extra query per relation: `WHERE id IN (...)` |
| Join done in | the database | Python |
| Risk | wide rows, big joins | big `IN` lists, memory |

PYTHON

```python
# N+1: 1 query for books + 1 per book for author
for b in Book.objects.all():
    print(b.author.name)

# 1 query (JOIN)
for b in Book.objects.select_related("author", "author__country"):
    print(b.author.name)

# 2 queries: authors, then all their books with WHERE author_id IN (...)
for a in Author.objects.prefetch_related("books"):
    print([b.title for b in a.books.all()])
```

**Prefetch objects** let you filter, order or `select_related` the inner query, and store the result in a list:

```python
from django.db.models import Prefetch

authors = Author.objects.prefetch_related(
    Prefetch(
        "books",
        queryset=Book.objects.filter(published=True).select_related("publisher"),
        to_attr="published_books",          # a plain list attribute
    )
)
for a in authors:                           # 2 queries total
    for b in a.published_books:
        print(b.publisher.name)
```

**Trap:** `a.books.filter(published=True)` inside the loop **ignores** the prefetch cache and runs a new query per author. Only `.all()` uses the cache. Filter in the `Prefetch` instead.

## Detecting N+1

↑ TOP

- **django-debug-toolbar**: SQL panel shows count, duplicates ("similar queries") and time per page.
- **Tests**: lock the query count so a regression fails CI.

```python
def test_book_list_queries(self):
    with self.assertNumQueries(2):
        self.client.get("/api/books/")
# pytest-django: django_assert_num_queries(2) fixture
```

- **Production**: Sentry performance ("N+1 query" detection), `nplusone`, or `django-silk` in staging. `connection.queries` works only with `DEBUG=True`.

## F() and Q()

↑ TOP

`F("field")` refers to a column value **inside SQL**, so the DB does the math atomically.

```python
# ❌ read-modify-write: two requests both read 10 and both write 9 (lost update)
p = Product.objects.get(pk=1)
p.stock -= 1
p.save()

# ✅ one atomic UPDATE: SET stock = stock - 1
Product.objects.filter(pk=1, stock__gt=0).update(stock=F("stock") - 1)  # returns rows changed

# F also compares columns
Order.objects.filter(paid_amount__lt=F("total"))
```

`Q()` builds `OR`, `NOT` and dynamic filters:

```python
from django.db.models import Q
Book.objects.filter(Q(title__icontains=s) | Q(author__name__icontains=s), ~Q(status="draft"))
```

## annotate, aggregate, Subquery, Exists

↑ TOP

- `aggregate()` → **one dict** for the whole queryset (`{"avg": 12.5}`). It ends the chain.
- `annotate()` → adds a computed **column per row** (`GROUP BY` when used with aggregates).

```python
from django.db.models import Count, Avg, Sum, Exists, OuterRef, Subquery

Book.objects.aggregate(avg=Avg("price"))                    # {'avg': Decimal('12.50')}
Author.objects.annotate(n=Count("books")).filter(n__gt=5)   # HAVING COUNT(...) > 5
```

**Trap: two `Count`s on different relations multiply rows** (JOIN fan-out):

```python
# ❌ wrong numbers: books × reviews
Author.objects.annotate(b=Count("books"), r=Count("reviews"))
# ✅ distinct=True (ok for counts) or Subquery (always correct, also for Sum)
Author.objects.annotate(b=Count("books", distinct=True), r=Count("reviews", distinct=True))
```

**Subquery + OuterRef** = a correlated subquery, e.g. "latest order date per customer":

```python
latest = Order.objects.filter(customer=OuterRef("pk")).order_by("-created")
Customer.objects.annotate(
    last_order_at=Subquery(latest.values("created")[:1]),
    has_open=Exists(Order.objects.filter(customer=OuterRef("pk"), status="open")),
).filter(has_open=True)
```

`Exists` is usually faster than `Count(...) > 0` or `__in` with a big list, because the DB can stop at the first match. In Django 3.0+ you can pass `Exists(...)` straight into `.filter()`.

## Loading less: values, values_list, only, defer

↑ TOP

| Call | Returns | Use |
|---|---|---|
| `values("id", "title")` | dicts | reports, JSON, grouping (`values(...).annotate(...)` = `GROUP BY`) |
| `values_list("id", flat=True)` | tuples / flat values | lists of IDs |
| `only("id", "title")` | model instances, other fields deferred | need methods but few columns |
| `defer("body")` | model instances without `body` | skip big text/JSON columns |

**Trap:** accessing a deferred field later runs **one extra query per object** (a new N+1).

## Bulk writes and update() vs save()

↑ TOP

| Operation | Queries | `save()` called? | `pre/post_save` signals? |
|---|---|---|---|
| `obj.save()` in a loop | N | yes | yes |
| `qs.update(x=...)` | 1 | no | no (also no `auto_now` update) |
| `bulk_create(objs, batch_size=1000)` | ~N/batch | no | no |
| `bulk_update(objs, ["x"], batch_size=1000)` | ~N/batch | no | no |
| `qs.delete()` | 1+ | `Model.delete()` not called | **yes**, `pre/post_delete` are sent (Django loads the objects to send them and handle cascades) |

- `bulk_create` sets PKs on the objects on PostgreSQL (uses `RETURNING`). Options: `ignore_conflicts=True`, or `update_conflicts=True, unique_fields=[...], update_fields=[...]` for upsert.
- `bulk_update` builds a big `CASE WHEN` statement; it is slower than people think on large sets. For one value on many rows, `update()` is best.
- If a field has `auto_now=True`, `update()` won't touch it; set it yourself: `update(status="x", updated_at=timezone.now())`.

↑ TOP

## Big tables: iterator()

```python
for row in Event.objects.filter(day=today).iterator(chunk_size=2000):
    process(row)
```

- Streams rows without filling the queryset cache, so memory stays flat. On PostgreSQL it uses a **server-side cursor**.
- With `prefetch_related`, you must pass `chunk_size` (prefetch runs per chunk).
- Behind **PgBouncer in transaction mode**, server-side cursors break → set `DISABLE_SERVER_SIDE_CURSORS: True` in the DB settings.
- For updates over millions of rows, process by **PK ranges** (`filter(pk__gt=last_id).order_by("pk")[:1000]`) in small transactions.

## get_or_create / update_or_create races

↑ TOP

```python
obj, created = Tag.objects.get_or_create(name="python", defaults={"color": "blue"})
```

How it works: `SELECT` → if missing, `INSERT` inside a savepoint → on `IntegrityError` it `SELECT`s again. So:

- **With** a unique constraint on `name`: two parallel calls are safe; one inserts, the other gets the `IntegrityError`, catches it, and reads the row.
- **Without** a unique constraint: both `SELECT`s miss, both `INSERT` → **duplicates**, and later `get()` raises `MultipleObjectsReturned`.

`update_or_create` locks the row with `select_for_update()` inside a transaction, then updates or creates (same unique-constraint rule for the create path). Django 5.0 added `create_defaults=` for values used only on create.

↑ TOP

## Indexes and constraints in Meta

PYTHON

```python
class Order(models.Model):
    customer = models.ForeignKey(Customer, on_delete=models.CASCADE)  # FK gets an index
automatically
    status = models.CharField(max_length=20)
    created = models.DateTimeField(db_index=True)

    class Meta:
        indexes = [
            models.Index(fields=["customer", "-created"], name="order_cust_created_idx"),
            models.Index(fields=["created"], name="order_open_idx",
                         condition=Q(status="open")),           # partial index (Postgres)
        ]
        constraints = [
            models.UniqueConstraint(fields=["customer", "external_id"], name="uniq_ext_id")
,
            models.CheckConstraint(condition=Q(total__gte=0), name="total_non_negative"),
        ]
```

Composite index column order matters (leftmost prefix). See indexes. Note: `CheckConstraint(condition=...)` is the Django 5.1+ name; older versions use `check=`.

## Raw SQL when the ORM is not enough

↑ TOP

Use it for window functions you can't express, CTEs, `DISTINCT ON` edge cases, or tuned reporting queries.

PYTHON

```python
# Model instances from raw SQL: always pass params, never f-strings
Book.objects.raw("SELECT * FROM shop_book WHERE price > %s", [min_price])

from django.db import connection
with connection.cursor() as cur:
    cur.execute("SELECT status, count(*) FROM shop_order WHERE created > %s GROUP BY 1", [s
ince])
    rows = cur.fetchall()
```

Also available: `RawSQL("...", params)` inside `annotate`, and `Func` / `Window` expressions (Django has `Window`, `RowNumber`, `Rank`, `Lag`). Check generated SQL with `print(qs.query)` or `qs.explain(analyze=True)`.

✓ KEY POINTS

- QuerySets are lazy and cached per object; `exists()` / `count()` for checks, not `if qs`.
- FK → `select_related`; M2M / reverse FK → `prefetch_related` (+ `Prefetch` to filter). Filtering a prefetched manager breaks the cache.
- Use `F()` for counters and balances; it's an atomic `UPDATE`, not read-modify-write.
- Bulk operations and `update()` skip `save()` and save-signals; `qs.delete()` still sends delete signals.
- Multiple `Count`s across joins give wrong numbers; use `distinct=True` or `Subquery`.
- Race safety comes from **DB constraints**, not from Python checks.

❝ INTERVIEW LINE

*"QuerySets are lazy; they hit the DB only on evaluation and cache the result. To avoid N+1 I use* `select_related` *for foreign keys, which is a JOIN, and* `prefetch_related` *for many-to-many or reverse relations, which is one extra* `IN` *query, and I lock the query count with* `assertNumQueries`*. For counters I use* `F()` *so the update is atomic in the database."*

⚠ COMMON TRAP

`obj.counter = F("counter") + 1; obj.save()` is atomic, but `obj.counter` now holds an expression, not a number. Saving the object again applies `+1` **again**. Call `obj.refresh_from_db()` after it.

↑ Back to top ☰ Contents

CHAPTER 8 · DJANGO

# Request Lifecycle, Middleware & Signals

WSGI ASGI gunicorn uvicorn request lifecycle middleware MIDDLEWARE order process_view process_exception process_template_response get_response URL resolver urlconf path converters function-based views class-based views as_view dispatch signals post_save pre_save m2m_changed request_finished AppConfig.ready dispatch_uid send_robust service layer

SEE ALSO Django ORM in Depth · Transactions & Migrations · Django in Production: Performance, Security, Async · Django REST Framework · Web & Networking Fundamentals · Django Interview Questions

ON THIS PAGE

1. WSGI vs ASGI · p. 166
2. The request path · p. 167
3. Writing middleware · p. 168
4. Function-based vs class-based views · p. 169
5. Signals · p. 169

**What:** how an HTTP request travels through Django, and the hooks (middleware, signals) you can plug into that path.

> **i WHY IT MATTERS**
> questions like "where do you put auth / logging / request-id?", "why didn't my signal fire?" and "WSGI or ASGI?" test whether you understand the framework or just use it.

## WSGI vs ASGI

↑ TOP

| | WSGI | ASGI |
|---|---|---|
| Model | sync, one request per worker thread | async, event loop; many connections per worker |
| Protocols | HTTP only | HTTP, WebSocket, long-polling, HTTP/2 via server |
| Servers | gunicorn, uWSGI | uvicorn, daphne, hypercorn (or gunicorn + uvicorn worker) |
| Entry | `wsgi.py` → `application` | `asgi.py` → `application` |
| Async views | work, but run in their own event loop per request: no benefit | real benefit for I/O-heavy views |

Under ASGI, **sync views and sync middleware run in a thread pool** (via `sync_to_async`). Each sync/async switch costs a bit, so a mostly-sync app gets nothing from ASGI. See async-await and concurrency-models.

↑ TOP

## The request path




1. The server calls the handler; Django sends `request_started`.
2. Middleware `__call__` "before" code runs **top → down**.
3. The URL resolver matches `urlpatterns` in order (first match wins; path converters like `<int:pk>`). No match → 404.
4. `process_view(request, view_func, args, kwargs)` hooks run (CSRF check lives here).
5. The view runs and returns an `HttpResponse`. If it raises, `process_exception` hooks run bottom → up.
6. If the response is a `TemplateResponse`, `process_template_response` runs, then it is rendered.
7. Middleware "after" code runs **bottom → up**.

↑ TOP

## Writing middleware

PYTHON

```python
import uuid, time, logging
logger = logging.getLogger(__name__)

class RequestIdMiddleware:
    def __init__(self, get_response):          # once, at startup
        self.get_response = get_response

    def __call__(self, request):               # every request
        request.id = request.headers.get("X-Request-ID") or uuid.uuid4().hex
        start = time.monotonic()
        response = self.get_response(request)  # calls the next layer (or the view)
        response["X-Request-ID"] = request.id
        logger.info("req %s %s %s %.0fms", request.id, request.path,
                    response.status_code, (time.monotonic() - start) * 1000)
        return response
```

- Return a response early (without calling `get_response`) to short-circuit, e.g. maintenance mode or IP block. Inner layers and the view don't run.
- Middleware runs for **every** request: keep it cheap, no DB queries if you can avoid them.
- `request.user` is a **lazy** object set by `AuthenticationMiddleware`; it hits the DB only when touched.
- DRF authenticates in the **view**, not in middleware. So in a custom middleware `request.user` is `AnonymousUser` for JWT/token requests.

**Order matters** (default order):

PYTHON

```python
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",          # HTTPS redirect, HSTS: first
    "django.contrib.sessions.middleware.SessionMiddleware",   # must come before auth
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",# needs the session
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

`GZipMiddleware` goes near the top (it must see the final response body). `CorsMiddleware` (django-cors-headers) goes as high as possible, before `CommonMiddleware`.

## Function-based vs class-based views

↑ TOP

PYTHON

```python
# FBV: explicit, easy to read
@require_http_methods(["GET", "POST"])
def order_list(request): ...

# CBV
class OrderDetail(LoginRequiredMixin, DetailView):
    model = Order
    def get_queryset(self):
        return super().get_queryset().filter(customer=self.request.user)

path("orders/<int:pk>/", OrderDetail.as_view())
```

- `as_view()` returns a function. Each request creates a **new instance**, so storing state on `self` is safe per request.
- `dispatch()` maps the HTTP method to `get()` / `post()` /...; unknown method → 405.
- Mixin order follows the MRO: put `LoginRequiredMixin` **left** of the base view.
- Trade-off: CBVs/generic views remove boilerplate but hide control flow; FBVs are clearer for unusual logic. DRF's `APIView` /viewsets are CBVs.

## Signals

↑ TOP

PYTHON

```python
# apps.py
class OrdersConfig(AppConfig):
    name = "orders"
    def ready(self):
        from . import signals      # import here so receivers get registered once

# signals.py
@receiver(post_save, sender=Order, dispatch_uid="order_created_notify")
def on_order_saved(sender, instance, created, **kwargs):
    if created:
        transaction.on_commit(lambda: send_receipt.delay(instance.pk))
```

Common built-ins: `pre_save` / `post_save`, `pre_delete` / `post_delete`, `m2m_changed`, `request_started` / `request_finished`, `user_logged_in`.

**How they really run:** `Signal.send()` calls every receiver **synchronously**, in the same thread, inside the **same DB transaction** as the caller. A receiver exception propagates to the caller (use `send_robust()` to collect errors instead). Django 5.0 added `asend()` / `asend_robust()` for async receivers.

**Pitfalls:**

| Pitfall | Detail |
|---|---|
| Hidden coupling | Reading `order.save()` gives no hint that an email, a cache purge and a webhook happen. Hard to debug and to test. |
| Not fired on bulk ops | `qs.update()`, `bulk_create()`, `bulk_update()` skip `pre/post_save`. Raw SQL skips everything. |
| Transaction timing | `post_save` runs **before commit**. A Celery task started there may run before the row is visible, or after a rollback for a row that never exists. Fix: `transaction.on_commit`. |
| Slow saves | Every receiver adds latency to every save, including in admin and scripts. |
| Double registration | Importing the module twice or bad `ready()` → receiver runs twice. Use `dispatch_uid`. |
| Ordering | Receiver order = registration order; don't depend on it. |

**Signals vs explicit service calls:**

PYTHON

```python
# services.py: business logic is visible and testable
def place_order(user, cart) -> Order:
    with transaction.atomic():
        order = Order.objects.create(user=user, total=cart.total)
        OrderItem.objects.bulk_create([...])
        transaction.on_commit(lambda: send_receipt.delay(order.pk))
    return order
```

Use signals for **decoupling across apps you don't own** (e.g. reacting to `user_logged_in`, or a reusable app that shouldn't import yours), and for cross-cutting things like cache invalidation or audit logs. Use explicit service functions for **business workflows** in your own code.

✓ KEY POINTS

- Middleware = onion around the view; order is part of correctness.
- URL resolution happens **after** request middleware, so middleware can change `request.urlconf`.
- `request_finished` is where Django closes old DB connections.
- Signals are sync, same-transaction, and skipped by bulk operations.
- Side effects from signals or services → `transaction.on_commit`.

❝ INTERVIEW LINE

*"A request goes from gunicorn or uvicorn into Django's handler, through the middleware stack top to bottom, then the URL resolver picks the view, and the response goes back through middleware in reverse. I avoid signals for business logic because they're synchronous, hidden, run before commit and are skipped by bulk updates; I prefer an explicit service function with `transaction.on_commit` for side effects."*

⚠ COMMON TRAP

thinking a `post_save` signal runs "after the data is saved" in the sense of committed. Inside `atomic()` it runs before commit, so a Celery task triggered there can read stale data or get `DoesNotExist`.

↑ Back to top ☰ Contents

CHAPTER 8 · DJANGO

# Transactions & Migrations

autocommit ATOMIC_REQUESTS transaction.atomic savepoint nested atomic TransactionManagementError select_for_update skip_locked nowait of no_key transaction.on_commit delay_on_commit captureOnCommitCallbacks F expression race makemigrations migrate django_migrations migration dependencies RunPython apps.get_model data migration squashmigrations zero-downtime migration expand and contract db_default SeparateDatabaseAndState AddIndexConcurrently atomic = False lock_timeout AddConstraintNotValid

SEE ALSO Django ORM in Depth · Request Lifecycle, Middleware & Signals · Django in Production: Performance, Security, Async · Transactions & Isolation Levels · Locking & Concurrency Control · Celery with Redis as Broker · Django Interview Questions

ON THIS PAGE

1. Transaction basics · p. 172
2. select_for_update · p. 173
3. transaction.on_commit · p. 174
4. Migrations: how they work · p. 174
5. Data migrations · p. 175
6. Squashing · p. 175
7. Zero-downtime migration rules · p. 176

**What:** how Django manages DB transactions and row locks, and how schema changes (migrations) are created and deployed safely.

> **i WHY IT MATTERS**
> money, stock and counters break under concurrency; a bad migration can lock a production table for minutes. Senior interviews always touch one of these.

## Transaction basics

↑ TOP

- **Autocommit (default):** each query is its own transaction.
- `ATOMIC_REQUESTS = True` (per database): wraps **each view** in one transaction; commits if the view returns, rolls back if it raises. Middleware and streaming response generators run **outside** it. Cost: transactions as long as the request, including slow external API calls → locks held longer. Opt out with `@transaction.non_atomic_requests`. Most teams prefer explicit `atomic()` where needed.
- `transaction.atomic()`: decorator or context manager.

```python
from django.db import transaction, IntegrityError

with transaction.atomic():                  # BEGIN
    order = Order.objects.create(...)
    try:
        with transaction.atomic():          # SAVEPOINT
            Coupon.objects.create(code=code, order=order)   # may raise IntegrityError
    except IntegrityError:                  # ROLLBACK TO SAVEPOINT: outer tx still OK
        order.note = "coupon already used"
    order.save()
                                            # COMMIT
```

**Rule: catch the exception *outside* an `atomic` block.** If you catch a DB error inside the same block and keep querying, PostgreSQL's transaction is broken and Django raises `TransactionManagementError: An error occurred in the current transaction...`. The inner `atomic()` is what makes recovery possible.

- Savepoints cost a round trip each; `atomic(savepoint=False)` skips them when you don't need partial rollback.
- Isolation level is the DB default (Postgres: READ COMMITTED). You can change it in `OPTIONS`. See transactions-isolation.

## select_for_update

↑ TOP

```python
with transaction.atomic():
    acc = Account.objects.select_for_update().get(pk=acc_id)   # SELECT ... FOR UPDATE
    if acc.balance < amount:
        raise InsufficientFunds
    acc.balance -= amount
    acc.save(update_fields=["balance"])
# lock released at COMMIT
```

- Must be inside `atomic()`; evaluating it in autocommit raises `TransactionManagementError`. Note it's lazy: the lock is taken when the queryset is evaluated.
- `nowait=True` → error immediately if locked (`DatabaseError`), instead of waiting.
- `skip_locked=True` → skip locked rows. Perfect for **DB-backed job queues**: many workers each take different rows.
- `of=("self",)` → with `select_related`, lock only the main table's rows.
- `no_key=True` (Postgres `FOR NO KEY UPDATE`) → weaker lock that doesn't block inserts of rows referencing this one by FK.
- Lock rows in a **consistent order** (e.g. by PK) to avoid deadlocks when locking several.

```python
# job queue: each worker takes up to 10 unclaimed jobs
with transaction.atomic():
    jobs = list(Job.objects.select_for_update(skip_locked=True)
                .filter(status="pending").order_by("id")[:10])
    Job.objects.filter(id__in=[j.id for j in jobs]).update(status="running")
```

**F() vs select_for_update:** for a single counter, a conditional `UPDATE` with `F()` is simpler and faster (no lock wait):

```python
updated = Product.objects.filter(pk=pid, stock__gte=qty).update(stock=F("stock") - qty)
if not updated:
    raise OutOfStock          # 0 rows changed = not enough stock, no race
```

Use `select_for_update` when the decision needs several reads or business logic in Python. See locking-concurrency.

## transaction.on_commit

↑ TOP

```python
with transaction.atomic():
    order = Order.objects.create(...)
    send_receipt.delay(order.pk)          # ❌ worker may run before COMMIT → DoesNotExist
                                          #    or after ROLLBACK → email for a ghost order
    transaction.on_commit(lambda: send_receipt.delay(order.pk))   # ✅ runs after COMMIT
```

- If the transaction rolls back, the callback is **dropped**. Outside a transaction, it runs immediately.
- Callbacks run in order; `robust=True` (Django 4.2+) keeps one failing callback from stopping the others.
- Celery 5.4+ has `task.delay_on_commit(...)` as a shortcut.
- In `TestCase` nothing really commits, so callbacks never run. Use `with self.captureOnCommitCallbacks(execute=True):`.
- `on_commit` gives "after commit" but not "exactly once": if the process dies between COMMIT and `delay()`, the task is lost. For strong guarantees use a **transactional outbox** table. See celery-redis-broker.

## Migrations: how they work

↑ TOP

- `makemigrations` loads all migration files, **replays them in memory** to build the "project state", diffs it against your current models, and writes new operations (`AddField`, `AlterField`...). It never looks at the real DB.
- `migrate` builds a dependency **graph** (`dependencies = [("orders", "0007_..."), ("users", "0003_...")]`), checks the `django_migrations` table for applied ones, and runs the rest in

order. On PostgreSQL each migration runs in **one transaction** by default (DDL is transactional there; not on MySQL).

- Useful commands: `showmigrations`, `migrate --plan`, `sqlmigrate app 0008` (see the real SQL), `makemigrations --check --dry-run` (CI: fail if models changed without migrations), `migrate app 0007` (roll back), `--fake` (mark as applied without running).
- Two branches creating `0008_*` in the same app → conflict; fix with `makemigrations --merge` or by renumbering before merge.

## Data migrations

↑ TOP

```python
from django.db import migrations

def forwards(apps, schema_editor):
    User = apps.get_model("users", "User")        # historical model, NOT `from users.models import User`
    for u in User.objects.filter(full_name="").iterator(chunk_size=2000):
        u.full_name = f"{u.first_name} {u.last_name}".strip()
        u.save(update_fields=["full_name"])

class Migration(migrations.Migration):
    dependencies = [("users", "0012_user_full_name")]
    operations = [migrations.RunPython(forwards, migrations.RunPython.noop)]
```

- Use `apps.get_model`: it gives the model **as it was at that migration**. Importing the real model breaks when the model changes later.
- Historical models have **no custom methods** and no custom `save()`; custom managers only if `use_in_migrations = True`. Signals still fire for them, but receivers expecting the real class may not match the historical one.
- Always give a reverse (`RunPython.noop` if nothing to undo), or the migration can't be rolled back.
- Large backfills inside a migration = one long transaction + long deploy. For big tables, run the backfill as a separate batched job/management command (or set `atomic = False` and batch).

## Squashing

↑ TOP

`squashmigrations app 0001 0040` creates one migration with `replaces = [...]`. New DBs run the squashed one; existing DBs that already applied the old ones keep working. After **every** environment has migrated past it, delete the old files and remove `replaces`. `RunPython` operations without `elidable=True` are kept, so squashes can stay big.

↑ TOP

## Zero-downtime migration rules

During a rolling deploy, **old code and new code run at the same time against the same schema**. Every migration must work with both.

| Change | Unsafe one-step way | Safe way |
|---|---|---|
| Add NOT NULL column | `AddField(null=False, default=x)`: Django sets the default only to fill existing rows and then **drops it from the DB**, so old code's `INSERT`s (without the column) fail | add `null=True` (or Django 5.0 `db_default=`, which stays in the DB) → deploy code that writes it → backfill in batches → then make NOT NULL |
| Rename column/table | `RenameField` → old code queries the old name → errors | add new column, write both, backfill, switch reads, drop old (several deploys); or `db_column=` to keep the DB name |
| Drop column | drop in the same deploy as the code change → old code still `SELECT`s it | deploy 1: remove field from code (make it nullable first if NOT NULL); deploy 2: drop column. `SeparateDatabaseAndState` helps split state vs DB change |
| Add index on big table | `AddIndex` → `CREATE INDEX` locks writes for the whole build | `AddIndexConcurrently` + `atomic = False` |
| Add NOT NULL / CHECK / FK on big table | full scan under a strong lock | Postgres: add constraint `NOT VALID`, then validate separately (`AddConstraintNotValid` + `ValidateConstraint` in `django.contrib.postgres.operations`) |
| Change column type | table rewrite under `ACCESS EXCLUSIVE` lock | new column + backfill + switch |

PYTHON

```python
from django.contrib.postgres.operations import AddIndexConcurrently

class Migration(migrations.Migration):
    atomic = False      # CREATE INDEX CONCURRENTLY can't run inside a transaction
    dependencies = [("orders", "0020_...")]
    operations = [
        AddIndexConcurrently("order", models.Index(fields=["created"], name="order_created_
idx")),
    ]
```

Extra safety:

- Set a short `lock_timeout` (e.g. `SET lock_timeout = '5s'`) for migrations. Even a fast `ALTER TABLE` waits behind a long-running query for its lock, and **every other query queues behind it** → outage. Better to fail and retry.
- Run migrations **before** the new code rolls out (as a release step), so the new code always finds its columns.

- Tools: `django-pg-zero-downtime-migrations`, or a linter like `django-migration-linter` in CI.

✓ KEY POINTS

- Nested `atomic` = savepoints; catch errors outside the inner block.
- `select_for_update` needs a transaction; `skip_locked` for queues; `nowait` to fail fast.
- Single-row counters: conditional `update()` with `F()`.
- Side effects (Celery, email, webhooks) → `on_commit`.
- Migrations come from replayed migration state; data migrations use `apps.get_model`.
- Every migration must be compatible with the **previous** code version.

❝ INTERVIEW LINE

*"I keep transactions short with explicit `atomic()` blocks, lock rows with `select_for_update` when the logic needs it, and trigger Celery tasks with `on_commit` so workers never see uncommitted data. For migrations I follow expand and contract: add nullable columns or use `db_default`, backfill in batches, add constraints later, create indexes concurrently, and never rename or drop in the same deploy as the code change."*

⚠ COMMON TRAP

`AddField(..., default=0)` on a NOT NULL column looks safe, but Django doesn't keep the default in the database. Old code still running during the deploy inserts rows without that column and gets `NOT NULL constraint` errors. Use `db_default` (Django 5.0+) or add it as nullable first.

↑ Back to top ☰ Contents

CHAPTER 8 · DJANGO

# Django REST Framework

DRF · Django REST Framework · Serializer · ModelSerializer · validation order · validate_field · validate · to_internal_value · to_representation · nested serializer · writable nested · SerializerMethodField · N+1 · ViewSet · ModelViewSet · GenericViewSet · router · DefaultRouter · action decorator · authentication · permissions · has_object_permission · 401 vs 403 · throttling · ScopedRateThrottle · pagination · CursorPagination · LimitOffsetPagination · django-filter · SearchFilter · OrderingFilter · versioning · get_queryset · idempotency key

SEE ALSO Django ORM in Depth · Request Lifecycle, Middleware & Signals · Django in Production: Performance, Security, Async · Web & Networking Fundamentals · HTTP/1.1 vs HTTP/2 vs HTTP/3 · Django Interview Questions

ON THIS PAGE

1. Request flow inside an APIView · p. 178
2. Serializers · p. 179
3. Views, viewsets and routers · p. 180
4. Authentication vs permissions · p. 181
5. Throttling · p. 182
6. Pagination · p. 182
7. Filtering and ordering · p. 182
8. Versioning · p. 182
9. Idempotency for POST · p. 183

**What:** DRF is the standard toolkit for JSON APIs on Django: serializers (validation + conversion), views/viewsets, routers, and plug-in policies for auth, permissions, throttling, pagination, filtering and versioning.

> **i WHY IT MATTERS**
> most Django jobs in Iran are API backends behind a mobile/web app. Interviewers check if you can keep DRF endpoints correct **and** fast.

## Request flow inside an APIView

↑ TOP

```
dispatch()
  initialize_request()   → wraps HttpRequest in DRF Request (parsers, authenticators)
  initial()
    perform_authentication()   → request.user (lazy, but forced here)
    check_permissions()        → has_permission() for each permission class
    check_throttles()          → 429 if over the rate
    (version determined here)
  handler: list() / create() / retrieve() ...
    get_object() → check_object_permissions() → has_object_permission()
  finalize_response()    → renderer (JSON), content negotiation
  handle_exception()     → APIException → proper status code
```

↑ TOP

## Serializers

```python
class OrderSerializer(serializers.ModelSerializer):
    customer_name = serializers.CharField(source="customer.name", read_only=True)
    items = OrderItemSerializer(many=True)

    class Meta:
        model = Order
        fields = ["id", "customer_name", "status", "total", "items"]
        read_only_fields = ["status"]

    def validate_total(self, value):                # field-level
        if value <= 0:
            raise serializers.ValidationError("must be positive")
        return value

    def validate(self, attrs):                      # object-level, after all fields
        if attrs["total"] > 10_000 and not self.context["request"].user.is_staff:
            raise serializers.ValidationError("limit exceeded")
        return attrs
```

**Validation order** for `serializer.is_valid()`:

1. For each field: `to_internal_value()` (type parsing) + field `validators=[...]` (e.g. `UniqueValidator`), then `validate_<field_name>()`.
2. Serializer-level `validators` (e.g. `UniqueTogetherValidator`, auto-added from `unique_together` / `UniqueConstraint`).
3. `validate(self, attrs)`.

Errors are collected per field; `is_valid(raise_exception=True)` returns a 400 with them.

**Writing:** `serializer.save()` calls `create()` (no instance) or `update()` (instance given). Pass server-side values: `serializer.save(owner=request.user)`.

**Nested writes:** `ModelSerializer.create()` raises an error if writable nested data is present. You write it yourself:

```python
    def create(self, validated_data):
        items = validated_data.pop("items")
        with transaction.atomic():
            order = Order.objects.create(**validated_data)
            OrderItem.objects.bulk_create([OrderItem(order=order, **i) for i in items])
        return order
```

For updates of nested lists you must decide: replace all, or match by id and diff. Often simpler: a separate endpoint for items.

**`SerializerMethodField` N+1 trap:**

```python
class AuthorSerializer(serializers.ModelSerializer):
    book_count = serializers.SerializerMethodField()
    def get_book_count(self, obj):
        return obj.books.count()          # ❌ 1 COUNT query per author: page of 50 = 51 queries

# ✅ compute it in the queryset and read the attribute
class AuthorViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = AuthorSerializer
    def get_queryset(self):
        return Author.objects.annotate(book_count=Count("books"))
# serializer: book_count = serializers.IntegerField(read_only=True)
```

Same for nested serializers: `OrderSerializer` with `items` and `customer` needs `.select_related("customer").prefetch_related("items__product")`.

Other notes: `to_representation()` customizes output; serializers are **slow** for big payloads (pure-Python per field), so for huge read-only lists use `.values()` or a lighter serializer.

## Views, viewsets and routers

↑ TOP

| Class | Gives you |
|---|---|
| `APIView` | `get/post/...` methods + DRF policies |
| `GenericAPIView` + mixins | `get_queryset`, `get_object`, pagination, filtering |
| `ListCreateAPIView` etc. | ready combinations |
| `ModelViewSet` | list, create, retrieve, update, partial_update, destroy |
| `ReadOnlyModelViewSet` / `GenericViewSet` + mixins | pick only what you expose |

```python
class OrderViewSet(viewsets.ModelViewSet):
    serializer_class = OrderSerializer
    permission_classes = [IsAuthenticated, IsOwner]

    def get_queryset(self):                       # per request, scoped to user, no N+1
        return (Order.objects.filter(customer=self.request.user)
                .select_related("customer").prefetch_related("items__product"))

    @action(detail=True, methods=["post"])
    def cancel(self, request, pk=None):           # POST /orders/{pk}/cancel/
        order = self.get_object()                 # runs object permissions
        order.cancel()
        return Response(status=204)

router = DefaultRouter()                          # DefaultRouter adds an API root view
router.register("orders", OrderViewSet, basename="order")
urlpatterns = router.urls
```

Use `get_queryset()`, not a class-level `queryset = Order.objects.filter(customer=...)`: the class attribute is built once at import and can't see `request.user`. Filtering by user in `get_queryset()` also makes other users' objects return **404** instead of 403 (no data leak).

## Authentication vs permissions

↑ TOP

- **Authentication** classes (`SessionAuthentication`, `TokenAuthentication`, JWT via `djangorestframework-simplejwt`) set `request.user` and `request.auth`. Failing all → `AnonymousUser`, not an error.
- **Permissions** (`IsAuthenticated`, `IsAdminUser`, `DjangoModelPermissions`, custom) decide access.
- **401 vs 403:** not authenticated → 401 if the first auth class sends a `WWW-Authenticate` header (Token/JWT do), otherwise 403 (Session). Authenticated but not allowed → 403.
- `SessionAuthentication` **enforces CSRF** for unsafe methods; token/JWT auth doesn't need CSRF.

```python
class IsOwner(BasePermission):
    def has_permission(self, request, view):          # every request, incl. list
        return request.user.is_authenticated
    def has_object_permission(self, request, view, obj):  # only via get_object()
        return obj.customer_id == request.user.id
```

**Trap:** `has_object_permission` is **not** called for `list()` or for objects you fetch manually. Scope the list in `get_queryset()`.

## Throttling

↑ TOP

`AnonRateThrottle`, `UserRateThrottle`, `ScopedRateThrottle` (`throttle_scope = "login"` + `"login": "5/min"`). State is kept in the **Django cache**, so use a shared cache (Redis), not local memory, with several workers. It's a soft limit (race between read and write), fine for abuse protection, not for billing. Behind a proxy set `NUM_PROXIES` so the client IP is read correctly from `X-Forwarded-For`. Real DoS protection belongs in nginx / the gateway.

## Pagination

↑ TOP

| Style | SQL | Pros | Cons |
|---|---|---|---|
| `PageNumberPagination` (`?page=3`) | `LIMIT/OFFSET` + `COUNT(*)` | page numbers, total | slow deep pages, `COUNT(*)` on big tables, duplicates/skips when rows are inserted |
| `LimitOffsetPagination` | same | flexible | same |
| `CursorPagination` | `WHERE created < :cursor ORDER BY created DESC LIMIT n` | O(1) per page with an index, stable under inserts | no total, no jump to page N; needs a **unique, fixed** ordering (default `-created`) |

Use cursor pagination for feeds, timelines and infinite scroll. Always set a default `PAGE_SIZE` and `max_page_size` so nobody can request 1M rows.

## Filtering and ordering

↑ TOP

```python
filter_backends = [DjangoFilterBackend, SearchFilter, OrderingFilter]
filterset_fields = ["status", "customer"]       # ?status=paid
search_fields = ["number", "customer__email"]   # ?search=... → ILIKE '%x%' (no index! use full-text/trigram for big tables)
ordering_fields = ["created", "total"]          # whitelist; never allow ordering by any field
```

Each filtered/ordered field should match an index.

## Versioning

↑ TOP

`URLPathVersioning` (`/api/v1/...`, most common), `NamespaceVersioning`, `AcceptHeaderVersioning` (`Accept: application/json; version=2`), `QueryParameterVersioning`. Read `request.version` and pick a serializer in `get_serializer_class()`. Keep old versions working until clients (old mobile app builds!) are gone.

↑ TOP

## Idempotency for POST

GET/PUT/DELETE are idempotent by definition; POST is not. Mobile networks retry, users double-click → duplicate orders or payments.

PYTHON

```python
def create(self, request, *args, **kwargs):
    key = request.headers.get("Idempotency-Key")
    if not key:
        return Response({"detail": "Idempotency-Key required"}, status=400)
    try:
        with transaction.atomic():
            rec = IdempotencyRecord.objects.create(user=request.user, key=key)  # unique
(user, key)
            response = super().create(request, *args, **kwargs)
            rec.status_code, rec.body = response.status_code, response.data
            rec.save()
            return response
    except IntegrityError:
        rec = IdempotencyRecord.objects.get(user=request.user, key=key)
        if rec.status_code is None:
            return Response({"detail": "in progress"}, status=409)
        return Response(rec.body, status=rec.status_code)      # replay the first result
```

The unique constraint is what makes it race-safe. Also store a hash of the request body to reject the same key with a different payload, and expire old keys. A natural unique key (e.g. `(user, client_order_id)`) works too.

✓ KEY POINTS

- Validation: fields → `validate_<field>` → validators → `validate()`.
- Nested writes are manual; wrap them in `atomic()`.
- Performance lives in `get_queryset()`: `select_related`, `prefetch_related`, `annotate`; test with `assertNumQueries`.
- Scope data in `get_queryset()`; object permissions don't protect lists.
- Cursor pagination for big or live lists.
- Throttles need a shared cache.

❝ INTERVIEW LINE

*"In DRF I keep views thin: `get_queryset()` scopes data to the user and fixes N+1 with `select_related` / `prefetch_related` or annotations, serializers only validate and shape data, and business logic goes to a service function. For big lists I use cursor pagination, and for create endpoints that clients retry I require an idempotency key backed by a unique constraint."*

⚠ COMMON TRAP

relying on `has_object_permission` for security on a list endpoint. It's never called there, so `GET /orders/` returns everyone's orders unless `get_queryset()` filters by user.

↑ Back to top ☰ Contents

CHAPTER 8 · DJANGO

# Django in Production: Performance, Security, Async

cache_page · template fragment cache · low-level cache API · cache.get_or_set · cache invalidation · cache stampede · RedisCache · django-redis · CONN_MAX_AGE · CONN_HEALTH_CHECKS · connection pooling · pgbouncer · DISABLE_SERVER_SIDE_CURSORS · gunicorn workers · gthread · uvicorn worker · collectstatic · WhiteNoise · ManifestStaticFilesStorage · STORAGES · django-storages · CSRF · XSS · mark_safe · SQL injection · extra · RawSQL · clickjacking · X_FRAME_OPTIONS · SECURE_SSL_REDIRECT · HSTS · SECURE_PROXY_SSL_HEADER · check --deploy · SECRET_KEY · async views · async ORM · sync_to_async · SynchronousOnlyOperation · Celery best practices · Sentry · structured logging · TestCase · TransactionTestCase · setUpTestData · factory_boy · pytest-django

SEE ALSO Django ORM in Depth · Request Lifecycle, Middleware & Signals · Transactions & Migrations · Django REST Framework · Celery with Redis as Broker · Async / Await · Concurrency Models (GIL, Threads, Processes, asyncio) · Django Interview Questions

ON THIS PAGE

1. Caching · p. 185
2. Database connections · p. 186
3. Gunicorn / uvicorn sizing · p. 187
4. Static and media files · p. 187
5. Security · p. 188
6. Async views and the async ORM · p. 189
7. Celery integration best practices · p. 189
8. Logging and monitoring · p. 190
9. Testing · p. 190

**What:** the things that decide whether a Django app survives real traffic: caching, DB connections, process model, static files, security settings, async limits, background jobs, observability and tests.

> **i WHY IT MATTERS**
> senior interviews ask "your endpoint is slow / your DB has too many connections / how do you deploy Django?". These answers show production experience.

## Caching

↑ TOP

PYTHON

```python
CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.redis.RedisCache",
# built in since Django 4.0
        "LOCATION": "redis://redis:6379/1",
        "TIMEOUT": 300,
        "KEY_PREFIX": "shop",
    }
}
```

`django-redis` is the popular third-party backend (extra features: `delete_pattern`, locks, compression). `LocMemCache` is **per process**: useless for shared state with many workers.

| Level | Example | Good for |
|---|---|---|
| Per-view | `@cache_page(60)` (+ `@vary_on_headers("Accept-Language")`) | public pages, same for everyone |
| Template fragment | `{% cache 300 sidebar request.user.id %}...{% endcache %}` | expensive parts of a page |
| Low-level | `cache.get_or_set(key, fn, 300)`, `get_many`, `incr` | query results, computed values, counters |
| HTTP | `Cache-Control`, `ETag` (`@condition`), CDN | static and public API responses |

PYTHON

```python
def product_detail_data(pid):
    key = f"product:v2:{pid}"                    # version in the key → bump to invalidate all
    return cache.get_or_set(key, lambda: build_product(pid), timeout=600)

def update_product(p, **changes):
    with transaction.atomic():
        ...
        transaction.on_commit(lambda: cache.delete(f"product:v2:{p.pk}"))  # after commit, not before
```

**Invalidation strategies:** short TTLs (accept staleness), delete on write (in `on_commit`, or the next read may cache old data), versioned keys / generation counters for groups (`products:gen` incremented on any change). Watch for **stampede** (many requests rebuild an expired hot key at once): use a lock (`cache.add(lock_key, 1, 10)`), random TTL jitter, or early refresh in the background.

**Trap:** `cache_page` on a view that depends on the user → one user's page served to others. Per-view cache is for anonymous/public content.

## Database connections

↑ TOP

- `CONN_MAX_AGE = 0` (default): open and close a connection **every request** (TCP + TLS + Postgres auth ≈ several ms, and a Postgres backend process fork).
- `CONN_MAX_AGE = 60` (or `None` = forever): reuse per thread; closed at `request_finished` when too old. Add `CONN_HEALTH_CHECKS = True` (4.1+) so a dead connection is detected before reuse.
- **Django 5.1+** with psycopg 3: built-in pool via `"OPTIONS": {"pool": True}` (or a dict of pool options). Don't combine with `CONN_MAX_AGE`.
- **Under ASGI**, Django docs say to **disable persistent connections** (`CONN_MAX_AGE = 0`) and use a pool (backend pool or pgbouncer), because async code runs ORM calls in different threads.

- **PgBouncer** in **transaction mode**: many app connections share few Postgres connections. Limits: no session state across transactions (session `SET`, advisory session locks, `LISTEN`), and **server-side cursors break** → `DISABLE_SERVER_SIDE_CURSORS = True`. Prepared statements need care (newer PgBouncer 1.21+ can handle protocol-level ones).

**Sizing math:** max DB connections = instances × gunicorn workers × threads (+ Celery workers × concurrency + cron). Example: 6 pods × 5 workers × 4 threads = 120, plus Celery 40 → 160, above Postgres' default `max_connections = 100` → "too many connections". Fix with a pooler, not by raising `max_connections` forever (each Postgres connection is a process with memory).

## Gunicorn / uvicorn sizing

↑ TOP

| Setup | When | Rule of thumb |
|---|---|---|
| `gunicorn --workers N` (sync) | CPU-light, simple | N ≈ 2 × CPU + 1 |
| `gunicorn --worker-class gthread --threads T` | I/O-heavy sync views (DB, HTTP calls) | fewer workers, 2–8 threads each; the GIL is released during I/O |
| `gunicorn -k uvicorn_worker.UvicornWorker` / `uvicorn --workers N` | ASGI, async views, websockets | ~1 worker per CPU |

- Set `--timeout` (default 30s), `--max-requests 1000 --max-requests-jitter 100` (recycle workers to contain memory leaks), and `--graceful-timeout` for clean deploys.
- In Kubernetes, prefer fewer workers per pod and scale pods; the CPU limit (not the node CPU count) is what matters.
- Slow external API calls inside a sync view block a whole worker/thread: move them to Celery or use timeouts.

## Static and media files

↑ TOP

- **Static** (your CSS/JS): `collectstatic` copies them to `STATIC_ROOT`; serve with nginx, a CDN, or **WhiteNoise** (middleware, simple for containers). Use hashed filenames (`ManifestStaticFilesStorage` / `CompressedManifestStaticFilesStorage`) for long cache headers. Since 4.2, configure with the `STORAGES` setting.
- **Media** (user uploads): never on the container disk in multi-instance setups. Use object storage (S3 / MinIO / ArvanCloud) via `django-storages`, ideally with **pre-signed URLs** so uploads bypass Django. Validate size and type; never serve uploads from the same domain as HTML without care (stored XSS via SVG/HTML).

↑ TOP

## Security

| Threat | Django protection | How it breaks |
|---|---|---|
| CSRF | `CsrfViewMiddleware` + `{% csrf_token %}`; DRF `SessionAuthentication` enforces it | `@csrf_exempt` on cookie-authenticated endpoints; `CSRF_TRUSTED_ORIGINS` too wide |
| XSS | templates **auto-escape** `{{ var }}` | `mark_safe(user_input)`, the `safe` filter on user data, `{% autoescape off %}`, building HTML in JS from API data |
| SQL injection | ORM and `params` are parameterized | f-strings/`%` in `raw()`, `cursor.execute()`, `extra()`, `RawSQL()`; user input as `order_by()` field without whitelist |
| Clickjacking | `XFrameOptionsMiddleware`, `X_FRAME_OPTIONS = "DENY"` (default) | removing the middleware |
| Host header attacks | `ALLOWED_HOSTS` | `ALLOWED_HOSTS = ["*"]` → poisoned password-reset links |
| Info leak | `DEBUG = False` | `DEBUG=True` in prod shows settings, SQL, stack traces |

PYTHON

```python
# ❌ SQL injection
User.objects.raw(f"SELECT * FROM auth_user WHERE username = '{name}'")
# ✅ parameterized
User.objects.raw("SELECT * FROM auth_user WHERE username = %s", [name])
```

**HTTPS settings** (behind a TLS-terminating proxy):

PYTHON

```python
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")   # only if the proxy sets/overwrites it
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31536000          # start small; HSTS is hard to undo
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_CONTENT_TYPE_NOSNIFF = True      # default True
```

**Secrets:** `SECRET_KEY`, DB passwords and API keys come from env vars / a secret manager, never git. `SECRET_KEY` signs sessions, password reset tokens and `signing` data; leaking it lets attackers forge them (`SECRET_KEY_FALLBACKS` allows rotation). Use Argon2 or the default PBKDF2 for passwords. Run `python manage.py check --deploy` in CI. Rate-limit login (throttles / `django-axes`).

## Async views and the async ORM

↑ TOP

PYTHON

```python
async def dashboard(request):
    user = await User.objects.aget(pk=request.user_id)
    orders = [o async for o in Order.objects.filter(user=user)[:20]]
    async with httpx.AsyncClient(timeout=3) as client:
        r1, r2 = await asyncio.gather(client.get(URL1), client.get(URL2))   # real
concurrency here
    return JsonResponse({...})
```

Limits to know (Django 5.x):

- Async ORM methods (`aget`, `acreate`, `acount`, `aupdate`, `async for`) exist, but DB drivers are still used **synchronously**: Django runs them through `sync_to_async` in a thread. You save threads for waiting on *other* I/O, not DB time.
- **Transactions don't work in async mode.** `transaction.atomic()` and `select_for_update` must run in a sync function: `await sync_to_async(place_order)(...)`.
- Calling the sync ORM directly in async code raises `SynchronousOnlyOperation`.
- Lazy relations: `order.customer` in async code triggers a sync query → error. Use `select_related` first or `await Customer.objects.aget(...)`.
- `sync_to_async(thread_sensitive=True)` (default) runs all such calls on **one shared thread** per request context, which serializes them. `thread_sensitive=False` is parallel but unsafe for DB connections.
- Any **sync middleware** in the stack forces a switch per request; check that your middleware is async-capable.
- Async pays off for **I/O fan-out** (calling several external APIs, websockets, SSE, long-polling); not for typical CRUD.

## Celery integration best practices

↑ TOP

- Pass **IDs**, not model instances (data goes stale; pickling is unsafe).
- Enqueue with `transaction.on_commit` (or `delay_on_commit`).
- Tasks must be **idempotent**; use `acks_late=True` for important work.
- `autoretry_for=(RequestException,), retry_backoff=True, retry_jitter=True, max_retries=5`.
- Set `soft_time_limit` / `time_limit`; separate **queues** for slow vs fast tasks so a backlog doesn't block emails.
- Never call `result.get()` inside a task (deadlock risk); chain instead.
- Celery workers hold DB connections too; close or reuse them correctly (Celery's Django fixup handles per-task cleanup).

See celery-redis-broker.

↑ TOP

## Logging and monitoring

- `LOGGING` dictConfig → JSON logs to stdout (container standard), with a **request ID** (middleware) and user ID in every line.
- **Sentry** (`sentry_sdk.init(integrations=[DjangoIntegration(), CeleryIntegration()], traces_sample_rate=0.1)`): exceptions with request context, performance traces, N+1 and slow-DB detection. Scrub PII (`send_default_pii=False`).
- Metrics: request latency p95/p99, error rate, DB query count/time per endpoint, Celery queue length (Prometheus via `django-prometheus`). Postgres side: `pg_stat_statements`, slow-query log.
- The `django.db.backends` logger prints SQL only when `DEBUG=True`.

## Testing

↑ TOP

| Class | DB | How it isolates | Use for |
|---|---|---|---|
| `SimpleTestCase` | no DB access | — | pure logic, forms, URL tests |
| `TestCase` | yes | wraps each test in a transaction and rolls back | almost everything (fast) |
| `TransactionTestCase` | yes | truncates tables after each test | real commits: `on_commit` behavior, `select_for_update` with threads, multiple connections |
| `LiveServerTestCase` | yes | runs a real server | Selenium/Playwright |

- `setUpTestData()` (class-level, created once per class) is much faster than `setUp()` per test.
- In `TestCase`, `on_commit` callbacks don't run → `self.captureOnCommitCallbacks(execute=True)`.
- **factory_boy** factories instead of fixtures (`OrderFactory(customer__name="x")`), readable and keep up with model changes.
- **pytest-django**: `@pytest.mark.django_db` (like `TestCase`), `@pytest.mark.django_db(transaction=True)` (like `TransactionTestCase`), fixtures `client`, `django_assert_num_queries`, `settings`.
- Mock external services at the boundary (`responses`, `respx`); run Celery tasks eagerly in tests or call the task function directly.
- Speed: `--reuse-db`, `pytest-xdist` (`-n auto`), fast password hasher (`MD5PasswordHasher`) in test settings.

✓ KEY POINTS

- Cache close to the data that's expensive; invalidate after commit; protect hot keys from stampede.
- Persistent connections or a pool; size total connections against Postgres `max_connections`.
- Threads (gthread) for I/O-heavy sync Django; ASGI only if you really use async.
- `DEBUG=False`, `ALLOWED_HOSTS`, HTTPS settings, parameterized SQL, `check --deploy`.
- Async ORM is a thread wrapper; transactions need sync code.
- `TestCase` for speed, `TransactionTestCase` for commit/locking behavior.

❝ INTERVIEW LINE

*"In production I run gunicorn with threads or a few workers per pod, keep DB connections persistent with health checks or behind pgbouncer, and count the total connections against Postgres' limit. I cache expensive reads in Redis with versioned keys and invalidate them in* `on_commit`*, monitor with Sentry and per-endpoint query counts, and I only use async views for I/O fan-out, because Django's async ORM still runs the DB driver in a thread and doesn't support transactions."*

⚠ COMMON TRAP

switching a CRUD app to ASGI + async views expecting a big speedup. The DB calls still run through `sync_to_async` threads, and sync middleware adds context switches, so it can be **slower**. The bottleneck is usually queries, not the server model.

↑ Back to top ☰ Contents

# 09

# System Design Case Studies

25 pages. Each card says what the page covers and its key lines; open the page for the full depth.

## V1·4 Rate Limiter · p. 201

- **Server-side middleware + Redis counters:** check the counter, reject with **HTTP 429** if over the limit, else forward and increment.
- **5 algorithms:** token bucket (bursts, Amazon/Stripe), leaking bucket (fixed outflow, Shopify), fixed window (edge bursts up to 2×), sliding log (exact, memory-heavy), sliding counter (approx, cheap).
- **Sliding window counter:** current + previous × overlap%; Cloudflare saw only **0.003%** wrong of 400M requests.
- **Distributed:** read-check-write races → **Lua script** or Redis sorted sets, not locks; share state in a **central Redis**, not sticky sessions.
- Tell clients via headers: `X-Ratelimit-Remaining`, `X-Ratelimit-Limit`, `X-Ratelimit-Retry-After`.

Read the full page →

## V1·5 Consistent Hashing · p. 207

- `hash(key) % N` **breaks on resize:** change N and most keys move → a storm of cache misses.
- **Hash ring:** hash servers and keys onto the same ring; a key lives on the **first server clockwise**. Resizing moves only **~k/n keys**.
- **Virtual nodes** fix uneven partitions and skewed keys: 100–200 vnodes → std dev **10%–5%** of the mean. Trade-off: more metadata.
- **Affected keys:** on add/remove, only the range from the node **anticlockwise to the previous server** moves.
- Used by Dynamo, Cassandra, Discord, Akamai, Maglev.

Read the full page →

## V1·6 Key-Value Store · p. 211

- **CAP:** partitions happen, so pick **CP** (block writes, e.g. banks) or **AP** (serve stale, sync later). CA can't exist in practice.
- **Partition** with consistent hashing; **replicate** to the first **N unique servers** clockwise, across data centers.
- **Quorum:** W + R > N → strong consistency (usually **N = 3, W = R = 2**). R = 1, W = N → fast reads; W = 1, R = N → fast writes.
- **AP toolkit (Dynamo/Cassandra):** eventual consistency + **vector clocks**, **gossip** failure detection, **sloppy quorum + hinted handoff**, **Merkle trees** for anti-entropy.
- **Node storage:** write = commit log → memory cache → flush to **SSTable**; read = memory → **bloom filter** → SSTables.

Read the full page →

## V1·7 Unique ID Generator · p. 218

- **Snowflake 64-bit ID:** 1 sign bit + 41 bits timestamp (ms) + 5 datacenter + 5 machine + 12 sequence.
- **Numbers:** 41 bits of ms ≈ **69 years** from the epoch; 12 bits = **4,096 IDs per ms per machine**.
- Rejected: **multi-master** (step by k, not time-ordered), **UUID** (128-bit, not sortable), **ticket server** (single point of failure).
- Sortable by time because the **timestamp sits in the high bits**; datacenter/machine IDs are fixed at startup.
- Weak spot: **clock sync** between machines (NTP); tune bit lengths for low-concurrency, long-lived apps.

Read the full page →

## V1·8 URL Shortener · p. 222

- **Short code = base 62 of a unique ID.** 62^7 ≈ 3.5 trillion, so **7 chars** cover 365 billion URLs.
- **Numbers:** 100M new URLs/day → ~1,160 writes/s, ~11,600 reads/s (10:1), 365B records in 10 years.
- **301 vs 302:** 301 is cached by the browser (less server load); 302 hits you every time (better analytics).
- **Hash + collision vs base 62:** hash needs no ID generator but must resolve collisions (bloom filter helps); base 62 never collides but the next URL is guessable.
- Redirect path: LB → web servers → **cache** → DB. Reads dominate, so cache the `<shortURL, longURL>` map.

Read the full page →

## V1·9 Web Crawler · p. 227

- **Loop:** seed URLs → frontier → download → parse → content seen? → extract links → filter → URL seen? → frontier.
- **Numbers:** 1B pages/month → **~400 QPS** (peak 800); 500 KB/page → **500 TB/month**, **30 PB** for 5 years.
- **BFS, not DFS**, but plain FIFO is impolite and ignores priority → **URL frontier**: front queues = priority, back queues = politeness (one host per queue).
- **Dedup twice:** content hash ("Content Seen?", ~29% of pages are duplicates) and URL set ("URL Seen?", bloom filter / hash table).
- Speed: distributed crawl, **DNS cache** (DNS is 10–200 ms), locality, short timeouts; obey **robots.txt**; cap URL length vs spider traps.

Read the full page →

## V1·10 Notification System · p. 232

- **Servers → one queue per channel → workers → third party** (APNs, FCM, Twilio, SendGrid). One channel's outage does not block the others.
- Scale: **10M push + 1M SMS + 5M email per day**. Soft real-time: a small delay under load is fine.
- **Never lose a notification:** persist it in a notification log, and retry from the queue on failure. Delay or reorder is OK.
- **Exactly-once is impossible:** dedupe by event ID to make duplicates rare.
- Before sending, check **opt-in settings** and a **per-user rate limit**. Authenticate callers with appKey/appSecret.

Read the full page →

## V1·11 News Feed System · p. 237

- **Two flows:** feed publishing (post → DB/cache → fanout to friends) and feed retrieval (read IDs from cache → hydrate).
- **Fanout on write (push)** = fast reads, but slow for users with many friends and wasted on inactive users. **Fanout on read (pull)** = no waste, but slow reads.
- **Hybrid:** push for most users; followers **pull celebrities'** posts on demand. Consistent hashing spreads hot keys.
- The feed cache stores only `<post_id, user_id>` **IDs**, with a configurable length limit. Hydrate from the user and post caches at read time.
- Scale: **10M DAU, ≤ 5,000 friends**, reverse chronological, media through a **CDN**.

Read the full page →

## V1·12 Chat System · p. 242

- **WebSocket** for both send and receive; plain **HTTP** for login, signup and profile. Chat servers are **stateful**, so pick one via **service discovery (ZooKeeper)**.
- Scale: **50M DAU**, groups ≤ **100**, text < **100,000 chars**, history kept **forever**. ~10 KB per connection → 1M connections ≈ 10 GB.
- History lives in a **KV store** (HBase, Cassandra): easy horizontal scaling, low latency, and a 1:1 read/write ratio.
- **message_id** must be unique and time-sortable: Snowflake, or a simpler **per-channel local sequence**. Devices sync with **cur_max_message_id**.
- Presence = **heartbeat** (e.g. every 5 s, offline after 30 s silence) + **pub/sub channel per friend pair**.

Read the full page →

## V1·13 Search Autocomplete · p. 248

- **Trie with top-k cached at every node** + max prefix length → lookup is O(1). Trade space for time.
- **Numbers:** 10M DAU × 10 searches × 20 chars ≈ **24K QPS** (peak ~48K), 0.4 GB new data/day, reply in < **100 ms**.
- **Don't update the trie on every query:** logs → aggregators → workers rebuild the trie **weekly** → Trie DB → Trie Cache.
- Speed tricks: AJAX, **browser cache** (`Cache-Control: private, max-age=3600`), log only 1 in N queries.
- Shard by first letter is uneven (c ⩾ x) → **shard map manager** built from historical distribution.

Read the full page →

## V1·14 YouTube · p. 253

- **Use the cloud:** blob storage for videos, **CDN** for streaming. API servers do everything else.
- **Numbers:** 5M DAU, 5 videos/day, 10% upload 300 MB → **150 TB/day** storage; CDN at $0.02/GB ≈ **$150K/day**.
- Upload: client → **pre-signed URL** → original storage → **transcoding (DAG)** → transcoded storage → CDN; a completion queue updates metadata.
- Transcoding pipeline: **preprocessor → DAG scheduler → resource manager → task workers**, with temp storage. Split by **GOP** for parallel, resumable uploads.
- Cut CDN cost via the **long tail**: only popular videos on CDN, others on video servers; encode rare short videos on demand.

Read the full page →

## V1·15 Google Drive · p. 259

- **Split files into ≤ 4 MB blocks** (Dropbox): block servers compress, encrypt, upload; **delta sync** sends only changed blocks.
- **Numbers:** 50M users × 10 GB = **500 PB** allocated; 10M DAU × 2 uploads → **~240 QPS**, peak 480.
- **Strong consistency** for metadata → relational DB (ACID) + invalidate cache on write.
- Notifications via **long polling**, not WebSocket: one-way, infrequent. Offline clients catch up via an offline backup queue.
- Conflicts: **first processed version wins**; the loser sees both copies and merges or overrides.

Read the full page →

## V2·1 Proximity Service · p. 266

- **Map 2D to 1D:** a lat/lng index only helps one dimension. Split the map into grids (**geohash**, **quadtree**, **S2**) and index the grid key.
- Scale: **100M DAU**, **200M businesses**, 5 searches/user/day → **~5,000 search QPS**. Read-heavy; writes are rare and take effect **next day**.
- **Geohash** = easy, fixed grid size, easy updates, needs **8 neighbor cells** for boundary cases. **Quadtree** = in-memory tree (~**1.71 GB**), adapts to density, good for **k-nearest**, harder to update.
- Geo index is small, so scale with **read replicas, not sharding**. Shard only the business table (by business_id).
- Cache by **geohash**, not raw lat/lng: geohash → business IDs (~**5 GB** for 3 precisions) and business_id → business object.

Read the full page →

## V2·2 Nearby Friends · p. 273

- Unlike proximity search, **locations change all the time**, so this is a **message-routing** problem, not an index problem.
- Scale: **100M DAU**, **10M concurrent**, update every **30 s** → **~334K updates/s**; × 400 friends × 10% online and nearby → **~13M pushes/s**.
- **WebSocket** servers + **Redis location cache** (user_id → lat/lng/ts, **TTL** = inactive timeout) + **Redis pub/sub, one channel per user**. Friends' handlers subscribe and filter by distance.
- Pub/sub bottleneck is **CPU, not memory**: ~200 GB of channels (≈2 servers) vs ~**130** servers at 100K pushes/s. Shard channels with a **hash ring in service discovery**.
- Treat pub/sub as a **stateful** cluster: over-provision, resize off-peak, because moving channels triggers mass resubscribes. **Erlang** is a strong alternative.

Read the full page →

## V2·3 Google Maps · p. 279

- Three features: **location updates**, **navigation + ETA**, **map rendering**. Mobile first, so **data and battery** matter.
- Scale: **1B DAU**; navigation ~**7,200 QPS** (peak 36K); GPS **batched every 15 s → 200K QPS** (peak **1M**). Map tiles ≈ **100 PB** across 21 zoom levels.
- **Map tiles:** pre-generated, static, one per geohash per zoom level, served from a **CDN**. ~**1.25 MB/min** per navigating user.
- **Routing tiles:** split the road graph into small **geohash-keyed tiles in 3 hierarchy levels** (local, arterial, highway), stored in **S3**. A* loads tiles on demand.
- Location: **Cassandra** (user_id partition key, timestamp clustering key; AP over C) + **Kafka** to feed traffic, routing-tile and ML updaters. Push reroutes over **WebSocket**.

Read the full page →

## V2·4 Distributed Message Queue · p. 285

- **Topic → partitions → brokers:** each partition is an append-only **FIFO log**; order holds only inside a partition; `hash(key) % numPartitions` picks it.
- **Storage = WAL segments on disk:** sequential I/O + OS page cache + **batching** everywhere + no message copying = high throughput.
- **Consumer group:** one partition → one consumer per group (keeps order). All consumers in one group = point-to-point; many groups = pub/sub.
- **Replication with ISR:** ACK=all (durable, slow), ACK=1 (leader only), ACK=0 (fastest, may lose). Pull model, long polling.
- **Delivery:** at-most once = commit offset *before* processing; at-least once = commit *after* (duplicates possible); exactly once = costly, for payments.

Read the full page →

## V2·5 Metrics Monitoring and Alerting · p. 293

- **Scale:** 1,000 pools × 100 machines × 100 metrics ≈ **10M metrics**; keep raw **7 days**, 1-min rollup **30 days**, 1-hour rollup **1 year**.
- **Workload:** constant heavy writes, spiky reads → use a **time-series DB** (InfluxDB, Prometheus), not MySQL or a generic NoSQL.
- **Pull vs push:** no winner. Pull (Prometheus) = easy debug + health check; push (CloudWatch, Graphite) = short-lived jobs, firewalls. Big orgs support both.
- **Pipeline:** collector → **Kafka** (partition by metric name) → consumers → TSDB, so data survives a DB outage.
- **Save space:** delta encoding, **downsampling**, cold storage. **Buy** visualization (Grafana) and alerting rather than build.

Read the full page →

## V2·6 Ad Click Event Aggregation · p. 300

- **Pipeline:** log watcher → Kafka → **aggregation service (Map → Aggregate → Reduce)** → Kafka → DB writer → aggregation DB. Raw events also go to a raw DB.
- **Numbers:** 1B clicks/day → **10K QPS avg, 50K peak**; 0.1 KB/event → **100 GB/day, ~3 TB/month**; 2M ads; +30%/year.
- Use **event time + watermark** (e.g. 15 s) for accuracy; very late events are fixed by **end-of-day reconciliation**.
- Billing needs **exactly-once**: save the offset only after downstream acks, and wrap send + save offset + ack in a **distributed transaction**.
- Store **both raw (backup, recalculation) and aggregated (fast queries)** data. Kappa: replay raw data through the same aggregation service.

Read the full page →

## V2·7 Hotel Reservation System · p. 307

- **Book a room type, not a room.** Core table: `room_type_inventory(hotel_id, room_type_id, date, total_inventory, total_reserved)`, one row per date, 2 years ahead (~73M rows).
- **Numbers:** 5,000 hotels, 1M rooms, 70% occupied, 3-day stays → ~240K bookings/day ≈ **3 TPS**; funnel 300 → 30 → 3 QPS.
- **Same user double-clicks:** `reservation_id` as **idempotency key** (primary key, unique constraint).
- **Two users, last room:** pessimistic lock (`SELECT … FOR UPDATE`) vs **optimistic lock (version)** vs **DB CHECK constraint**. Low contention → optimistic or constraint.
- Keep reservation + inventory in **one relational DB** for ACID. Pure microservices need **2PC or Saga**, which is too complex here.

Read the full page →

## V2·8 Distributed Email Service · p. 313

- **Numbers:** 1B users; send **100K QPS**; 40 emails/day received, 50 KB metadata → **730 PB/year**; 20% with 500 KB attachments → **1,460 PB/year**.
- Webmail → **web servers (HTTPS)** + **real-time servers (WebSocket, long-poll fallback)** → storage: **metadata DB, attachment store (S3), Redis cache, search store**.
- Send/receive both go through **message queues** (outgoing / incoming) so SMTP workers and mail processing (spam, virus) scale on their own.
- Metadata DB: partition by **user_id**, sort by **TIMEUUID email_id**; **denormalize** into `read_emails` / `unread_emails` because NoSQL can't filter on `is_read`.
- **Consistency over availability:** one primary per mailbox. Deliverability needs dedicated IPs, warm-up (2–6 weeks), feedback loops, **SPF/DKIM/DMARC**.

Read the full page →

## V2·9 S3-like Object Storage · p. 321

- **Split metadata (mutable) from data (immutable):** metadata store maps name → UUID; data store only knows UUIDs.
- **Numbers:** 100 PB/year, 6 nines durability, 4 nines availability → ~**0.68B objects**, ~0.68 TB metadata.
- **Pack small objects into big append-only files** (WAL-style); per-node SQLite maps UUID → (file, offset, size).
- **3× replication** ≈ 6 nines, 200% overhead; **(8+4) erasure coding** ≈ 11 nines, 50% overhead but slower reads/writes.
- Shard object metadata by **hash(bucket_name, object_name)**; listing uses a separate table sharded by bucket.

Read the full page →

## V2·10 Real-time Gaming Leaderboard · p. 329

- **Redis sorted set per month:** `ZINCRBY` on a win, `ZREVRANGE 0 9` for top 10, `ZREVRANK` for a user's rank, all **O(log n)**.
- **Numbers:** 5M DAU, 25M MAU, 10 games/day → **~500 score QPS**, peak **2,500**; **~650 MB** in Redis (26 B × 25M).
- SQL rank needs a sort over millions of changing rows: **10s of seconds**, and caching does not help.
- At 500M DAU (65 GB, 250K QPS) shard: **fixed score-range partitions** beat Redis-cluster hash slots (rank stays easy).
- Scores are set **server-side** (game service), never by the client; MySQL point table rebuilds Redis after failure.

Read the full page →

## V2·11 Payment System · p. 335

- **Pay-in** (buyer card → site's bank) and **pay-out** (site's bank → seller). A PSP moves the money; we never store card numbers.
- Only **10 TPS** (1M tx/day). The interview is about **correctness**, not throughput. Pick a proven **ACID relational DB**.
- **Exactly-once = at-least-once (retry) + at-most-once (idempotency key).** The payment_order_id is the PSP's dedupe key.
- **Reconciliation** against the PSP's nightly settlement file is the last line of defense. Never assume the external system is right.
- Money as **string**, not double. **Double-entry ledger:** every tx debits one account and credits another; the sum is always 0.

Read the full page →

## V2·12 Digital Wallet · p. 342

- **1M transfers/s = 2M DB ops/s.** At 1,000 TPS per node that is **2,000 nodes**, so raise per-node TPS.
- **Redis shards** are fast but not atomic across two nodes. A crash between "A−$1" and "C+$1" loses money.
- Distributed tx: **2PC** (DB-level, holds locks, coordinator SPOF) vs **TC/C** and **Saga** (app-level compensation). Always **deduct first**, then add.
- **Event sourcing:** command → validate → immutable event → state. Replay events to rebuild any past balance (**reproducibility** for auditors). CQRS read side.
- Final design: **file-based** event log (mmap, RocksDB, snapshots), **Raft** groups per partition, a **reverse proxy** that pushes status, and a **Saga/TC/C coordinator** across partitions.

Read the full page →

## V2·13 Stock Exchange · p. 349

- **1B orders/day over 6.5 h ≈ 43K QPS, peak ~215K.** 100 symbols, tens of thousands of users, limit orders only.
- Critical path: **gateway → order manager (risk, wallet) → sequencer → matching engine**. Market data and reporting are off the path.
- **Sequencer** stamps every order and fill with a gap-free ID, so the **deterministic** matching engine can replay and recover.
- **Order book:** map of price → PriceLevel (**doubly linked list** of orders) + `orderId → order` map, giving **O(1)** place, match, cancel.
- Low latency: **one server**, components talk over an **mmap** event store (sub-µs), **single-threaded app loops pinned to cores**. HA: hot-warm engines, then Raft across servers.

Read the full page →

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·4 Rate Limiter

rate limiter · throttling · token bucket · leaking bucket · fixed window · sliding window log · sliding window counter · Redis · INCR EXPIRE · HTTP 429 · API gateway · race condition · Lua script · distributed rate limiting

SEE ALSO Locking & Concurrency Control

**Source:** *System Design Interview* Vol 1, Ch 4. · Your write-up: Rate Limiter

## Problem

↑ TOP

Design a rate limiter. It limits how many requests a client can send in a time window and blocks the excess calls.

Why it matters: it stops DoS attacks (Twitter: 300 tweets per 3 hours; Google Docs: 300 reads per user per 60 s), it cuts cost (fewer servers, fewer paid third-party API calls), and it keeps servers from overloading.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Client-side or server-side API rate limiter? | Server-side API rate limiter |
| Throttle by IP, user ID, or something else? | Flexible: support different sets of throttle rules |
| Scale? Startup or big company? | Must handle a large number of requests |
| Distributed environment? | Yes |
| Separate service or in application code? | Your design decision |
| Tell throttled users? | Yes |

**Requirements:**

- Functional: accurately limit excessive requests; support flexible rules; show clear exceptions to throttled users.
- Non-functional:
- Low latency: must not slow down HTTP response time.
- Use as little memory as possible.
- Distributed: shared across many servers or processes.
- High fault tolerance: if the limiter has a problem (e.g. a cache server goes offline), the whole system keeps working.

**Estimates:** the book gives none.

> Try it first: decide where the limiter sits, pick an algorithm, and sketch where the counters live and how two limiter servers share them.

SOLUTION

## High-level design

**Where to put it:**

- **Client-side:** unreliable. Requests can be forged, and you may not control the client.
- **Server-side:** inside the API servers.
- **Middleware:** a rate limiter in front of the API servers. In microservices this is often the **API gateway** (which also does SSL termination, auth, IP whitelisting).

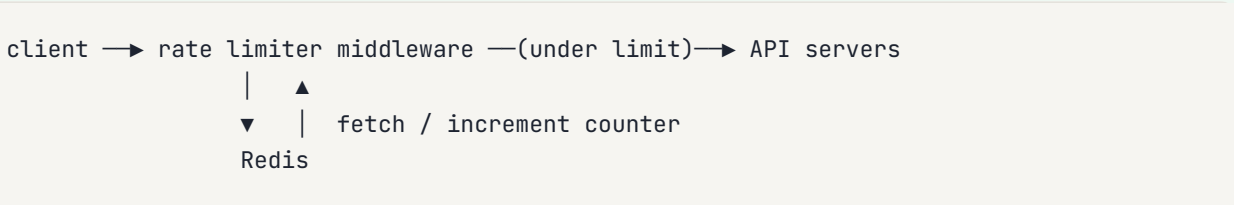
Example: the limit is 2 requests/second and a client sends 3. The first two reach the API servers. The middleware throttles the third and returns **HTTP 429** (too many requests).

Server-side vs gateway has no single answer. Guidelines:

- Check your stack: is your language efficient enough to do it server-side?
- Server-side gives full control of the algorithm. A third-party gateway may limit your choice.
- Already have an API gateway for auth and IP whitelisting? Add the limiter there.
- Building your own takes time. Without enough engineers, use a commercial API gateway.

**Where to store counters:** not the database (disk is slow). Use an **in-memory cache** with time-based expiry. **Redis** has two useful commands:

- `INCR`: increase the counter by 1.
- `EXPIRE`: set a timeout; the counter is deleted when it expires.




- The client sends a request to the middleware.
- The middleware fetches the counter from the right bucket in Redis and checks the limit.
- Limit reached → reject the request.
- Limit not reached → forward to the API servers, and increment the counter in Redis.

## Deep dives

### Rate limiting algorithms

**Token bucket** (used by Amazon and Stripe)

- A bucket has a fixed capacity. A refiller adds tokens at a preset rate. Extra tokens overflow.
- Each request takes one token. No token → the request is dropped.
- Example: size 4, refill 4 per minute. At 1:00:00 one request passes (3 left). At 1:00:05 three requests pass (0 left). At 1:00:20 a request is dropped. At 1:01:00 the bucket is refilled to 4.
- Two parameters: **bucket size** and **refill rate**.
- How many buckets? One per endpoint per user (1 post/s, 150 friends/day, 5 likes/s → 3 buckets per user). One per IP if throttling by IP. One global bucket for a system-wide cap like 10,000 req/s.

**Leaking bucket** (used by Shopify)

- A FIFO queue. If the queue is full, the request is dropped; otherwise it is queued.
- Requests are pulled from the queue and processed at a fixed rate.
- Two parameters: **bucket size** (queue size) and **outflow rate**.

**Fixed window counter**

- Split the timeline into fixed windows, one counter each. Each request adds 1. At the threshold, drop until the next window.
- Problem: bursts at window edges. Limit 5/min, reset on the minute. 5 requests in 2:00:30–2:01:00 and 5 in 2:01:00–2:01:30 → **10 requests in one rolling minute**, twice the limit.

**Sliding window log**

- Keep request timestamps (e.g. in a Redis **sorted set**).
- On a new request: remove timestamps older than the window start, add the new timestamp, accept if the log size ≤ the limit.
- Example, 2/min: 1:00:01 accepted; 1:00:30 accepted (size 2); 1:00:50 rejected (size 3, but the timestamp stays in the log); 1:01:40 removes 1:00:01 and 1:00:30 → size 2 → accepted.

**Sliding window counter** (hybrid of fixed window + sliding log)

- Requests in rolling window = current window + previous window × overlap % of the rolling window with the previous window.
- Example, limit 7/min: 5 requests last minute, 3 this minute, new request at 30% into the minute → 3 + 5 × 0.7 = 6.5 → rounded down to 6 → allowed. One more request hits the limit.

| Algorithm | Pros | Cons |
|---|---|---|
| Token bucket | Easy; memory efficient; allows short bursts | Bucket size and refill rate are hard to tune |
| Leaking bucket | Memory efficient (bounded queue); stable outflow rate | A burst fills the queue with old requests, so new ones are dropped; two params hard to tune |
| Fixed window counter | Memory efficient; easy; reset at window end fits some cases | Spikes at window edges let up to 2× the quota through |
| Sliding window log | Very accurate: never exceeds the limit in any rolling window | Uses a lot of memory; rejected requests' timestamps are still stored |
| Sliding window counter | Smooths spikes; memory efficient | Approximation (assumes the previous window was even); Cloudflare: only 0.003% of 400M requests wrong |

### Rate limiting rules

Example from Lyft's open-source limiter: rules live in config files on disk.

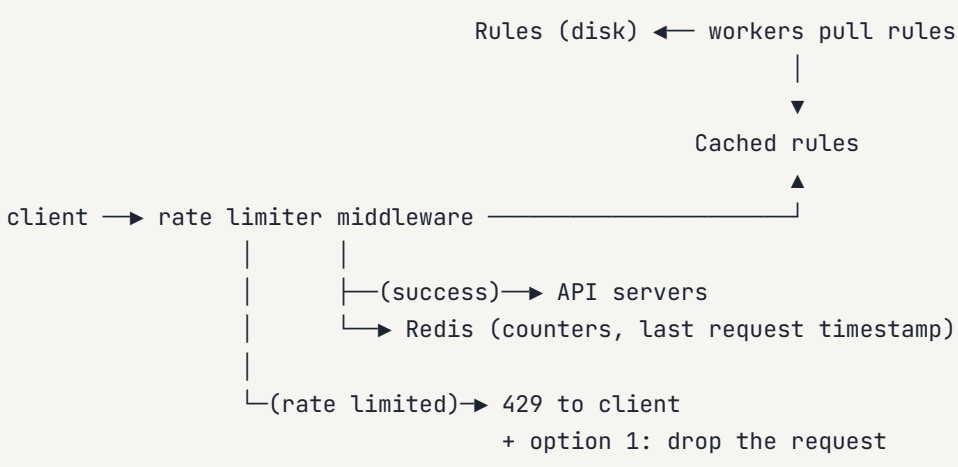
YAML

```yaml
domain: messaging
descriptors:
  - key: message_type
    value: marketing
    rate_limit:
      unit: day
      requests_per_unit: 5      # max 5 marketing messages per day
---
domain: auth
descriptors:
  - key: auth_type
    value: login
    rate_limit:
      unit: minute
      requests_per_unit: 5      # max 5 logins per minute
```

### Exceeding the limit

- Return **HTTP 429**. Depending on the use case, enqueue the request for later (e.g. orders throttled during overload).
- Response headers:
- `X-Ratelimit-Remaining`: requests left in the window.
- `X-Ratelimit-Limit`: calls allowed per window.
- `X-Ratelimit-Retry-After`: seconds to wait before retrying.
- A throttled user gets 429 plus `X-Ratelimit-Retry-After`.

### Detailed design



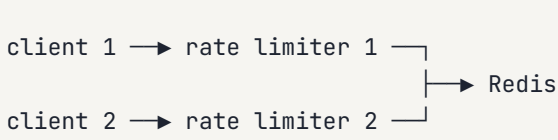

- Workers pull rules from disk into a cache.
- The middleware loads rules from the cache and counters + last request timestamp from Redis.
- Not limited → forward to API servers. Limited → 429, and drop or enqueue the request.

### Distributed environment

**Race condition.** The flow is read counter → check `counter + 1` → write. Two concurrent requests both read 3 and both write 4; the correct value is 5. Locks fix it but slow the system a lot. Use a **Lua script** or **Redis sorted sets** instead.

**Synchronization.** Many limiter servers are needed. The web tier is stateless, so a client's requests can hit different limiters; without sync, limiter 1 knows nothing about client 2.

- Sticky sessions: not advisable; neither scalable nor flexible.
- **Centralized store (Redis)** that all limiters share: the book's choice.




### Performance optimization

- **Multi-data center / edge servers:** latency is high for users far away. Route traffic to the closest edge (Cloudflare had 194 edge locations as of 5/20 2020).
- **Synchronize data with eventual consistency** (see Ch 6, Key-Value Store).

### Monitoring

Check that the **algorithm** and the **rules** are effective. Too strict rules drop valid requests → relax them. The limiter fails during sudden bursts like flash sales → switch to an algorithm that supports bursts; **token bucket** fits.

## Wrap-up

- **Hard vs soft limiting:** hard never exceeds the threshold; soft can exceed it for a short time.
- **Other layers:** this chapter is layer 7 (HTTP). You can also limit by IP at layer 3 with iptables.
- **Avoid being rate limited (client side):** cache on the client, know the limits, catch errors to recover gracefully, add enough back-off time to retries.

✓ KEY POINTS

- Put the limiter in middleware or the API gateway; reject with 429 and tell the client via `X-Ratelimit-*` headers.
- Store counters in Redis ( `INCR` + `EXPIRE` ), not a database.
- Compare the 5 algorithms; pick by need: bursts → token bucket, steady outflow → leaking bucket, accuracy → sliding log, cheap and close → sliding counter.
- Rules live in config files, cached by workers; the middleware reads them from the cache.
- Distributed: fix read-check-write races with Lua scripts or sorted sets, and share one central Redis instead of sticky sessions.

⚠ COMMON TRAP

using a plain fixed window and missing that a burst across a window edge lets twice the limit through. Also: wrapping the counter in a lock, which kills latency.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·5 Consistent Hashing

consistent hashing · hash ring · rehashing problem · modulo hashing · virtual nodes · vnodes · partitioning · sharding · SHA-1 · cache misses · hotspot · Dynamo · Cassandra · horizontal scaling

SEE ALSO SQL vs NoSQL & Scaling Databases

**Source:** *System Design Interview* Vol 1, Ch 5.

## Problem

↑ TOP

Design a way to distribute keys across servers so that adding or removing a server moves only a few keys.

This is a technique, not a product. The setting is a pool of cache (or data) servers that must scale horizontally and spread load evenly.

**Clarifying questions → answers:** the chapter has no Q&A. These are the assumptions it works with:

| Question | Answer |
|---|---|
| What is being distributed? | Cache keys (or data) across a pool of servers |
| Does the pool change? | Yes: servers are added and go offline |
| What hurts when keys move? | Clients hit the wrong server → a storm of cache misses |

**Requirements:**

- Functional: map any key to exactly one server; look it up without a central table.
- Non-functional:
- Adding or removing a server moves only a small fraction of keys.
- Keys spread evenly; no server gets most of the data.
- Scales horizontally; helps with hotspot keys.

**Estimates:** none. The only number to know is the goal: on resize, only **k/n** keys move on average (k = keys, n = slots).

> Try it first: show why `hash(key) % N` fails when one of 4 servers dies, then sketch a scheme where only that server's keys move.

**SOLUTION**

## High-level design

**The rehashing problem.** With N servers, the simple method is `serverIndex = hash(key) % N`. It works only while N is fixed.

4 servers, 8 keys. Then server 1 goes offline, so N = 3:

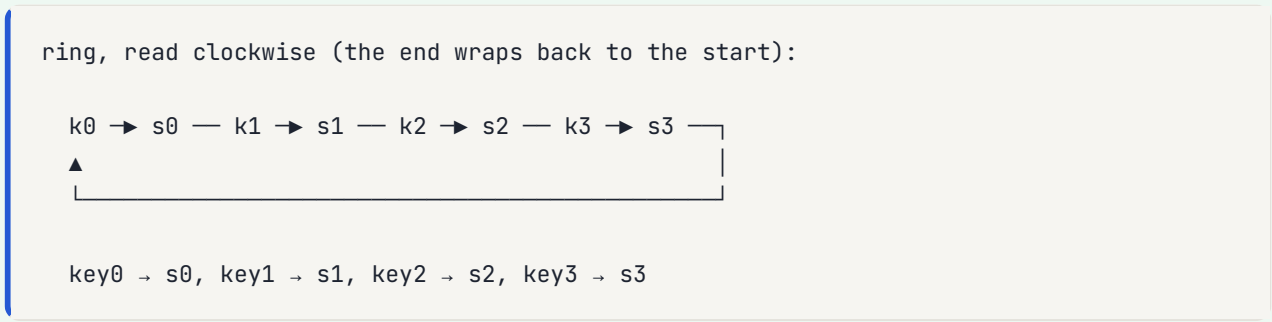
| key | hash | hash % 4 | hash % 3 |
|---|---|---|---|
| key0 | 18358617 | 1 | 0 |
| key1 | 26143584 | 0 | 0 |
| key2 | 18131146 | 2 | 1 |
| key3 | 35863496 | 0 | 2 |
| key4 | 34085809 | 1 | 1 |
| key5 | 27581703 | 3 | 0 |
| key6 | 38164978 | 2 | 1 |
| key7 | 22530351 | 3 | 0 |

With the 3 remaining servers re-indexed (0 = server 0, 1 = server 2, 2 = server 3), key0, key3, key4, key5 and key7 all change server. Only key0 and key4 were on server 1. Most keys move, not just the dead server's keys → cache-miss storm.

**Consistent hashing** (Karger et al., MIT): when the table is resized, only **k/n** keys need to move on average. Traditional hashing remaps nearly all keys.

**The hash ring:**

- Take a hash function f, e.g. SHA-1. Its space is 0 … 2^160 − 1.
- Join both ends of the space → a ring.
- Hash each **server** (by IP or name) onto the ring with f.
- Hash each **key** onto the same ring with f. There is **no modulo**.
- **Lookup:** from the key's position, go **clockwise** to the first server. That server stores the key.




- **Add a server:** s4 lands between k0 and s0 ( `k0 → s4 — s0` ). Only key0 moves (s0 → s4). k1, k2, k3 stay.

- **Remove a server:** s1 is removed. Only key1 moves (s1 → s2). The rest stay.

## Deep dives

### Two issues with the basic approach

1. **Uneven partitions.** A partition is the hash space between two adjacent servers. Servers come and go, so partition sizes drift. Example: remove s1, and s2's partition becomes twice as large as s0's and s3's.
2. **Non-uniform key distribution.** If the servers cluster on one part of the ring, most keys land on one server. Example: s0, s1, s2, s3 bunched together → most keys go to server 2, and servers 1 and 3 get none.

### Virtual nodes

- Each real server appears on the ring as **many virtual nodes**. Server 0 is s0_0, s0_1, s0_2; server 1 is s1_0, s1_1, s1_2. (3 is arbitrary; real systems use many more.)
- Each server now owns many small partitions spread around the ring.
- **Lookup:** go clockwise to the first **virtual** node, then map it to its server. Example: k0 → s1_1 → server 1.
- More virtual nodes → smaller standard deviation → more balanced data.

| Virtual nodes per server | Std dev of load (vs mean) |
|---|---|
| 100 | ~10% |
| 200 | ~5% |

- **Trade-off:** more virtual nodes need more space to store vnode metadata. Tune the count to your requirements.

### Find affected keys

- **Server added (s4):** start at s4 and go **anticlockwise** until you hit a server (s3). Keys between s3 and s4 move to s4.
- **Server removed (s1):** start at s1 and go anticlockwise until you hit a server (s0). Keys between s0 and s1 move to s2 (the next server clockwise).

```
add s4:     s3 ──[keys here move to s4]── s4 ── s0 ...
remove s1:  s0 ──[keys here move to s2]── s1(x) ── s2
```

## Wrap-up

Benefits:

- Very few keys are redistributed when servers are added or removed.

- Easy to scale horizontally because data is spread more evenly.
- Mitigates **hotspot keys**: e.g. Katy Perry, Justin Bieber and Lady Gaga's data are less likely to land on the same shard.

Real-world users:

- Amazon Dynamo (partitioning)
- Apache Cassandra (data partitioning across the cluster)
- Discord chat
- Akamai CDN
- Google Maglev network load balancer

✓ KEY POINTS

- Start with why `% N` fails: changing N remaps most keys and floods the backend with misses.
- Servers and keys share one hash space (e.g. SHA-1, 0 … 2^160 − 1); a key goes to the first server clockwise.
- Adding or removing a server moves only the keys in one arc, about k/n of them.
- Basic ring has uneven partitions and skew; virtual nodes fix both (100 → ~10%, 200 → ~5% std dev), at the cost of metadata.
- Affected range = from the changed node anticlockwise to the previous server.

⚠ COMMON TRAP

saying "consistent hashing balances load" without virtual nodes. With one point per server, partitions are uneven and one server can own most keys.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·6 Key-Value Store

key-value store · distributed hash table · CAP theorem · quorum · eventual consistency · vector clock · gossip protocol · sloppy quorum · hinted handoff · Merkle tree · SSTable · bloom filter · Dynamo · Cassandra · put get

SEE ALSO SQL vs NoSQL & Scaling Databases

**Source:** *System Design Interview* Vol 1, Ch 6.

## Problem

↑ TOP

Design a distributed key-value store that supports two operations: `put(key, value)` and `get(key)`.

Keys are unique. A short key works better (plain text like `last_logged_in_at` or a hash like `253DDEC4`). The value is an opaque object: a string, list, object, etc.

**Clarifying questions → answers:** the book lists the scope directly. As Q&A:

| Question | Answer |
|---|---|
| How big is a key-value pair? | Small: less than 10 KB |
| How much data? | Big data: more than one server can hold |
| Availability? | High: respond quickly, even during failures |
| Scaling? | High scalability, and automatic: add/remove servers based on traffic |
| Consistency? | Tunable |
| Latency? | Low |

**Requirements:**

- Functional: `put(key, value)`, `get(key)`.
- Non-functional: store big data, high availability, high and automatic scalability, tunable consistency, low latency.

**Estimates:** the book gives none. A single server fills up fast even with compression and keeping only hot data in memory (the rest on disk), so the store must be distributed.

> Try it first: pick CP or AP, then sketch how keys are partitioned, how many copies you keep, and how a read and a write reach the replicas.

**SOLUTION**

### High-level design

**API:** `get(key)` and `put(key, value)`.

**Start from CAP.** A distributed system can give at most two of:

- **Consistency:** all clients see the same data at the same time, whatever node they hit.
- **Availability:** every request gets a response, even if some nodes are down.
- **Partition tolerance:** the system keeps working when nodes can't talk to each other.

| Type | Keeps | Gives up | Note |
|---|---|---|---|
| CP | Consistency + partition tolerance | Availability | e.g. bank balances: return an error until consistent |
| AP | Availability + partition tolerance | Consistency | keep serving reads (maybe stale) and accepting writes; sync later |
| CA | Consistency + availability | Partition tolerance | cannot exist: network failure is unavoidable |

Example: data on n1, n2, n3; n3 goes down. CP blocks all writes to n1 and n2. AP keeps accepting writes on n1 and n2 and syncs n3 when the partition heals. Discuss the choice with the interviewer. The book's design goes **AP with eventual consistency**, like Dynamo and Cassandra.

**Architecture:**

```
ring of nodes n0 … n7 (consistent hashing); key's replicas = n0, n1, n2 (N = 3)

client ──read/write──► n6 (coordinator) ──► n0, n1, n2
       ◄──response─── n6                ◄── ACKs / data
```

- Clients use a simple API: `get(key)` and `put(key, value)`.
- A **coordinator** node is a proxy between the client and the store.
- Nodes sit on a ring via consistent hashing.
- The system is fully **decentralized**, so adding and moving nodes can be automatic.
- Data is replicated on multiple nodes.
- **No single point of failure:** every node has the same jobs: client API, failure detection, conflict resolution, failure repair, replication, storage engine.

## Deep dives

### Data partition

Two challenges: spread data evenly, and move little data when nodes join or leave. **Consistent hashing** solves both (see V1·5): put servers on a ring, hash the key, store it on the first server clockwise.

- **Automatic scaling:** servers can be added and removed based on load.
- **Heterogeneity:** virtual nodes per server are proportional to its capacity. Bigger servers get more.

### Data replication

- Replicate **asynchronously** to **N** servers (N is configurable).
- Pick them by walking clockwise from the key and taking the first N servers. N = 3: key0 → s1, s2, s3.
- With virtual nodes, the first N nodes may belong to fewer than N physical servers. Choose only **unique servers** on the walk.
- Nodes in one data center fail together (power, network, disasters). Place replicas in **different data centers** linked by fast networks.

### Consistency: quorum consensus

- **N** = number of replicas.
- **W** = write quorum: a write succeeds after W replicas acknowledge.
- **R** = read quorum: a read waits for at least R replicas.
- W = 1 does **not** mean data is written to one server. The coordinator sends to all N and waits for just 1 ACK.
- Trade-off: W or R = 1 is fast (wait for any one replica). W or R > 1 is more consistent but slower (wait for the slowest replica).

| Setting | Effect |
|---|---|
| R = 1, W = N | Optimized for fast reads |
| W = 1, R = N | Optimized for fast writes |
| W + R > N | Strong consistency: at least one overlapping node has the latest data (usually N = 3, W = R = 2) |
| W + R ≤ N | Strong consistency not guaranteed |

**Consistency models:**

- **Strong:** a read always returns the latest write. Usually done by blocking reads/writes on a replica until all replicas agree on the current write. Bad for high availability.
- **Weak:** later reads may not see the latest value.

- **Eventual:** a weak form; given enough time, all replicas converge. **The book's choice** (as in Dynamo and Cassandra). Concurrent writes can create conflicting values; the client reads them and reconciles.

### Inconsistency resolution: vector clocks

How conflicts arise: n1 and n2 both hold `name = john`. Server 1 sets `johnSanFrancisco` and server 2 sets `johnNewYork` at the same time → two conflicting versions v1 and v2, with no clear winner.

**Versioning:** every change is a new immutable version. A **vector clock** is a list of `[server, version]` pairs on a data item: `D([S1, v1], [S2, v2], …)`. On a write of D to server Si: increment vi if `[Si, vi]` exists, else add `[Si, 1]`.

```
D1([Sx,1])               write handled by Sx
   │
D2([Sx,2])               write handled by Sx (overwrites D1)
   ├─────────────────────┐
D3([Sx,2],[Sy,1])     D4([Sx,2],[Sz,1])    two clients update D2 via Sy and Sz → conflict
   └─────────┬───────────┘
D5([Sx,3],[Sy,1],[Sz,1])  client reconciles; written by Sx
```

- **Ancestor (no conflict):** every counter in Y ≥ the matching counter in X. `D([s0,1],[s1,1])` is an ancestor of `D([s0,1],[s1,2])`.
- **Sibling (conflict):** some counter in Y is less than in X. `D([s0,1],[s1,2])` vs `D([s0,2],[s1,1])`.
- Downsides:
- The client must implement conflict-resolution logic.
- The pairs can grow fast. Cap the length and drop the oldest pairs. That can make reconciliation inaccurate, but per the Dynamo paper Amazon has not hit this in production.

### Failure detection: gossip

- One server saying "X is down" is not enough. Usually you need **at least two independent sources**.
- All-to-all multicasting works but is inefficient with many servers.
- **Gossip protocol** (decentralized):
- Each node keeps a membership list: member ID, heartbeat counter, time.
- Each node increments its own heartbeat counter periodically.
- Each node sends heartbeats to random nodes, which pass them on.
- Nodes update their lists with the latest info.
- If a member's heartbeat has not increased for a set period, it is offline.

- Example: s0's list shows member 2 stuck at 9908 since 11:58:02 while others are around 10,230 at 12:00. s0 gossips this to random nodes; once others confirm, s2 is marked down and the news spreads.

#### Temporary failures: sloppy quorum + hinted handoff

- A strict quorum can block reads and writes when replicas are down.
- **Sloppy quorum:** use the first **W healthy** servers for writes and the first **R healthy** servers for reads on the ring. Ignore offline servers.
- **Hinted handoff:** if s2 is down, s3 handles its reads and writes for now. When s2 is back, s3 hands the data back to s2.

#### Permanent failures: anti-entropy with Merkle trees

- **Anti-entropy:** compare data on replicas and update each to the newest version.
- A **Merkle tree** (hash tree): each non-leaf node is the hash of its children. It finds differences while moving little data.
- Building one, key space 1–12: 1. Split the key space into buckets (4 here) to limit tree depth. 2. Hash each key in a bucket. 3. Make one hash node per bucket. 4. Build up to the root by hashing children.
- **Compare:** if root hashes match, the replicas are the same. If not, compare left child, then right child, and walk down to the buckets that differ. Sync only those.
- Data to sync is proportional to the **differences**, not the total data. Real config example: 1 million buckets for 1 billion keys → 1,000 keys per bucket.

#### Data center outage

Replicate data **across multiple data centers**. If one is fully offline, users read through the others.

#### Write path (based on Cassandra)

```
client —write—▶ node
                1. append to commit log (disk)
                2. save in memory cache
                3. when cache is full / hits threshold → flush to SSTable (disk)
```

An **SSTable** (sorted-string table) is a sorted list of `<key, value>` pairs.

### Read path

```
client —read—▶ node
                1. in memory cache? → return it
                2. no → check bloom filter
                3. bloom filter says which SSTables might have the key
                4. SSTables return the data
                5. return result to client
```

## Wrap-up

The book's summary table:

| Goal / problem | Technique |
|---|---|
| Ability to store big data | Consistent hashing to spread load across servers |
| High availability reads | Data replication; multi-data center setup |
| Highly available writes | Versioning and conflict resolution with vector clocks |
| Dataset partition | Consistent hashing |
| Incremental scalability | Consistent hashing |
| Heterogeneity | Consistent hashing |
| Tunable consistency | Quorum consensus |
| Handling temporary failures | Sloppy quorum and hinted handoff |
| Handling permanent failures | Merkle tree |
| Handling data center outage | Cross-data center replication |

✓ KEY POINTS

- Open with CAP: CA is impossible, so choose CP or AP with the interviewer. This design is AP with eventual consistency.
- Partition with consistent hashing; replicate to N unique servers clockwise, spread across data centers.
- Tune consistency with N, W, R; W + R > N gives strong consistency (N = 3, W = R = 2).
- Resolve conflicting writes with vector clocks; the client reconciles siblings.
- Failures: gossip to detect, sloppy quorum + hinted handoff for temporary, Merkle trees for permanent, cross-DC replication for outages.
- Each node: commit log → memory cache → SSTables; reads use a bloom filter to skip SSTables.

> ⚠ COMMON TRAP
> thinking W = 1 means the data is written to only one server. The coordinator still sends to all N replicas; W is only how many ACKs it waits for.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·7 Unique ID Generator

unique ID generator · distributed ID · Twitter snowflake · 64-bit ID · UUID · ticket server · Flickr · multi-master replication · auto_increment · sortable ID · timestamp bits · sequence number · custom epoch · clock synchronization · NTP

SEE ALSO SQL vs NoSQL & Scaling Databases

**Source:** *System Design Interview* Vol 1, Ch 7.

## Problem

↑ TOP

Design a unique ID generator for a distributed system. A database `auto_increment` key does not work here: one DB server is not big enough, and keeping IDs unique across many DBs with low delay is hard.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| What are the characteristics of the IDs? | Unique and sortable |
| Does each new ID go up by exactly 1? | No. IDs grow with time, not always by 1. Evening IDs > morning IDs on the same day. |
| Numbers only? | Yes |
| Length? | Must fit in 64 bits |
| Scale? | 10,000 IDs per second |

Example IDs from the book: `1227238262110117894`, `1241107244890099715`, `1567981766075453440`.

**Requirements:**

- IDs are unique.
- IDs are numeric only.
- IDs fit in 64 bits.
- IDs are ordered by date.
- Generate over 10,000 unique IDs per second.

**Estimates:** none beyond the 10,000 IDs/s target.

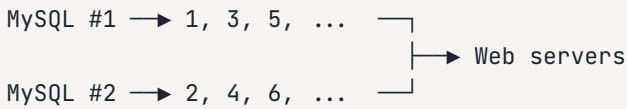
> Try it first: list 3–4 ways to generate IDs, check each against the 5 requirements, then draw the bit layout of a 64-bit ID that meets them all.

SOLUTION

## High-level design

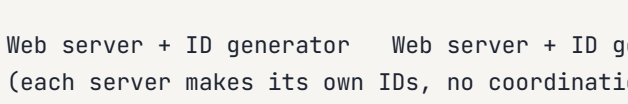
The book compares four options. Only the last one meets every requirement.

### 1. Multi-master replication



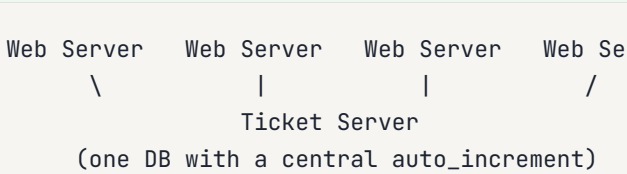

- Use each DB's `auto_increment`, but step by **k** (k = number of DB servers) instead of 1.
- IDs scale with the number of DB servers.
- Cons: hard to scale across multiple data centers; IDs do **not** go up with time across servers; adding or removing a server breaks the scheme.

### 2. UUID




- 128-bit number, e.g. `09c93e62-50b4-468d-bf8a-c07e1040bfb2`.
- Collision chance is tiny: you need 1 billion UUIDs per second for ~100 years to reach a 50% chance of one duplicate.
- Pros: simple; no coordination or sync between servers; scales with the web servers.
- Cons: 128 bits (we need 64); not time-ordered; can be non-numeric.

### 3. Ticket server (Flickr)




- Pros: numeric IDs; easy to build; fine for small to medium scale.
- Cons: **single point of failure**. More ticket servers fix that but bring data sync problems.

### 4. Twitter snowflake (chosen)

Split the 64-bit ID into sections ("divide and conquer"):

```
| 1 bit | 41 bits    | 5 bits        | 5 bits     | 12 bits         |
|   0   | timestamp  | datacenter ID | machine ID | sequence number |
```

- **Sign bit (1):** always 0. Reserved for future use (e.g. signed vs unsigned).
- **Timestamp (41):** ms since the epoch. The book uses Twitter's default epoch `1288834974657` (Nov 04, 2010, 01:42:54 UTC).
- **Datacenter ID (5):** 2^5 = 32 datacenters.
- **Machine ID (5):** 2^5 = 32 machines per datacenter.
- **Sequence (12):** +1 for each ID made on that machine/process; resets to 0 every ms.

## Deep dives

### Fixed vs runtime parts

- Datacenter ID and machine ID are chosen at **startup** and normally stay fixed.
- Change them only with careful review. An accidental change can cause ID conflicts.
- Timestamp and sequence are produced at **runtime**.

### Timestamp

- The 41 timestamp bits are the most significant part. Time grows, so IDs sort by time.
- Decoding example (Figure 7-7):

```
0-00100010101001011010011011000101101011000-01010-01100-000000000000
        41-bit timestamp → decimal 297616116568
        + Twitter epoch 1288834974657 = 1586451091225 ms
        → Apr 09 2020 16:51:31 UTC
```

- Max value: 2^41 − 1 = 2,199,023,255,551 ms ≈ **69 years**.
- A custom epoch close to today delays overflow. After 69 years you need a new epoch or an ID migration.

### Sequence number

- 12 bits = 2^12 = **4,096** values.
- It stays 0 unless one server makes more than one ID in the same ms.
- So one machine can make up to 4,096 IDs per ms in theory. That is far above 10,000/s.

**How the options compare** (summary of the book's pros/cons):

| Option | 64-bit numeric | Time-ordered | Scales / no SPOF |
|---|---|---|---|
| Multi-master | Yes | No | Hard across DCs, hard to add/remove servers |

| Option | 64-bit numeric | Time-ordered | Scales / no SPOF |
|---|---|---|---|
| UUID | No (128-bit, may be non-numeric) | No | Yes |
| Ticket server | Yes | Not discussed | No, single point of failure |
| Snowflake | Yes | Yes | Yes |

## Wrap-up

Snowflake is chosen because it meets all the use cases and scales in a distributed setup. Extra talking points:

- **Clock synchronization.** The design assumes all ID servers share one clock. That may be false across cores or machines. NTP is the most popular fix. The book leaves the details out of scope.
- **Section length tuning.** Fewer sequence bits and more timestamp bits suit low-concurrency, long-lived apps.
- **High availability.** The ID generator is mission-critical, so it must be highly available.

✓ KEY POINTS

- Start by saying why `auto_increment` fails: one DB is not enough, and many DBs can't agree cheaply.
- Walk through the 4 options and reject each with one requirement it breaks.
- Draw the 1/41/5/5/12 layout and explain each field.
- Quote the numbers: ~69 years of timestamps, 4,096 IDs per ms per machine, 32 × 32 machines.
- Point out the IDs sort by time because the timestamp is in the high bits.
- End with clock sync (NTP), bit tuning, and high availability.

⚠ COMMON TRAP

Picking UUID because it is "simple and unique". It fails two hard requirements: 64 bits and time ordering. Also, forgetting that snowflake depends on synced clocks.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·8 URL Shortener

URL shortener TinyURL short URL 301 vs 302 redirect base 62 hash collision CRC32 MD5 SHA-1 bloom filter unique ID generator read-heavy cache redirect hashValue length

SEE ALSO Web & Networking Fundamentals · SQL vs NoSQL & Scaling Databases

**Source:** *System Design Interview* Vol 1, Ch 8. · Your write-up: URL Shortener

## Problem

↑ TOP

Design a URL shortening service like TinyURL. Given a long URL, return a short alias. Clicking the alias redirects to the original URL.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Example? | `https://www.systeminterview.com/q=chatsystem&c=loggedin&v=v3&l=long` → `https://tinyurl.com/y7keocwj` |
| Traffic? | 100 million URLs generated per day |
| How short? | As short as possible |
| Allowed characters? | `0-9`, `a-z`, `A-Z` |
| Can short URLs be deleted or updated? | No (keep it simple) |

**Requirements:**

- Functional: shorten (long URL → short URL); redirect (short URL → original URL).
- Non-functional: high availability, scalability, fault tolerance.

**Estimates:**

- Writes: 100M / 24 / 3600 ≈ **1,160/s**.
- Reads (10:1 read:write): **11,600/s**.
- Records over 10 years: 100M × 365 × 10 = **365 billion**.
- Average URL length: 100 bytes.
- Storage over 10 years: the book says **365 TB**. (Note: 365B × 100 bytes is ~36.5 TB; the book multiplies by 10 years twice. Either way, it is not small.)

> Try it first: write the two APIs, decide 301 or 302, compute the short-code length, and pick how to turn a long URL into that code.

SOLUTION

## High-level design

### API (REST):

```
POST api/v1/data/shorten
  request:  {longUrl: longURLString}
  returns:  shortURL

GET api/v1/shortUrl
  returns:  longURL for HTTP redirection
```

### Redirect flow:

```
Client ── GET https://tinyurl.com/qtj5opu ──▶ tinyurl server
Client ◀── 301, Location: <long URL> ──────── tinyurl server
Client ── GET <long URL> ──────────────────▶ amazon.com server
```

### 301 vs 302:

| | 301 Moved Permanently | 302 Found (temporary) |
|---|---|---|
| Browser caches it? | Yes | No |
| Later requests | Go straight to the long URL | Go through the shortener again |
| Best for | Lower server load | Analytics (click rate, click source) |

- Simplest model: a hash table of `<shortURL, longURL>`; redirect = `hashTable.get(shortURL)`.
- Short URL format: `www.tinyurl.com/{hashValue}`.
- The hash function must map each long URL to one hashValue, and each hashValue must map back to its long URL.

## Deep dives

### Data model

- A hash table in memory is too costly at this size.
- Store the mapping in a **relational DB** table `url`:

```
url
  id        PK, auto increment
  shortURL
  longURL
```

### Hash value length

- 62 characters: 10 digits + 26 lower + 26 upper.
- Find the smallest n with 62^n ≥ 365 billion.

| n | 62^n |
|---|---|
| 5 | 916,132,832 |
| 6 | 56,800,235,584 |
| 7 | 3,521,614,606,208 ≈ 3.5 trillion |

- So **n = 7**.

### Option A: hash + collision resolution

- Use CRC32, MD5 or SHA-1. Even the shortest is too long for `https://en.wikipedia.org/wiki/Systems_design`:

| Hash | Hex value |
|---|---|
| CRC32 | `5cb54054` |
| MD5 | `5a62509a84df9ee03fe1230b9df8b84e` |
| SHA-1 | `0eeae7916c06853901d9ccbefbfcaf4de57ed85b` |

- Take the first 7 characters. That can collide. On a collision, append a predefined string to the long URL and hash again, until there is no collision:

```
longURL → hash → first 7 chars → exists in DB?
   ▲                                  │ yes (collision)
   └──── longURL + predefined ◄──────┘
                                      │ no
                                   save to DB
```

- Cost: a DB lookup per request. A **bloom filter** (fast, space-efficient, probabilistic set check) makes this cheaper.

### Option B: base 62 conversion

- Map digits: 0–9 → `0-9`, 10–35 → `a-z`, 36–61 → `A-Z`.

- Example: $11157 = 2 \times 62^2 + 55 \times 62^1 + 59 \times 62^0 \rightarrow [2, 55, 59] \rightarrow$ `2TX`.

```
11157 / 62 = 179  rem 59 → X
  179 / 62 = 2    rem 55 → T
    2 / 62 = 0    rem 2  → 2      → read upward: 2TX
```

- Short URL: `https://tinyurl.com/2TX`.

### Comparison

| Hash + collision resolution | Base 62 conversion |
|---|---|
| Fixed short URL length | Length grows with the ID |
| No unique ID generator needed | Needs a unique ID generator |
| Collisions possible; must resolve | No collisions; IDs are unique |
| Next short URL can't be guessed | Next URL is easy to guess if IDs go +1 (security concern) |

**Book's choice: base 62.**

### URL shortening flow

```
1. input longURL
2. longURL in DB? ── yes ──▶ 3. return existing shortURL
        │ no
4. generate a new unique ID (distributed ID generator, see Ch 7)
5. convert ID to shortURL with base 62
6. save (id, shortURL, longURL) to DB
```

- Example: `https://en.wikipedia.org/wiki/Systems_design` → ID `2009215674938` → `zn9edcu`.
- Stored row: `id=2009215674938, shortURL=zn9edcu, longURL=https://en.wikipedia.org/wiki/Systems_design`.

### URL redirecting flow

```
User ─(1) GET tinyurl.com/zn9edcu─▶ Load balancer ─(2)─▶ Web servers
                                                          │(3)      │(4)
                                                        Cache    Database
User ◀──(5) return long URL ──────────────────────────────┘
```

1. User clicks `https://tinyurl.com/zn9edcu`.
2. The load balancer forwards to a web server.
3. If the short URL is in the cache, return the long URL.

4. On a cache miss, read the DB. If it is not in the DB, the short URL is probably invalid.
5. Return the long URL to the user.

- Reads far outnumber writes, so the cache is the key performance piece.

## Wrap-up

- **Rate limiter:** stop abusers who flood the shorten API (filter by IP or other rules; see Ch 4).
- **Web tier scaling:** it is stateless, so add or remove servers freely.
- **DB scaling:** replication and sharding.
- **Analytics:** how many clicks, and when.
- **Availability, consistency, reliability** (Ch 1).

✓ KEY POINTS

- Do the math first: 1,160 writes/s, 11,600 reads/s, 365B URLs → 7 base-62 chars.
- Explain 301 vs 302 and pick based on load vs analytics.
- Compare hash+collision with base 62; choose base 62 with a distributed ID generator.
- Check "long URL already exists?" before creating a new row, so one long URL keeps one short URL.
- Put a cache in front of the DB for redirects.
- Mention rate limiting and the guessable-ID security concern.

⚠ COMMON TRAP

Truncating MD5/SHA-1 to 7 chars and ignoring collisions. Also, using 301 when the product needs click analytics: the browser caches it and you never see repeat clicks.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·9 Web Crawler

web crawler · spider · URL frontier · politeness · front queues · back queues · BFS vs DFS · robots.txt · DNS cache · content seen · URL seen · bloom filter · spider trap · duplicate content · freshness · PageRank

SEE ALSO Web & Networking Fundamentals

**Source:** *System Design Interview* Vol 1, Ch 9.

## Problem

↑ TOP

Design a web crawler for a search engine. It starts from some URLs, downloads the pages, extracts links, and keeps going. Other uses are web archiving, web mining and web monitoring.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Main purpose? | Search engine indexing |
| Pages per month? | 1 billion |
| Content types? | HTML only |
| New or edited pages? | Yes, handle them |
| Store crawled HTML? | Yes, up to 5 years |
| Duplicate content? | Ignore pages with duplicate content |

**Requirements:**

- Functional: crawl 1B HTML pages/month; pick up new and edited pages; store HTML for 5 years; skip duplicates.
- **Scalability:** the web is huge, so crawl in parallel.
- **Robustness:** handle bad HTML, dead servers, crashes, malicious links.
- **Politeness:** don't send too many requests to one site in a short time.
- **Extensibility:** add new content types (e.g. images) without a redesign.

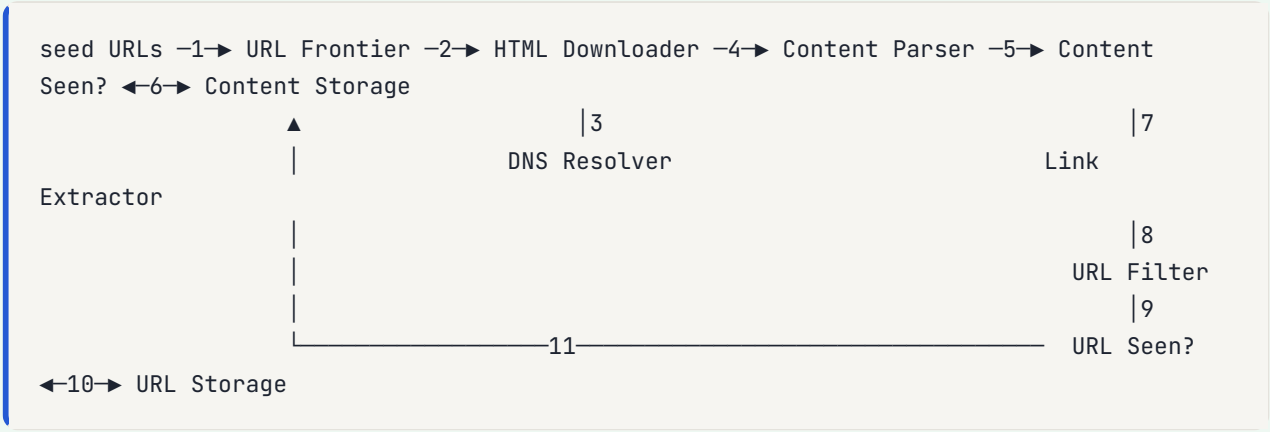
**Estimates:**

- QPS: 1,000,000,000 / 30 days / 24 h / 3600 s ≈ **400 pages/s**. Peak = 2 × = **800**.
- Average page: 500 KB → 1B × 500 KB = **500 TB/month**.
- 5 years: 500 TB × 12 × 5 = **30 PB**.

> Try it first: draw the crawl loop as boxes (where URLs wait, where pages are fetched, where duplicates are dropped), then think how to stop one host from being flooded.

SOLUTION

## High-level design




- **Seed URLs:** the starting points. Split the URL space by locality (country) or by topic (shopping, sports, ...). No perfect answer; think out loud.
- **URL Frontier:** URLs still to download. Think of it as a FIFO queue (refined in the deep dive).
- **HTML Downloader:** fetches pages; asks the **DNS Resolver** for the IP first.
- **Content Parser:** parses and validates pages. It is a separate component so it doesn't slow down crawl servers.
- **Content Seen?:** drops pages whose content was already stored. Compare **hashes** of pages, not characters. About 29% of web pages are duplicates.
- **Content Storage:** mostly on disk (too big for memory); popular content in memory for low latency.
- **URL Extractor:** pulls links; turns relative paths into absolute URLs (e.g. add `https://en.wikipedia.org`).
- **URL Filter:** drops some content types, file extensions, error links and blacklisted sites.
- **URL Seen?:** skips URLs already visited or already in the frontier. Prevents extra load and infinite loops. Built with a **bloom filter** or **hash table**.
- **URL Storage:** stores visited URLs.

Workflow: (1) seeds go into the frontier → (2) downloader takes URLs → (3) resolves DNS and downloads → (4) parser validates → (5–6) Content Seen? drops known content → (7) links extracted → (8) filtered → (9–10) URL Seen? drops known URLs → (11) new URLs go back into the frontier.

## Deep dives

### DFS vs BFS

- The web is a directed graph: pages are nodes, links are edges.
- DFS is a bad fit because the depth can be huge. Crawlers use **BFS** with a FIFO queue.
- Plain FIFO BFS has two problems:
- **Impolite:** most links on a page point to the same host (e.g. all internal wikipedia links). Parallel downloads then flood that host.
- **No priority:** every URL is treated the same, but pages differ in quality and importance.

### URL frontier

The frontier fixes both problems. It handles **politeness**, **priority** and **freshness**.

**Politeness (back queues)**

- Download one page at a time per host, with a delay between downloads.
- Keep a map from hostname to a queue; each worker thread reads from one queue.

```
        Queue router ──► Mapping table (host → queue)
       /     |      \       wikipedia.com → b1
    b1     b2 ...    bn     apple.com     → b2
       \     |      /       nike.com      → bn
        Queue selector
       /     |      \
  Worker 1  Worker 2 ... Worker N   (one host each, one page at a time)
```

- Queue router: puts only one host's URLs in each queue b1…bn.
- Queue selector: maps each worker thread to one queue.

**Priority (front queues)**

- A random forum post about Apple matters less than the Apple home page.
- **Prioritizer** scores URLs by usefulness: PageRank, web traffic, update frequency, etc.
- Queues f1…fn each have a priority. The queue selector picks randomly, biased toward high-priority queues.

**Full frontier = front queues + back queues:**

```
input URLs → Prioritizer → f1 … fn → Front queue selector
           → Back queue router (+ mapping table) → b1 … bn → Back queue selector
           → Worker threads
```

- Front queues manage **priority**. Back queues manage **politeness**.

**Freshness**

- Pages are added, edited and deleted, so recrawl. Recrawling everything is too costly.
- Recrawl based on each page's update history.
- Recrawl important pages first and more often.

**Frontier storage**

- Real frontiers hold hundreds of millions of URLs. All in memory is not durable or scalable. All on disk is too slow.
- **Hybrid:** most URLs on disk; in-memory buffers for enqueue/dequeue, flushed to disk periodically.

### HTML downloader

**Robots.txt** (Robots Exclusion Protocol)

- Sites list what crawlers may fetch. Check it before crawling a site and follow it.
- Cache the file and re-download it periodically.
- Example from amazon.com: `User-agent: Googlebot`, `Disallow: /creatorhub/*`, `Disallow: /rss/people/*/reviews`, ...

**Performance optimizations**

1. **Distributed crawl:** many servers, many threads each. Partition the URL space so each downloader owns a subset.
2. **DNS cache:** DNS takes 10–200 ms and many DNS interfaces are synchronous, so one lookup can block other threads. Keep a domain → IP cache, refreshed by cron jobs.
3. **Locality:** place crawl servers (and cache, queue, storage) near the website hosts.
4. **Short timeout:** set a max wait. If a host doesn't answer in time, move on to other pages.

### Robustness

- **Consistent hashing** spreads load across downloaders and makes adding/removing one easy (Ch 5).
- **Save crawl state and data** so a failed crawl restarts from saved state.
- **Exception handling:** errors are normal at scale; don't crash.
- **Data validation** to prevent system errors.

### Extensibility

- Plug in new modules after "Content Seen?", next to the Link Extractor.
- Examples: a **PNG Downloader** module for PNG files; a **Web Monitor** module for copyright and trademark checks.

### Problematic content

| Problem | Fix |
|---|---|
| Redundant content (~30% duplicates) | Hashes or checksums |
| Spider traps (e.g. `/foo/bar/foo/bar/...` forever) | Max URL length; spot sites with an unusually large page count; manual review → exclude site or custom URL filter |
| Data noise (ads, code snippets, spam URLs) | Exclude when possible |

## Wrap-up

- **Server-side (dynamic) rendering:** JavaScript/AJAX builds links on the fly. Render the page before parsing to see them.
- **Filter unwanted pages:** an anti-spam component saves storage and crawl resources.
- **DB replication and sharding** for the data layer.
- **Horizontal scaling:** hundreds or thousands of download servers; keep them **stateless**.
- **Availability, consistency, reliability** (Ch 1).
- **Analytics** for fine-tuning.

✓ KEY POINTS

- Clarify scope first: purpose, 1B pages/month, HTML only, 5-year storage, dedup.
- State the four qualities: scalable, robust, polite, extensible.
- Draw the 11-step loop and name both dedup checks (content hash and URL seen).
- Explain the frontier: front queues for priority, back queues for one-host-per-worker politeness.
- List the speed-ups: distributed crawl, DNS cache, locality, short timeouts; and obey robots.txt.
- Mention spider traps and freshness (recrawl by update history and importance).

⚠ COMMON TRAP

Using a plain BFS queue with many threads. Most links point to the same host, so you end up hammering one site. Also, forgetting DNS as a bottleneck.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·10 Notification System

notification system · push notification · APNs · FCM · SMS · email · message queue · workers · retry · dedupe · notification template · opt-out · rate limiting · analytics · soft real-time

SEE ALSO Kafka vs RabbitMQ · Celery with Redis as Broker

**Source:** *System Design Interview* Vol 1, Ch 10. · Your write-up: Notification System

## Problem

↑ TOP

Design a scalable system that sends notifications to users as mobile push, SMS, and email. It must send millions of notifications a day.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Which notification types? | Push, SMS, and email |
| Real-time? | Soft real-time: as soon as possible, but a slight delay under high load is OK |
| Which devices? | iOS, Android, and laptop/desktop |
| What triggers a notification? | Client apps, or a schedule on the server side |
| Can users opt out? | Yes. Opted-out users get nothing |
| How many per day? | 10M push, 1M SMS, 5M email |

**Requirements:**

- Functional: send push (iOS/Android), SMS, and email. Accept triggers from other services and from schedules. Respect opt-out.
- Non-functional: soft real-time, highly reliable (no lost notifications), scalable, and easy to add or remove a third-party provider.

**Estimates:** 16M notifications per day in total (10M push, 1M SMS, 5M email). The book gives no further math.

> Try it first: sketch how one internal service's request becomes an iOS push, an SMS and an email. Then add what keeps one slow provider from blocking the rest, and what happens when a send fails.

**SOLUTION**

### High-level design

**How each channel works.** We never deliver to devices ourselves. A third-party service does it.

- **iOS push:** Provider → **APNs** → iOS device. The provider sends a **device token** (the unique device ID) and a **JSON payload**, e.g. `{"aps": {"alert": {"title": "Game Request", "body": "Bob wants to play chess", "action-loc-key": "PLAY"}, "badge": 5}}`.
- **Android push:** Provider → **FCM** → Android device.
- **SMS:** Provider → an SMS service such as **Twilio** or **Nexmo** → phone.
- **Email:** Provider → a commercial email service such as **SendGrid** or **Mailchimp** → inbox. These give better delivery rates and analytics than running your own mail servers.

**Contact info gathering.** When a user installs the app or signs up, the API servers store their contact info.

```
user:   user_id bigint, email varchar, country_code int, phone_number int, created_at
timestamp
device: id bigint, device_token varchar, user_id bigint, last_logged_in_at timestamp
        (one user → many devices; push goes to all of them)
```

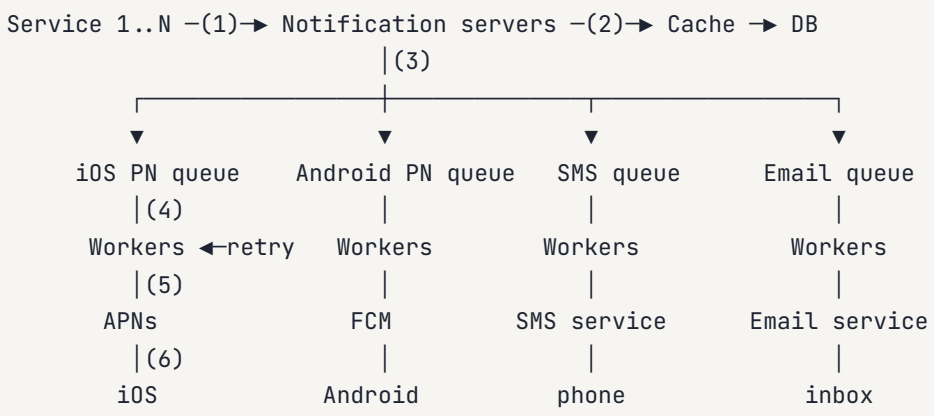
**Send API** (internal or verified clients only, to prevent spam). The book's email example (its URL says `sms`, but the body is an email):

```
POST https://api.example.com/v/sms/send
{ "to": [{"user_id": 123456}],
  "from": {"email": "from_address@example.com"},
  "subject": "Hello, World!",
  "content": [{"type": "text/plain", "value": "Hello, World!"}] }
```

**Initial design:** Service 1..N → one notification server → APNs/FCM/SMS/email → devices. It has three problems:

- **Single point of failure:** one server.
- **Hard to scale:** the DB, the cache and the processing parts cannot scale independently.
- **Performance bottleneck:** building HTML and waiting on third parties is slow. One box overloads at peak.

**Improved design** (Figure 10-10): move the DB and cache out, add more notification servers with auto horizontal scaling, and decouple with message queues.




1. A service calls the notification servers' API.
2. The servers validate (email, phone number), then fetch user info, device tokens and settings from the cache or DB.
3. The servers put an event on the queue for that channel (e.g. iOS push → iOS PN queue).
4. Workers pull events from the queues.
5. Workers call the third-party service.
6. The third party delivers to the device.

- **One queue per channel:** an outage at one provider does not affect the other channels. The queues also buffer traffic spikes.
- **Cache:** user info, device info, and notification templates. **DB:** users, notifications, settings.
- **Extensibility:** it must be easy to plug a provider in or out. A provider may not exist in a new market. For example, FCM is not available in China, so you use Jpush or PushY there.

## Deep dives

### Reliability: no data loss

- Notifications may be delayed or reordered, but **never lost**.
- Workers write every notification to a **notification log DB** (Figure 10-11) and use a retry mechanism.

### Reliability: exactly once?

- **No.** Most of the time a notification arrives once, but distribution can cause duplicates.
- **Dedupe:** when an event arrives, check its **event ID**. If it was seen before, drop it. Otherwise send it.
- This makes duplicates rare. It does not make exactly-once possible.

### Notification template

- Most notifications share a format. A template is a preformatted message with parameters, styling and tracking links.

- Example: `BODY: You dreamed of it. We dared it. [ITEM NAME] is back — only until [DATE]. CTA: Order Now. Or, Save My [ITEM NAME]`
- Benefits: a consistent format, fewer errors, and less time spent.

#### Notification setting

- Users get fine-grained control. Table: `user_id bigint, channel varchar (push/email/SMS), opt_in boolean`.
- **Before sending, check that the user opted in** to this channel.

#### Rate limiting

- Cap how many notifications one user receives. If we send too often, users turn notifications off completely.

#### Retry mechanism

- If a third party fails, the worker puts the notification **back on the queue** and retries a set number of times.
- If the failure persists, **alert the developers**.

#### Security

- Push APIs are secured with an **appKey/appSecret** pair. Only authenticated or verified clients can send.

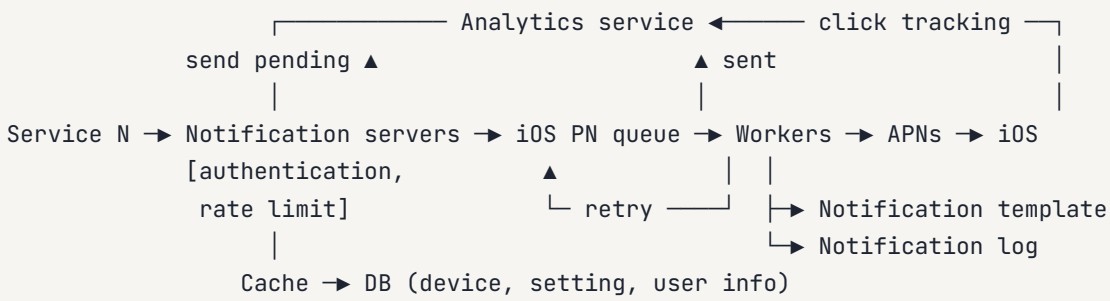
#### Monitor queued notifications

- Key metric: **the total number of queued notifications**. If it is large, workers are too slow. Add more workers.

#### Events tracking

- An analytics service tracks open rate, click rate and engagement.
- Event lifecycle (Figure 10-13): `start → pending → sent → deliver → click | unsubscribe`. `pending` and `sent` can go to `error`.

### Updated design (Figure 10-14)




- Notification servers gain **authentication** and **rate limiting**.
- Failed sends go back to the queue, and workers retry a predefined number of times.
- **Templates** make creating notifications consistent and fast.
- **Monitoring and tracking** give health checks and data for improvements.

## Wrap-up

- We support push, SMS and email, and use message queues to decouple the parts.
- **Reliability:** a robust retry mechanism keeps the failure rate low.
- **Security:** an appKey/appSecret pair lets only verified clients send.
- **Tracking and monitoring** at every stage of the flow.
- **Respect user settings:** check opt-out before sending.
- **Rate limiting:** cap the frequency per user.

✓ KEY POINTS

- Third parties deliver; your job is reliable, decoupled hand-off to them.
- Give each channel its own queue and worker pool, so one provider's outage stays isolated.
- Persist to a notification log and retry from the queue: "delayed or reordered, never lost."
- Say exactly-once delivery is impossible, then show the event-ID dedupe.
- Gate every send on opt-in settings and a per-user rate limit.
- Watch queue depth, and scale workers when it grows.

⚠ COMMON TRAP

one notification server that calls APNs/SMS/email inline. It is a single point of failure, and one slow provider stalls everything. Put queues between the servers and the senders.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·11 News Feed System

news feed · timeline · Facebook feed · Twitter timeline · Instagram feed · fanout on write · fanout on read · push vs pull · hybrid fanout · celebrity problem · hotkey · graph database · news feed cache · hydration · CDN

SEE ALSO The 4-Step Framework (Alex Xu, with a News Feed Example)

**Source:** *System Design Interview* Vol 1, Ch 11. The worked example in The 4-Step Framework uses this same design.

## Problem

↑ TOP

Design a news feed like Facebook's: a constantly updating list of posts from friends on the home page. Similar asks: Instagram feed, Twitter timeline.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Mobile app, web app, or both? | Both |
| Most important features? | Publish a post, and see friends' posts on the news feed page |
| Feed order: reverse chronological or ranked (e.g. close friends score higher)? | Reverse chronological, to keep it simple |
| How many friends can a user have? | 5,000 |
| Traffic? | 10M DAU |
| Text only, or media? | Images and videos too |

**Requirements:**

- Functional: publish a post; view a feed of friends' posts, newest first. Posts can hold images and videos.
- Non-functional: fetching the feed must be fast. Scale to 10M DAU and users with up to 5,000 friends.

**Estimates:** the book does no math here. Work from 10M DAU and up to 5,000 friends, which means one post can fan out to 5,000 feeds.

> Try it first: sketch the two flows (publish a post, read a feed). Then decide when a post gets copied into friends' feeds: at write time or at read time.

SOLUTION

## High-level design

**APIs** (HTTP):

```
POST /v1/me/feed   params: content (post text), auth_token
GET  /v1/me/feed   params: auth_token
```

**Feed publishing** (Figure 11-2):

```
User ──POST /v1/me/feed?content=Hello&auth_token=…──► Load balancer ► Web servers
   Web servers ─► Post service ─► Post cache ─► Post DB
               ─► Fanout service ─► News feed cache
               ─► Notification service
```

**News feed building** (Figure 11-3):

```
User ──GET /v1/me/feed──► Load balancer ► Web servers ► News feed service ► News feed
cache
```

- **Load balancer** spreads traffic over the web servers. **Web servers** route requests to internal services.
- **Post service** saves the post in the DB and the cache.
- **Fanout service** pushes the new post into friends' feeds, which live in a cache for fast reads.
- **Notification service** tells friends there is new content and sends push notifications.
- **News feed service** reads the feed from the **news feed cache**, which holds the IDs needed to render it.

## Deep dives

### Feed publishing (Figure 11-4)

```
User ─► Load balancer ─► Web servers [authentication, rate limiting]
          ├─► Post service ─► Post cache ─► Post DB
          ├─► Notification service
          └─► Fanout service ─(1) get friend IDs──► Graph DB
                    ├─(2) get friends' data──► User cache ─► User DB
                    └─(3)─► Message queue ─(4)─► Fanout workers ─(5)─► News feed cache
```

**Web servers** also enforce:

- **Authentication:** only signed-in users with a valid `auth_token` can post.

- **Rate limiting:** cap posts per user per time window, to stop spam and abuse.

**Fanout service.** Fanout delivers a post to all friends. The book compares two models:

| | Fanout on write (push) | Fanout on read (pull) |
|---|---|---|
| When the feed is built | At write time; post goes into friends' caches right away | At read time, on demand when the user loads the home page |
| Pros | Real-time; feed can be pushed to friends at once. Fast reads, since the feed is precomputed | No wasted work on inactive users. No hotkey problem, since nothing is pushed |
| Cons | **Hotkey problem:** a user with many friends makes fetching the list and writing all feeds slow. Wasted work for inactive users | Slow reads, since the feed is not precomputed |

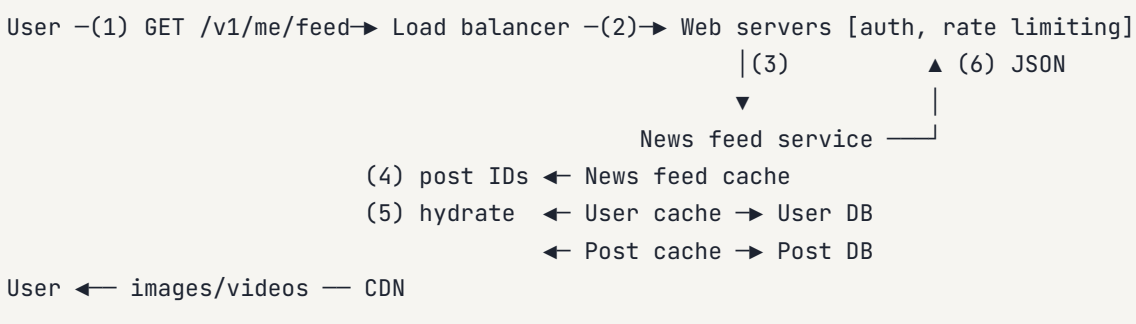
**The book's choice: hybrid.** Fast feed reads are crucial, so use **push for most users**. For **celebrities** or users with very many friends/followers, followers **pull** that content on demand, which avoids overload. **Consistent hashing** helps with hotkeys by spreading requests and data more evenly.

**Fanout steps** (Figure 11-5):

1. Get friend IDs from the **graph DB**. Graph DBs suit friend relations and friend recommendations.
2. Get friends' info from the **user cache**. Filter by user settings: a muted friend's posts stay hidden, and a post may be shared only with chosen friends.
3. Send the friend list and the new post ID to the **message queue**.
4. **Fanout workers** read from the queue.
5. Workers append `<post_id, user_id>` to the **news feed cache** (Figure 11-6).

- Store **only IDs**. Whole user and post objects would use too much memory.
- Keep a **configurable limit** on feed length. Few users scroll through thousands of posts; most want the latest. So the cache miss rate stays low.

### News feed retrieval (Figure 11-7)




1. The user sends `GET /v1/me/feed`.

2. The load balancer sends it to a web server.
3. The web server calls the news feed service.
4. The service gets a list of **post IDs** from the news feed cache.
5. A feed needs more than IDs: username, profile picture, post text, post image. The service **hydrates** the feed with full user and post objects from the user cache and post cache.
6. The fully hydrated feed goes back to the client as **JSON**.

Media (images, videos) is stored in a **CDN** for fast retrieval.

### Cache architecture (Figure 11-8)

The cache is central to a feed system. The book splits it into 5 layers:

| Layer | Holds |
|---|---|
| News feed | IDs of news feed items |
| Content | Every post's data. Popular posts go in a **hot cache**, the rest in normal |
| Social graph | Relationships: follower, following |
| Action | Whether a user liked, replied to, or did something else to a post |
| Counters | Like, reply, follower, following, and other counts |

## Wrap-up

There is no perfect design; fit it to the company's constraints and know your trade-offs. If time is left, talk about scale.

- **Scaling the database:** vertical vs horizontal scaling, SQL vs NoSQL, master-slave replication, read replicas, consistency models, sharding.
- **Other points:** keep the web tier stateless, cache as much as you can, support multiple data centers, loosely couple components with message queues.
- **Monitor key metrics:** for example, peak-hour QPS and the latency when users refresh their feed.

✓ KEY POINTS

- Split the problem into two flows, publishing and retrieval, and design each one.
- Compare push and pull out loud, then choose **hybrid**: push for normal users, pull for celebrities.
- Fan out **async** through a message queue and workers, so the post request returns fast.
- Store only IDs in the feed cache. Cap its length, and hydrate from the user and post caches at read time.
- Put authentication and rate limiting at the web servers, and media on a CDN.

> ⚠ COMMON TRAP
> pure fanout on write. One celebrity post then becomes millions of cache writes (the hotkey problem). Say "hybrid" and explain why.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·12 Chat System

chat system messenger WhatsApp WebSocket long polling presence heartbeat service discovery ZooKeeper key-value store message ID message sync queue group chat multi-device sync push notification

SEE ALSO Web & Networking Fundamentals · Redis Pub/Sub vs RabbitMQ vs Kafka

**Source:** *System Design Interview* Vol 1, Ch 12. · Your write-up: Chat System

## Problem

↑ TOP

Design a chat app like Facebook Messenger. It must support 1-on-1 chat, small group chat, and online status.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| 1-on-1 or group? | Both |
| Mobile, web, or both? | Both |
| Scale? | 50M DAU |
| Group member limit? | 100 people |
| Key features? Attachments? | 1-on-1 chat, group chat, online indicator. Text only |
| Message size limit? | Text under 100,000 characters |
| End-to-end encryption? | Not now; discuss if time allows |
| How long to keep history? | Forever |

**Requirements:**

- Functional: 1-on-1 chat with low delivery latency; small group chat (max 100); online presence; multiple devices per account logged in at once; push notifications.
- Non-functional: 50M DAU, low latency, chat history stored forever.

**Estimates:** if each connection needs ~10 KB of server memory, 1M concurrent users need only ~10 GB. In theory one box could hold them all, but a single server is a **single point of failure**. Start with one server only if you say it is a starting point. Messenger and WhatsApp together process ~60B messages a day.

> Try it first: decide how the server pushes a message to a receiver who sent no request. Then sketch where messages are stored, and how a sender's message reaches an online receiver and an offline one.

SOLUTION

## High-level design

**The chat service must:** receive messages from clients, find the recipients and relay to them, and hold messages for offline recipients until they come online.

**Protocol choice.** Sending is client-initiated, so HTTP with keep-alive works (early Facebook used it). Receiving is the hard part, because the server must push:

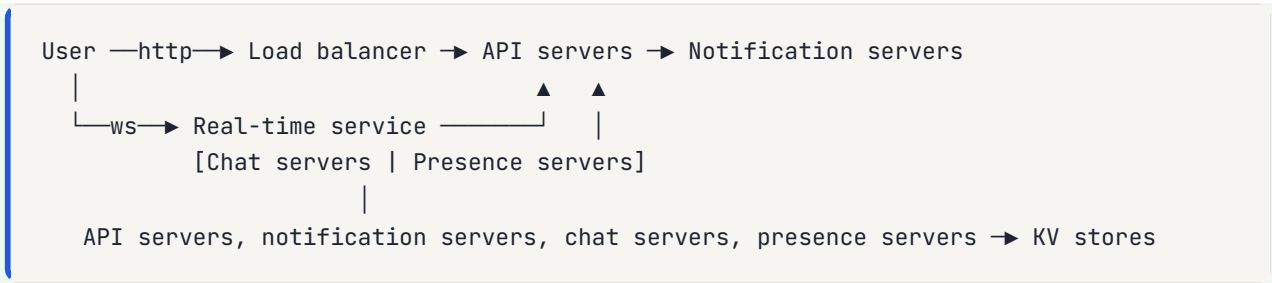
| Technique | How | Problems |
|---|---|---|
| Polling | Client asks "anything new?" on a timer | Wastes server resources; most answers are "no" |
| Long polling | Client holds the request open until a message arrives or a timeout, then reconnects | Sender and receiver may hit different stateless servers; the server cannot tell when a client disconnected; still reconnects after timeouts for quiet users |
| **WebSocket** | Starts as HTTP, then upgrades. Bi-directional and persistent | Connections are stateful, so you must manage them carefully on the server |

**Choice: WebSocket for both sending and receiving.** It uses ports 80/443, so it usually passes firewalls. One protocol on both sides keeps the client and server simpler. Everything else (signup, login, profile) stays plain HTTP request/response.

**Three groups of services** (Figure 12-7):

- **Stateless** (behind a load balancer, HTTP): service discovery, authentication, group management, user profile. Service discovery gives the client the DNS host names of the chat servers it can use.
- **Stateful:** the **chat service**. Each client keeps a persistent connection to one chat server and stays on it while that server is up.
- **Third party: push notification**, to reach users when the app is not running (see Ch 10).

**Adjusted design** (Figure 12-8):




- **Chat servers** send and receive messages. **Presence servers** manage online/offline status.
- **API servers** handle login, signup, profile changes and the rest.
- **Notification servers** send push notifications.
- **KV store** holds chat history, so a user who comes online sees all past messages.

**Storage.** There are two kinds of data:

- **Generic data** (profiles, settings, friend lists): a relational DB, with replication and sharding.
- **Chat history:** huge volume; mostly recent chats are read, but random access is needed for search, mentions and jump-to-message; the read/write ratio is about 1:1 for 1-on-1 chat.

Choice for chat history: a **key-value store**. It scales horizontally with ease and has very low latency. Relational DBs handle the long tail of data badly: when indexes grow large, random access is expensive. Proven in practice: Facebook Messenger uses **HBase**, Discord uses **Cassandra**.

**Data models:**

```
message (1-on-1):   message_id bigint PK, message_from bigint, message_to bigint,
                    content text, created_at timestamp
group_message:      channel_id bigint, message_id bigint   PK = (channel_id,
message_id)
                    user_id bigint, content text, created_at timestamp
                    channel_id is the partition key: every group query is per channel
```

- Order by `message_id`, not `created_at`: two messages can have the same timestamp.

**Message ID.** It must be **unique** and **sortable by time** (newer rows get higher IDs). Options:

| Option | Verdict |
|---|---|
| `auto_increment` (MySQL) | NoSQL stores usually do not have it |
| Global 64-bit generator like **Snowflake** (Ch 7) | Works |
| **Local sequence number**, unique only within a channel | Enough, since order only matters inside one 1-on-1 or group channel. Easier than a global ID |

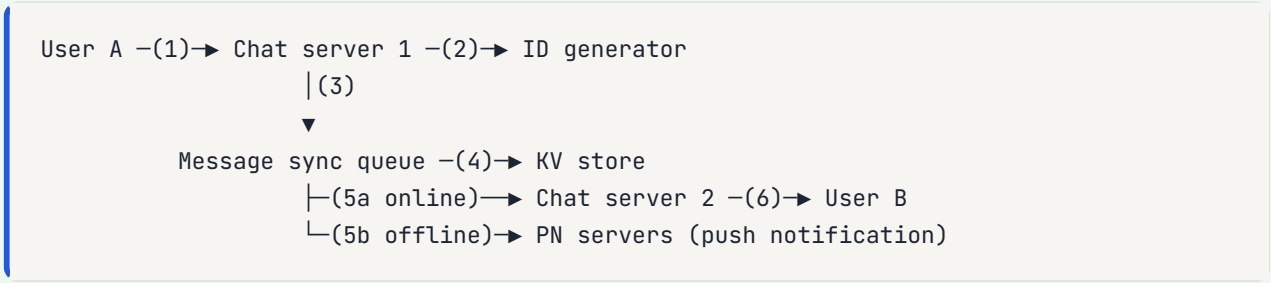
## Deep dives

### Service discovery (Figure 12-11)

It picks the best chat server for a client by criteria such as location and server capacity. **Apache ZooKeeper** registers all chat servers and chooses one.

1. User A logs in.
2. The load balancer sends the login to the API servers.
3. After authentication, service discovery picks a chat server (say server 2) and returns its info.
4. User A opens a WebSocket to chat server 2.

### 1-on-1 message flow (Figure 12-12)




1. User A sends a message to chat server 1.
2. Chat server 1 gets a message ID from the ID generator.
3. It sends the message to the message sync queue.
4. The message is stored in the KV store.
5. If B is online, the message goes to chat server 2, where B is connected. If B is offline, the PN servers send a push notification.
6. Chat server 2 sends it to B over B's WebSocket.

### Multi-device sync (Figure 12-13)

- User A's phone and laptop each hold a WebSocket to chat server 1.
- Each device keeps its own `cur_max_message_id`: the latest message ID it has (e.g. phone 653, laptop 842).
- A message is new for a device if its **recipient ID = the logged-in user** and its **message ID > cur_max_message_id**.
- So each device just pulls its new messages from the KV store.

### Small group chat (Figures 12-14, 12-15)

- When A sends to a group {A, B, C}, the message is **copied into each member's message sync queue**: one for B, one for C.
- Think of the sync queue as each **recipient's inbox**. It holds messages from many senders and groups.

- Good for small groups: each client checks only its own inbox, and a copy per member is cheap when groups are small.
- WeChat does this and caps groups at **500**. For very large groups, a copy per member is not acceptable.

#### Online presence

- **Login:** after the WebSocket is up, save A's `online` status and `last_active_at` in the KV store.
- **Logout:** set the status to `offline` in the KV store.
- **Disconnection:** marking offline on every drop makes the indicator flicker (e.g. a user in a tunnel). Use a **heartbeat** instead. The client sends one periodically, e.g. **every 5 s**. If no heartbeat arrives within **x seconds** (the book's example: **30 s**), the user is offline.
- **Status fanout:** presence servers use **pub/sub with one channel per friend pair**. A's change is published to channels A-B, A-C and A-D, which B, C and D subscribe to. Clients get updates over WebSocket.
- This works for small groups (WeChat caps at 500). For a group of 100,000 members, each change would make 100,000 events. Instead, **fetch status only when a user enters a group or refreshes the friend list**.

### Wrap-up

The design: WebSocket for real-time traffic, chat servers, presence servers, push notification servers, KV stores for history, and API servers for the rest. Extra talking points:

- **Media files:** much larger than text. Discuss compression, cloud storage and thumbnails.
- **End-to-end encryption:** as in WhatsApp, only sender and recipient can read messages.
- **Client-side message cache** to cut data transfer.
- **Faster load time:** Slack built a geo-distributed cache for users' data and channels.
- **Chat server failure:** a server may hold hundreds of thousands of connections. If it dies, service discovery (ZooKeeper) gives clients a new server to reconnect to.
- **Message resend:** use retry and queueing.

✓ KEY POINTS

- Compare polling, long polling and WebSocket, then pick WebSocket for both directions and keep HTTP for everything else.
- Split services into stateless (behind a load balancer), stateful (chat servers, chosen by service discovery) and third party (push).
- Store history in a KV store and justify it: scale, latency, long tail, a 1:1 read/write ratio, and HBase/Cassandra in production.
- Order by a time-sortable `message_id`, not a timestamp. A per-channel local sequence is enough.
- Presence = heartbeat with a timeout + pub/sub per friend pair. Fall back to fetch-on-open for huge groups.

⚠ COMMON TRAP

marking a user offline on every socket drop. Flaky networks make the indicator flicker. Use a heartbeat with a grace timeout.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·13 Search Autocomplete

search autocomplete typeahead top k trie prefix tree data gathering aggregation trie cache shard map manager browser cache data sampling query service trending queries

**Source:** *System Design Interview* Vol 1, Ch 13.

## Problem

↑ TOP

Design a search autocomplete system ("typeahead", or "top k most searched queries"). As the user types, show the most popular queries that start with what they typed.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Match only at the start of the query, or in the middle too? | Only at the start |
| How many suggestions? | 5 |
| How do we pick the 5? | By popularity: historical query frequency |
| Spell check / autocorrect? | No |
| Language? | English. Multi-language if time allows |
| Capitals and special characters? | No. All queries are lowercase a–z |
| Users? | 10M DAU |

**Requirements:**

- Functional: given a prefix, return the top 5 queries by frequency, sorted.
- Fast: results within **100 ms** (Facebook's figure), or typing stutters.
- Relevant and sorted by popularity (or another ranking model).
- Scalable to high traffic. Highly available when parts fail or slow down.

**Estimates:**

- 10M DAU, 10 searches per user per day.
- A query is 4 words × 5 chars = **20 bytes** (ASCII, 1 byte per char).
- One request per typed character: about **20 requests per search** (`search?q=d`, `search?q=di`, ... `search?q=dinner`).
- QPS = 10M × 10 × 20 / 86,400 ≈ **24,000**. Peak ≈ **48,000**.
- 20% of queries are new: 10M × 10 × 20 B × 20% = **0.4 GB new data per day**.

> Try it first: sketch the data structure that returns the top 5 for a prefix in O(1), and the pipeline that keeps it fresh without slowing reads.

SOLUTION

## High-level design

Two services:

- **Data gathering service:** collects user queries and counts them. The naive version updates a frequency table in real time.
- **Query service:** given a prefix, return the 5 most searched queries.

Naive version: one frequency table `(query, frequency)`. Example rows: twitter 35, twitch 29, twilight 25, twin peak 21, twitch prime 18, twitter search 14, twillo 10, twin peak sf 8. Typing "tw" returns the top 5 of these.

```sql
SELECT * FROM frequency_table
WHERE query LIKE 'prefix%'
ORDER BY frequency DESC
LIMIT 5;
```

- This works for a small data set.
- At scale the database becomes the bottleneck. The deep dive fixes both services.

## Deep dives

### Trie data structure

A trie (prefix tree, from "re**trie**val") stores strings compactly.

- The root is the empty string. Each node is one character and has up to 26 children.
- Each node is a word or a prefix. Add the **frequency** to nodes that end a real query.

Basic top-k algorithm (p = prefix length, n = nodes, c = children of a node):

| Step | Cost |
|---|---|
| 1. Find the prefix node | O(p) |
| 2. Traverse its subtree to collect all valid queries | O(c) |
| 3. Sort them and take top k | O(c log c) |

Example, k = 2, prefix "tr": valid nodes are tree:10, true:35, try:29 → return true, try.

This is too slow: in the worst case you walk the whole trie. Two optimizations:

1. **Limit the prefix length.** Users rarely type long queries. Cap p at about 50. Finding the prefix becomes O(1).
2. **Cache the top k queries at every node.** Node "be" stores `[best:35, bet:29, bee:20, be:15, beer:10]`. This costs a lot of space. Fast response is worth it.

After both: find prefix O(1) + return cached top k O(1) = **O(1)**.

### Data gathering service

Updating the trie on every query is impractical:

- Users send billions of queries a day. Constant updates slow down the query service.
- Top suggestions do not change much once the trie is built.

How fresh it must be depends on the product. Twitter needs near real time. Many Google keywords barely change day to day. The data source is the same either way: analytics logs.




- **Analytics logs:** raw `(query, time)` rows. Append-only, not indexed.
- **Aggregators:** turn huge raw logs into counts. Short windows for real-time apps; weekly is enough for many. Ask the interviewer. The book assumes a **weekly rebuild**.
- **Aggregated data:** `(query, time = week start, frequency)`, e.g. tree / 2019-10-01 / 12000.
- **Workers:** async jobs at fixed intervals. They build the trie and store it in Trie DB.
- **Trie Cache:** distributed in-memory cache of the trie. Takes a weekly snapshot of the DB.
- **Trie DB:** persistent storage. Two options:

| Option | How |
|---|---|
| Document store (e.g. MongoDB) | Serialize the weekly trie snapshot and store it |
| Key-value store | Each prefix is a key; the node's data (its top-k list) is the value |

Key-value example: `b → [be:15, bee:20, beer:10, best:35]`, `bee → [bee:20, beer:10]`, `best → [best:35]`.

### Query service

```
User (web / mobile)
   | 1
   ▼
Load balancer —2→ API servers —3→ Trie Cache —4 (on miss)→ Trie DB
```

1. The query goes to the load balancer.
2. It routes to an API server.
3. The API server reads trie data from Trie Cache and builds the suggestions.
4. On a cache miss, reload the data from Trie DB into the cache. Misses happen when a cache server is out of memory or offline.

Optimizations:

- **AJAX requests:** fetch results without reloading the page.
- **Browser caching:** suggestions change slowly. Google caches them in the browser for an hour: `Cache-Control: private, max-age=3600`. `private` = one user only, no shared caches.
- **Data sampling:** logging every query is expensive. Log only 1 out of every N requests.

### Trie operations

- **Create:** workers build it from aggregated data (from the analytics log/DB).
- **Update, option 1:** rebuild weekly and replace the old trie. This is the default.
- **Update, option 2:** update a single node in place. Slow, so avoid it. OK only if the trie is small. You must also update **every ancestor up to the root**, because ancestors cache the top queries of their children. Example: "beer" 10 → 30 changes "beer" in the node and in all its ancestors.
- **Delete:** remove hateful, violent, sexually explicit or dangerous suggestions. Put a **filter layer** between Trie Cache and API servers. Filter rules are flexible. Delete the bad data from the DB asynchronously, so the next weekly build is clean.

```
Trie Cache —→ Filter Layer —→ API servers
```

### Scale the storage

When the trie does not fit on one server, shard it.

- **First-level sharding by first letter:** 2 servers = a–m, n–z. 3 servers = a–i, j–r, s–z. Up to 26 servers.
- **Beyond 26:** shard on the 2nd or 3rd letter. Example: "a" split into aa–ag, ah–an, ao–au, av–az.
- **Problem:** far more words start with "c" than "x". Shards are uneven.

- **Fix:** analyze historical distribution. A **shard map manager** keeps a lookup DB of which shard holds which prefixes. Example: if "s" has as many queries as u–z combined, use one shard for "s" and one for "u–z".




## Wrap-up

- **Multiple languages:** store Unicode characters in trie nodes.
- **Different top queries per country:** build a trie per country. Store tries in CDNs for speed.
- **Trending (real-time) queries:** the weekly design fails. Workers are not scheduled yet, and a rebuild takes too long. Ideas:
- Shard to shrink the working data set.
- Change the ranking model to weight recent queries more.
- Treat data as streams: Hadoop MapReduce, Spark Streaming, Storm, Kafka. The book does not go deeper.

✓ KEY POINTS

- Split the problem into a **read path** (query service) and a **write path** (data gathering). Optimize them separately.
- Trie + capped prefix length + **top k cached per node** makes reads O(1). Say the space cost out loud.
- Rebuild the trie **offline** (weekly) from aggregated logs. Serve it from an in-memory Trie Cache backed by Trie DB.
- Cut load with browser caching, AJAX, and log sampling.
- Shard by prefix using a **shard map manager** based on real distribution, not plain alphabet ranges.

⚠ COMMON TRAP

updating the trie on every search. It kills read latency, and an in-place update must rewrite every ancestor's cached top k.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·14 YouTube

YouTube video streaming Netflix video upload transcoding encoding DAG GOP blob storage CDN pre-signed URL MPEG-DASH HLS DRM long tail completion queue resumable upload

SEE ALSO Kafka vs RabbitMQ

**Source:** *System Design Interview* Vol 1, Ch 14.

## Problem

↑ TOP

Design YouTube: creators upload videos and viewers watch them. The same design applies to Netflix or Hulu.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Most important features? | Upload a video and watch a video |
| Clients? | Mobile apps, web browsers, smart TV |
| DAU? | 5 million |
| Average daily time on the product? | 30 minutes |
| International users? | Yes, a large share |
| Supported resolutions / formats? | Most resolutions and formats |
| Encryption? | Yes |
| File size limit? | Small and medium videos; max **1 GB** |
| Use existing cloud (AWS, GCP, Azure)? | Yes, recommended. Don't build everything from scratch |

**Requirements:**

- Functional: upload videos fast; stream smoothly; change video quality.
- Clients: mobile, web, smart TV.
- Non-functional: low infrastructure cost; high availability, scalability and reliability.

**Estimates:**

- 5M DAU. Each user watches 5 videos per day.
- 10% of users upload 1 video per day. Average video = 300 MB.
- Storage per day: 5M × 10% × 300 MB = **150 TB**.
- CDN (Amazon CloudFront, all traffic from the US, average $0.02/GB): 5M × 5 × 0.3 GB × $0.02 = **$150,000 per day**. Streaming only.

- So CDN cost is the big number. The deep dive reduces it.

> Try it first: sketch the upload path (where the raw file goes, how it becomes many formats, how metadata learns it is ready) and the watch path.

SOLUTION

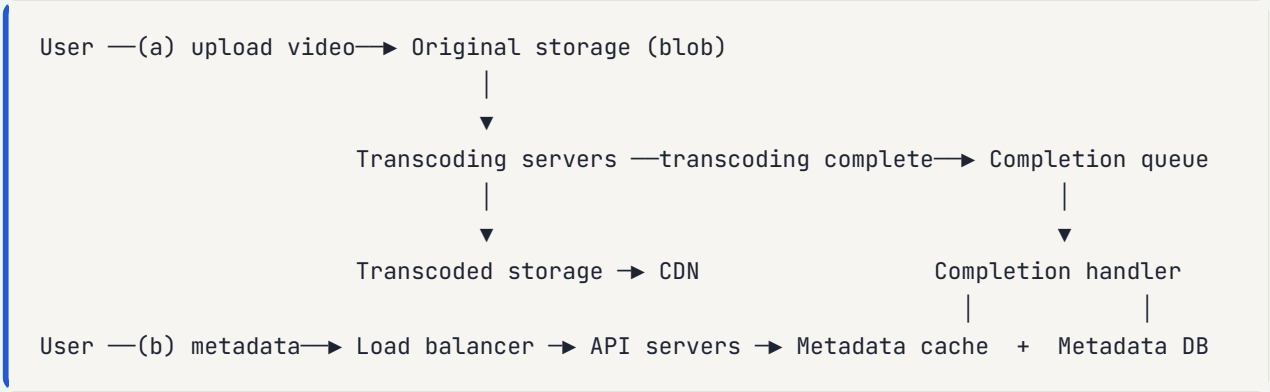
## High-level design

Use existing cloud services: **blob storage** and **CDN**. In an interview, picking the right tool matters more than explaining how it works inside. Even Netflix (AWS) and Facebook (Akamai CDN) do not build everything.

Three parts:

- **Client:** computer, phone, smart TV.
- **CDN:** stores videos and streams them when you press play.
- **API servers:** everything else. Feed recommendations, upload URLs, metadata DB and cache, sign-up.

**Upload flow** (two processes run in parallel):




- **Flow a, upload the video:** 1. Upload to original storage. 2. Transcoding servers fetch it and transcode. 3. In parallel: 3a. transcoded files go to transcoded storage, then to the CDN. 3b. a completion event goes to the completion queue. Completion-handler workers pull events and update the metadata DB and cache. 4. API servers tell the client the video is uploaded and ready to stream.
- **Flow b, update metadata:** while the file uploads, the client sends metadata (file name, size, format, ...). API servers write it to the metadata cache and DB.
- **Metadata DB** is sharded and replicated. **Metadata cache** holds video metadata and user objects.

**Streaming flow:**

```
Client (web / mobile / TV) ──streaming video──► CDN (nearest edge server)
```

- Streaming = the client loads a little data at a time and plays right away. Downloading = copy the whole file first.
- Streaming protocols: **MPEG-DASH**, **Apple HLS**, Microsoft Smooth Streaming, Adobe HDS. Names don't matter much. What matters: each supports different encodings and players, so pick one that fits.
- Video comes from the nearest CDN edge, so latency is low.

## Deep dives

### Video transcoding

Why transcode:

- Raw video is huge. One hour of HD at 60 fps can be a few hundred GB.
- Devices and browsers support only some formats.
- Send high resolution to high-bandwidth users and low resolution to low-bandwidth users.
- Networks change, especially on mobile. Switch quality automatically or manually to keep playback smooth.

An encoding format has two parts:

- **Container:** the "basket" for video, audio and metadata. Seen in the extension: .avi, .mov, .mp4.
- **Codec:** compression algorithm. Most used: **H.264, VP9, HEVC**.

### DAG model

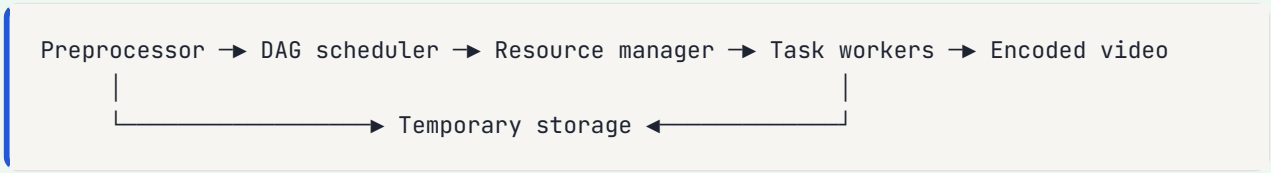
Transcoding is slow and expensive, and creators want different steps (watermarks, own thumbnails, HD or not). So let programmers define tasks as a **directed acyclic graph** (like Facebook's streaming video engine). Tasks run in stages, in sequence or in parallel.

```
                ┌─► Video ──► [Inspection | Video transcoding | Thumbnail | … |
Watermark] ─┐
Original video ─┼─► Audio ──► Audio encoding
────────────────────────────────────────────┼─► Assemble
                └─► Metadata
```

- **Inspection:** check quality; reject malformed files.
- **Video encoding:** many resolutions, codecs and bitrates, e.g. 360p, 480p, 720p, 1080p, 4K .mp4.
- **Thumbnail:** uploaded by the user or generated.

- **Watermark:** image overlay with identifying info.

### Video transcoding architecture




- **Preprocessor:** 1. Splits the video into **GOP** (Group of Pictures) chunks. A GOP is a few seconds of frames that play on their own. 2. Does the splitting for old clients that cannot split. 3. **Generates the DAG** from config files programmers write. Example: task `download-input` (type Download, output `inputVideo`, next `transcode`) → task `transcode` (type Transcode, input `inputVideo` + `transConfig`). That is 2 nodes and 1 edge. 4. **Caches** GOPs and metadata in temporary storage, so a failed encode can retry from them.
- **DAG scheduler:** splits the DAG into stages and puts tasks in the resource manager's task queue. Example: stage 1 = video, audio, metadata. Stage 2 = video encoding + thumbnail (from video), audio encoding (from audio).
- **Resource manager:** three queues + a task scheduler.
- Task queue: priority queue of tasks.
- Worker queue: priority queue of worker utilization.
- Running queue: current task/worker pairs.
- Scheduler loop: take the highest-priority task → pick the best worker → tell it to run → record in running queue → remove when done.
- **Task workers:** run the DAG tasks. Different worker types: watermark, encoder, thumbnail, merger, ....
- **Temporary storage:** pick by data type, size, access rate, lifetime. Metadata is small and hot → memory cache. Video/audio → blob storage. Freed when processing finishes.
- **Encoded video:** the output, e.g. `funny_720p.mp4`.

### Speed optimizations

- **Parallelize uploads:** the client splits the video into GOP chunks. Upload is faster and **resumable** after a failure.
- **Upload centers near users:** use the CDN as upload centers (North America, Asia, ...).
- **Parallelism everywhere:** original storage → download module → encoding module → upload module → encoded storage → CDN. Each step waits on the last. Put a **message queue** between every pair. Then, e.g., the encoding module no longer waits for the download module; it processes queued events in parallel.

### Safety optimizations

- **Pre-signed upload URL:** only authorized users upload, and only to the right place. 1. Client: `POST /upload` to API servers. 2. API servers return a pre-signed URL (S3 term;

Azure calls it "Shared Access Signature"). 3. Client uploads the video to original storage with that URL.

- **Protect videos** (pick one):

- **DRM:** Apple FairPlay, Google Widevine, Microsoft PlayReady.
- **AES encryption:** encrypt the video with an authorization policy; decrypt on playback.
- **Visual watermark:** logo or company name overlaid on the video.

### Cost-saving optimizations

YouTube views follow a **long-tail** distribution: a few videos get most views, many get almost none.

1. Serve **only popular videos from the CDN**. Serve the rest from high-capacity **video servers**.
2. For unpopular content, store fewer encoded versions. Encode **short videos on demand**.
3. Some videos are popular in one region only. Don't distribute them elsewhere.
4. Build your **own CDN** and partner with ISPs (Comcast, AT&T, Verizon), like Netflix. A huge project, but it makes sense for large streaming companies.

Analyze historical viewing patterns before doing any of this.

### Error handling

- **Recoverable** (e.g. a segment fails to transcode): retry a few times. If it keeps failing, return an error code.
- **Non-recoverable** (e.g. malformed video): stop the video's tasks and return an error code.

| Component | Playbook |
|---|---|
| Upload | Retry a few times |
| Split video | Old client can't split by GOP → send the whole file; split on the server |
| Transcoding | Retry |
| Preprocessor | Regenerate the DAG |
| DAG scheduler | Reschedule the task |
| Resource manager queue down | Use a replica |
| Task worker down | Retry the task on a new worker |
| API server down | Stateless → route to another server |
| Metadata cache down | Data is replicated; read other nodes, start a new cache server |
| Metadata DB master down | Promote a slave to master |

| Component | Playbook |
| --- | --- |
| Metadata DB slave down | Read from another slave; replace the dead one |

## Wrap-up

- **Scale the API tier:** stateless, so scale horizontally.
- **Scale the database:** replication and sharding.
- **Live streaming:** also needs upload, encoding and streaming, but:
- Stricter latency → maybe a different streaming protocol.
- Less need for parallelism; small chunks are processed in real time.
- Different error handling; slow recovery is not acceptable.
- **Video takedowns:** remove copyright, pornographic or illegal videos. Catch some at upload; others via user flagging.

✓ KEY POINTS

- Lean on managed **blob storage + CDN**. Only API traffic hits your servers; video bytes go to/from storage and CDN.
- Upload and metadata update run **in parallel**. A **completion queue** tells metadata when transcoding is done.
- Transcoding is a **DAG** of tasks on GOP chunks, run by a scheduler, resource manager and worker pool.
- Put **message queues** between pipeline stages to decouple them and run them in parallel.
- **Pre-signed URLs** for safe uploads; DRM / AES / watermark to protect content.
- CDN is the main cost. Use the **long tail** to decide what goes on the CDN.

⚠ COMMON TRAP

streaming every video through the CDN without looking at the cost. The estimate shows ~$150K/day, and most videos are rarely watched.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V1·15 Google Drive

Google Drive Dropbox file sync cloud storage block server delta sync compression S3 cold storage metadata database strong consistency sync conflict long polling resumable upload file versions deduplication

SEE ALSO Transactions & Isolation Levels · Web & Networking Fundamentals

**Source:** *System Design Interview* Vol 1, Ch 15.

## Problem

↑ TOP

Design Google Drive: a file storage and sync service. Users store files in the cloud, reach them from any device, and share them.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Most important features? | Upload and download files, file sync, notifications |
| Mobile, web, or both? | Both |
| File formats? | Any type |
| Encryption? | Yes, files in storage must be encrypted |
| File size limit? | Yes, **10 GB** or smaller |
| Users? | **10M DAU** |

**Requirements:**

- Functional: add files (drag and drop); download files; sync across devices; see file revisions; share files; notify when a file is edited, deleted or shared with you.
- Out of scope: Google Docs-style real-time collaborative editing.
- Non-functional:
- **Reliability:** data loss is unacceptable.
- **Fast sync.** Slow sync makes users leave.
- **Low bandwidth use**, especially on mobile data.
- Scalability and high availability.

**Estimates:**

- 50M signed-up users, 10M DAU. Each user gets 10 GB free.
- 2 uploads per user per day, average file 500 KB. Read:write = 1:1.
- Total space allocated: 50M × 10 GB = **500 PB**.

- Upload QPS: 10M × 2 / 86,400 ≈ **240**. Peak ≈ **480**.

> Try it first: sketch how a file gets from one device to the cloud and to the user's other devices, and what you store as metadata vs. as file content.

SOLUTION

## High-level design

The book starts with **one server** and scales it up.

**Single server:** Apache web server, MySQL for metadata, and a `drive/` directory with 1 TB. Under `drive/` there is one **namespace** (folder) per user. A file is identified by namespace + relative path, e.g. `drive/user1/recipes/…`.

**APIs** (all need authentication and HTTPS/SSL):

| API | Params |
|---|---|
| `https://api.example.com/files/upload?uploadType=resumable` | `uploadType=resumable`, `data` (local file) |
| `https://api.example.com/files/download` | `path`, e.g. `/recipes/soup/best_soup.txt` |
| `https://api.example.com/files/list_revisions` | `path`, `limit` (e.g. 20) |

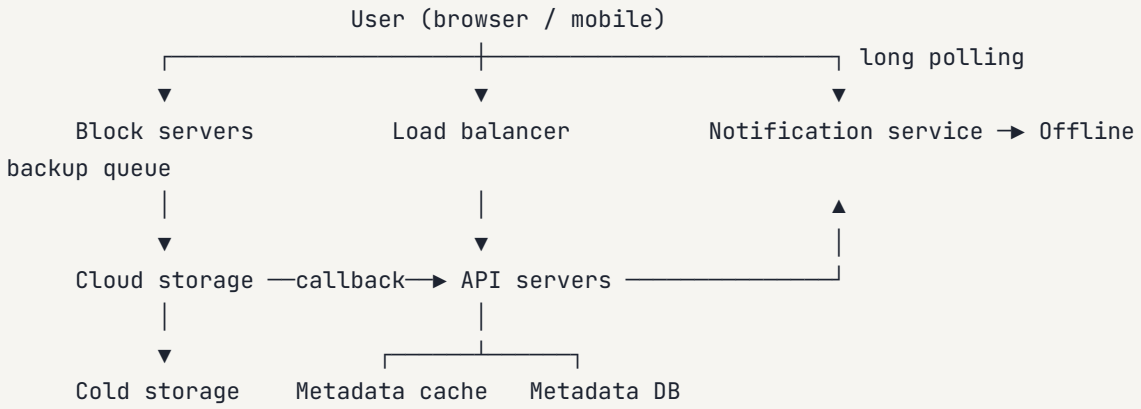
- Two upload types: **simple** (small files) and **resumable** (large files, flaky network).
- Resumable upload: 1) get a resumable URL, 2) upload and monitor state, 3) resume if interrupted.

**Move away from a single server:**

- Disk fills up (10 MB left). First fix: **shard storage by** `user_id`.
- Still risky: a server outage can lose data. Move files to **Amazon S3**. Use same-region and **cross-region replication** (a bucket is like a folder).
- **Load balancer** in front; add/remove **web (API) servers** with load.
- **Metadata DB** moves off the server, with replication and sharding.
- **File storage** = S3, replicated in two regions.

**Sync conflicts:** two users change the same file at the same time. **The first version processed wins.** The later one gets a conflict. The system shows user 2 both copies (local and latest server version). User 2 merges them or overrides one.

**Final high-level design:**




- **Block servers:** split files into blocks and upload them to cloud storage. Each block has a hash, stored in the metadata DB. Blocks are joined in order to rebuild a file. Max block size **4 MB** (Dropbox).
- **Cloud storage:** holds the blocks. **Cold storage:** holds files not used for a long time.
- **API servers:** everything except uploading: auth, user profile, file metadata.
- **Metadata DB + cache:** users, files, blocks, versions. Only metadata; content lives in the cloud.
- **Notification service:** pub/sub. Tells clients when a file is added, edited or removed elsewhere, so they pull changes.
- **Offline backup queue:** holds changes for offline clients until they reconnect.

## Deep dives

### Block servers

Sending the whole file on every edit wastes bandwidth. Two fixes:

- **Delta sync:** upload only the changed blocks (rsync-style algorithm). Example: only block 2 and block 5 changed → only those two upload.
- **Compression:** per file type. gzip/bzip2 for text; other algorithms for images and video.

Block server pipeline for a new file:

```
file —split→ Block 1..N —compress→ —encrypt→ Cloud storage
```

Block servers do the heavy lifting: split, compress, encrypt, upload only modified blocks.

### High consistency requirement

- **Strong consistency by default.** Different clients must not see a file differently at the same time. This applies to both the metadata cache and DB.
- Memory caches are eventually consistent by default. To fix this: keep cache replicas consistent with the master, and **invalidate the cache on every DB write**.

- Relational DBs give **ACID** natively. NoSQL does not; you would have to build it into the sync logic. **Choice: relational database.**

### Metadata database

Simplified schema:

| Table | Key fields |
|---|---|
| user | user_id, user_name, created_at |
| device | device_id (uuid), user_id, last_logged_in_at (plus push_id for mobile push) |
| workspace (namespace) | id, owner_id, is_shared, created_at |
| file | id, file_name, relative_path, is_directory, latest_version, checksum, workspace_id, created_at, last_modified |
| file_version | id, file_id, device_id, version_number, last_modified |
| block | block_id, file_version_id, block_order |

- user 1 → * devices; user 1 → * workspaces; workspace 1 → * files; file 1 → * versions; version 1 → * blocks.
- **Namespace / workspace** = a user's root directory.
- **file** holds the latest version. **file_version** holds history; its rows are **read-only** to protect history.
- Any version can be rebuilt by joining its blocks in `block_order`.

### Upload flow

Client 1 sends two requests **in parallel**:

```
Add metadata:
 1. Client 1 —add file metadata—▶ API servers
 2. API servers —▶ Metadata DB: status = "pending"
 3. —▶ Notification service
 4. Notification service —▶ Client 2: "file is being uploaded"

Upload content:
 2.1 Client 1 —upload file—▶ Block servers
 2.2 Block servers: chunk, compress, encrypt —▶ Cloud storage
 2.3 Cloud storage —upload completion callback—▶ API servers
 2.4 API servers —▶ Metadata DB: status = "uploaded"
 2.5 —▶ Notification service
 2.6 Notification service —▶ Client 2: "file fully uploaded"
```

Editing a file follows the same flow.

### Download flow

A client learns about changes in one of two ways:

- **Online:** the notification service tells it to pull.
- **Offline:** changes are saved (in the cache); it pulls them when it comes back online.

```
1. Notification service → Client 2: changes exist
2. Client 2 —get changes→ API servers
3-4. API servers ↔ Metadata DB
5. API servers → Client 2: metadata
6. Client 2 —download blocks→ Block servers
7-8. Block servers ↔ Cloud storage
9. Block servers → Client 2: blocks; client rebuilds the file
```

### Notification service

Options: **long polling** (Dropbox uses it) or **WebSocket** (persistent, bi-directional).

**Choice: long polling**, because:

- Communication is one-way: server tells client about changes, not the reverse.
- WebSocket suits real-time two-way traffic like chat. Drive notifications are infrequent, with no bursts.

How it works: each client holds a long-poll connection. When a change is detected, the connection closes and the client fetches the latest changes from the metadata server. After a response or a timeout, the client immediately opens a new long-poll request.

### Save storage space

Many versions across many data centers fill storage fast. Three techniques:

- **De-duplicate blocks** at the account level. Same hash = same block.
- **Smart backup strategy:**
- **Set a limit** on versions per file. At the limit, the oldest is replaced.
- **Keep valuable versions only.** A heavily edited doc could save 1000+ versions in a short time. Weight recent versions more. Experiment to find the right number.
- **Move cold data to cold storage** (not touched for months or years). S3 Glacier is much cheaper than S3.

### Failure handling

| Failure | Handling |
|---|---|
| Load balancer | Secondary takes over. LBs check each other via heartbeat |
| Block server | Other servers pick up unfinished or pending jobs |

| Failure | Handling |
|---|---|
| Cloud storage | S3 buckets are replicated across regions; fetch from another region |
| API server | Stateless; LB redirects traffic |
| Metadata cache | Replicated; read other nodes, start a replacement |
| Metadata DB master | Promote a slave to master; add a new slave |
| Metadata DB slave | Read from another slave; replace the failed one |
| Notification service | Each server holds many long-poll connections (Dropbox 2012: **1M+ per machine**). If one dies, all its clients must reconnect elsewhere. Reconnecting them all is slow |
| Offline backup queue | Queues are replicated; consumers re-subscribe to the backup queue |

## Wrap-up

- The design has two flows: **manage file metadata** and **file sync**. The notification service (long polling) keeps clients current.
- What makes it interesting: strong consistency + low bandwidth + fast sync.
- **Alternative: upload straight from the client to cloud storage.** Faster: the file travels once, not client → block server → cloud. Drawbacks:
- Chunking, compression and encryption must be built on every platform (iOS, Android, web). Error-prone and costly. Block servers keep it in one place.
- Clients can be hacked, so client-side encryption is not ideal.
- **Presence service:** move online/offline tracking out of the notification servers, so other services can use it.

✓ KEY POINTS

- Separate **metadata** (relational DB + cache) from **content** (blocks in S3).
- Files become **≤ 4 MB blocks**; block servers compress and encrypt; **delta sync** sends only changed blocks.
- Upload metadata and content **in parallel**; status goes pending → uploaded via the storage callback.
- **Strong consistency**: ACID DB, invalidate cache on write.
- **Long polling** for change notifications; offline backup queue for offline clients.
- Save space with block **dedup**, version limits, and **cold storage**.

⚠ COMMON TRAP

re-uploading the whole file on every edit. It wastes bandwidth, which is one of the stated requirements. Also: saying "WebSocket" for notifications without noting that traffic is one-way and infrequent.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·1 Proximity Service

proximity service · Yelp · nearby search · location-based service · LBS · geospatial index · geohash · quadtree · Google S2 · Hilbert curve · k-nearest · read replicas · Redis cache · geofencing

SEE ALSO Indexes · SQL vs NoSQL & Scaling Databases

**Source:** *System Design Interview* Vol 2, Ch 1.

## Problem

↑ TOP

Design a proximity service like Yelp's "nearby": given a user's location and a radius, return nearby businesses (restaurants, hotels, gas stations).

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Can the user set a radius? Expand if too few results? | Yes. Only care about the radius; expanding is a bonus topic |
| Max radius? | 20 km (12.5 miles) |
| Radius options in the UI? | 0.5 km, 1 km, 2 km, 5 km, 20 km |
| Must business adds/edits/deletes show in real time? | No. Agreed upfront: changes take effect the next day |
| Refresh results as the user moves? | No. Assume users move slowly |

**Requirements:**

- Functional: return businesses for a (latitude, longitude) + radius; owners add, update or delete businesses (not real-time); customers view business details.
- Non-functional: low latency; **data privacy** (location is sensitive; comply with GDPR, CCPA); high availability and scalability for peak hours in dense areas.

**Estimates:** 100M DAU, 200M businesses. A day ≈ 10^5 seconds. 5 searches per user per day → search QPS = 100M × 5 / 10^5 = **5,000**.

> Try it first: write the search API, then explain how you would index 200M points so "everything within 1 km of me" does not scan the whole table.

SOLUTION

### High-level design

**API:**

```
GET /v1/search/nearby?latitude=..&longitude=..&radius=..   (radius optional, default
5000 m)
  → { "total": 10, "businesses": [ {business object} ] }   (paginate in real life)

GET    /v1/businesses/{:id}   business details
POST   /v1/businesses         add a business
PUT    /v1/businesses/{:id}   update a business
DELETE /v1/businesses/{:id}   delete a business
```

The search result object has enough to render the list. The detail page (photos, reviews, rating) needs a second call.

**Data model.** Reads are high (search, view details); writes are low. A relational DB like **MySQL** fits.

```
business:      business_id PK, address, city, state, country, latitude, longitude
geo index:     covered in the deep dive (geohash table)
```

**Architecture** (Figure 2):

```
              User
               |
         Load balancer
    /search/nearby   /businesses/{:id}
       |                    |
     LBS            Business service
      | read               | write
  ┌───▼─── Database cluster ──▼────┐
  | Replica  Replica  Replica  Primary|  (primary replicates to replicas)
  └────────────────────────────────┘
```

- **Load balancer:** one DNS entry point; routes by URL path.
- **LBS (location-based service):** finds nearby businesses. Read-only, high QPS at peak, stateless, so it scales horizontally.
- **Business service:** owner writes (low QPS) and detail views (high QPS at peak).
- **DB cluster:** primary takes writes, replicas take reads. Replication lag is fine, since updates need not be real-time.
- Both services are stateless: add servers at mealtime, remove them at night. Use multiple regions and AZs.

**Algorithms to fetch nearby businesses.** Real systems use Redis geohash or PostGIS. In the interview, explain how the index works instead of naming a DB.

| Option | Idea | Verdict |
|---|---|---|
| 1. 2D search | `WHERE lat BETWEEN lat±r AND lng BETWEEN lng±r` | Full scan. Indexes on lat and lng each return a huge set; intersecting them is slow. An index helps only one dimension |
| 2. Even grid | Split the world into equal cells | Data is very uneven (NYC vs ocean). Finding neighbor cells is hard |
| 3. **Geohash** | Recursively halve lat and lng, interleave bits, base32-encode | Simple, widely used |
| 4. **Quadtree** | In-memory tree; split a cell into 4 until ≤ 100 businesses | Adapts to density |
| 5. **Google S2** | Map the sphere to 1D on a **Hilbert curve** (close on the curve = close in space) | Powerful but hard to explain |

The shared idea: **divide the map into small areas and index them** (hash-based: even grid, geohash, cartesian tiers; tree-based: quadtree, S2, R-tree).

**Geohash.** Split the planet at the equator and prime meridian: lat [-90,0]→0, [0,90]→1; lng [-180,0]→0, [0,180]→1. Keep splitting, alternating longitude and latitude bits. Encode in base32. Example: Google HQ = `9q9hvu`, Facebook HQ = `9q9jhr`. There are 12 lengths; only **4–6** are useful here.

| Geohash length | Grid size |
|---|---|
| 4 | 39.1 km × 19.5 km |
| 5 | 4.9 km × 4.9 km |
| 6 | 1.2 km × 609.4 m |
| 7 | 152.9 m × 152.4 m |

Pick the **shortest geohash whose cell covers the whole search circle**:

| Radius | 0.5 km | 1 km | 2 km | 5 km | 20 km |
|---|---|---|---|---|---|
| Geohash length | 6 | 5 | 5 | 4 | 4 |

**Geohash boundary issues:**

- A longer shared prefix means closer. The reverse is false. **Issue 1:** two places can be close with **no shared prefix** (they sit on either side of the equator or prime meridian). La Roche-Chalais (`u000`) is 30 km from Pomerol (`ezzz`). So `WHERE geohash LIKE '9q8zn%'` misses results.
- **Issue 2:** two points can share a long prefix yet sit in different cells.

- **Fix:** query the current cell **plus its 8 neighbors**. Neighbor geohashes are computed in constant time.
- **Not enough results?** Option 1: return only what is in the radius (easy, but may be too few). Option 2: **drop the last geohash character** to widen the cell, and repeat until there are enough.

**Quadtree.**

- Root = the whole world. Split any node with **> 100 businesses** into 4 children (`buildQuadtree` recursion). It is an **in-memory structure on each LBS server**, built at start-up, not a database.
- Leaf node: 4 coordinates (32 B) + up to 100 IDs × 8 B = **832 B**. Internal node: 32 B coords + 4 child pointers (32 B) = **64 B**.
- Leaves ≈ 200M / 100 = **2M**; internal nodes ≈ 2M / 3 ≈ **0.67M**. Memory ≈ 2M × 832 B + 0.67M × 64 B ≈ **1.71 GB**. Fits in one server, but you may still need several for CPU or bandwidth.
- Build time ≈ (n/100) log(n/100): **a few minutes** for 200M.
- Search: walk from the root to the leaf with the user's location. If it has 100 businesses, return it; otherwise add neighbors until there are enough.
- **Operations:** a server cannot serve while it builds. Roll out to a few servers at a time. Blue/green would make a whole new cluster load 200M rows from the DB at once. Updates: rebuild incrementally (some servers stale for a while, acceptable given the next-day rule), or update on the fly (needs locking in a multi-threaded tree, much harder). A nightly cache refresh can invalidate many keys at once and load the cache servers.

**S2** advantages: great for **geofencing** (cover any area at varying levels and notify users who leave it), and the **Region Cover** algorithm (set min level, max level, max cells instead of one fixed precision).

**Who uses what:** Geohash: Bing Maps, Redis, MongoDB, Lyft. Quadtree: Yext. Both: Elasticsearch. S2: Google Maps, Tinder. **In an interview, pick geohash or quadtree.**

| | Geohash | Quadtree |
|---|---|---|
| Implementation | Easy, no tree | Harder, must build a tree |
| Query type | Within a radius | **k-nearest** (e.g. nearest k gas stations, however far) |
| Grid size | Fixed per precision | Adapts to density (Denver example) |
| Updates | Easy: delete the (geohash, business_id) row | O(log n) walk from root; locking with threads; rebalancing (over-allocate ranges) |

## Deep dives

### Scale the database

- **Business table:** too big for one server. **Shard by business_id**: even load, easy to operate.
- **Geo index table** (geohash as the example). Option 1: one row per geohash holding a **JSON array** of business IDs. Updates must scan the array, check duplicates, and lock the row. Option 2 (**chosen**): **one row per business**, compound key **(geohash, business_id)**. Adds and removes are simple, with no locks.

```
geohash | business_id
32feac  | 343
32feac  | 347
f3lcad  | 112
```

- **Don't rush to shard the geo index.** It is small (≈1.71 GB for quadtree, similar for geohash) and fits in one DB's working set. Sharding adds app-level logic for no strong reason. Use **read replicas** to spread the read load.

### Caching

- First ask if a cache is even a win. The dataset fits in memory, so queries are not I/O bound, and replicas can add read throughput. Justify it with benchmarks and cost.
- **Cache key:** not raw lat/lng. Phone coordinates are noisy and change slightly as users move. Use the **geohash**, so nearby points map to the same key.
- Two caches:

| Key | Value | Name in final diagram |
|---|---|---|
| geohash | list of business IDs in the grid | "Geohash" Redis |
| business_id | business object (name, address, image URLs) | "Business info" Redis |

- On a miss: `SELECT business_id FROM geohash_index WHERE geohash LIKE '{geohash}%'`, then `Cache.set(key, ids, "1d")`. On add/edit/delete, update the DB and invalidate the cache. Easy, since writes are rare and geohash needs no locks.
- Cache all three precisions (geohash_4, _5, _6). Memory = 8 B × 200M × 3 ≈ **5 GB**. One Redis server is enough by memory, but **deploy the same copy globally** for availability and lower cross-continent latency.

### Regions and availability zones

- Users connect to a nearby region (US West, Europe).
- Spread dense areas (Japan, Korea) into their own regions or several AZs.

- **Privacy laws:** some countries require local data. Put a region there and use DNS routing to keep that country's requests inside it.

### Filter by time or type

"Open now" or "restaurants only": each grid returns few businesses, so return IDs, hydrate the objects, then filter in memory. This assumes opening time and type are in the business table.

### Final design (Figure 21)

```
User → Load balancer ─┬─ /search/nearby ──→ LBS ──read──→ Redis cluster [Business
info | Geohash]
                      └─ /businesses/{:id} → Business service ─read→ Redis cluster
                                                          └─write→ DB primary →
replicas
                                                DB cluster ──sync──→ Redis cluster
```

**Search flow** (restaurants within 500 m):

1. Client sends lat/lng (37.776720, -122.416730) and radius 500 m to the load balancer.
2. The load balancer forwards to the LBS.
3. LBS maps 500 m to geohash length 6.
4. LBS computes the 8 neighbors: `[my_geohash, neighbor1 … neighbor8]`.
5. LBS fetches business IDs for each geohash from "Geohash" Redis, **in parallel**.
6. LBS hydrates businesses from "Business info" Redis, computes distances, ranks, and returns.

**View/edit flow:** the business service reads "Business info" Redis, falling back to the DB and filling the cache. Because changes take effect next day, a **nightly job** updates cached business data.

## Wrap-up

- Covered indexes: 2D search, even grid, geohash, quadtree, S2. Geohash was used as the worked example.
- Caching cuts latency: what to cache and how to use it.
- Scale the DB with replication (geo index) and sharding (business table).
- Multi-region and multi-AZ deployment for availability, proximity to users, and local privacy laws.

✓ KEY POINTS

- Explain why a 2D lat/lng index fails, then map 2D → 1D with grid cells.
- Know geohash: precision vs radius, the boundary issues, the 8-neighbor fix, and dropping a character to widen the search.
- Know quadtree: the 100-per-leaf rule, ~1.71 GB memory, start-up build time, and that it suits k-nearest.
- The geo index is small: use read replicas, not sharding. Shard the business table by ID.
- Cache by geohash, not exact coordinates.

⚠ COMMON TRAP

jumping to sharding the geo index, or querying only the user's own geohash cell. The index fits in memory, and a prefix-only query misses businesses just across a cell edge.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·2 Nearby Friends

nearby friends · location sharing · WebSocket · Redis pub/sub · location cache · TTL · consistent hashing · hash ring · service discovery · etcd · ZooKeeper · Cassandra · geohash channel · Erlang · fan-out · stateful servers

SEE ALSO Redis Pub/Sub vs RabbitMQ vs Kafka · Concurrency Models (GIL, Threads, Processes, asyncio)

**Source:** *System Design Interview* Vol 2, Ch 2.

## Problem

↑ TOP

Design the backend for a "Nearby Friends" feature. An opt-in user sees a list of friends who are close by, with distance and last-updated time.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| What counts as "nearby"? | 5 miles; configurable |
| Straight-line distance? | Yes (ignore rivers and roads) |
| How many users? | 1B users; 10% use this feature |
| Store location history? | Yes, e.g. for machine learning |
| Inactive friends? | Hide a friend inactive for more than 10 minutes |
| Privacy laws (GDPR, CCPA)? | Ignore for now |

**Requirements:**

- Functional: see nearby friends, each with a distance and a last-updated timestamp; the list refreshes every few seconds.
- Non-functional: low latency; reliable overall, but **losing an occasional data point is fine**; **eventual consistency** (replicas a few seconds behind is fine).

**Estimates:**

- Refresh every **30 s** (walking is 3–4 mph, so 30 s barely changes anything).
- 100M DAU; concurrent = 10% → **10M**. 400 friends per user on average; 20 shown per page.
- Location update QPS = 10M / 30 ≈ **334,000**.
- Forwarding: if ~10% of friends are online and nearby, 333K × 400 × 10% ≈ **13M updates/s** to forward.

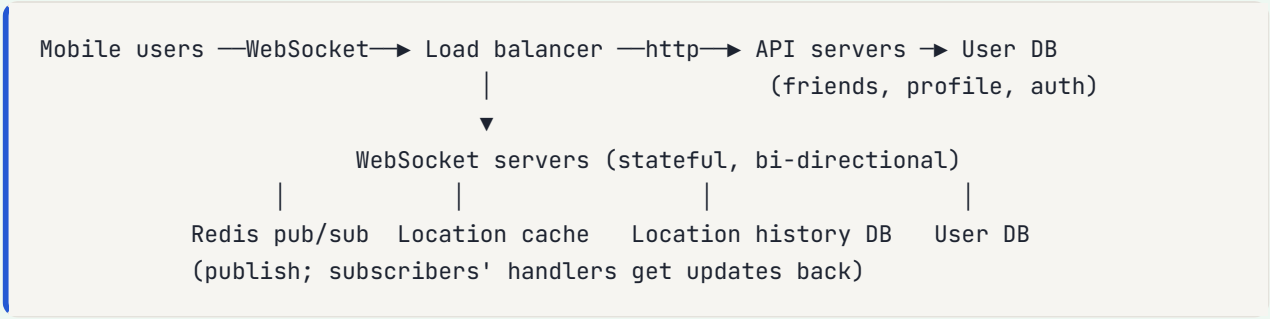
> Try it first: decide how a location update from one user reaches each online friend within seconds, and where the "current location" lives. Then estimate how many pub/sub servers you need.

SOLUTION

## High-level design

The book designs the high level **before** the API, because the protocol is not plain HTTP: the server must push.

Peer-to-peer (each phone connects to each nearby friend) is not practical on flaky mobile networks and a tight battery budget. So a **shared backend** does three things: receive every active user's updates; for each update, find the active friends who should get it; and drop it if the distance is over the threshold.




- **Load balancer:** in front of the stateless REST API servers and the stateful WebSocket servers.
- **REST API servers:** add/remove friends, update profiles. Standard work.
- **WebSocket servers:** one persistent connection per client. They push friends' updates and **seed a new client** with nearby online friends.
- **Redis location cache:** latest location per active user, with a **TTL**. Each update refreshes the TTL. When it expires, the user is inactive and drops out.
- **User DB:** profiles and friendships (relational or NoSQL).
- **Location history DB:** not on the critical path.
- **Redis pub/sub:** a very light message bus. Channels are cheap; a server with GBs of memory can hold **millions of channels**. Each user publishes to **their own channel**. Each online friend's WebSocket handler subscribes to it.

**Periodic location update (Figure 7):**

1. The client sends an update to the load balancer.
2. The load balancer forwards it to the client's persistent connection on a WebSocket server.
3. The server saves it to the **location history DB**.
4. The server updates the **location cache** (refreshing the TTL) and stores the location in the connection handler's variable.

5. The server publishes it to the user's channel in **Redis pub/sub**. Steps 3–5 can run **in parallel**.
6. Redis broadcasts it to all subscribers: the handlers of the user's online friends.
7. Each friend's handler computes the distance between the sender (from the message) and the friend (from the handler's variable).
8. If within the radius, it sends the new location and timestamp to the friend's client. Otherwise it drops the update.

With 400 friends and 10% online and nearby, each update fans out to about **40** friends.

**API (WebSocket):**

| API | Request | Response |
|---|---|---|
| Periodic location update | lat, lng, timestamp | nothing |
| Client receives updates | (server push) friend location + timestamp | — |
| WebSocket init | lat, lng, timestamp | friends' locations |
| Subscribe to a new friend | (server sends) friend ID | friend's latest lat, lng, timestamp |
| Unsubscribe a friend | (server sends) friend ID | nothing |

HTTP APIs (add/remove friends, profiles) are standard.

**Data model:**

```
Location cache (Redis):   key = user_id   value = {latitude, longitude, timestamp}
Location history:         user_id | latitude | longitude | timestamp
```

- **Why not a DB for current location?** Only the current location matters, one per user. Redis is very fast and has **TTL** for auto-purging inactive users. It need not be durable. If Redis dies, start an empty one and let updates refill it. Users miss an update cycle or two, which is acceptable.
- **Location history:** heavy writes, must scale horizontally. **Cassandra** is a good fit. A relational DB also works but must be **sharded by user ID**.

## Deep dives

### How each component scales

- **API servers:** stateless; auto-scale on CPU, load or I/O.
- **WebSocket servers:** auto-scaling is easy, but they are **stateful**. To remove a node, mark it **"draining"** at the load balancer so no new connections arrive. Remove it when its connections close (or after a long wait). Deploys need the same care. Good cloud load balancers handle this well.

- **User DB:** profiles and friendships will not fit one instance. **Shard by user ID**. At this scale a dedicated team may expose it through an internal API. Same function and performance either way.
- **Location cache:** 10M users × ≤ 100 B fits easily in one Redis by memory. But **334K writes/s** is too much for one server. **Shard by user ID** (each user's data is independent). Replicate each shard to a standby for availability.

### Client initialization

When a client opens its WebSocket and sends its first location, the handler:

1. Updates the user's location in the location cache.
2. Saves it in a handler variable.
3. Loads all friends from the user DB.
4. **Batch-fetches** friends' locations from the cache. Inactive friends have expired (TTL = inactivity timeout), so they are missing.
5. For each friend within the radius, returns profile, location and timestamp to the client.
6. **Subscribes to every friend's channel**, active or not. Inactive channels cost a little memory but **no CPU or I/O** until the friend publishes.
7. Publishes the user's current location to the user's own channel.

### Redis pub/sub: why and how many

- A channel is created on first subscribe. Publishing to a channel with no subscribers just drops the message. A channel costs a small hash table and linked list, and **no CPU while idle**.
- So: **one channel per user**, and subscribe to all friends at start-up. The backend never has to subscribe or unsubscribe as friends go online or offline. This trades **more memory for a simpler design**.

| Resource | Math | Servers |
|---|---|---|
| Memory | 100M channels × 100 active friends × 20 B per subscriber ≈ **200 GB** | ≈ **2** (100 GB each) |
| CPU | 13M pushes/s ÷ ~100K pushes/s per server (conservative, gigabit NIC) | ≈ **130** |

- The bottleneck is **CPU, not memory**. We need a **distributed** pub/sub cluster.

### Distributed pub/sub cluster

- Channels are independent, so **shard by the publisher's user ID**.
- Add **service discovery** (etcd or ZooKeeper). It only needs to store a config value and let clients subscribe to changes:

```
Key:   /config/pub_sub_ring
Value: ["p_1", "p_2", "p_3", "p_4"]      ← consistent-hash ring of pub/sub servers
```

- To publish (Figure 10): (1) the WebSocket server looks up `hash(channel)` on the ring. It keeps a **local cached copy** of the ring and subscribes to ring updates. (2) It publishes to that pub/sub server. Subscribing uses the same lookup.

#### Scaling and operating the pub/sub cluster

- Messages are not stored, so the data path is stateless. But each channel's **subscriber list is state**. If a channel moves, every subscriber must unsubscribe from the old server and resubscribe on the new one.
- So treat it as a **stateful cluster**, like storage. **Over-provision** for daily peak plus headroom. Do not scale it up and down every day.
- **Resizing** (e.g. `[p_1..p_4]` → `[p_1..p_6]`) moves many channels and causes a **flood of resubscribes**. Some updates are missed. Do it at the **lowest-traffic** time. Steps: provision new servers, update the ring key, watch for a CPU spike on the WebSocket cluster.
- **Replacing a dead server** is much safer: only its channels move. The on-call operator swaps `p_1` → `p_1_new` in the ring. WebSocket servers are notified. Each handler checks its subscribed channels against the new ring and resubscribes only the moved ones.

#### Adding or removing friends

The app registers callbacks. On "friend added", the client asks the WebSocket server to subscribe to the friend's channel. The server returns the friend's latest location if active. On "friend removed", it unsubscribes. The same callbacks handle a friend opting in or out.

#### Users with many friends

Friend counts are capped (Facebook: **5,000**), and friendships are two-way, unlike a celebrity follower model. A user's subscribers spread across many WebSocket servers. Their channel adds some load to one of tens of pub/sub servers. These "whales" are spread out, so there are no hotspots.

#### Nearby random person (extra credit)

- Add a pool of **pub/sub channels per geohash** cell.
- On update, the handler computes the user's geohash and publishes to that cell's channel. Everyone in the cell (except the sender) receives it.
- For border cases, each client subscribes to **its cell plus the 8 surrounding cells**.

#### Alternative: Erlang

- The book argues **Erlang is better** than Redis pub/sub here, but it is niche and hard to hire for.

- The Erlang ecosystem: Erlang/Elixir language, BEAM VM, OTP libraries. An Erlang process is ~**300 bytes**, with millions per server. It uses no CPU when idle.
- Design: write the WebSocket service in Erlang and replace the Redis pub/sub cluster with a distributed Erlang app. **Each user is an Erlang process.** It gets the user's updates and subscribes to friends' processes (native in OTP), forming a mesh that routes updates.

### Wrap-up

Core parts: **WebSocket** for real-time traffic, **Redis** for fast location reads and writes, and **Redis pub/sub** as the routing layer. We built a small-scale design, then scaled the API servers, WebSocket servers, data layer and pub/sub servers. We also covered the Erlang alternative, users with many friends, and "nearby random person".

✓ KEY POINTS

- Say why this differs from proximity search: dynamic locations make it a fan-out problem.
- Put current locations in Redis with a TTL equal to the inactivity timeout. Keep history in Cassandra.
- One pub/sub channel per user. The subscriber's handler does the distance filter, so far-away updates are dropped on the server.
- Do the memory vs CPU math: CPU (~13M pushes/s) drives the server count.
- Shard channels with a consistent-hash ring in etcd/ZooKeeper. Resize rarely and off-peak; replacing a node is cheap.

⚠ COMMON TRAP

treating the pub/sub cluster as stateless and auto-scaling it daily. Each resize moves channels and forces mass resubscribes. Also, storing current locations in a durable DB is unnecessary.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·3 Google Maps

Google Maps navigation ETA map rendering map tiles routing tiles zoom levels CDN geohash geocoding A* search Dijkstra Cassandra Kafka adaptive ETA rerouting vector tiles Web Mercator

SEE ALSO Kafka vs RabbitMQ · SQL vs NoSQL & Scaling Databases

**Source:** *System Design Interview* Vol 2, Ch 3.

## Problem

↑ TOP

Design a simple Google Maps for mobile phones: track user locations, give directions with an ETA, and render the map.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| DAU? | 1 billion |
| Which features? | Location update, navigation, ETA, map rendering |
| Road data? | We already have it from several sources: TBs of raw data |
| Consider traffic? | Yes, it is key to accurate ETAs |
| Travel modes? | Support several (driving, walking, bus...) |
| Multi-stop directions? | Nice to have; not the focus |
| Businesses and photos? | Out of scope |

**Requirements:**

- Functional: user location update; navigation service including ETA; map rendering. Main clients are mobile phones.
- Non-functional: **accuracy** (no wrong directions); **smooth** map rendering; **low data and battery usage**; general availability and scalability.

**Map 101 (the concepts the design needs):**

- **Projection:** 3D globe → 2D plane always distorts. Google uses **Web Mercator**.
- **Geocoding:** address → lat/lng (reverse geocoding goes the other way). One method is **interpolation** over GIS street data.
- **Geohashing:** recursively split a flat 20,000 × 10,000 km map into 4 sub-grids, each named 0–3, until a size threshold. Used here for tiling.
- **Map tiles:** the world is cut into 256×256 PNG tiles per **zoom level**. The client downloads only the tiles it needs and stitches them. Fully zoomed out = one tile.

- **Routing:** routing algorithms (Dijkstra and A *variants) run on a graph: intersections = nodes, roads = edges. Their speed depends heavily on graph size. So split the road network into routing tiles: geohash-like grids, each a small graph with references to the tiles it connects to**. Map tiles are PNGs; routing tiles are binary road data.
- **Hierarchical routing tiles:** 3 levels. Small tiles with **local roads**; bigger tiles with **arterial roads**; large tiles with **highways**. Edges link levels (e.g. local street A → freeway F). Cross-country routing uses the coarse level.

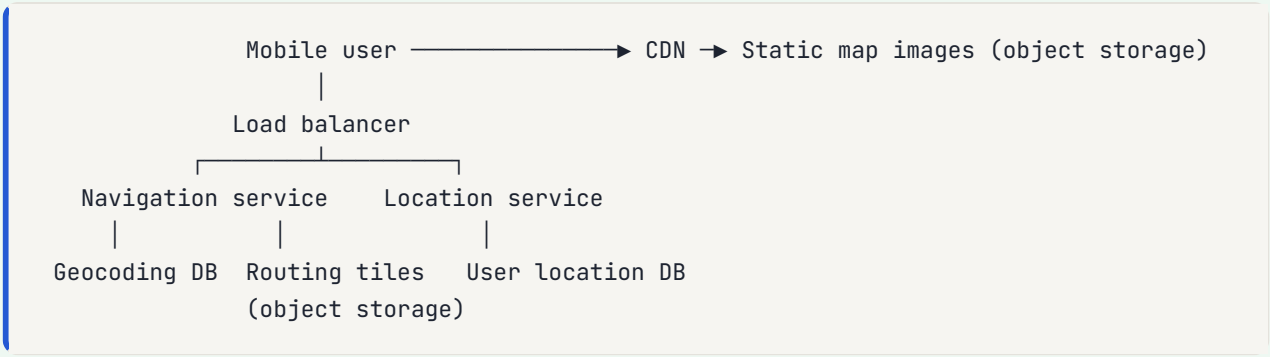
**Estimates:**

- **Map storage:** zoom 21 has ~4.4 trillion tiles. × 100 KB ≈ **440 PB**. About 90% of the world is ocean, desert or mountains, which compress well, so cut by 80–90% → 44–88 PB, call it **50 PB**. Each lower level has 4× fewer tiles: 50 + 50/4 + 50/16 + ... ≈ **67 PB**, so roughly **100 PB** in total. Routing tiles: TBs. Metadata: negligible.
- **Navigation:** 1B DAU × 5 sessions/week, 35 min/week → 5B minutes/day. QPS = 1B × 5 / 7 / 10^5 ≈ **7,200**; peak (5×) **36,000**.
- **Location updates:** sending GPS every second = 5B min × 60 = 300B/day = **3M QPS**. **Batch on the client and send every 15 s → 200K QPS**; peak **1M QPS**. Send less often when the user is stuck in traffic.

> Try it first: sketch how the phone gets map images without the server drawing them, how a route is computed over a road graph too big for memory, and where 1M location writes per second go.

SOLUTION

## High-level design




**Location service.**

- The client records GPS **every second** but sends a **batch every 15 s**. This cuts traffic a lot.
- Why collect it: live traffic, detecting new or closed roads, personalization, better ETAs and rerouting.
- Writes stay huge, so use a write-optimized, scalable DB (**Cassandra**). Also log to **Kafka** for stream processing.

- Protocol: **HTTP with keep-alive** (efficient).

```
POST /v1/locations
  locs: JSON array of (latitude, longitude, timestamp) tuples
```

**Navigation service.** Find a reasonably fast route from A to B. Some latency is fine. It need not be the fastest, but it must be **accurate**.

```
GET /v1/nav?origin=1355+market+street,SF&destination=Disneyland
→ { distance: {text: "0.2 mi", value: 259}, duration: {text: "1 min", value: 83},
    start_location, end_location, html_instructions, polyline: {points: "..."},
    geocoded_waypoints: [...], travel_mode: "DRIVING" }
```

Rerouting and traffic changes are handled later by **adaptive ETA**.

**Map rendering.** 100 PB cannot live on the client. Fetch tiles on demand when the user **zooms or pans**, or **moves into a nearby tile** during navigation.

| Option | How | Verdict |
|---|---|---|
| 1. Build tiles on the fly | Server renders per location + zoom | Huge server load; endless combinations make caching hard |
| 2. **Pre-generated static tiles** | Fixed grid per zoom level, one tile per geohash, served by a **CDN** | **Chosen.** Served from the nearest POP; static files cache very well |

**Data usage per client** (no client cache): moving at 30 km/h, with each 256×256 tile (~100 KB) covering 200 m × 200 m. 1 km$^2$ = 25 tiles = 2.5 MB. 30 km/h → 75 MB/h ≈ **1.25 MB/min**. Daily routes repeat, so client caching cuts this a lot.

**CDN traffic:** 5B nav minutes/day × 1.25 MB = 6.25B MB/day → **62,500 MB/s**. With **200 POPs**, each serves only a few hundred MB/s.

**Tile URLs.** The client can compute the geohash from lat/lng + zoom, e.g. zoom-7 tile at Google HQ `9q9heb` → `https://cdn.map-provider.com/tiles/9q9heb.png`.

- **Trade-off:** that algorithm is hardcoded in every client. Changing the encoding means shipping app updates on every platform, which is slow and risky.
- **Alternative:** a tiny **map tile service** builds the URLs. It adds operational flexibility. Flow: (1) the client calls the map tile service via the load balancer, (2) the LB forwards it, (3) the service takes location + zoom and returns **9 tile URLs** (the tile plus 8 neighbors), (4) the client downloads them from the CDN.

## Deep dives

### Data model

| Data | Store | Why |
|---|---|---|
| **Routing tiles** | Binary adjacency lists in **object storage (S3)**, keyed by geohash; cached aggressively on the routing service | Too big for memory (else no tiling needed). A DB would only be costly storage; no DB features needed. Geohash gives fast lookup by lat/lng |
| **User location** | **Cassandra** | Write-heavy, scales horizontally |
| **Geocoding** | **Redis** (KV) | Frequent reads, rare writes. Converts origin and destination to lat/lng |
| **Precomputed map images** | **CDN backed by S3** | Rendering roads into images is heavy and repeated. Compute once per zoom level |

- A **routing tile processing service** is an offline pipeline that runs periodically. It turns the raw road data (names, county, lat/lng; not a graph) into the 3 levels of routing tiles. Collected user locations improve the data over time.

### Location service

- **1M writes/s**, and a location is stale as soon as the next one arrives. Favor **availability over consistency** (AP in CAP). **Cassandra** fits.
- Key = **(user_id, timestamp)**: `user_id` is the **partition key**, `timestamp` the **clustering key**. You can read a user's latest position fast, and a time-range read is efficient because rows are stored together, sorted by time.

```
user_id | timestamp | lat  | long | user_mode | navigation_mode
51      | 132053000 | 21.9 | 89.8 | active    | driving
```

- **Also write every update to Kafka.** Consumers include:

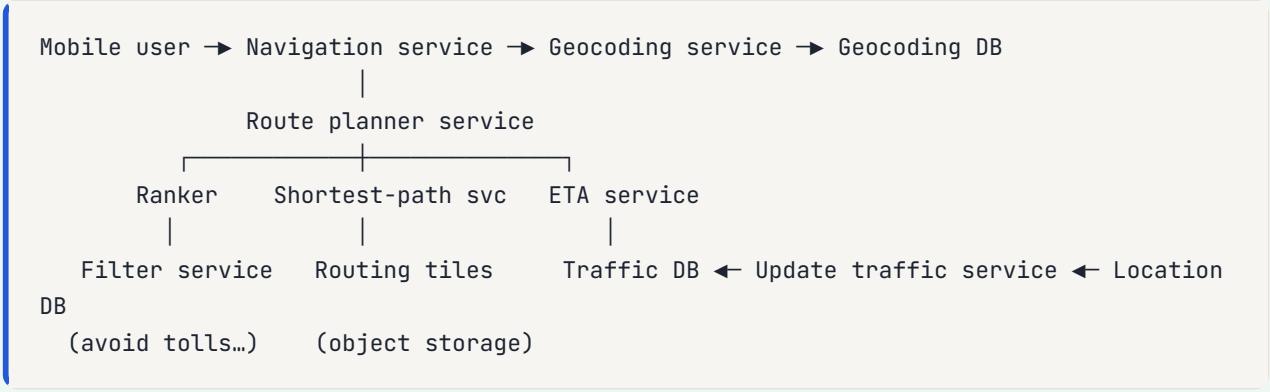
```
Location service → User location DB
        └→ Kafka ─┬→ Traffic update service → Traffic DB
                  ├→ ML personalization → Personalization DB
                  ├→ Routing tile processing → Routing tiles (S3): new/closed roads
                  └→ Analytics → Analytics DB
```

### Map rendering

- **21 zoom levels** (as Google uses). Level 0 = 1 tile of 256×256. Each level doubles the tiles in both directions: level 1 = 2×2 (512×512 px), level 2 = 4×4 (1024×1024 px). Each level has 4× the pixels. The client picks the level that matches its viewport and zoom.

- **Optimization: vector tiles.** With WebGL, send **paths and polygons** instead of images, and the client draws them. Vectors **compress much better** (big bandwidth savings). Zooming is also **smooth**: raster tiles stretch and pixelate between levels, while vectors scale each element.

### Navigation service (Figure 17)




- **Geocoding service:** resolves a place name or address to lat/lng (like Google's geocode API: `address=1600+Amphitheatre+Parkway…` → lat 37.4224764, lng -122.0842499). It runs on origin and destination first.
- **Route planner:** finds a route optimized for travel time under current traffic, by calling the services below.
- **Shortest-path service:** returns the **top-k shortest paths without traffic**. It depends only on road structure, which rarely changes, so **cache routes**. It runs an **A* variant** on routing tiles: convert origin and destination to geohashes, load those tiles, then traverse and **hydrate neighbor tiles** from S3 (or a local cache) as the search grows. Links between levels let it "enter" highway-only tiles.
- **ETA service:** ML prediction from **live and historical traffic** for each candidate route. The hard part is predicting traffic 10–20 minutes ahead (out of scope).
- **Ranker:** applies user filters (avoid tolls, avoid freeways), sorts fastest first, and returns the top-k.
- **Updater services** consume Kafka: routing tile processing (new roads and closures → more accurate paths) and traffic update (→ live traffic DB → better ETAs).

### Adaptive ETA and rerouting

The server must track **active navigating users** and update their ETA when traffic changes. How do we find affected users fast among millions of routes?

- **Naive:** store each user's route as a list of routing tiles (`user_1: s_1, s_2, …, s_k`). On an incident in `s_2`, scan every row: **O(n × m)** (n users, m tiles per route).
- **Better:** for each user, store the **current tile and its enclosing tiles at the coarser levels**: `user_1, s_1, super(s_1), super(super(s_1)), …`. An incident tile is checked against the **last (largest) tile** of each row. If it is not inside, the user is unaffected. This filters out most users quickly.

- **When traffic clears:** keep all possible routes per user, recompute ETAs regularly, and notify the user if a faster route appears.

**Delivery protocol (server → client):**

| Option | Verdict |
|---|---|
| Mobile push notification | Payload too small (**4,096 bytes** on iOS); no web support |
| Long polling | Heavier on servers than WebSocket |
| Server-Sent Events | Works, but one-way only |
| **WebSocket** | **Chosen:** light on servers and **bi-directional** (features like last-mile delivery may need it) |

**Final design** = Figure 17 plus an **Adaptive ETA and rerouting** service. It is called by the navigation service, reads the traffic DB, and keeps an **Active users** store.

## Wrap-up

We designed location updates, ETAs, route planning and map rendering. Extension: **multi-stop navigation** for enterprise customers (find the best order to visit a set of destinations under live traffic), useful for DoorDash, Uber, Lyft.

✓ KEY POINTS

- Learn the vocabulary: projection, geocoding, geohash, map tiles vs routing tiles, zoom levels.
- Pre-generate static map tiles per zoom level and serve them from a CDN. Discuss client-side URL computation vs a map tile service.
- Break the road graph into hierarchical routing tiles in object storage. A* loads them lazily and jumps to highway tiles for long trips.
- Batch GPS on the client (15 s). Store it in Cassandra keyed by (user_id, timestamp), and fan it out through Kafka to traffic, routing-tile and ML updaters.
- Separate shortest path (static, cacheable) from ETA (traffic-aware ML) and ranking (user filters).

⚠ COMMON TRAP

rendering map tiles on demand on the server, or loading the whole world's road graph into memory. Both fail at this scale. Pre-render to a CDN and tile the graph.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·4 Distributed Message Queue

message queue · event streaming · Kafka · topic · partition · broker · consumer group · offset · write-ahead log · segment · in-sync replicas · ISR · ack · pull vs push · rebalancing · delivery semantics · exactly once

SEE ALSO Kafka vs RabbitMQ

**Source:** *System Design Interview* Vol 2, Ch 4.

## Problem

↑ TOP

Design a distributed message queue. It should also have event-streaming features: long retention and repeated consumption of messages (like Kafka or Pulsar).

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Message format and size? | Text only, in the KB range |
| Can messages be consumed repeatedly? | Yes, by different consumers (an added feature; a traditional MQ drops a message once delivered) |
| Consumed in the order produced? | Yes (added feature; traditional MQs usually don't guarantee order) |
| Persist data? Retention? | Yes, 2 weeks (added feature) |
| How many producers and consumers? | The more the better |
| Delivery semantics? | At-least-once for sure; ideally all three, configurable |
| Throughput and latency? | High throughput for log aggregation; low latency for classic MQ use cases |

**Requirements:**

- Functional: producers send messages; consumers consume them; messages can be consumed repeatedly or once; old data can be truncated; KB-sized messages; delivery in the order added; configurable at-least / at-most / exactly once.
- Non-functional: high throughput **or** low latency (configurable per use case); scalable and able to absorb sudden surges; persistent and durable (on disk, replicated across nodes).

**Estimates:** the book gives none.

**Traditional MQ note:** RabbitMQ-style queues keep messages in memory only until consumed (small on-disk overflow) and usually don't keep order. That makes the design much simpler.

> Try it first: sketch how one topic is split across servers, how a message is stored on disk, how consumers share work without breaking order, and how a message survives a broker crash.

SOLUTION

## High-level design

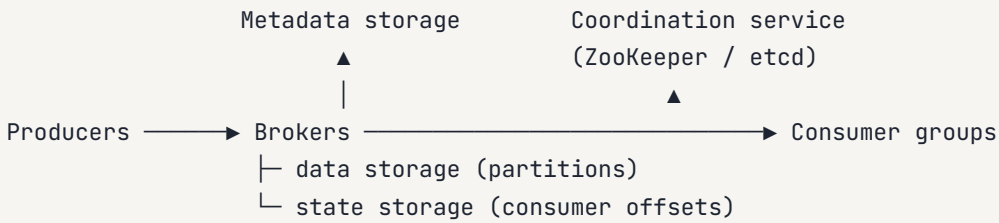
**Messaging models:**

- **Point-to-point:** a message goes to a queue and **one** consumer gets it. It is removed after the ACK. No retention.
- **Publish-subscribe:** a message goes to a **topic** and **every** subscriber gets it.
- This design supports both. Topics give pub/sub. **Consumer groups** simulate point-to-point.

**Core concepts:**

- **Topic:** a named category of messages, unique across the service.
- **Partition:** a topic is sharded into partitions. Each one is a FIFO queue, so order is kept *inside* a partition. Add partitions to scale a topic.
- **Offset:** the position of a message in its partition.
- **Broker:** a server that holds partitions.
- **Message key:** same key (e.g. user ID) → same partition. No key → random partition.
- **Consumer group:** consumers that share the work for a topic. Each group keeps its own offsets (e.g. one group for billing, one for accounting).
- **Rule:** a partition is consumed by **only one consumer in a group**. Two consumers on one partition would break order. If a group has more consumers than partitions, some sit idle.
- Put all consumers in one group → point-to-point. Allocate many more partitions than consumers up front, then scale by adding consumers.

**Architecture:**




- **Producer** pushes messages to a topic. **Consumer group** subscribes and consumes.
- **Broker** holds partitions.
- **Data storage:** messages, per partition. **State storage:** consumer state (offsets). **Metadata storage:** topic config and properties.

- **Coordination service:** service discovery (which brokers are alive) and leader election. One broker is the **active controller**; it assigns partitions.

## Deep dives

Three choices drive throughput: an on-disk structure built for **sequential access** and the OS disk cache; a message format that passes from producer to queue to consumer **with no changes (no copying)**; and **batching** everywhere.

### Data storage

Traffic: write-heavy and read-heavy, no updates or deletes, mostly sequential access.

| Option | Verdict |
|---|---|
| Database (SQL table per topic, or NoSQL collection) | Hard to support write-heavy + read-heavy at scale; becomes a bottleneck |
| **Write-ahead log (WAL)** | **Chosen.** Append-only file, pure sequential read/write (like MySQL redo log, ZooKeeper WAL) |

- New messages are appended to the tail of a partition with an increasing offset.
- A file can't grow forever, so split it into **segments**. Only the **active segment** takes writes. When it hits a size limit, a new active segment starts. Old segments serve reads only and are truncated past the retention or capacity limit.
- Layout: `Topic-A/Partition-1/segment-1, segment-2, …`.
- **Disk note:** rotational disks are slow only for random access. Sequential access on RAID disks gives several hundred MB/s, cheaply. The OS also caches disk data in free memory very aggressively.

### Message data structure

A shared contract between producer, queue, and consumer. If any part disagrees, messages get mutated, which means expensive copying.

| Field | Type | Note |
|---|---|---|
| key | byte[] | picks the partition: `hash(key) % numPartitions` (or a custom mapping) |
| value | byte[] | payload: plain text or compressed binary |
| topic | string | |
| partition | integer | |
| offset | long | topic + partition + offset locate a message |
| timestamp | long | when stored |
| size | integer | |

| Field | Type | Note |
|---|---|---|
| crc | integer | integrity check |

- The key is **not** the partition number. Partition numbers are internal and not exposed to clients.
- Unlike a KV store, message keys need not be unique or even present.
- Optional fields (e.g. tags) can be added for features like filtering.

### Batching

- Groups messages into one network request, so fewer round trips.
- The broker writes large chunks: bigger sequential writes and bigger contiguous disk-cache blocks.
- **Trade-off:** bigger batch = higher throughput but higher latency. For a low-latency queue, use smaller batches. When tuned for throughput, you may need more partitions per topic.

### Producer flow

- **Option 1: routing layer.** The producer sends to a router. The router reads the replica plan from metadata storage (cached) and forwards to the partition's **leader** replica. Followers pull from the leader. Once "enough" replicas sync, the leader commits and replies. Drawbacks: an extra network hop, and no batching.
- **Option 2 (chosen): routing + buffer inside the producer client library.** Fewer hops (lower latency), the producer can choose its own partition logic, and the buffer sends larger batches (higher throughput).
- Batch size is the classic throughput vs latency knob; producers tune it per use case.

### Consumer flow: push vs pull

The consumer gives an offset in a partition and gets back a chunk of events from that position.

| | Push | Pull |
|---|---|---|
| Pros | Low latency: broker sends at once | Consumer controls rate; real-time and batch consumers can coexist; slow consumers scale out or catch up; better for aggressive batching (pull everything after the current offset) |
| Cons | Consumers can be overwhelmed; hard to serve consumers with different speeds | Wasted pulls when no data; fix with **long polling** |

**Choice: pull** (what most MQs do). Pull flow:

1. A new consumer joins group 1 for topic A. It finds its broker by **hashing the group name**. That broker is the group's **coordinator** (not the same as the cluster coordination service).

2. The coordinator confirms membership and assigns a partition (round-robin, range, etc.).
3. The consumer fetches from the last consumed offset in state storage.
4. It processes messages and commits the offset. The order of process vs commit sets the delivery semantics.

#### Consumer rebalancing

Decides which consumer owns which partitions. Triggers: a consumer joins, leaves, crashes, or partitions change.

- The coordinator gets heartbeats from consumers and manages their offsets.
- When the member list changes, the coordinator elects a **group leader** (a consumer). The leader builds the **partition dispatch plan**; the coordinator broadcasts it.
- **Join:** B asks to join → coordinator tells A (on its next heartbeat) to rejoin → all rejoin → leader elected → leader sends plan → everyone consumes new partitions.
- **Leave:** A asks to leave → B is told to rejoin → same steps.
- **Crash:** no heartbeat from A within the timeout → A marked dead → rebalance.

#### State and metadata storage

- **State storage:** partition ↔ consumer mapping, and each group's last consumed offset per partition (e.g. group 1 at 6, group 2 at 13). A replacement consumer resumes from it.
- Access: frequent small reads/writes, frequent updates, rare deletes, random access, consistency matters → a KV store like **ZooKeeper**. (Kafka later moved offsets into the brokers.)
- **Metadata storage:** partition count, retention, replica distribution. Small, rarely changes, needs strong consistency → ZooKeeper.
- **Simplified design:** ZooKeeper holds metadata, state, and coordination (leader election). Brokers keep only message data.

#### Replication and in-sync replicas

- Each partition has **3 replicas** on different brokers: one leader, two followers.
- Producers write only to the leader. Followers **pull** from it. The leader ACKs once "enough" replicas have the message.
- The **replica distribution plan** (e.g. "topic A partition 1: leader broker 1, followers 2 and 3") is made by the controller broker and saved in metadata storage.
- **ISR (in-sync replicas):** followers that are close enough to the leader, set by topic config (e.g. `replica.lag.max.messages = 4`: up to 3 behind is still in sync). The leader is always in ISR. The leader tracks each replica's lag.
- **Committed offset:** every message up to it is on all ISRs. Example: leader has 10–15, committed = 13; replica-2 and replica-3 are in ISR; replica-4 (only 10–11) is out until it catches up.
- Why ISR: waiting for *all* replicas is safest, but one slow replica would stall the partition.

| ACK setting | Producer gets ACK when | Trade-off | Use case |
|---|---|---|---|
| ACK=all | all ISRs have it (needs ISR count ≥ `min.insync.replicas`) | slowest, strongest durability | no loss allowed |
| ACK=1 | leader persisted it | lost if leader dies before followers copy | low latency, rare loss OK |
| ACK=0 | never; no retry | lowest latency, may lose data | metrics, logging |

- **Consumers read from the leader.** It is simple, one consumer per partition per group keeps connections low, and hot topics scale by adding partitions. Exception: a consumer in another data center can read from the closest ISR.

### Scalability

- **Producers:** no group coordination; add or remove instances.
- **Consumers:** groups are isolated; rebalancing handles add, remove, and crash.
- **Broker failure:** 4 brokers, broker 3 crashes. The controller sees it and makes a new plan. Each partition gets a new replica on a live broker (e.g. topic B partition 1: broker 4 becomes leader, new replica on broker 2). New replicas catch up as followers.
- Higher min ISR is safer but slower.
- Never put replicas of one partition on the same node.
- Cross-data-center replicas are safer but add latency and cost. Data mirroring is out of scope.
- **Adding a broker:** don't just reshuffle. Temporarily allow **more replicas than configured**. E.g. partition 2 moves to brokers (2, 3, 4); broker 4 copies from leader 2; then the extra replica on broker 1 is removed. No data loss. Removing a broker works the same way.
- **Adding partitions:** old messages stay where they are (no migration); new messages spread over all partitions. Producers learn of it from any broker; consumers rebalance.
- **Removing partitions:** harder. The decommissioned partition stops receiving writes but stays readable until **retention expires**; then it is truncated and groups rebalance. It doesn't free space quickly.

### Delivery semantics

| Semantic | Producer | Consumer | Result | Use case |
|---|---|---|---|---|
| At-most once | async, ack=0, no retry | commits offset **before** processing | may lose, never duplicates | monitoring metrics |
| At-least once | ack=1 or all, retries on failure/ timeout | commits offset **after** processing | no loss, possible duplicates | most cases; dedupe with a unique key in the DB |

| Semantic | Producer | Consumer | Result | Use case |
|---|---|---|---|---|
| Exactly once | guaranteed once | guaranteed once | no loss, no duplicates; high cost in performance and complexity | payments, trading, accounting, when downstream isn't idempotent |

### Advanced features

- **Message filtering:** e.g. payments only want checkout and refund events from the orders topic.
- A dedicated topic per consumer wastes storage and couples producers to consumers.
- Filtering in the consumer wastes traffic.
- **Choice: filter on the broker by tags in the message metadata.** The broker must not decrypt or deserialize the payload (slow, and sensitive data should stay unreadable). Several tags allow multi-dimension filters. Complex formulas would need a parser, which is too heavy.
- **Delayed / scheduled messages:** e.g. close an order if unpaid after 30 minutes. Send the message now to **temporary storage** (special topics) on the broker; deliver it to the real topic when time is up. Timing options: fixed delay levels (RocketMQ: 1s, 5s, 10s, 30s, 1m ... 2h) or a **hierarchical time wheel**. Scheduled messages work the same way.

## Wrap-up

- **Protocol:** must cover produce, consume, heartbeat; move large volumes; verify integrity. Examples: AMQP, Kafka protocol.
- **Retry consumption:** send failed messages to a dedicated **retry topic** so they don't block new ones.
- **Historical archive:** for replaying truncated data, archive old messages to HDFS or object storage.

✓ KEY POINTS

- Say early whether you're designing a classic queue or a streaming platform; retention and ordering change the whole design.
- Partitions are the unit of scale and of order. Key → partition; one consumer per partition per group.
- Store messages as segmented append-only logs on disk. Sequential I/O, OS cache, zero-copy format, and batching give throughput.
- Replicate each partition (leader + followers), track ISR, and let producers pick ACK=0/1/all.
- Consumers pull with long polling; the group coordinator runs rebalancing.
- Delivery semantics come from ACK settings plus when the consumer commits its offset.

> ⚠ COMMON TRAP
> expecting global order across a topic. Order holds only within a partition, and only if a single consumer in the group reads that partition.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·5 Metrics Monitoring and Alerting

metrics monitoring · alerting · time-series database · InfluxDB · Prometheus · pull vs push · service discovery · metrics collector · Kafka · downsampling · delta encoding · alert manager · Grafana · observability

SEE ALSO Kafka vs RabbitMQ

**Source:** *System Design Interview* Vol 2, Ch 5.

## Problem

↑ TOP

Design a scalable metrics monitoring and alerting system for internal use at a large company (like Datadog or Prometheus, but in-house).

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| In-house or SaaS? | Internal use only |
| Which metrics? | Operational: CPU load, memory, disk; also requests/sec, running server count. No business metrics |
| Scale? | 100M DAU, 1,000 server pools, 100 machines per pool |
| Retention? | 1 year |
| Lower resolution for old data? | Raw for 7 days; 1-minute resolution for 30 days; 1-hour after that |
| Alert channels? | Email, phone, PagerDuty, webhooks |
| Logs (error/access)? | No |
| Distributed tracing? | No |

**Requirements:**

- Functional: collect many kinds of metrics (CPU, request count, memory, queue message count); store them with the retention policy; alert to several channels; visualize.
- Non-functional: **scalable** (growing metrics and alerts), **low query latency** for dashboards and alerts, **reliable** (never miss a critical alert), **flexible** (easy to plug in new tech).
- Out of scope: log monitoring (ELK) and distributed tracing.

**Estimates:** 1,000 pools × 100 machines × 100 metrics per machine ≈ **10 million metrics**. 1-year retention with the downsampling policy above.

> Try it first: sketch the path from a server's CPU number to a stored data point and to an alert. Decide pull or push, what database stores it, and how you survive a database outage.

SOLUTION

## High-level design

**Five components:** data collection, data transmission, data storage, alerting, visualization.

**Data model:** a time series is identified by a name and optional labels. Each holds `<value, timestamp>` pairs.

| Part | Type |
|---|---|
| Metric name | String |
| Tags/labels | List of `<key:value>` pairs |
| Values | Array of `<value, timestamp>` pairs |

Example point: `cpu.load`, labels `host:i631,env:prod`, timestamp `1613707265`, value `0.29`.

Average CPU on us-west web servers over 10 minutes = average the values of lines like this (the **line protocol**, used by Prometheus and OpenTSDB):

```
CPU.load host=webserver01,region=us-west 1613707265 50
CPU.load host=webserver02,region=us-west 1613707265 43
...
```

**Access pattern:** writes are heavy and constant (10M metrics, many at high frequency). Reads are **spiky** (dashboards and alert queries come in bursts).

**Storage choice:**

| Option | Verdict |
|---|---|
| Relational DB (MySQL) | Needs expert tuning at this scale; rolling-window queries are messy SQL; an index per tag; weak under constant heavy writes |
| General NoSQL (Cassandra, Bigtable) | Works, but you need deep internals knowledge to design a time-series schema |
| **Time-series DB** | **Chosen.** Far fewer servers for the same data; time-series query languages; built-in retention and aggregation |

- OpenTSDB runs on Hadoop/HBase (extra complexity). Twitter uses MetricsDB; Amazon has Timestream.

- Most popular: **InfluxDB** and **Prometheus**. Both use an in-memory cache plus on-disk storage. InfluxDB with 8 cores and 32 GB RAM handles **over 250,000 writes/sec**.
- A good TSDB indexes labels for fast lookup. Keep each label **low cardinality** (few possible values).
- In the interview you don't need TSDB internals. Say "metrics are time series, so use a TSDB like InfluxDB."

**Architecture:**

```
Metrics source ──► Metrics collector ──► Time-series DB ◄── Query service ◄──┐
Alerting system ──► email / phone / PagerDuty / webhooks                      │
(app servers,                                                                 └──
Visualization system
 SQL DBs, queues)
```

- **Metrics source:** app servers, SQL databases, message queues, etc.
- **Metrics collector:** gathers metrics and writes them to the TSDB.
- **Time-series DB:** stores series, offers a custom query interface, indexes labels.
- **Query service:** a thin wrapper over the TSDB; could be replaced by the TSDB's own interface.
- **Alerting system:** sends notifications. **Visualization system:** graphs and charts.

## Deep dives

### Metrics collection: pull vs push

For counters and CPU usage, losing a point now and then is fine, so clients can fire and forget.

**Pull model:** dedicated collectors pull from each app's HTTP endpoint (e.g. `/metrics`, exposed by a client library).

1. The collector gets endpoint config from **service discovery** (etcd, ZooKeeper): pull interval, IPs, timeout, retries.
2. It pulls metrics over HTTP.
3. It registers for change events (or polls) so it sees new and removed servers.

- A static file of IPs is hard to keep current as servers come and go; service discovery fixes that.
- One collector can't handle thousands of servers, so use a pool. To stop two collectors pulling the same server, put collectors on a **consistent hash ring** and map each server by name. Each server then has exactly one collector (e.g. collector 2 owns servers 1 and 5).

**Push model:** a **collection agent** on every server pushes metrics to the collector.

- The agent can pre-aggregate (e.g. a counter per minute) to cut volume.

- If the collector rejects pushes, the agent can buffer locally and resend. In auto-scaling groups servers disappear, so buffered data can be lost.
- Put collectors behind a **load balancer** in an **auto-scaling cluster** (scale on CPU) so they don't fall behind.

| | Pull | Push |
|---|---|---|
| Easy debugging | `/metrics` can be viewed any time, even on a laptop. **Pull wins** | |
| Health check | No response to a pull → server is down. **Pull wins** | Missing metrics might just be a network issue |
| Short-lived jobs | Jobs may end before a pull (fix: push gateway) | **Push wins** |
| Firewalls / complex networks | Every endpoint must be reachable; hard across data centers | Collector behind LB + auto-scaling receives from anywhere. **Push wins** |
| Performance | Usually TCP | Usually UDP, lower latency (counter: TCP setup is small next to the payload) |
| Data authenticity | Targets are listed in config, so data is authentic | Anyone can push; fix with whitelists or auth |

- Examples: pull = **Prometheus**; push = **Amazon CloudWatch**, **Graphite**.
- Knowing the trade-offs matters more than picking a winner. A large org probably needs both, especially with serverless, where you may not be able to install an agent.

### Scaling the transmission pipeline

- The collector cluster auto-scales in both models. But if the TSDB is down, data is lost.
- **Add a queue:** collector → **Kafka** → consumers / stream processors (Storm, Flink, Spark) → TSDB.
- Kafka is reliable and scalable.
- It decouples collection from processing.
- It keeps data while the DB is unavailable.
- **Scale through Kafka partitions:**
- Set the partition count from throughput needs.
- Partition by **metric name** so consumers can aggregate per metric.
- Further partition by tags/labels.
- Categorize and prioritize so important metrics go first.
- **Alternative:** running Kafka at production scale is hard. Facebook's **Gorilla** in-memory TSDB has no queue and stays write-available during partial network failures; arguably as reliable.

### Where aggregation happens

| Place | How | Trade-off |
|---|---|---|
| Collection agent | Simple logic only, e.g. aggregate a counter each minute | Limited |
| Ingestion pipeline | Stream processing (Flink) before writing | Much less write volume; but late events are hard, and raw data and flexibility are lost |
| Query side | Aggregate raw data at query time | No data loss; slower queries over the whole dataset |

### Query service

- A cluster of query servers between the TSDB and the visualization/alert systems. It decouples them so either side can be swapped.
- Add a **cache layer** for query results to cut TSDB load.
- **The case against it:** most visualization and alert tools already have plugins for popular TSDBs, and a good TSDB needs no extra cache.
- **Query language:** Prometheus and InfluxDB don't use SQL. An exponential moving average is hard to read in SQL but one line in Flux:

```
from(db:"telegraf") ▷ range(start:-1h) ▷ filter(fn: (r) ⇒ r._measurement == "foo")
▷ exponentialMovingAverage(size:-10s)
```

### Storage layer

- **Pick the TSDB carefully:** per a Facebook paper, **at least 85%** of queries hit data from the **past 26 hours**. A TSDB that exploits this performs much better.
- **Delta encoding and compression:** `1610087371` and `1610087381` differ by 10 s, which needs 4 bits, not 32. Store a base and deltas: `1610087371, 10, 10, 9, 11`.
- **Downsampling:** convert old high-resolution data to lower resolution. Rules per metric, e.g. 7 days raw; 30 days at 1 minute; 1 year at 1 hour.
- Example (10 s → 30 s, host-a cpu): values 10, 16, 20 at 19:00:00–19:00:20 → **19** at 19:00:00; values 30, 20, 30 at 19:00:30–19:00:50 → **25** at 19:00:30.
- **Cold storage:** move rarely used data to much cheaper storage.

### Alert system




1. Alert rules live as config files (YAML) and are loaded into a cache. Example:

```yaml
- name: instance_down
  rules:
  # Alert for any instance that is unreachable for >5 minutes.
  - alert: instance_down
    expr: up == 0
    for: 5m
    labels:
      severity: page
```

1. The **alert manager** reads rule configs from the cache.
2. It queries the query service on a set interval. A threshold violation creates an alert event. It also: - Filters, merges, and dedupes (e.g. three `disk_usage > 90%` events on instance 1 → **one alert**). - Enforces access control on alert management. - Retries, checking alert state, so each notification goes out **at least once**.
3. The **alert store** (a KV DB like Cassandra) keeps each alert's state: inactive, pending, firing, resolved.
4. Eligible alerts go into **Kafka**.
5. **Alert consumers** pull from Kafka.
6. They send notifications to email, text, PagerDuty, or HTTP endpoints.

**Build vs buy:** many off-the-shelf alert systems integrate tightly with popular TSDBs and channels. Building your own is hard to justify. For senior roles, be ready to defend the choice.

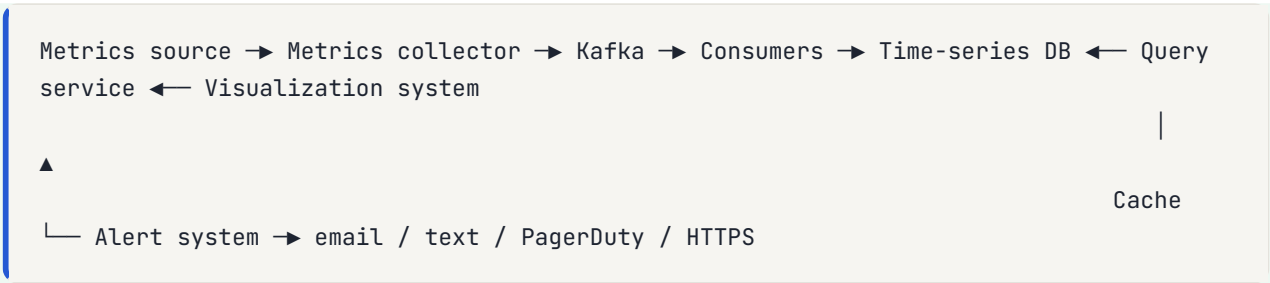
### Visualization

Metrics and alerts dashboards over different time ranges. A good one is hard to build, so the case for buying is strong: **Grafana** integrates with many popular TSDBs.

## Wrap-up

Topics covered: pull vs push collection, scaling with Kafka, choosing a TSDB, downsampling, and build vs buy for alerting and visualization.

Final design:




✓ KEY POINTS

- Pin the scope first: operational metrics only, no logs, no tracing. ~10M metrics, 1-year retention with rollups.
- Metrics are time series with labels; the load is write-heavy and read-spiky, so use a TSDB with low-cardinality labels.
- Compare pull vs push with the trade-off table. Pull uses service discovery plus a consistent hash ring of collectors; push uses agents plus an auto-scaled, load-balanced collector cluster.
- Put Kafka between collectors and the TSDB, partitioned by metric name, so a DB outage loses nothing.
- Cut storage with delta encoding, downsampling, and cold storage. Recent data (past 26 h) gets most queries.
- Alerting: YAML rules → alert manager (dedupe/merge, state in a KV store) → Kafka → channel consumers. Prefer buying alerting and dashboards.

⚠ COMMON TRAP

declaring pull or push the "right" answer. The book's point is that both are used widely; explain the trade-offs and note a big company likely needs both.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·6 Ad Click Event Aggregation

ad click aggregation · real-time bidding · RTB · stream processing · MapReduce · Kafka · Flink · watermark · tumbling window · sliding window · exactly-once · lambda architecture · kappa architecture · reconciliation · hotspot · star schema · top N

SEE ALSO Kafka vs RabbitMQ

**Source:** *System Design Interview* Vol 2, Ch 6.

## Problem

↑ TOP

Design an ad click event aggregation system at Facebook or Google scale. Its results drive ad billing and real-time bidding (RTB), so they must be correct.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Input format? | Log files on many servers. New click events are appended. Fields: `ad_id`, `click_timestamp`, `user_id`, `ip`, `country` |
| Data volume? | 1 billion clicks per day, 2 million ads. Clicks grow 30% per year |
| Queries? | (1) clicks for one ad in the last M minutes; (2) top 100 most clicked ads in the past 1 minute (both configurable, aggregate every minute); (3) filter both by `ip`, `user_id` or `country` |
| Edge cases? | Late events, duplicate events, any part may fail. Handle all of them |
| Latency? | A few minutes end to end. (RTB itself needs < 1 s, but aggregation is for billing and reporting) |

**Requirements:**

- Functional: count clicks of an `ad_id` in the last M minutes; return the top 100 most clicked ads every minute; filter by attributes; Facebook/Google scale.
- Non-functional: correct results (used for RTB and billing); handle late and duplicate events; resilient to partial failures; end-to-end latency of a few minutes at most.

**Estimates:**

- 1B DAU, 1 click each per day → 1B events/day.
- QPS = 10^9 / 10^5 s = **10,000**. Peak = 5× = **50,000 QPS**.
- 0.1 KB per event → **100 GB/day**, about **3 TB/month**.

> Try it first: sketch the path from log files to a dashboard, how you count per minute and get the top N, and how you avoid counting a click twice after a crash.

SOLUTION

## High-level design

**Query API** (clients are dashboard users: data scientists, advertisers, PMs). Filtering is a query parameter, so two APIs cover all three use cases.

| API | Params | Response |
|---|---|---|
| `GET /v1/ads/{:ad_id}/aggregated_count` | `from` (start minute, default now – 1 min), `to` (end minute, default now), `filter` (e.g. `001` = drop non-US clicks) | `ad_id`, `count` |
| `GET /v1/ads/popular_ads` | `count` (top N), `window` (M minutes), `filter` | `ad_ids` (array) |

**Data model:**

- Raw event: `[AdClickEvent] ad001, 2021-01-01 00:00:01, user 1, 207.148.22.22, USA`. Columns: `ad_id, click_timestamp, user_id, ip, country`.
- Aggregated: `ad_id | click_minute | count`, e.g. `ad001 | 202101010000 | 5`.
- With filters: `ad_id | click_minute | filter_id | count`. A separate filter table maps `filter_id` to criteria (region, IP, user_id), e.g. `0012` = region US.
- Top N: `most_clicked_ads(window_size int, update_time_minute timestamp, most_clicked_ads array of ad IDs in JSON)`.

**Raw vs aggregated data:**

| | Raw data only | Aggregated data only |
|---|---|---|
| Pros | Full data set; supports filters and recalculation | Smaller data set; fast query |
| Cons | Huge storage; slow query | Data loss: it is derived (10 entries → 1) |

Store **both**. Raw data is the backup: use it for debugging and to recalculate after a bug. Move old raw data to cold storage. Aggregated data is the active data, tuned for queries.

**Database choice:**

- Raw data: write-heavy (10K–50K QPS), rarely read. Relational DBs struggle to scale writes. **Cassandra** or InfluxDB fit (write-optimized, time-range queries). Another option: S3 with columnar files (ORC, Parquet, AVRO), capped at ~10 GB per file. The book uses Cassandra.
- Aggregated data: time series, read- and write-heavy. Dashboards query every ad every minute (2M ads). The service writes every minute. Same DB type works.

**Architecture.** A synchronous chain breaks when producers outrun consumers (OOM, shutdowns) or when one part dies. So add **Kafka** to decouple stages and scale them independently.




- MQ #1 holds raw click events: `ad_id, click_timestamp, user_id, ip, country`.
- MQ #2 holds per-minute results: `ad_id, click_minute, count` and `update_time_minute, most_clicked_ads`.
- **Why MQ #2 instead of writing to the DB directly?** It gives end-to-end **exactly-once** (atomic commit across MQ #1 → aggregation → MQ #2).

**Aggregation service = a DAG of Map / Aggregate / Reduce nodes** (MapReduce, really map-reduce-reduce):

- **Map:** reads, cleans, normalizes, and routes by `ad_id` (e.g. `ad_id % 3`). Why not let Kafka partition it? Input may need cleaning, and you may not control producers, so the same `ad_id` can land in different partitions.
- **Aggregate:** counts clicks per `ad_id` in memory every minute.
- **Reduce:** merges results from all Aggregate nodes.
- Nodes talk over TCP (separate processes) or shared memory (threads).

Use cases:

1. **Click count:** map by `ad_id`, each Aggregate node counts its ads.
2. **Top N:** each Aggregate node keeps a **heap** of its local top 3. Reduce merges 3 × 3 = 9 candidates into the global top 3 each minute.
3. **Filtering:** pre-aggregate per dimension, e.g. `ad001 | 202101010001 | USA | 100`, `GPB | 200`, `others | 3000`. This is a **star schema**; filter fields are dimensions. Simple, reuses the same service, fast reads. Cost: many more buckets and records.

## Deep dives

### Streaming vs batching

| | Services (online) | Batch (offline) | Streaming (near real-time) |
|---|---|---|---|
| Responsiveness | Respond fast | No client response | No client response |

| | Services (online) | Batch (offline) | Streaming (near real-time) |
|---|---|---|---|
| Input | User requests | Bounded, large | Unbounded streams |
| Output | Responses | Materialized views, aggregates | Materialized views, aggregates |
| Measured by | Availability, latency | Throughput | Throughput, latency |
| Example | Online shopping | MapReduce | Flink |

- **Lambda:** batch path + streaming path. Downside: two codebases.
- **Kappa:** one streaming engine does both real-time and reprocessing. The design uses **Kappa**.
- **Data recalculation (replay):** after a bug, (1) a recalculation service reads raw data in a batch job, (2) sends it to a **dedicated** aggregation service so live traffic is not affected, (3) writes results to MQ #2 → aggregation DB.

### Time: event time vs processing time

| | Pros | Cons |
|---|---|---|
| Event time | More accurate; the client knows when the click happened | Client clocks can be wrong or forged |
| Processing time | Server clock is reliable | Wrong if the event arrives much later (e.g. 5 hours) |

- Choice: **event time**, because accuracy matters.
- **Watermark:** extend each window by e.g. **15 s** (tunable) to catch slightly late events. Longer = more accurate but more latency.
- Very late events are rare. Do not build complex logic for them; fix with **end-of-day reconciliation**.

### Aggregation window

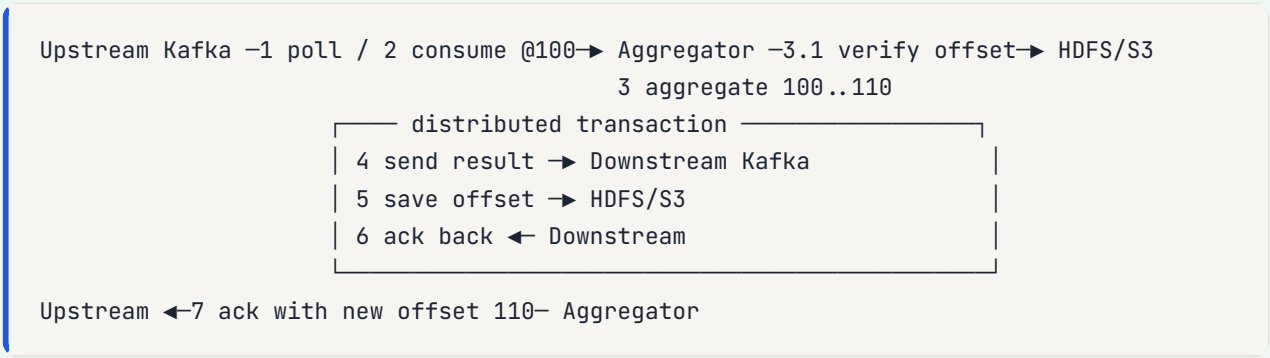
Four types (Kleppmann): tumbling, hopping, sliding, session.

- **Tumbling (fixed):** equal, non-overlapping chunks. Fits "clicks per minute" (use case 1).
- **Sliding:** a window slides over the stream, may overlap. Fits "top N in the last M minutes" (use case 2).

### Delivery guarantees and deduplication

- At-least-once is normally fine, but here a few percent error = **millions of dollars**. Use **exactly-once**.
- Duplicate sources: clients resending (malicious ones go to ad fraud / risk control), and **server outages**.

- Outage case: Aggregator consumes offsets 100–110, sends results downstream, gets the ack, then dies before acking upstream with offset 110. A new node re-reads from 100 → duplicates.
- Fix 1: store the offset in HDFS/S3 and check it before processing. If the offset is saved **before** sending downstream and the send fails, events 100–110 are **lost**.
- Fix 2: save the offset **after** the downstream ack. Crash before saving → resend → duplicates again.
- Final: put **send result + save offset + ack** in one **distributed transaction**. Any failure rolls back all.




### Scale the system

Growth is 30%/year (2× every ~3 years). MQ, aggregation service and DB are decoupled, so scale each alone.

- **Kafka consumers:** add consumers to the group; rebalance spreads partitions. With hundreds of consumers, rebalance can take minutes, so add them off-peak.
- **Kafka brokers:**
- Hash key = `ad_id`, so one ad's events go to one partition.
- **Pre-allocate enough partitions.** Changing the count remaps `ad_id`s.
- Shard topics by geography (`topic_north_america`, ...) or business (`topic_web_ads`, `topic_mobile_ads`). Pro: more throughput, faster rebalances. Con: more complexity and maintenance.
- **Aggregation service:** horizontal. Two ways to raise throughput: (1) different `ad_id`s on different threads (simpler); (2) run nodes on a resource provider like **Hadoop YARN** (multi-processing). Option 2 is more common because you just add compute.
- **Database:** Cassandra scales horizontally with **virtual nodes** on a hash ring. A new node triggers automatic rebalancing. No manual resharding.
- **Hotspots:** big advertisers get far more clicks. Example: a node handles 100 events but gets 300 for ad1. It asks the **resource manager** for more nodes, splits the events into 3 groups of 100, and the reduced result returns to the original node. More advanced: Global-Local Aggregation, Split Distinct Aggregation (Flink).

### Fault tolerance

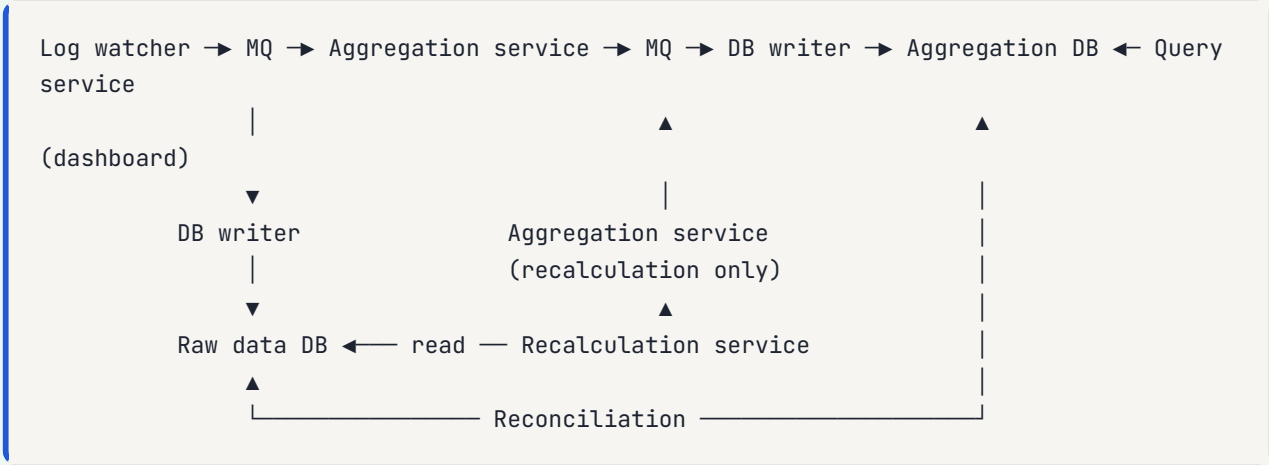
- Aggregation is in memory, so a crash loses state. Replaying Kafka from the start is slow.

- Save a **snapshot** of system state: the upstream offset **plus** state like the top-N data (e.g. top 3 of the last 5 minutes with per-minute counts per ad).
- Failover: start a new node, load the latest snapshot, replay events after the snapshot from Kafka.

### Monitoring and reconciliation

- Monitor: **latency** per stage (track timestamps as events flow); **Kafka records-lag** (not "queue size"); CPU, disk, JVM on aggregation nodes.
- **Reconciliation:** no third party to compare with. A batch job sorts each partition's events by event time at end of day and compares with the real-time result. Use a smaller window (e.g. 1 hour) for more accuracy. Small mismatches remain because of late events.

**Final design:**




**Alternative design:** Log watcher → MQ → risk control engine → **Hive** (+ ElasticSearch for data scientist queries) and **ClickHouse** (or Druid) OLAP for merchant-facing analytics. In a generalist interview a generic design with clear trade-offs is enough.

## Wrap-up

Covered: data model and API; MapReduce aggregation; scaling MQ, aggregation service and DB; hotspots; monitoring; reconciliation; fault tolerance. Knowing Kafka, Flink or Spark makes this design easier.

✓ KEY POINTS

- Two Kafka queues decouple ingestion, aggregation and storage, and enable **exactly-once** end to end.
- Aggregate with **Map → Aggregate → Reduce**; use local heaps + a reducer for top N.
- Keep **raw data** for replay and reconciliation; serve queries from **aggregated data**.
- **Event time + watermark**; tumbling window for counts, sliding window for top N.
- Partition by `ad_id`, pre-allocate partitions, handle hot ads with extra nodes.
- **Snapshots** (offset + in-memory state) make failover fast.

⚠ COMMON TRAP

settling for at-least-once delivery. Duplicates here are billing errors worth millions. Also, saving the offset before the downstream ack loses data.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·7 Hotel Reservation System

hotel reservation · Marriott · Airbnb · booking · double booking · idempotency key · overbooking · room type inventory · pessimistic locking · optimistic locking · database constraint · SELECT FOR UPDATE · race condition · sharding · Redis cache · CDC · 2PC · saga

SEE ALSO Transactions & Isolation Levels · Locking & Concurrency Control

**Source:** *System Design Interview* Vol 2, Ch 7.

## Problem

↑ TOP

Design a hotel reservation system for a hotel chain like Marriott. The same ideas apply to Airbnb, flight booking and movie ticket booking.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Scale? | Hotel chain website: 5,000 hotels, 1 million rooms |
| When do customers pay? | In full, when they reserve |
| Booking channels? | Hotel website or app only (no phone) |
| Can customers cancel? | Yes |
| Anything else? | Allow **10% overbooking** (sell more rooms than exist, expecting cancellations) |
| Room search? | Out of scope |
| Pricing? | Dynamic. The price can differ each day, based on expected occupancy |

**Requirements:**

- Functional: show hotel page; show room detail page; reserve a room; admin panel to add/remove/update hotel or room info; support overbooking.
- Non-functional: **high concurrency** (many people booking the same popular room in peak season); **moderate latency** (a few seconds to process a reservation is fine).

**Estimates:**

- Daily reservations: 1M × 70% / 3 days ≈ 233,333 → **~240,000**.
- Reservations per second: 240,000 / 10^5 ≈ **3 TPS**. Not high.
- Funnel (10% move to the next step, no prefetching): view hotel/room detail **300 QPS** → order confirmation page **30 QPS** → reserve **3 QPS**.

> Try it first: sketch the services, the tables (think: what does a guest actually reserve?), and how you stop two people from getting the last room.

SOLUTION

## High-level design

**API** (RESTful; search APIs are out of scope):

| API | Detail |
|---|---|
| `GET /v1/hotels/ID` | Hotel details |
| `POST / PUT / DELETE /v1/hotels[/ID]` | Add / update / delete a hotel (staff only) |
| `GET /v1/hotels/ID/rooms/ID` | Room details |
| `POST / PUT / DELETE /v1/hotels/ID/rooms[/ID]` | Add / update / delete a room (staff only) |
| `GET /v1/reservations` | Reservation history of the logged-in user |
| `GET /v1/reservations/ID` | Reservation details |
| `POST /v1/reservations` | Make a reservation |
| `DELETE /v1/reservations/ID` | Cancel a reservation |

`POST /v1/reservations` body: `startDate`, `endDate`, `hotelID`, `roomID`, `reservationID`. The `reservationID` is the **idempotency key** that prevents double booking.

**Database: relational.**

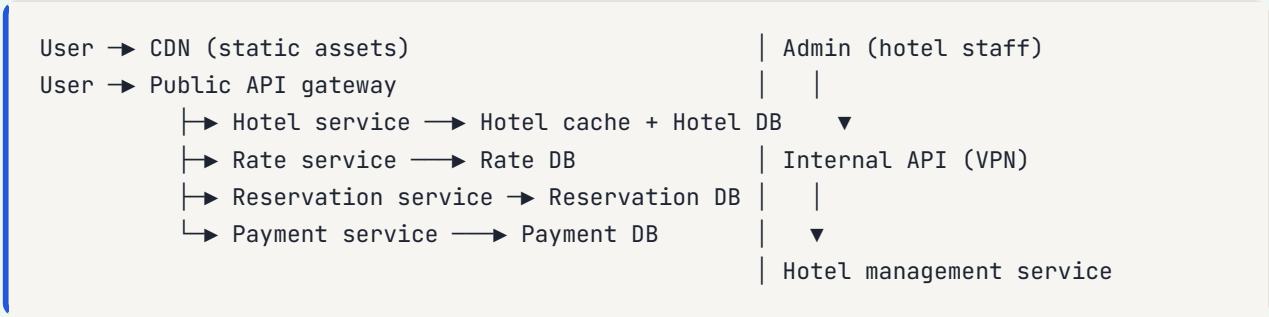
- Read-heavy, write-light: far more people browse than book. NoSQL is usually optimized for writes.
- **ACID** matters: it prevents negative balance, double charges and double reservations, and keeps app code simple.
- The data is clearly relational and stable (hotel, room, room_type, ...).

**First schema** (one service per box):

- Hotel service: `hotel(hotel_id PK, name, address, location)`, `room(room_id PK, room_type_id, floor, number, hotel_id, name, is_available)`.
- Rate service: `room_type_rate(hotel_id PK, date PK, rate)`.
- Reservation service: `reservation(reservation_id PK, hotel_id, room_id, start_date, end_date, status, guest_id)`.
- Guest service: `guest(guest_id PK, first_name, last_name, email)`.
- Reservation status: `pending → paid → refunded`, or `pending → cancelled`, or `pending → rejected`.

**Problem with this schema:** it works for Airbnb, where the user picks a specific listing. Hotel guests reserve a **room type** (standard, king, two queens). The room number is assigned at check-in. Fixed in the deep dive.

**Architecture (microservices):**




- **API gateway:** managed; rate limiting, auth, routes by endpoint.
- **Hotel service:** hotel and room info. Mostly static, so cache it.
- **Rate service:** room prices for future dates (price depends on expected occupancy).
- **Reservation service:** takes reservations and tracks room inventory on book/cancel. Calls the rate service to compute the total charge.
- **Payment service:** charges the customer; sets status to `paid` on success or `rejected` on failure.
- **Hotel management service:** staff only; view upcoming reservations, book for a customer, cancel. Forwards changes to the service that owns the data.
- Services talk over an RPC framework like **gRPC**.

## Deep dives

### Improved data model

- The reservation API uses `roomTypeID` instead of `roomID`, plus `roomCount`.
- `reservation` now stores `room_type_id` instead of `room_id`.
- New table in the reservation service:

```
room_type_inventory
  hotel_id          ┐
  room_type_id      ├ composite primary key
  date              ┘
  total_inventory   -- total rooms minus rooms taken off for maintenance
  total_reserved    -- rooms booked for this hotel, room type and date
```

- One row per date makes date-range bookings and queries easy.
- Rows are **pre-populated for 2 years**. A daily job adds new dates.
- Size: 5,000 hotels × 20 room types × 2 years × 365 days = **73M rows**. One DB is enough; replicate across regions/AZs for availability.

Check availability (input: start/end date, room type, hotel, number of rooms):

```sql
SELECT date, total_inventory, total_reserved
FROM room_type_inventory
WHERE room_type_id = ${roomTypeId} AND hotel_id = ${hotelId}
  AND date BETWEEN ${startDate} AND ${endDate}
-- then, for every row:
--   total_reserved + ${numberOfRoomsToReserve} ⩽ total_inventory
-- with 10% overbooking:
--   total_reserved + ${numberOfRoomsToReserve} ⩽ 110% * total_inventory
```

If reservation data outgrows one DB:

- Keep only current and future reservations. Archive history, some to cold storage.
- **Shard by** `hotel_id` (`hash(hotel_id) % servers`): both making and looking up a reservation start from a hotel.

### Concurrency 1: the same user clicks "book" twice

- **Client side:** gray out / disable the button. Not reliable (JavaScript disabled, network retries).
- **Idempotency key (chosen):** 1. User fills details → reservation service generates a **reservation order**. 2. It returns a review page with a unique `reservation_id` from a global ID generator. 3. 3a: submit with `reservation_id`. 3b: a second submit hits the **unique constraint** on the primary key and fails.
- The key doesn't have to be `reservation_id`; it is used because it already exists.

### Concurrency 2: two users want the last room

Race (isolation below serializable; 100 rooms, 99 reserved): T1 and T2 both check `total_reserved + 1 ⩽ 100` → both true. Both update `total_reserved` to 100 and commit. Two bookings for one room.

Reserve-room pseudo-code (one transaction):

```
# step 1: check inventory
SELECT date, total_inventory, total_reserved FROM room_type_inventory WHERE ... date
range
if ((total_reserved + ${numberOfRoomsToReserve}) > 110% * total_inventory)
{ Rollback }
# step 2: reserve
UPDATE room_type_inventory SET total_reserved = total_reserved + $
{numberOfRoomsToReserve}
WHERE room_type_id = ${roomTypeId} AND date BETWEEN ${startDate} AND ${endDate}
Commit
```

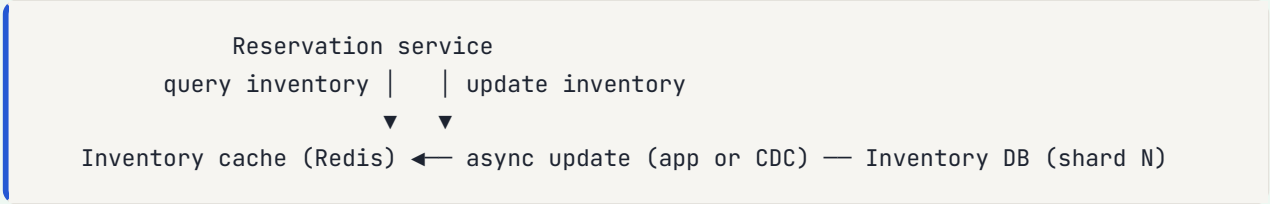
Three fixes:

| Option | How | Pros | Cons |
|---|---|---|---|
| **Pessimistic locking** | `SELECT … FOR UPDATE` locks the rows; T2 waits for T1, then sees 100 and rolls back | Easy; serializes updates; good under heavy contention | **Deadlocks** with many locked resources; **not scalable**, long locks hurt DB performance |
| **Optimistic locking** | Add a `version` column. Read version, write version + 1. DB rejects the write if the version is not exactly current + 1; client retries | No DB locks (app logic only); fast when conflicts are rare | Poor under heavy contention: everyone reads the same version, one wins, the rest retry again and again → bad UX |
| **Database constraint** | `CHECK (total_inventory - total_reserved ⩾ 0)` ; T2 makes it 101 → violates → rollback | Easy; fine under low contention | Many failures under contention ("rooms shown, then none"); constraints are hard to version-control; not all DBs support them (migration risk) |

- Version numbers beat timestamps for optimistic locking, because server clocks drift.
- **Book's choice:** not pessimistic. **Optimistic locking or the DB constraint** are both good, because reservation QPS is low.

### Scalability (e.g. booking.com / expedia.com, 1,000× QPS)

- Services are stateless → add servers. The **database** is the bottleneck.
- **Sharding by `hotel_id`:** e.g. 16 shards; 30,000 QPS / 16 = **1,875 QPS per shard**, fine for one MySQL server.
- **Inventory cache (Redis):** only current and future inventory matters, so use **TTL** to expire old data and **LRU** eviction.
- Key: `hotelID_roomTypeID_{date}` → value: number of available rooms.
- Reservation service **queries inventory in the cache**, **updates inventory in the DB**. The DB then updates the cache **asynchronously** (from app code or via **CDC**, e.g. Debezium).
- Most ineligible requests stop at the cache. Reads outnumber writes by an order of magnitude.
- The cache can be stale. That is fine: the **DB is the source of truth** and re-validates every booking. Worst case, the user sees "someone just booked the last room."
- Pros: much lower DB load; fast in-memory reads. Con: keeping cache and DB consistent is hard; think about its UX impact.



### Data consistency among services

- The design is **hybrid**: the reservation service owns both the reservation and inventory tables in **one relational DB**, so ACID handles the concurrency issues above.
- A microservice purist wants one DB per service. Then one logical operation spans services and DBs; no single transaction. If the reservation DB update fails, the inventory DB change must be rolled back. One happy path, many failure paths.
- Options:
- **Two-phase commit (2PC):** atomic commit across nodes. **Blocking**: one node failure stalls everyone. Not performant.
- **Saga:** a chain of local transactions, each publishing a message to trigger the next. On failure, run **compensating transactions**. Eventually consistent.
- Book's choice: not worth the complexity here. Keep reservation and inventory in the same relational DB.

## Wrap-up

Covered: requirements and estimates; API, first data model, architecture; switching to room-type-level reservations; race conditions and three fixes (pessimistic, optimistic, constraint); scaling with sharding and a Redis cache; data consistency across microservices (2PC, Saga).

✓ KEY POINTS

- Guests reserve a **room type**. Model inventory as `room_type_inventory`, one row per (hotel, room type, date).
- Load is tiny (~3 TPS) but bursty and contended. Pick a **relational DB for ACID**.
- `reservation_id` as an **idempotency key** stops double submits.
- Low contention → **optimistic locking or a CHECK constraint**; avoid pessimistic locks (deadlocks, poor scaling).
- Overbooking is just `110% * total_inventory` in the check.
- At booking.com scale: shard by `hotel_id`, cache inventory in Redis, but **always re-check in the DB**.

⚠ COMMON TRAP

modeling reservations per specific `room_id` (the Airbnb model). Hotels sell room types, and room numbers are assigned at check-in. Another trap: trusting the cache count without re-validating in the DB.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·8 Distributed Email Service

email service Gmail Outlook SMTP IMAP POP MX record webmail WebSocket attachment store S3 metadata database Bigtable denormalization read unread conversation threads deliverability SPF DKIM DMARC Elasticsearch LSM tree

SEE ALSO SQL vs NoSQL & Scaling Databases

**Source:** *System Design Interview* Vol 2, Ch 8.

## Problem

↑ TOP

Design a large-scale email service like Gmail, Outlook or Yahoo Mail.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| How many users? | 1 billion |
| Features? | Send and receive email; fetch all emails; filter by read/unread; search by subject, sender, body; anti-spam and anti-virus. Authentication is out of scope |
| How do clients connect? | Traditionally SMTP, POP, IMAP and vendor protocols. Here, assume **HTTP** |
| Attachments? | Yes |

**Requirements:**

- Functional: the feature list above.
- Non-functional:
- **Reliability:** never lose email data.
- **Availability:** replicate email and user data across nodes; keep working through partial failures.
- **Scalability:** performance must not drop as users and emails grow.
- **Flexibility and extensibility:** easy to add features. POP/IMAP are too limited, so custom protocols may be needed.

**Estimates:**

- Send QPS: 10^9 users × 10 emails / 10^5 s = **100,000**.
- Received: 40 emails/user/day. Metadata (everything except attachments) = 50 KB.
- Metadata per year: 1B × 40 × 365 × 50 KB = **730 PB**.
- 20% of emails have an attachment, avg 500 KB. Attachments per year: 1B × 40 × 365 × 20% × 500 KB = **1,460 PB**.

- So: a distributed database is needed.

> Try it first: sketch the send flow and the receive flow (with queues and spam checks), then the metadata tables for "folders", "emails in a folder" and "unread emails".

SOLUTION

## High-level design

**Email 101:**

- **SMTP:** sends mail between servers.
- **POP:** downloads mail to one device and deletes it from the server. Downloads the whole email, even big attachments.
- **IMAP:** keeps mail on the server, downloads a message only when opened (headers first). Works on many devices. Most used for personal accounts.
- **HTTPS:** not a mail protocol, but used by webmail (e.g. Outlook's ActiveSync for mobile).
- **DNS MX records:** find the recipient's mail server. Lower priority number = preferred (e.g. `gmail-smtp-in.l.google.com` priority 5, then `alt1…` priority 10).
- **Attachments:** Base64-encoded, sent with **MIME**. Limits: Outlook 20 MB, Gmail 25 MB (June 2021).

**Traditional mail servers** (Alice on Outlook → Bob on Gmail):

1. Outlook client → Outlook mail server (SMTP).
2. Outlook server looks up Gmail's SMTP server via DNS and sends the email (SMTP).
3. Gmail server stores it.
4. Bob's client fetches it via IMAP/POP.

- Storage was **Maildir**: one file per email in per-user directories. Fine for few users. At billions of emails, disk I/O becomes the bottleneck, backups are hard, and a dead disk loses data.
- Old protocols were not built for threading, search, labels or billions of users.

**Webmail API** (HTTP):

| Endpoint | Purpose |
|---|---|
| `POST /v1/messages` | Send to recipients in To, Cc, Bcc |
| `GET /v1/folders` | All folders: `[{id, name, user_id}]`. Defaults (RFC 6154): All, Archive, Drafts, Flagged, Junk, Sent, Trash |
| `GET /v1/folders/{folder_id}/messages` | Messages in a folder. Real version needs consecutive paging (1–50, 51–100) and range paging (73–87) |

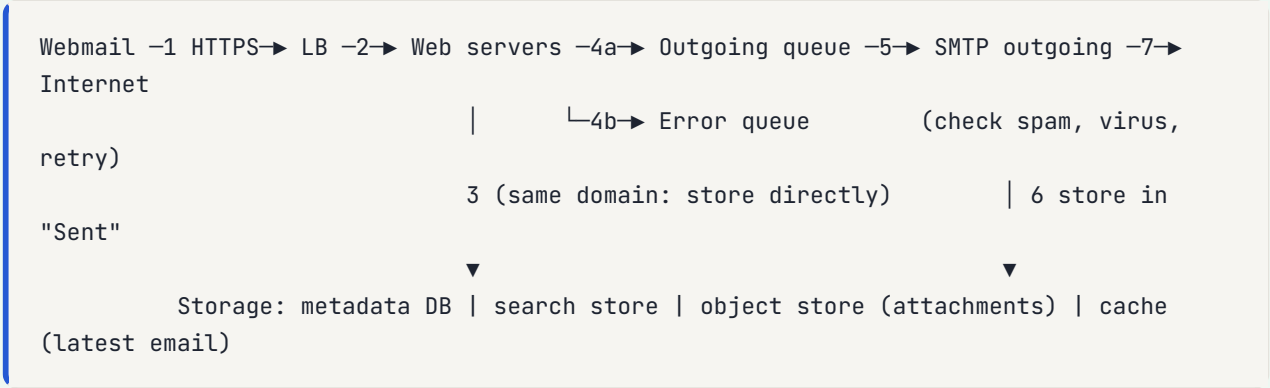
| Endpoint | Purpose |
|---|---|
| `GET /v1/messages/{message_id}` | One message: `user_id, from {name, email}, to [{name, email}], subject, body, is_read` |

**Distributed architecture:**




- **Web servers:** all API requests (send, load folders, load emails) plus login, signup, profile.
- **Real-time servers:** push new mail. **Stateful** (persistent connections). WebSocket, with **long polling as fallback** for old browsers. Example: Apache James uses JMAP over WebSocket.
- **Metadata DB:** subject, body, from, to, ... (chosen in the deep dive).
- **Attachment store: S3.** Attachments up to 25 MB. Not Cassandra: its blob limit is 2 GB in theory but **< 1 MB in practice**, and big blobs prevent using the row cache.
- **Distributed cache: Redis.** Recent emails are loaded again and again. Redis has lists and scales easily.
- **Search store:** distributed document store with an **inverted index** for full-text search.

**Sending flow:**

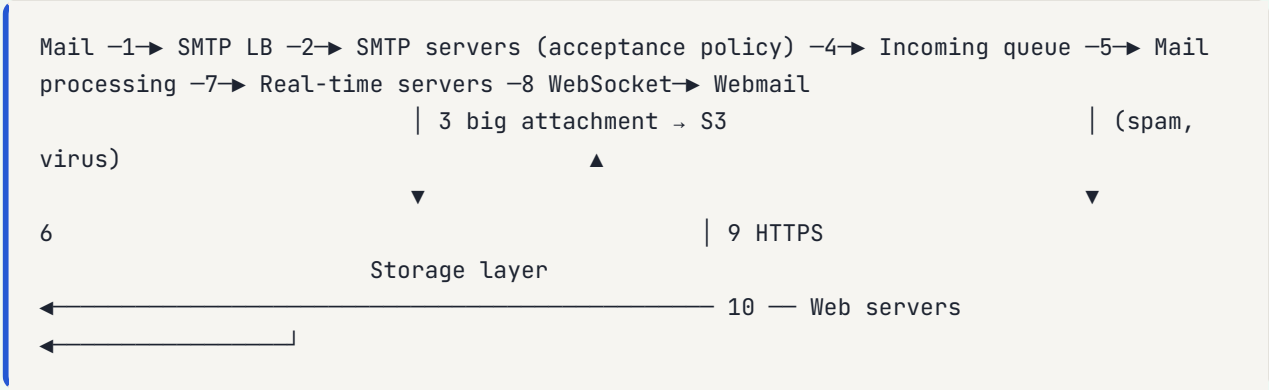



1. User sends → load balancer.
2. LB applies the **rate limit** and routes to web servers.
3. Web servers do basic validation (e.g. size limit). If the recipient is on **the same domain**, store directly in DB, cache and object store; done.
4. Valid → **outgoing queue**; invalid → **error queue**.
5. SMTP outgoing workers pull, check spam and viruses.
6. Store the email in the **Sent** folder.

7. Send to the recipient's mail server.

- The queue decouples web servers from SMTP workers, so workers scale alone. Each message holds all metadata to build the email.
- **Monitor the outgoing queue size.** Stuck mail means: the recipient server is down (retry with **exponential backoff**) or too few consumers (add more).

**Receiving flow:**




- **1–2:** LB spreads mail across SMTP servers. An **acceptance policy** at SMTP-connection level bounces invalid mail early.
- **3:** attachments too big for the queue go to S3.
- **4:** the **incoming queue** decouples SMTP servers from processing workers and buffers surges.
- **5:** workers filter spam and viruses.
- **6:** store in mail storage, cache and object store.
- **7–8:** recipient online → push via real-time (WebSocket) servers.
- **9–10:** recipient offline → mail waits in storage. When back, the client calls web servers (REST), which pull new mail.

## Deep dives

### Metadata database

Characteristics:

- Headers: small, read often. Bodies: any size, read rarely (usually once).
- Operations are **per user**: a user only touches their own mail.
- Recency matters: **82% of reads are for data younger than 16 days**.
- **No data loss** allowed.

Options:

| Option | Verdict |
|---|---|
| Relational (MySQL, PostgreSQL) | Indexes make simple search fast, but emails are big (100 KB+ with HTML). Search over BLOBs is slow. **Not a good fit** |
| Distributed object storage (S3) | Good for backup. Hard to support mark-as-read, keyword search, threading |
| NoSQL | Gmail uses **Bigtable** (not open source). Cassandra could work, but no big provider is known to use it |

Big providers build **custom DBs** to cut IOPS. In an interview, describe the needed properties instead:

- A single column can be single-digit MB.
- **Strong consistency.**
- Designed to reduce disk I/O.
- Highly available and fault-tolerant.
- Easy incremental backups.

### Data model

- Partition key = **user_id**: one user's data sits on one shard (messages are not shared between users, which is fine here). Partition key spreads data; clustering key sorts inside a partition.

```
folders_by_user:   user_id UUID (K), folder_id UUID, folder_name TEXT
emails_by_folder:  user_id UUID (K), folder_id UUID (K), email_id TIMEUUID (C↓),
                   from TEXT, subject TEXT, preview TEXT, is_read BOOLEAN
emails_by_user:    user_id UUID (K), email_id TIMEUUID (C↓), from TEXT, to LIST<TEXT>,
                   subject TEXT, body TEXT, attachments LIST<filename|size>
attachments:       email_id TIMEUUID, filename TEXT, url TEXT
read_emails / unread_emails: user_id (K), folder_id (K), email_id TIMEUUID (C↓), from,
subject, preview
```

- **Q1 folders for a user:** `folders_by_user`, partition `user_id`.
- **Q2 emails in a folder:** composite partition `<user_id, folder_id>`. `email_id` is a **TIMEUUID** clustering key, so emails sort newest first.
- **Q3 get an email:** `SELECT * FROM emails_by_user WHERE email_id = 123;` Attachments are fetched by `email_id + filename`.
- **Q4 read/unread:** `WHERE … AND is_read = true` is fine in SQL, but NoSQL only queries partition and clustering keys, so it rejects this. Fetching the whole folder and filtering in the app does not scale. Fix: **denormalize** into `read_emails` and `unread_emails`. Mark as read = delete from `unread_emails` + insert into `read_emails`. Query: `SELECT * FROM unread_emails WHERE user_id = ? AND folder_id = ? ORDER BY email_id;` More complex code, much faster reads.

- **Bonus, threads:** headers carry `Message-Id` (set by the sending client), `In-Reply-To` (parent's Message-Id) and `References` (IDs in the thread). With the reply chain preloaded, the client rebuilds the thread (e.g. **JWZ algorithm**).

**Consistency trade-off:** correctness matters, so each mailbox has a **single primary**. During failover the mailbox is unavailable and sync pauses. **Consistency over availability.**

#### Email deliverability

Sending is easy; landing in the inbox is hard. Over **50% of all email is spam**, and a new server has no reputation.

- **Dedicated IPs** for sending; providers distrust new IPs with no history.
- **Classify emails:** send marketing and important mail from different IPs, or ISPs may mark everything promotional.
- **Warm up** new IPs slowly with Gmail, Office365, Yahoo: about **2–6 weeks** (Amazon SES).
- **Ban spammers quickly** before they hurt reputation.
- **Retries:** after max retries, park the event in a queue and monitor it.
- **Feedback loops with ISPs:** route to separate queues for **hard bounces** (invalid address), **soft bounces** (temporary, e.g. ISP busy) and **complaints** ("report spam").
- **Email authentication** against phishing (93% of breaches, Verizon 2018): **SPF, DKIM, DMARC**.

#### Search

Email search reindexes on every send, receive and delete, but runs only when the user clicks search. So it is **write-heavy**.

| | Scope | Sorting | Accuracy |
|---|---|---|---|
| Google search | Whole internet | Relevance | Indexing takes time; new items can lag |
| Email search | User's own mailbox | Time, has attachment, date, unread, ... | Near real-time and accurate |

- **Option 1: Elasticsearch.** Partition documents by `user_id`. Search is **sync** (user waits). Send/receive/delete events go **async** through **Kafka** → consumers → reindex. Challenge: keep the primary store and ES in sync.
- **Option 2: custom search in the datastore.** Main bottleneck is **disk I/O** (PB/day of new data; accounts with 500K+ emails). Use an **LSM tree**: writes go to an in-memory level 0, then merge down; only sequential writes. Also separate data that changes often (folders, filter rules) from data that doesn't (email bodies).

| Feature | Elasticsearch | Custom search engine |
|---|---|---|
| Scalability | Scalable to some extent | Easier; tuned for email |

| Feature | Elasticsearch | Custom search engine |
|---|---|---|
| System complexity | Two systems: datastore + ES | One system |
| Data consistency | Two copies; hard to keep in sync | Single copy in the metadata store |
| Data loss possible | No; rebuild index from primary storage | No |
| Development effort | Easy to integrate; large scale may need a dedicated ES team | Significant engineering effort |

- Rule of thumb: small scale → **Elasticsearch**. Gmail/Outlook scale → **native search embedded in the DB**.

### Scalability and availability

- Users' access patterns are independent, so most components **scale horizontally**.
- Replicate data across **multiple data centers**. Users talk to the nearest one. During a network partition, they reach their mail through another data center.

## Wrap-up

Extra talking points:

- **Fault tolerance:** node failures, network issues, event delays.
- **Compliance:** store EU PII per **GDPR**; lawful intercept.
- **Security:** phishing protection, safe browsing, proactive alerts, account safety, confidential mode, encryption.
- **Optimization:** a group email stores the same attachment many times in S3. Check if it already exists before the expensive save.

✓ KEY POINTS

- Storage dominates: **~2.2 EB/year** (730 PB metadata + 1,460 PB attachments). Attachments go to **S3**, not the metadata DB.
- Queues on both send and receive paths decouple web/SMTP servers from workers and absorb spikes.
- Real-time servers push new mail over **WebSocket**; offline users pull via REST.
- Partition metadata by **user_id**, sort by TIMEUUID, and **denormalize** for read/unread.
- **Single primary per mailbox** → consistency over availability.
- Deliverability and search (Elasticsearch vs native, LSM for writes) are the hard parts at scale.

⚠ COMMON TRAP

treating email as "just SMTP + a database." The hard parts are storage scale, keeping mail out of spam (reputation, warm-up, SPF/DKIM/DMARC), and write-heavy search indexing.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·9 S3-like Object Storage

S3 object storage blob storage bucket metadata store data store placement service replication erasure coding checksum durability multipart upload versioning garbage collection compaction sharding

SEE ALSO SQL vs NoSQL & Scaling Databases · Indexes

**Source:** *System Design Interview* Vol 2, Ch 9.

## Problem

↑ TOP

Design an object storage service like Amazon S3: users create buckets and store objects of any size through a RESTful API.

**Background: three kinds of storage**

| | Block storage | File storage | Object storage |
|---|---|---|---|
| Mutable content | Yes | Yes | No (versioning yes, in-place update no) |
| Cost | High | Medium to high | Low |
| Performance | Medium to very high | Medium to high | Low to medium |
| Consistency | Strong | Strong | Strong |
| Data access | SAS / iSCSI / FC | CIFS/SMB, NFS | RESTful API |
| Scalability | Medium | High | Vast |
| Good for | VMs, databases | General file access | Binary, unstructured data |

Object storage trades performance for durability, scale and low cost. Data lives as objects in a **flat** structure (no directories).

**Terms: bucket** = logical container, globally unique name. **Object** = data (payload) + metadata (name-value pairs). **Versioning** = keep many variants of an object, enabled per bucket. Each resource has a **URI**.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Features? | Create bucket; upload/download objects; object versioning; list objects in a bucket (like `aws s3 ls`) |
| Data size? | Both huge objects (GBs) and many small ones (tens of KB) |

| Question | Answer |
| --- | --- |
| Data per year? | **100 PB** |
| Durability / availability? | **6 nines** (99.9999%) / **4 nines** (99.99%) |

**Requirements:**

- Functional: the four features above.
- Non-functional: 100 PB; 6 nines durability; 4 nines availability; **storage efficiency** (lower cost, still reliable and fast).

**Estimates:** bottleneck is disk capacity or disk IOPS.

- Object mix: 20% small (< 1 MB, median 0.5 MB), 60% medium (1–64 MB, median 32 MB), 20% large (> 64 MB, median 200 MB).
- One SATA 7200 rpm disk: **100–150 IOPS**.
- At 40% storage usage: (10^11 MB × 0.4) / (0.2×0.5 + 0.6×32 + 0.2×200) ≈ **0.68 billion objects**.
- Metadata ~1 KB per object → **~0.68 TB** of metadata.

> Try it first: sketch the upload and download paths, and decide where object names live versus where the bytes live.

SOLUTION

## High-level design

Properties that shape the design: **immutable** objects (replace whole, no incremental change); a **key-value store** (URI → data); **write once, read many** (LinkedIn: 95% reads); both small and large objects.

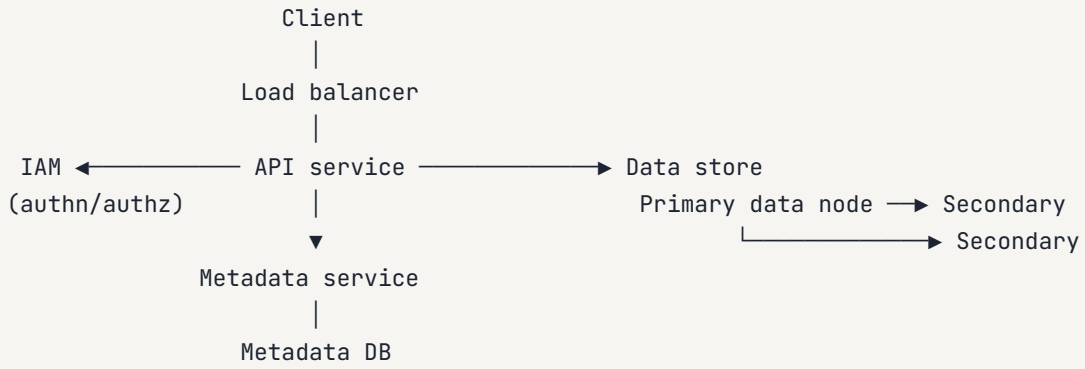
**Like a UNIX file system:** the inode holds metadata and block pointers; data sits elsewhere on disk. Here, the **metadata store** plays the inode, and the **data store** plays the disk. The metadata store finds data by object ID over the network. Separation lets each part be built and tuned on its own: data is immutable, metadata is mutable.

**APIs:**

```
PUT /bucket-to-share/script.txt HTTP/1.1
Host: foo.s3example.org
Authorization: authorization string
Content-Type: text/plain
Content-Length: 4567
x-amz-meta-author: Alex

[4567 bytes of object data]

GET /bucket-to-share/script.txt HTTP/1.1
Host: foo.s3example.org
Authorization: authorization string
```




- **API service:** stateless; calls IAM, metadata store and data store. **IAM:** authentication + authorization.
- **Data store:** stores and fetches bytes by **object ID (UUID)** only. **Metadata store:** object metadata.
- Both stores are logical; Ceph RGW, for example, keeps everything as Rados objects.

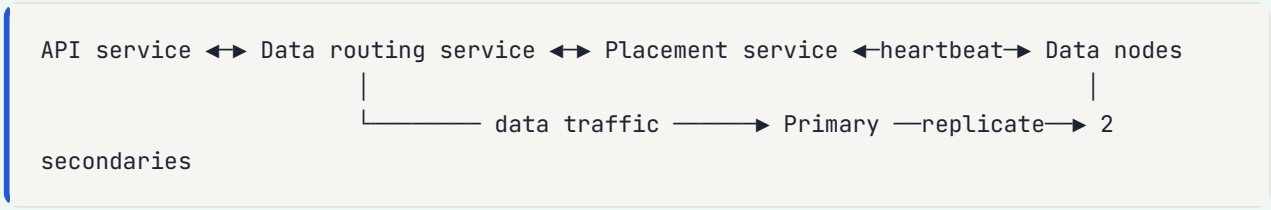
**Upload (7 steps):**

1. `PUT` create bucket `bucket-to-share` → API service.
2. IAM checks WRITE permission.
3. Metadata store creates the bucket entry → success.
4. `PUT` object `script.txt`.
5. IAM checks WRITE on the bucket.
6. API service sends the payload to the data store; it stores it and returns a **UUID**.
7. Metadata store inserts a row: `object_name=script.txt, object_id=<UUID>, bucket_id=<UUID>`.

**Download:** no real folders, but names like `bucket-to-share/script.txt` imply a hierarchy. `GET /bucket-to-share/script.txt` → IAM checks READ → metadata store maps name → UUID → data store returns bytes by UUID → API service returns them.

## Deep dives

### Data store




- **Data routing service:** stateless (REST/gRPC). Asks placement for the best node, reads and writes data nodes.
- **Placement service:** picks primary + replicas. Keeps a **virtual cluster map** (root → datacenter → host → partition) so replicas are physically apart. Node is "down" after a **15 s** missed heartbeat. Critical, so run **5 or 7 nodes with Paxos/Raft**; 7 nodes survive 3 failures.
- **Data node:** a daemon sends heartbeats (number of disks, data per disk). On first heartbeat, placement assigns an ID and returns the cluster map and where to replicate.

**Persistence flow:** API → routing service generates the UUID → placement returns the replication group (deterministic lookup, e.g. **consistent hashing**) → primary saves and replicates to 2 secondaries → UUID returned.

**Consistency vs latency** (when is a write "done"):

| Ack after | Consistency | Latency |
|---|---|---|
| All 3 nodes | Best | Highest (wait for slowest) |
| Primary + 1 secondary | Medium | Medium |
| Primary only | Worst | Lowest |

The last two are eventual consistency. The book's flow waits for all secondaries (strong).

### How data is organized

One file per object breaks with many small files:

- **Wasted blocks:** a disk block is ~4 KB; a tiny file still takes a full block.
- **Inode exhaustion:** inode count is fixed at format time; the OS handles huge inode counts badly.

**Fix:** append small objects into one big **read-write file**, like a WAL. At a threshold (a few GBs) it becomes **read-only** and a new read-write file starts. Writes to one file must be serialized, which kills throughput on many cores, so use **one read-write file per core**.

**Object lookup:** `object_mapping(object_id, file_name, start_offset, object_size)`.

- RocksDB (SSTable): fast writes, slower reads. RDBMS (B+ tree): fast reads, slower writes. Access is write once, read many → **relational**.
- Mapping is local to each node, so no shared cluster: run **SQLite on each data node**.

Updated write: append "object 4" to `/data/c` → insert into `object_mapping` → return UUID.

### Durability

- **Replication:** disk annual failure rate ~0.81%. 3 copies → $1 - 0.0081^3 \approx$ **0.999999** (rough).
- **Failure domains:** node, rack (shared switch/power), row/room, **availability zone**. Spreading replicas across AZs does not raise durability on paper but survives outages, cooling failures and disasters.

**Erasure coding:** split data into chunks, compute parities, rebuild lost chunks from survivors.

- (4+2) example: d1..d4 + p1, p2; lose d3, d4 → rebuild from d1, d2, p1, p2.
- **(8+4):** 12 equal pieces in 12 failure domains; survives **4** lost nodes. Reads need at least **8** healthy nodes.
- Space: 1 GB → 3 GB with 3× replication (**200%** overhead) vs 1.5 GB with EC (**50%**).

| | Replication | Erasure coding |
|---|---|---|
| Durability | 6 nines (3 copies) | **11 nines** (8+4) — EC wins |
| Storage overhead | 200% | **50%** — EC wins |
| Compute | None — replication wins | Parity math |
| Write | Copy to nodes — replication wins | Slower: compute parities first |
| Read | One replica; unaffected by failure — replication wins | Reads from many nodes; slower in failure (rebuild) |

Replication suits latency-sensitive systems; EC minimizes cost but complicates the data node. **This design focuses on replication.**

**Correctness (checksums):** in-memory corruption is common at scale. Compare checksums across process boundaries; mismatch = corrupted. Choice: **MD5**. Append a checksum after each object, and a whole-file checksum before a file turns read-only. On read with (8+4) EC: fetch data + checksum, recompute; on mismatch read from another failure domain; repeat until 8 good pieces, then rebuild.

### Metadata data model

Queries: (1) object ID by name, (2) insert/delete by name, (3) list objects in a bucket by prefix.

```
bucket: bucket_name, bucket_id, owner_id, enable_versioning
object: bucket_name, object_name, object_version, object_id
```

- **bucket table:** 1M customers × 10 buckets × 1 KB = **10 GB**. Fits one server; add read replicas for load.
- **object table:** too big; shard it.

| Shard key | Problem |
|---|---|
| `bucket_id` | Hot shards; a bucket can hold billions of objects |
| `object_id` | Even load, but queries 1 and 2 are by URI |
| **hash(bucket_name, object_name)** | Chosen: most operations are by URI. Listing gets harder |

### Listing objects in a bucket

Path `s3://mybucket/abc/d/e/f/file.txt` → bucket `mybucket`, object name `abc/d/e/f/file.txt`, prefix `abc/d/e/f/`. Prefixes look like folders but are not.

Three uses: `aws s3 list-buckets`; `aws s3 ls s3://mybucket/abc/` (deeper names roll up, e.g. `CA/ NY/ federal.txt`); `--recursive` (no rollup).

**Single DB:**

```sql
SELECT * FROM bucket WHERE owner_id = {id};
SELECT * FROM object WHERE bucket_id = "123" AND object_name LIKE 'a/b/%'
  ORDER BY object_name OFFSET 0 LIMIT 10;
```

Rollup is done in app code. Paging: the server returns a **cursor** with the offset encoded; next page uses `OFFSET 10`.

**Sharded DB:** query every shard and merge. Paging is hard: each shard returns a different count, so the server must track **one offset per shard** in the cursor (hundreds of shards → hundreds of offsets).

**Choice:** listing speed is rarely a priority (all commercial stores are sub-optimal here). **Denormalize listing data into a separate table sharded by bucket ID.** One DB per listing query; fine even for billions of objects.

### Object versioning

- Without versioning: the new metadata replaces the old; the old object is marked deleted and the GC reclaims it.
- With versioning (enabled on the bucket): a PUT inserts a **new row** with the same `bucket_id` + `object_name`, a new `object_id`, and `object_version` = **TIMEUUID**. Current version = largest TIMEUUID.

- **Delete:** keep all versions and insert a **delete marker** as the newest version. GET then returns **404**.

### Optimizing uploads of large files (multipart)

```
1. Client —initiate→ Data store       2. ←— uploadID
3. Client —part i + uploadID→          4. ←— ETag i (MD5 of the part)
5. Client —complete(uploadID, part numbers, ETags)→
6. Data store reassembles by part number (may take minutes) —→ success
```

Example: 1.6 GB file → 8 parts × 200 MB. A failed part is retried alone. Old parts become garbage after reassembly.

### Garbage collection

Garbage sources: **lazy deletion** (marked, not removed), **orphan data** (half or abandoned multipart uploads), **corrupted data** (failed checksum). It is cleaned later by periodic **compaction**, on all replicas (or all 12 nodes for (8+4) EC).

- Compaction: copy live objects from `/data/b` to `/data/d`, skipping ones with the delete flag. Then update `file_name` and `start_offset` in `object_mapping` **in one transaction** (`obj_id`, `object_size` stay).
- Wait for many read-only files and merge them into a few large ones, to avoid many small files.

## Wrap-up

The book compares block, file and object storage, then covers upload, download, listing and versioning. Object storage = **data store + metadata store**. Deep dives: data persistence, replication vs erasure coding, multipart upload, schema design, and sharding the metadata store.

✓ KEY POINTS

- Separate **metadata** (name → UUID, mutable) from **data** (UUID → bytes, immutable).
- Placement service with a cluster map + Paxos/Raft; primary replicates to secondaries across failure domains.
- Pack small objects into large append-only files; per-node SQLite for UUID → (file, offset, size).
- Replication (fast, 200% overhead) vs **(8+4) erasure coding** (11 nines, 50% overhead, slower). Checksums catch corruption.
- Shard object metadata by **hash(bucket, name)**; use a bucket-sharded listing table for prefix listing.
- Versioning via TIMEUUID rows + delete markers; multipart upload; compaction-based GC.

⚠ COMMON TRAP

storing each object as its own file on the data node. Many small objects waste disk blocks and run out of inodes. Also: sharding metadata by bucket, which creates hot shards.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·10 Real-time Gaming Leaderboard

leaderboard ranking Redis sorted set ZINCRBY ZREVRANGE ZREVRANK skip list top K sharding Redis cluster hash slot scatter-gather DynamoDB write sharding percentile AWS Lambda

SEE ALSO SQL vs NoSQL & Scaling Databases · Big-O Cheat Sheet

**Source:** *System Design Interview* Vol 2, Ch 10.

## Problem

↑ TOP

Design a real-time leaderboard for an online mobile game. It shows the top players and where the current user ranks.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| How is score calculated? | +1 point per match won; each user has a total score |
| All players included? | Yes |
| Time segment? | A new tournament and new leaderboard **each month** |
| Only top 10? | Top 10 + a user's rank. Bonus: 4 players above and below a user |
| Users? | **5M DAU**, **25M MAU** |
| Matches? | **10 per player per day** on average |
| Ties? | Same score = same rank. Tie-breaking is a bonus topic |
| Real-time? | Yes, or as close as possible. No batched results |

**Requirements:**

- Functional: top 10 players; a user's rank; 4 places above and below a user (bonus).
- Non-functional: real-time score updates reflected on the leaderboard; scalable, available, reliable.

**Estimates:**

- Users: 5M / 10^5 s ≈ **50 users/s**. Peak 5× → **250 users/s**.
- Score QPS: 50 × 10 = **~500**. Peak 5× → **2,500**.
- Top-10 fetch QPS: one game open per user per day → **~50**.

> Try it first: pick the data structure that makes "add 1 point" and "what is my rank" both fast with 25M players, and sketch who is allowed to update scores.

SOLUTION

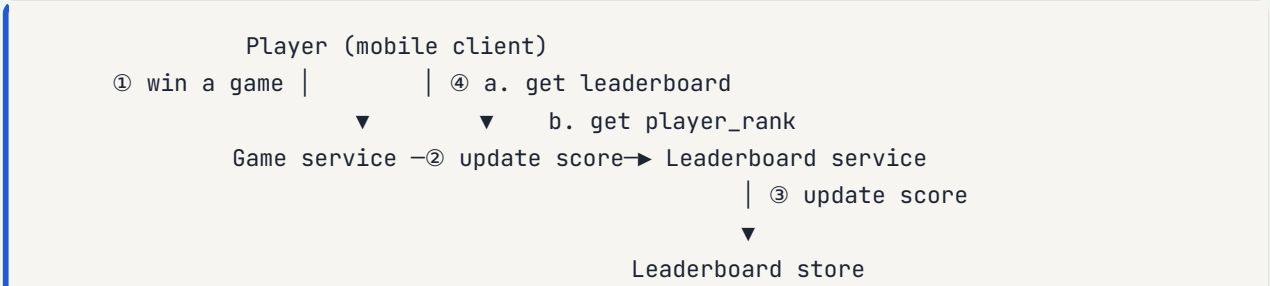
## High-level design

### APIs:

| API | Purpose |
|---|---|
| `POST /v1/scores` | Update score after a win. Params: `user_id`, `points`. **Internal**: only game servers call it. 200 OK / 400 Bad Request |
| `GET /v1/scores` | Top 10 players |
| `GET /v1/scores/{:user_id}` | Rank of one user |

```
GET /v1/scores →
{ [ { "user_id": "user_id1", "user_name": "alice", "rank": 1, "score": 976 },
    { "user_id": "user_id2", "user_name": "bob",   "rank": 2, "score": 965 }, ... ],
  "total": 10 }

GET /v1/scores/user5 →
{ "user_info": { "user_id": "user5", "score": 1000, "rank": 6 } }
```




- Game service validates the win, then calls the leaderboard service.
- The client reads the leaderboard directly from the leaderboard service.
- **Client must not set the score.** A man-in-the-middle proxy could change it. In server-authoritative games (e.g. online poker) the game server sets it without the client.
- **Message queue?** Only if other consumers need the scores (analytics, push notifications, turn-based/multiplayer updates). Then: game service → **Kafka** → leaderboard / analytics / push services. Not required here, so **no queue**.

### Data model option 1: relational database

Table per month: `leaderboard(user_id varchar, score int)`.

```sql
INSERT INTO leaderboard (user_id, score) VALUES ('mary1934', 1);
UPDATE leaderboard SET score = score + 1 WHERE user_id = 'mary1934';

SELECT (@rownum := @rownum + 1) AS rank, user_id, score
FROM leaderboard ORDER BY score DESC;          -- add LIMIT 10 for top 10
```

Why it fails at scale:

- Rank needs **every** player sorted; duplicate scores mean rank is not just row position.
- Ranking over millions of rows takes **10s of seconds**. Data changes constantly, so caching does not help.
- Batch ranking works but breaks the real-time requirement.
- Index + `LIMIT 10` helps top 10, but a user's rank still needs a table scan; no easy answer for users far from the top.
- Rank for one user needs a nested query: `SELECT *, (SELECT COUNT(*) FROM leaderboard lb2 WHERE lb2.score ⩾ lb1.score) RANK FROM leaderboard lb1 WHERE lb1.user_id = {:user_id};`

### Data model option 2: Redis sorted sets (chosen)

A **sorted set**: unique members, each with a score; ordered by score. Built from a **hash table** (user → score) and a **skip list** (score → users).

**Skip list:** a sorted linked list (O(n) search) plus index levels, each skipping every other node of the level below. Search jumps like binary search. Example with 5 index levels: **11** nodes visited instead of **62**. Insert, update and find are **O(log n)**, and elements stay in order on every write.

| Command | Use | Cost |
|---|---|---|
| `ZADD` | Insert user or set score | O(log n) |
| `ZINCRBY` | Add to score (starts at 0 if new) | O(log n) |
| `ZRANGE` / `ZREVRANGE` | Range by rank, asc / desc | O(log n + m) |
| `ZRANK` / `ZREVRANK` | Position of a user, asc / desc | O(log n) |

Workflow (one sorted set per month; old months move to historical storage):

```
ZINCRBY  leaderboard_feb_2021 1 'mary1934'        # user wins a match
ZREVRANGE leaderboard_feb_2021 0 9 WITHSCORES     # top 10 with scores
ZREVRANK leaderboard_feb_2021 'mary1934'          # user's rank
ZREVRANGE leaderboard_feb_2021 357 365            # Mallow007 at rank 361: 4 above, 4
below
```

**Storage:** worst case all 25M MAU have an entry. 24-char id + 2-byte score = 26 B → **~650 MB**. Double it for skip list + hash overhead; **one Redis server** is enough. 2,500 updates/s is easy for one node.

**Persistence:** Redis can persist, but restarting a big instance from disk is slow. Use a **read replica**; promote it on failure and attach a new replica.

**MySQL alongside Redis:** `user` (id, display name) and `point` (user_id, score, timestamp of each win). Used for play history and to **rebuild Redis** after a failure. Optionally cache user details for the top 10.

## Deep dives

### Cloud or own services

**Own services:**

```
Client → Load balancer → Web servers ─┬→ Redis: leaderboard sorted set
                                      ├→ MySQL: user profile + user points
                                      └→ Redis: user profile cache (top 10)
```

The API servers read the sorted set, then fetch names and profile images from MySQL (or the top-10 cache).

**On AWS:** API Gateway maps REST endpoints to Lambda functions.

| API | Lambda |
|---|---|
| `GET /v1/scores` | LeaderboardFetchTop10 |
| `GET /v1/scores/{:user_id}` | LeaderboardFetchPlayerRank |
| `POST /v1/scores` | LeaderboardUpdateScore |

(The book's table swaps the two GET rows; the mapping above follows the function names.)

- Score a point: leaderboard service → API Gateway → Lambda → `ZINCRBY` in Redis + insert into MySQL point table.
- Get leaderboard: API Gateway → Lambda → ① `ZREVRANGE` in Redis → ② user details from MySQL.

**Choice:** serverless if building from scratch. Lambda auto-scales; no servers to set up or maintain.

### Scaling Redis

At **500M DAU** (100×): **65 GB** and **250,000 QPS**. Needs sharding.

**Fixed partition (by score range):** e.g. scores 1–1000 in 10 shards: [1,100], [101,200], ..., [901,1000]. Each shard is a sorted set; sharding is done in app code.

- Ranges must give an even spread; adjust them if not.
- To find a user's shard: read the score from MySQL, or better, a **secondary cache user_id → score**. When a score crosses a range, remove the user from the old shard and update the cache.
- Top 10: read the top of the highest shard ([901, 1000]).
- User rank = local rank in the shard + count of players in all higher shards. A shard's size comes from `info keyspace` in **O(1)**.

**Hash partition (Redis cluster):** good if scores are clumped. **16,384 hash slots**; slot = `CRC16(key) % 16384`. Not consistent hashing, but nodes can still be added or removed without moving all keys. Example: node 1 = [0, 5500], node 2 = [5501, 11000], node 3 = [11001, 16383].

- Update: just the user's shard.
- Top 10: **scatter-gather** — top 10 from every shard (in parallel), app merges and sorts.
- Limits: large top-K is slow (many entries to sort); many partitions → wait for the slowest; **no simple way to get a user's rank**.

**Choice: fixed partition.**

**Sizing a node:** write-heavy apps need extra memory for snapshots during failure; **allocate 2× memory**. Benchmark with `redis-benchmark`.

### Alternative: NoSQL (DynamoDB)

Want: optimized for writes, and efficient sorting by score inside a partition. DynamoDB, Cassandra or MongoDB fit. Design: leaderboard service → API Gateway → Lambda → DynamoDB (replaces Redis + MySQL).

1. Denormalized table (user_id PK; score, email, profile_pic, leaderboard_name like `chess#2020-02`). Works, but top scores need a **full scan**.
2. Global secondary index: partition key `game_name#{year-month}`, sort key `score`. Works, but the current month is one **hot partition** (DynamoDB places items by consistent hashing on the partition key).
3. **Write sharding:** partition key `game_name#{year-month}#p{user_id % N}`, sort key `score`. Each partition is locally sorted. Top 10 = **scatter-gather** over N partitions.

Trade-off: more partitions = less load each but more read fan-out. Pick N by write volume or DAU, and **benchmark**.

Exact rank is still hard. Offer a **percentile** instead ("top 10–20%" beats "rank 1,200,001"). Assume shards share a score distribution; a **cron job** computes and caches cut-offs, e.g. 10th percentile = score < 100, 20th = < 500, ..., 90th = < 6500.

## Wrap-up

- MySQL rejected: does not scale to millions of users in real time. Redis sorted sets chosen; sharded for 500M DAU; DynamoDB as an alternative.
- **Faster retrieval:** a Redis **hash** of user_id → user object avoids DB lookups for display.
- **Break ties:** also store user_id → timestamp of the latest win; on equal score, the **older timestamp ranks higher**.
- **Failure recovery:** replay the MySQL point table (one row per win, with timestamp) and call `ZINCRBY` per entry to rebuild the leaderboard offline.

✓ KEY POINTS

- One **Redis sorted set per month**; ZINCRBY / ZREVRANGE / ZREVRANK are all O(log n).
- Skip list + hash table is why rank queries are fast; SQL must sort the whole table.
- ~650 MB and 2,500 QPS fit **one Redis node** plus a replica; MySQL keeps the durable win log.
- Only the **game service** updates scores; add Kafka only if other consumers need them.
- At 100× scale, **range-partition by score** so top 10 and rank stay cheap; hash slots need scatter-gather and lose rank.

⚠ COMMON TRAP

ranking with `ORDER BY` in SQL and hoping a cache fixes it. Scores change constantly, so the cache is always stale. Also: letting the client post its own score.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·11 Payment System

payment system · pay-in · pay-out · PSP · Stripe · hosted payment page · idempotency key · exactly-once · reconciliation · settlement file · double-entry ledger · retry queue · dead letter queue · webhook · PCI DSS

SEE ALSO Transactions & Isolation Levels · Kafka vs RabbitMQ

**Source:** *System Design Interview* Vol 2, Ch 11. · Your write-up: Wallet and Ledger

## Problem

↑ TOP

Design the payment backend for an e-commerce site like Amazon. It handles everything related to money movement when a customer places an order.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| What kind of payment system? | A payment backend for an e-commerce app like Amazon.com |
| Payment options? | All in real life. Use credit cards as the example |
| Do we process cards ourselves? | No. Use third-party processors (Stripe, Braintree, Square) |
| Do we store card data? | No. Security and compliance rules are too strict. The PSP holds it |
| Global? Many currencies? | Global, but assume one currency |
| How many transactions? | 1 million per day |
| Pay sellers too (pay-out)? | Yes |
| Anything else? | Many internal and external services can drift apart. Reconciliation is required |

**Requirements:**

- Functional: **pay-in** (receive money from customers on behalf of sellers) and **pay-out** (send money to sellers around the world).
- Non-functional: reliability and fault tolerance (handle failed payments carefully). An async **reconciliation** process between internal services (payment, accounting) and external ones (PSPs).

**Estimates:** 1,000,000 tx / 10^5 s = **10 TPS**. Any database handles that. The focus is correct handling of each payment, not throughput.

> Try it first: sketch the services between "place order" and the card network, and the tables you would keep. Then explain how you stop a double charge when the user clicks "pay" twice or the PSP response is lost.

SOLUTION

## High-level design

Money flow: `Buyer credit card —pay-in→ e-commerce bank account —pay-out→ seller bank account`. The site is only a custodian of the seller's money, for a fee. Pay-out happens later, e.g. when the product is delivered.

**Pay-in flow (Figure 2):**

```
user —(1) payment event→ Payment Service —(3) payment order→ Payment Executor —(5)→
PSP —→ Card schemes
                              |(2) DB            |(4) DB          (internal |
external)  (PayPal,    (Visa,
                            (8) |   (6)
Stripe,     MasterCard)
                                ▼    ▼
Adyen)
                           Ledger  Wallet
                            |(9)    |(7)
                            DB      DB
```

1. "Place order" creates a **payment event**. The payment service stores it (after a risk check for AML/CFT, usually by a third party).
2. One event can hold several **payment orders** (one per seller). The service calls the **payment executor** for each.
3. The executor stores the order and calls the **PSP** to charge the card.
4. On success, the service updates the **wallet** (seller balance), then the **ledger** (append-only financial record).

**APIs:**

```
POST /v1/payments        execute a payment event
  buyer_info        json
  checkout_id       string   globally unique per checkout
  credit_card_info  json     encrypted card info or a PSP token
  payment_orders    list of { seller_account string, amount string,
                              currency string (ISO 4217), payment_order_id string }
GET  /v1/payments/{:id}  status of one payment order
```

- `payment_order_id` is globally unique. The PSP uses it as the **idempotency key**.

- `amount` is a **string**, not a double. Precision differs across protocols and hardware, which causes rounding errors. Numbers can be huge (Japan's GDP ≈ 5×10^14 yen) or tiny (1 satoshi = 10^-8). Parse only for display or math.

**Data model.** Choose storage for **proven stability** (5+ years at big financial firms), **tooling**, and a **mature DBA job market**, not speed. So: a relational DB with ACID transactions over NoSQL/ NewSQL.

```
payment_event:  checkout_id string PK, buyer_info, seller_info, credit_card_info,
is_payment_done bool
payment_order:  payment_order_id string PK, buyer_account, amount string, currency,
                checkout_id string FK, payment_order_status, ledger_updated bool,
wallet_updated bool
```

- `payment_order_status`: `NOT_STARTED → EXECUTING → SUCCESS | FAILED`.
- On SUCCESS: update the wallet, set `wallet_updated`; then update the ledger, set `ledger_updated`. When all orders of a checkout succeed, set `is_payment_done`.
- A scheduled job watches in-flight orders and alerts if one does not finish within a threshold.

**Double-entry ledger.** Record every transaction in two accounts with the same amount: buyer **debit $1**, seller **credit $1**. The sum of all entries must be 0. One cent lost means someone else gained a cent. This gives end-to-end traceability.

**Hosted payment page.** To avoid PCI DSS, the PSP's widget, iframe or SDK page collects the card. Card data never touches our system.

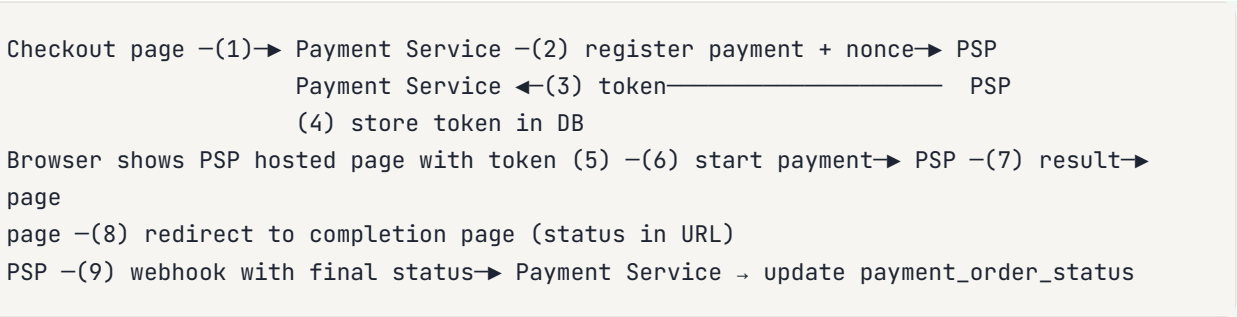
**Pay-out** works like pay-in, but a third-party accounts-payable provider (e.g. Tipalti) moves money from the site's bank to the seller's.

## Deep dives

### PSP integration

- Direct links to banks or card schemes are rare. Only very large companies justify them.
- Option 1: PSP **API**. We build the payment pages and store card data ourselves.
- Option 2: PSP **hosted page**. The PSP collects and stores card data. **Most companies choose this.**

Hosted payment flow (Figure 4):




- The **nonce** (usually the payment order ID) makes registration exactly-once. The **token** is the PSP's ID for that registration.
- The hosted page needs the token (to look up the amount) and a **redirect URL**. The redirect URL is not the webhook URL.

### Reconciliation

- Async messages may never arrive, and responses may be lost. Reconciliation periodically compares states between services. It is **the last line of defense**.
- Each night the PSP or bank sends a **settlement file** (balance + the day's transactions). We parse it and compare with the ledger. We also compare ledger vs wallet internally.
- Mismatches go to three buckets:

| Case | Handling |
|---|---|
| Classifiable, fix can be automated | Program classifies and adjusts automatically |
| Classifiable, automation too costly | Job queue; finance team fixes manually |
| Unclassifiable | Special job queue; finance team investigates |

### Handling payment processing delays

- Some payments take hours or days: high-risk manual review, or 3D Secure needing more card-holder details.
- The PSP returns **pending**. The client shows it and offers a status page.
- The PSP tracks it and notifies us by **webhook** (or we **poll** it). Then we update and ship.
- Three parties are involved, so every call must be **idempotent**, and reconciliation keeps them eventually consistent.

### Communication among internal services

| | Synchronous (HTTP) | Asynchronous |
|---|---|---|
| Performance | One slow service slows the whole chain | Decoupled |
| Failure isolation | PSP down → client gets no response | Queue buffers it |
| Coupling | Sender must know the receiver | Loose |

| | Synchronous (HTTP) | Asynchronous |
|---|---|---|
| Scaling | No buffer for spikes | Queue absorbs spikes |

- **Single receiver:** a shared message queue. One consumer processes a message, then it is removed.
- **Multiple receivers:** Kafka. Messages stay, so the payment, analytics and billing services all read the same payment event.
- **Choice: async.** It trades design simplicity and consistency for scalability and resilience. That fits a large system with many third-party dependencies.

### Handling failed payments

- **Track payment state** in an append-only table. After a failure, the state tells you whether to retry or refund.
- **Retry queue + dead letter queue (Figure 10):**

```
failure → retryable? —no→ DB (e.g. invalid input)
              |yes
              ▼
         retry queue → payment system → fails again → retries left? —yes→ retry
queue
                                                                    |no
                                                                    ▼
                                                          dead letter queue (inspect)
```

### Exactly-once delivery

Double charging is among the worst bugs. **Exactly-once = at-least-once + at-most-once.**

**Retry (at-least-once).** Strategies: immediate, fixed interval, incremental interval, **exponential backoff** (1s, 2s, 4s...), or cancel. Use exponential backoff when the issue will not clear soon. Too aggressive a retry wastes resources and overloads services. Return an error code with a `Retry-After` header.

**Idempotency (at-most-once).** The client sends a UUID in the header `idempotency-key: <value>` (Stripe and PayPal do this). It expires after a while.

- **Scenario 1: user clicks "pay" twice.** The key is the shopping cart ID right before checkout. The second request is seen as a retry, and we return the latest status of the first. Concurrent requests with the same key: process one, return **429 Too Many Requests** to the rest.
- Implementation: use the key as the table's **primary key**. Insert succeeds → new request. Insert fails on a duplicate key → seen before, do not process.
- **Scenario 2: PSP charged the card, but the response was lost, and the user clicks again.** The nonce maps to the payment order, and the token maps to the nonce. The retry carries

the same token. The PSP uses the token as its idempotency key and returns the earlier result.

### Consistency

- Stateful parts: payment service (nonce, token, order, status), ledger, wallet, PSP, and DB replicas.
- **Between internal services:** exactly-once processing.
- **Internal vs PSP:** idempotency (reuse the same key on retry) **plus reconciliation**, because the external system is not always right.
- **Replica lag:** (1) read and write from the primary only. Simple, but replicas waste resources. Or (2) keep replicas in sync with Paxos/Raft, or use a consensus DB like YugabyteDB or CockroachDB.

### Payment security

| Problem | Solution |
|---|---|
| Eavesdropping | HTTPS |
| Data tampering | Encryption and integrity monitoring |
| Man-in-the-middle | SSL with certificate pinning |
| Storing passwords | Salted hashing |
| Data loss | Replicate across regions, take snapshots |
| DDoS | Rate limiting and a firewall |
| Card theft | Tokenization: store and use tokens, not real card numbers |
| PCI compliance | Follow PCI DSS |
| Fraud | Address verification, CVV, user behavior analysis |

## Wrap-up

- Covered: pay-in, pay-out, retry, idempotency, consistency, error handling, security.
- More to discuss: **monitoring** (e.g. acceptance rate per payment method), **alerting**, **debugging tools** for "why did this payment fail?", **currency exchange**, regional payment methods, **cash payments** (India, Brazil), and Google/Apple Pay integration.

✓ KEY POINTS

- Say early that 10 TPS is tiny. The hard part is correctness under failure.
- Split a checkout into a payment event and per-seller payment orders, each with a status enum.
- Use a hosted payment page so card data never touches you (avoids PCI DSS scope).
- Exactly-once = retry with backoff + idempotency key (DB unique key; payment_order_id/token at the PSP).
- Prefer async messaging (Kafka) with a retry queue and a DLQ.
- Reconcile against the PSP settlement file every night, and between ledger and wallet.

⚠ COMMON TRAP

assuming a retry is safe, or that the PSP is always right. Without an idempotency key a retry can double charge. Without reconciliation, lost webhooks and silent mismatches are never found.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·12 Digital Wallet

digital wallet · balance transfer · distributed transaction · two-phase commit · 2PC · TC/C · try-confirm-cancel · saga · compensating transaction · event sourcing · CQRS · reproducibility · Raft · snapshot · RocksDB · mmap

SEE ALSO Transactions & Isolation Levels · Locking & Concurrency Control

**Source:** *System Design Interview* Vol 2, Ch 12. · Your write-up: Wallet and Ledger

## Problem

↑ TOP

Design the backend of a digital wallet (like PayPal) that supports balance transfers between two wallets on the same platform.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Which features? | Balance transfer between two wallets only |
| Throughput? | 1,000,000 TPS |
| Are transactional guarantees enough for correctness? | Yes |
| Must we prove correctness? | Reconciliation only shows that a difference exists, not how it happened. We want **reproducibility**: rebuild any historical balance by replaying data from the start |
| Availability? | 99.99% |
| Foreign exchange? | Out of scope |

**Requirements:**

- Functional: transfer balance between two wallets.
- Non-functional: 1M TPS, 99.99% reliability, transactional guarantees, reproducibility.

**Estimates:** a relational DB node does a few thousand TPS; assume 1,000. Each transfer is two operations (debit and credit), so we need **2M TPS → 2,000 nodes**. Higher per-node TPS means fewer nodes and lower cost:

| Per-node TPS | Nodes |
|---|---|
| 100 | 20,000 |
| 1,000 | 2,000 |
| 10,000 | 200 |

> Try it first: start with sharded balances and show where a transfer between two shards breaks. Then pick a way to make it atomic, and a way to answer an auditor's "what was A's balance last Tuesday, and prove it".

SOLUTION

## High-level design

**API:**

```
POST /v1/wallet/balance_transfer
  from_account    string   debit account
  to_account      string   credit account
  amount          string   (string, not double: see the payment chapter)
  currency        string   ISO 4217
  transaction_id  uuid     for deduplication
→ { "Status": "success", "Transaction_id": "01589980-2664-11ec-9621-0242ac130002" }
```

The book discusses three designs, each fixing the previous one's flaw.

### 1. In-memory sharding (Redis)

```
Wallet service (stateless) ──► Redis 1 (A)   Redis 2 (B)   Redis 3 (C)
   │  partition = accountID.hashCode() % partitionNumber
   └─ Zookeeper: partition count + Redis node addresses
```

- The wallet service receives the command, validates it, and updates two balances, usually on two Redis nodes.
- **Flaw:** no atomicity. If the service crashes after "A −$1" but before "C +$1", the transfer is incomplete. Also, data is not durable.

### 2. Distributed transactions on sharded relational DBs

Replace each Redis node with a transactional DB. Each local update is now safe, but the two updates still live on two databases. We need a **distributed transaction**.

**Two-phase commit (2PC), low-level (in the DB):**

1. The coordinator (wallet service) reads and writes on both DBs. Both rows are **locked**.
2. Phase 1: the coordinator asks every DB to **prepare**.
3. Phase 2: all say yes → **commit**. Any says no → **abort**.

- Needs special DB support (X/Open XA).

- **Slow:** locks are held while waiting for other nodes. The **coordinator is a single point of failure**; if it crashes, the DBs wait.

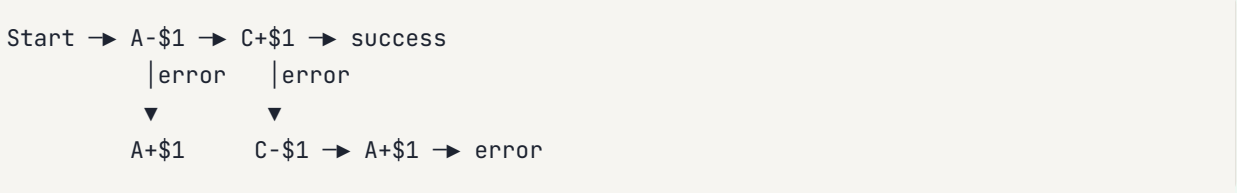
**Try-Confirm/Cancel (TC/C), high-level (in the app).** A compensating transaction. Each phase is its own local transaction. Transfer $1 from A to C:

| Phase | Operation | Account A | Account C |
|---|---|---|---|
| 1 | Try | −$1 | do nothing (NOP) |
| 2 | Confirm | do nothing | +$1 |
| 2 | Cancel | +$1 (undo) | do nothing |

| | First phase | Second phase: success | Second phase: fail |
|---|---|---|---|
| 2PC | Local transactions not done yet (locked) | Commit all | Abort all |
| TC/C | Local transactions done (unlocked) | Run new local transactions if needed | Reverse the committed effect ("undo") |

- Pro: **database-agnostic**; any DB with transactions works. Con: the app owns all the complexity.
- **Phase status table:** survives a wallet-service restart. It stores the tx ID and content, the Try status per DB (not sent / sent / response received), the second phase name (Confirm or Cancel), its status, and an **out-of-order flag**. Keep it in the DB of the account being debited.
- **Unbalanced state:** after Try, A lost $1 and C has not gained it, so the sum is short by $1. That is fine. The app sees intermediate state that a DB-level transaction would hide.
- **Valid order:** only **deduct first** works. Add-first (NOP on A, +$1 on C) can fail: someone may spend the $1 from C before Cancel takes it back. Doing both at once creates messy partial failures.
- **Out-of-order execution:** C's DB may get Cancel before Try (network delay). Allow Cancel without Try: it leaves a flag. Try checks the flag and fails if it is set.

**Saga.** The de facto standard in microservices.




- Operations run **in linear order**, each a local transaction. On failure, run compensations in reverse order. So n operations need 2n definitions.
- **Choreography:** services react to each other's events. Fully decentralized, but each service needs its own state machine. Hard with many services.

- **Orchestration:** one coordinator drives the order. **Preferred for a wallet.**

| | TC/C | Saga |
|---|---|---|
| Compensating action | In Cancel phase | In rollback phase |
| Central coordination | Yes | Yes (orchestration) |
| Execution order | Any | Linear |
| Parallel execution | Yes | No |
| Partial inconsistent state visible | Yes | Yes |
| Logic lives in | Application | Application |

- **Choice:** few services and no latency need → either; go with Saga to follow the microservice trend. Latency-sensitive with many operations → TC/C.
- **Remaining flaw:** no audit trail. If a user enters a wrong operation, we cannot trace the root cause, and old balances are overwritten.

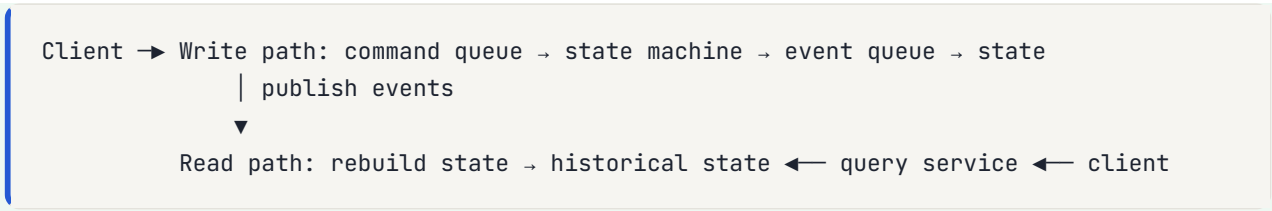
### 3. Event sourcing

Auditors ask: what was the balance at any time? Are past and current balances correct? Is the logic still correct after a code change? Event sourcing (from DDD) answers all three.

- **Command:** an intended action from outside ("transfer $1 A→C"). It may be invalid. Commands go in a FIFO queue (e.g. Kafka).
- **Event:** a validated fact, in past tense ("transferred $1"). It must be applied. One command makes zero or more events. Event *generation* may use randomness or I/O, but applying an event must be **deterministic**.
- **State:** the balances, a map `account → balance` (KV store or relational DB).
- **State machine:** validates commands into events and applies events to state. It must have no randomness.

```
Command queue: [A-$1→C] [A-$1→B] [A-$1→D]
   (1) read command → (2) read state (A's balance) → (3) validate → events
Event queue:   [A:-$1][C:+$1] [A:-$1][B:+$1] [A:-$1][D:+$1]
   (4) read event → (5) update state
```

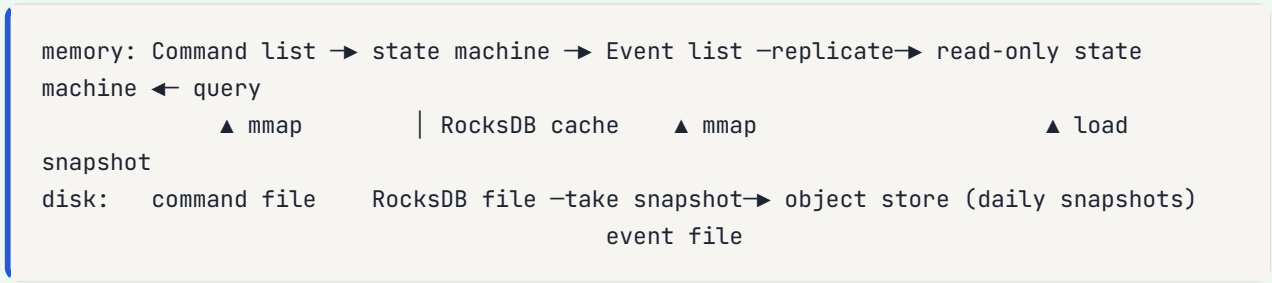
- **Reproducibility:** the event list is immutable and the logic is deterministic, so replaying from the start always gives the same states. Answers: replay up to a time; recompute balances from events; rerun events with the new code and compare.
- **CQRS:** do not publish state. Publish the **events**. One write state machine, many read-only state machines that build their own views: a balance DB for queries, or a per-hour view to investigate double charges. Read side is **eventually consistent**.




## Deep dives

### High-performance event sourcing

- **File-based command and event lists** on local disk instead of remote Kafka. No network hop. Append-only means sequential writes, which are fast even on HDDs; sequential disk can beat random memory access.
- **Cache recent commands and events in memory.** Use **mmap**: the file is mapped to memory, the OS caches it, and appends almost always stay in memory.
- **File-based state:** a local store instead of a networked DB. SQLite or **RocksDB**; the book picks RocksDB because its LSM tree is write-optimized, with recent data cached for reads.
- **Snapshots:** periodically save the state to a file, so replay starts from the last snapshot, not the beginning. Finance often wants one at **00:00** each day. Snapshots are large binaries; store them in object storage like HDFS.




### Reliable high-performance event sourcing

Local files make the node stateful and a **single point of failure**.

- Computation can rerun anywhere. Only **data** needs protection.
- State and snapshots can be rebuilt from events. Commands cannot rebuild events, because event generation may be non-deterministic. So **only the event list needs strong reliability**.
- Replicate it with **consensus (Raft)**: no data loss and the same order on every node. Works while a majority is up: 3 nodes tolerate 1 failure, 5 tolerate 2.
- Roles: one **leader**, the rest **followers** (and candidates during elections). The leader accepts commands, makes events, and Raft replicates them. **Every node applies events**, so all states match.
- **Leader crash:** Raft elects a new leader. If the crash happened before the command became an event, the client sees a timeout or error and **resends to the new leader**.
- **Follower crash:** Raft retries until it is restarted or replaced.

### Distributed event sourcing

Two limits remain: CQRS clients do not know when their transfer finished, and one Raft group has limited capacity.

**Pull vs push:**

- **Pull:** the client polls the read side. Not real-time, and frequent polling can overload the service.
- **Reverse proxy + pull:** the proxy forwards commands and polls for the client. Simpler client, still not real-time.
- **Reverse proxy + push:** the read-only state machine **pushes** status to the proxy as soon as it gets the event. Feels real-time. This makes the async framework look synchronous.

**Distributed transaction across partitions.** Shard accounts (book example: hash % 2), one Raft group per partition, and reuse TC/C or Saga.

```
user ◄──► TC/C or Saga coordinator ── Phase status table (DB)
              │ command            ▲ response (pushed)
              ▼                    │
   Partition 1 (A): reverse proxy → Raft leader [commands → events] ⇄ followers → read
path
   Partition 2 (C): reverse proxy → Raft leader [commands → events] ⇄ followers → read
path
```

Saga happy path for A→C $1:

1. User sends the tx (A−$1, C+$1) to the Saga coordinator. It writes a record in the phase status table.
2. Deduct first: send **A−$1** to partition 1. Its Raft leader stores the command, validates it, makes an event, replicates it with Raft, then applies it.
3. CQRS syncs to partition 1's read path, which **pushes** the status back to the coordinator. The coordinator records success.
4. Send **C+$1** to partition 2. The same steps run. The coordinator records success.
5. All steps succeeded. The coordinator replies to the caller.

## Wrap-up

- About 2,000 nodes are needed at 1,000 TPS each, so per-node throughput matters.
- Redis: not durable or atomic. Transactional DBs + 2PC, TC/C or Saga: atomic, but hard to audit.
- Event sourcing with external Kafka and a DB: auditable but slow. Local files (mmap, RocksDB, snapshots): fast.
- One node is a SPOF, so replicate the event list with Raft.

- CQRS + a pushing reverse proxy make it feel synchronous. TC/C or Saga coordinates across Raft groups.

✓ KEY POINTS

- Do the node math out loud: one transfer = two writes, so 2M ops/s.
- Explain why a non-atomic cross-shard update loses money, then compare 2PC, TC/C and Saga with their trade-offs.
- Always deduct before you add, and handle Cancel-before-Try with a flag.
- Use event sourcing for reproducibility and audits: an immutable event log plus a deterministic state machine.
- Only the event log needs consensus replication. State and snapshots can be rebuilt.

⚠ COMMON TRAP

saying "use a DB transaction" for a transfer between shards. A local transaction cannot span two databases. Another trap: replicating commands instead of events. Event generation may be non-deterministic, so only events give reproducibility.

↑ Back to top ☰ Contents

CHAPTER 9 · SYSTEM DESIGN CASE STUDIES

# V2·13 Stock Exchange

stock exchange · matching engine · order book · limit order · sequencer · order manager · market data · candlestick · FIX · event sourcing · mmap · CPU pinning · application loop · hot-warm · Raft · p99 latency · multicast

SEE ALSO Concurrency Models (GIL, Threads, Processes, asyncio) · Big-O Cheat Sheet

**Source:** *System Design Interview* Vol 2, Ch 13. · Your write-up: Order Book and Matching Engine

## Problem

↑ TOP

Design an electronic stock exchange that matches buyers and sellers. Clients place and cancel orders, receive matched trades in real time, and view the live order book.

**Clarifying questions → answers:**

| Question | Answer |
|---|---|
| Which securities? | Stocks only |
| Which operations and order types? | Place and cancel. **Limit orders** only |
| After-hours trading? | No, normal hours only |
| Scale? | Tens of thousands of concurrent users, at least 100 symbols, billions of orders per day |
| Risk checks? | Simple ones, e.g. max 1M shares of Apple per user per day |
| Wallet? | Yes. Check funds when an order is placed, and **withhold** funds while the order waits in the book |

**Requirements:**

- Functional: place and cancel limit orders; real-time fills; real-time order book; risk checks; fund checks.
- Non-functional: **availability ≥ 99.99%** (seconds of downtime hurt reputation); fault tolerance with fast recovery; **millisecond round-trip latency, focus on p99**; security (KYC before opening an account, DDoS protection for public market data). Small-to-medium scale, but it must be extensible.

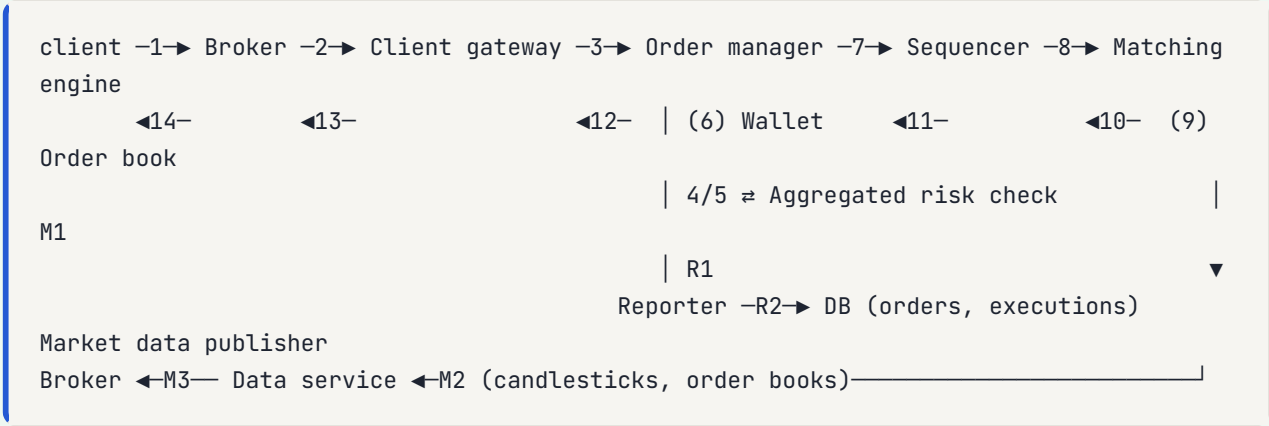
**Estimates:** 100 symbols, 1B orders/day. NYSE is open 9:30–16:00, 6.5 hours. **QPS = 1B / 6.5 / 3600 ≈ 43,000. Peak = 5 × QPS ≈ 215,000** (volume spikes at open and close).

Try it first: draw the order's path from the broker to the matching engine and back. Then design the order book so place, match and cancel are O(1), and explain how a crashed matching engine recovers with exactly the same state.

SOLUTION

## High-level design

**Business basics:** retail clients trade through **brokers** (Schwab, Robinhood). **Institutional clients** need order splitting or very low latency. A **limit order** has a fixed price and may fill partly or not at all. A **market order** has no price and fills at once. Market data: **L1** = best bid/ask and quantities; **L2** = more price levels; **L3** = price levels plus the queued quantity at each level. **Candlestick** = open, close, high, low per interval (1 min to 1 month). **FIX** = a vendor-neutral protocol for securities transactions (1991).




- **Trading flow (critical path, steps 1–14).** The gateway validates, rate-limits, authenticates and normalizes. The order manager runs risk checks and the wallet check. The sequencer stamps the order. The matching engine matches it and emits **two executions (fills)**, one for each side. Fills flow back the same way.
- **Market data flow (M1–M3).** The publisher builds candlesticks and order books from the fill stream and sends them to the data service. Brokers read from it.
- **Reporting flow (R1–R2).** The reporter merges fields from orders and executions (client_id, price, quantity, filled/remaining) and writes them to a DB.
- Only the trading flow has strict latency needs.

**Matching engine (cross engine):** keeps an order book per symbol, matches buy and sell orders, and publishes the fill stream. It must be **deterministic**: the same input sequence must give the same fill sequence on replay. That is the base of high availability.

**Sequencer:** an **inbound** one stamps every order and an **outbound** one stamps every pair of fills. IDs are consecutive, so gaps are easy to spot. Reasons: timeliness and fairness, fast recovery and replay, and exactly-once. It is also a message queue and an event store, like two Kafka streams. Kafka would work if its latency were lower and more predictable.

**Order manager:** sends orders to the risk check and the wallet check, then to the sequencer, with only the attributes the engine needs (smaller messages). It gets fills back and returns them to brokers. Managing order state transitions is its main complexity; event sourcing fits it well.

**Client gateway:** auth, validation, rate limit, normalization, FIXT support. Keep it light because it sits on the critical path; push complex logic to the matching engine and risk check. Different gateways serve retail and institutions. **Colocation:** a broker's engine on rented servers in the exchange's data center; latency is about light travel time.

**APIs** (broker ↔ gateway, REST; institutions may need a faster protocol):

```
POST /v1/order                 symbol, side, price (long), orderType, quantity (long)
  → id, creationTime, filledQuantity, remainingQuantity, status (new/canceled/filled)
GET /execution?symbol=&orderId=&startTime=&endTime=
  → executions[]: id, orderId, symbol, side, price, orderType, quantity
GET /marketdata/orderBook/L2?symbol=&depth=          → bids[], asks[] (price, size)
GET /marketdata/candles?symbol=&resolution=&startTime=&endTime=
  → candles[]: open, close, high, low
```

**Data models:**

- **Product:** type, symbol, lot size, tick size, currencies. Static and highly cacheable.
- **Order** (inbound) and **execution/fill** (outbound): an order has 0..n executions.
- On the critical path, orders and executions are **not stored in a DB**. They live in memory and in the sequencer, and are archived after market close. The reporter writes them to a DB. The market data processor uses them to rebuild order books and candlesticks.
- Market data goes to an in-memory columnar DB (e.g. KDB) for real-time analytics, then to a historical DB after the close.

## Deep dives

### Order book

Needs: constant-time lookup of volume at a price; O(1) add, cancel and execute; fast update; best bid/ask query; iterate price levels.

Example: a market buy for 2,700 Apple shares takes the whole best-ask queue at 100.10 (200 + 400 + 1100 + 100) and the first order at 100.11 (900). Best ask moves up to 100.11 and the spread widens.

```
class PriceLevel { Price limitPrice; long totalVolume; List<Order> orders; }
class Book<Side>  { Side side; Map<Price, PriceLevel> limitMap; }
class OrderBook   { Book<Buy> buyBook; Book<Sell> sellBook;
                    PriceLevel bestBid; PriceLevel bestOffer;
                    Map<OrderID, Order> orderMap; }
```

A plain list is not O(1) for removal. Use a **doubly linked list** for `orders`:

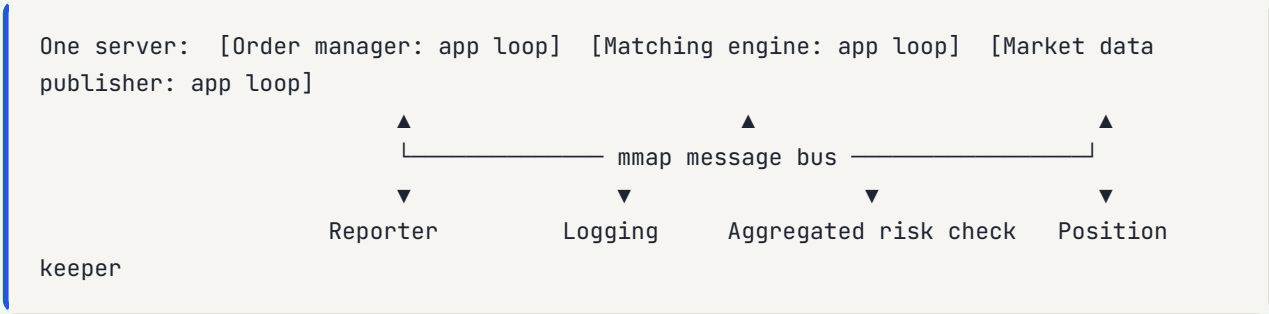
- **Place:** append to the tail of the PriceLevel. O(1).
- **Match:** remove from the head. O(1).
- **Cancel:** find the order with `orderMap`, then unlink it. O(1).

The market data processor uses the same structure to rebuild L1/L2/L3 from fills.

**Candlesticks:** a new Candlestick is added to the chart's linked list each interval. Many symbols × intervals use a lot of memory. Fixes: **pre-allocated ring buffers** (fewer allocations) and keeping only recent sticks in memory, the rest on disk.

### Performance: latency

- `Latency = Σ execution time along the critical path`. Reduce the number of tasks, or shorten each one (less network and disk, faster code). Even logging is off the critical path.
- With separate servers, each network round trip is about **500 µs**, so the hops add up to single-digit ms. The sequencer's disk writes add **tens of ms**. Total: tens of ms. Modern exchanges target **tens of µs**.
- **Put everything on one server:**




- **Application loop:** a while loop that polls for tasks. It is **single-threaded and pinned to one CPU core**. Gains: **no context switches** and **no locks**, so a low p99. Cost: harder code; no task may hog the loop.
- **mmap** over a file in `/dev/shm` (memory-backed) gives shared memory with **no disk access**. A message on this bus takes **under 1 µs**.

### Event sourcing

- Instead of storing current state (Order V1 New → V2 Filled), store an **immutable log of events** (seq 100 NewOrderEvent, seq 101 OrderFilledEvent). Replay to recover any state.

```
FIX ─► Gateway (FIX → SBE) ─NewOrderEvent─► Event store (mmap) ─► Matching engine
                                            entry = [sequence | event type | SBE
payload]
                                              │       (embedded order manager
validates,
                                              │        updates state, sends to
matching core)
                                              │  ◄── OrderFilledEvent
                                              ├─► Market data publisher
                                              └─► Reporter (embedded order manager)
```

- The gateway converts FIX to **FIX over SBE** (Simple Binary Encoding) for fast, compact messages.
- The **order manager becomes a library** embedded in each component. A central service would add latency for everyone. Event sourcing keeps each copy identical and replayable.
- **The sequencer becomes a single writer.** It pulls events from each component's local ring buffer, stamps a sequence ID, and writes to the event store. Multiple sequencers would fight over the write lock. Keep backup sequencers for HA.
- This looks like Kafka pub-sub. Without strict latency needs, Kafka could be used.

### High availability

- 99.99% means **8.64 s of downtime per day**, so recovery must be almost instant.
- Stateless parts (gateway) scale horizontally. Stateful parts (order manager, matching engine) need replicated state.
- **Hot-warm matching engine:** the warm one reads the same events but publishes nothing. On primary failure it takes over at once. A restarted warm engine recovers from the event store. Detect failure with monitoring and **heartbeats** from the engine.
- Across servers or data centers, the whole server is hot or warm, and the event store is replicated, e.g. over **reliable UDP** (see Aeron).

### Fault tolerance

- What if the warm instances die too? Replicate core data to data centers in several cities.
- **When to fail over:** false alarms cause needless failovers, and the same bug can kill the backup too. Start with **manual failover**; automate after you gain operational confidence. Use **chaos engineering**.
- **Who takes over: Raft**. In a 5-server cluster each server has its own mmap event store. The leader replicates events over RPC; quorum is N/2 + 1 = 3. Empty AppendEntries are heartbeats. A follower that times out becomes a candidate and sends RequestVote; a majority makes it leader. A split vote times out and restarts the election. Terms divide time.
- **RTO** (allowed downtime): seconds, so failover must be automatic. Rank services by priority and define degraded modes.

- **RPO** (allowed data loss): **near zero**. Raft keeps many consistent copies, so the new leader works immediately.

### Matching algorithm

```
handleOrder(book, event):
  if event.sequenceId ≠ nextSequence → ERROR(OUT_OF_ORDER)
  if !validateOrder(symbol, price, quantity) → ERROR(INVALID_ORDER)
  NEW    → handleNew: BUY matches sellBook, SELL matches buyBook
  CANCEL → if orderId not in orderMap → ERROR(CANNOT_CANCEL_ALREADY_MATCHED)
           else remove, set CANCELED
match(book, order):
  walk orders at limitMap[order.price] while leavesQuantity > 0:
    matched = min(...); update matchedQuantity; remove; generateMatchedFill()
```

- This is **FIFO**: the first order at a price level fills first. Other algorithms exist, e.g. **FIFO with LMM** (a Lead Market Maker gets a negotiated share ahead of the queue), mostly in futures. Dark pools also use matching algorithms.

### Determinism

- **Functional determinism:** the sequencer and event sourcing give the same results on replay. Only event **order** matters, not wall-clock time, so replay can squeeze out idle gaps and run fast.
- **Latency determinism:** almost the same latency for every trade. Measure **p99 or p99.99** (e.g. with HdrHistogram). Investigate spikes; in Java, **safe points** (Stop-the-World GC, deoptimization, stack dumps) pause all threads.

### Market data publisher

- Detailed L2/L3 data is valuable; hedge funds record it themselves. Tiers: retail sees **5 levels of L2** by default and pays for 10.
- Candlesticks for 1 min, 1 hour and 1 day each live in a **ring buffer holding the recent 100 ticks**, then go to persistence and the data service.
- **Ring buffer:** fixed size, pre-allocated, no object creation, lock-free. **Padding** keeps the sequence number alone in its cache line.

### Fairness, multicast, colocation

- Lower latency is like seeing the future. If the first subscriber always gets data first, clients race to connect first. Fixes: **multicast over reliable UDP** (one source to a group of hosts, all at about the same time), or a random subscriber order.
- Unicast = one to one; broadcast = one to a whole subnet; multicast = one to a group across subnets. UDP can drop datagrams, so add retransmission.
- **Colocation** is a paid VIP service. It does not break fairness.

### Network security (DDoS)

1. Isolate public services from private ones; serve read-only copies.
2. Cache data that rarely changes.
3. Harden URLs: `/data/recent` instead of `/data?from=123&to=456`, which attackers can vary endlessly; it also caches at the CDN.
4. Safelists and blocklists at the network gateway.
5. Rate limiting.

## Wrap-up

- Some big exchanges really run everything on one huge server, or even one process.
- Many crypto exchanges run on the cloud. Some DeFi projects use **AMM** (automatic market making) and need no order book.

✓ KEY POINTS

- Do the QPS math, and note the open and close peaks (5×).
- Separate the critical trading path from market data and reporting.
- Sequencer + deterministic matching engine + event log = replay, recovery and exactly-once.
- O(1) order book: price → doubly linked list of orders, plus an orderId index for cancels.
- For µs latency: one server, an mmap bus, single-threaded loops pinned to cores, nothing extra on the path.
- HA: hot-warm engine first, then Raft across servers; RPO ≈ 0, RTO in seconds.

⚠ COMMON TRAP

a typical microservice design with Kafka and a DB on the critical path. Each network hop costs ~500 µs and disk costs ms, so p99 never reaches exchange levels. Another trap: a list-based price level, which makes cancel O(n).

↑ Back to top ☰ Contents

# Design Problems

6 pages. Each card says what the page covers and its key lines; open the page for the full depth.

### Chat System · p. 358

- Separate the **stateful gateway** (WebSockets) from **stateless** chat logic; estimate connections per server.
- Store the message **before** acking "sent"; real-time push is best-effort and backed by **sync from last seq**.
- **Per-conversation sequence numbers** for ordering, not client clocks; no need for global order.
- Explain cross-server fan-out: **session registry (user → gateway)** + Redis Pub/Sub per gateway (or a broker), with the trade-off.

Read the full page →

### Notification System · p. 364

- Accept with **202** and process async; make the **idempotency key required** for callers.
- **Queue per channel and per priority** so OTPs never wait behind a campaign.
- Retries with **exponential backoff + jitter**, separate retryable vs non-retryable errors, **DLQ** with replay.
- Explain the three sources of duplicates and how each is handled; admit exactly-once to an external provider isn't possible.

Read the full page →

### Order Book and Matching Engine · p. 369

- Define **price-time priority** and trade at the **maker's price**.
- Data structures: **sorted price levels + FIFO doubly linked list per level + order_id hash map** (O(1) cancel); give complexities.
- **Single-threaded engine per market** fed by **one ordered log** (sequencer) → no locks, deterministic.
- **Hold funds before matching**; release/adjust after fills or cancel.

Read the full page →

### Rate Limiter · p. 376

- Compare **all five algorithms** in one table and name the fixed-window **edge burst** problem.
- Pick **token bucket** (bursts OK, 2 fields per key) and say why not sliding log (memory).
- Show the **race condition** and solve it with an **atomic Lua script**; mention `EVALSHA` and single-slot keys in Redis Cluster.
- Use **one clock** (Redis `TIME`) to avoid skew between servers.

Read the full page →

### URL Shortener · p. 382

- Do the math: **7 base62 chars ≈ 3.5 trillion codes**; 100:1 read:write; hot set fits in RAM.
- Compare ID strategies (hash / random / counter / Snowflake) and explain why **counter ranges + shuffle** avoid both collisions and guessable codes.
- Explain **301 vs 302** and connect it to the analytics requirement.
- Cache-aside on the redirect path, plus **negative caching** for unknown codes and an in-process cache for hot links.

Read the full page →

### Wallet and Ledger (Crypto Exchange) · p. 388

- **Double-entry**: every transfer sums to zero; balances are a cache of entries and can be rebuilt.
- **Never use float**; integer smallest units or `NUMERIC`.
- Prevent double spend with **atomic conditional UPDATE** (or `SELECT FOR UPDATE` in fixed order) + `CHECK (amount ⩾ 0)`.
- **Idempotency key UNIQUE in the same transaction** as the entries.

Read the full page →

DESIGN PROBLEMS

# Chat System

ON THIS PAGE

1. 1. Requirements · p. 358
2. 2. Estimates · p. 358
3. 3. API · p. 359
4. 4. Data model · p. 359
5. 5. High-level design · p. 360
6. 6. Deep dives · p. 361
7. 7. Wrap-up · p. 362
8. Interview checklist · p. 363

Design a chat app like WhatsApp/Telegram: 1:1 and small group chats, online/offline delivery, receipts and presence. Follows the 7-step framework.

## 1. Requirements

↑ TOP

**Functional:**

- 1:1 chat and group chat (up to ~500 members).
- Real-time delivery when the receiver is online; stored and delivered later when offline (+ mobile push).
- Message history (scroll back), multi-device per user.
- **Sent / delivered / read** receipts.
- Presence: online / last seen.

**Non-functional:**

- **Low latency:** < 200 ms end to end within a region.
- **Ordering:** messages in one conversation appear in the same order for everyone.
- **Durability:** a message that got "sent ✓" is never lost.
- **High availability**; eventual consistency is fine for presence and receipts.
- Scale: 50M DAU.

**Out of scope:** end-to-end encryption details (mention it), voice/video calls, media processing (upload to object storage, send a link), search.

## 2. Estimates

↑ TOP

- 50M DAU × 40 messages/day = **2B messages/day** → $2\times10^9$ / $10^5$ s ≈ **20k msg/s avg**, peak ~60k/s.
- Fan-out: average 1:1, groups bigger → deliveries ≈ 2–3× messages → **~150k deliveries/s at peak**.
- **Concurrent connections:** ~30% of DAU online at peak = **15M WebSockets**. One tuned gateway server holds ~100k–500k idle connections (memory ~10–20 KB each); with 100k each → **~150 gateway servers**.

- **Storage:** 2B × ~200 B (text + metadata) = **400 GB/day** → ~150 TB/year (before replication ×3). Media goes to object storage, not here.
- Reads: users open a chat → load the last 50 messages; read-heavy on recent data.

Conclusion: write-heavy, append-only, huge data → a wide-column store; connections need a separate stateful tier.

## 3. API

↑ TOP

Real-time over **WebSocket** (`wss://chat.example.com/ws`, authenticated with a token on connect):

```
client → server
  send_message   {client_msg_id, conversation_id, body, sent_at}
  ack            {conversation_id, seq}          (delivered up to seq)
  read           {conversation_id, seq}          (read up to seq)
  typing         {conversation_id}
server → client
  message        {conversation_id, seq, message_id, sender_id, body, created_at}
  send_ack       {client_msg_id, message_id, seq}   ← this is the "sent ✓"
  receipt        {conversation_id, user_id, type: delivered|read, seq}
  presence       {user_id, status, last_seen}
```

REST for the rest:

```
GET  /conversations?cursor=                     → list with last message + unread count
GET  /conversations/{id}/messages?before_seq=&limit=50   → history page
POST /conversations  {member_ids}               → create group
```

`client_msg_id` (UUID made on the device) is the **idempotency key**: if the client retries after a network drop, we don't store the message twice.

## 4. Data model

↑ TOP

**messages** (ScyllaDB / Cassandra):

| Column | Note |
|---|---|
| conversation_id | **partition key** (with a time bucket, see below) |
| bucket | e.g. month — keeps partitions < ~100 MB |
| seq | **clustering key DESC** — per-conversation sequence number |
| message_id, sender_id, body, created_at, client_msg_id | |

Query "last 50 messages of conversation X" = one partition, already sorted → very fast.

**conversations / members** (Postgres): `conversations(id, type, created_at)`, `members(conversation_id, user_id, role, last_read_seq, last_delivered_seq)`, index on `(user_id)` for "my chats".

**inbox** (ScyllaDB): `user_conversations(user_id, last_activity DESC, conversation_id)` for the chat list.

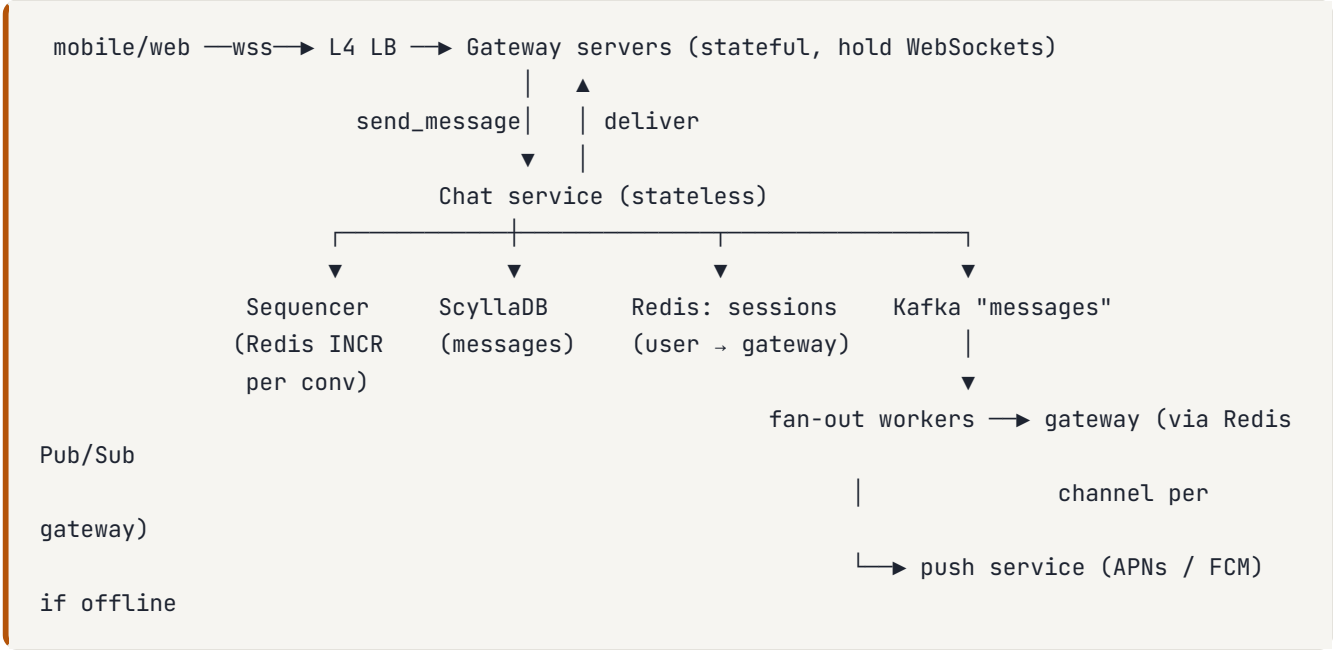
**Presence / sessions** (Redis): `session:{user_id}` → set of `{device_id → gateway_id}`, TTL refreshed by heartbeat.

**DB choice:**

- **ScyllaDB/Cassandra** for messages: LSM-tree, very high write throughput, linear scaling, partitioning by conversation matches the main query. ScyllaDB is the C++ rewrite with lower tail latency (Discord moved to it). Trade-off: no joins, no multi-row transactions, must design tables per query.
- **Postgres alternative:** works up to a few billion rows with **partitioning by time** + index `(conversation_id, seq)`, and sharding by `conversation_id` later (Citus). Simpler for a smaller company; harder at 400 GB/day.
- **Postgres** for membership: small, relational, needs consistency.

## 5. High-level design

↑ TOP




**Send flow (A → B):**

1. A's gateway receives `send_message`, forwards to the chat service.
2. Chat service checks membership, dedups by `client_msg_id`, gets `seq = INCR seq:{conversation_id}`.

3. Writes the message to ScyllaDB (quorum write). **Only now** sends `send_ack` → A sees "sent ✓".
4. Publishes to Kafka topic `messages` (key = `conversation_id`, so per-conversation order is kept).
5. Fan-out worker reads members, looks up each member's sessions in Redis, publishes to the channel of each gateway that holds a member's connection.
6. B's gateway pushes `message` to B's devices. B's client sends `ack(seq)` → "delivered ✓✓" goes back to A.
7. Members with no live session → push notification via APNs/FCM.

**Reconnect / sync:** client sends its last known `seq` per conversation; server returns everything after it from ScyllaDB. This makes delivery **reliable even if a real-time push was lost**.

## 6. Deep dives

↑ TOP

### 6.1 Connection gateway and cross-server fan-out

**Problem:** 15M connections on ~150 servers. B is connected to gateway 87; the message was processed somewhere else. How does it reach gateway 87?

**Options:**

- **Redis Pub/Sub**, one channel per gateway (`gw:87`): fan-out worker looks up `session:B` → `gw:87` → `PUBLISH`. Fast (sub-ms), simple. It is fire-and-forget: if the gateway is down, the message is dropped — OK because the message is **already stored** and the client re-syncs by `seq`. (Redis Pub/Sub notes)
- **Kafka/broker per gateway**: durable, but thousands of consumers and higher latency for a "best-effort push" that is already backed by storage.
- **Direct gRPC** from worker to gateway (service discovery by gateway ID): fewest hops; more connection management.

**Choice:** Kafka for the durable "message accepted" stream (ordering, replay, multiple consumers: fan-out, push, search indexing, analytics), and **Redis Pub/Sub per gateway** for the last hop. Trade-off: two systems to run, but each does what it's good at — durable log vs low-latency fire-and-forget.

Gateway notes: keep them **thin** (auth, heartbeats, forward); use a **L4** load balancer (long-lived TCP); on deploy, drain connections gradually so 100k clients don't reconnect at once (add jitter to client reconnect).

### 6.2 Ordering per conversation

**Problem:** two users in a group send at the same moment; clocks on phones are wrong; messages must show in one order for all.

**Choice:** a **server-assigned, per-conversation sequence number** (`INCR seq:{conv}` in Redis, or a counter column updated in the DB). The sequence is the order of truth; client timestamps are only for display. Kafka key = `conversation_id` keeps the order through fan-out. Clients sort by `seq` and detect **gaps** (got 41, 43 → fetch 42).

Trade-off: the per-conversation counter is a tiny serialization point — fine, since a single conversation rarely exceeds a few msg/s. If Redis loses the counter, recover from `max(seq)` in ScyllaDB (persist with AOF to reduce risk). We don't need **global** ordering, only per conversation — saying that is a key point.

### 6.3 Delivery and read receipts

- **Sent ✓:** server stored it (step 3).
- **Delivered ✓✓:** recipient device acked `seq`. Store `last_delivered_seq` per (member, conversation) — **one number, not one row per message**.
- **Read (blue):** client sends `read(seq)`; store `last_read_seq`. Unread count = `latest_seq - last_read_seq`.
- In large groups, receipts are many (500 members × each message). Batch them (send at most every ~1 s), and show "read by N" lazily on request.
- Delivery semantics: **at-least-once** with client dedup by `message_id` — retries and re-sync may send a message twice; the client ignores duplicates.

### 6.4 Presence

- Gateway sets `presence:{user}` = online with **TTL 60 s**, refreshed by heartbeats every 30 s. Disconnect or missing heartbeat → key expires → "last seen = t".
- Fan-out of presence changes is expensive (a user with 500 contacts). So: push presence only to users who have **that chat open** (subscribe on open), others fetch on demand. Debounce flapping (bad mobile network) by waiting a few seconds before showing "offline".
- Trade-off: presence is eventually consistent and slightly delayed — users accept that.

## 7. Wrap-up

↑ TOP

- **Bottlenecks:** gateway count (memory per connection), ScyllaDB write throughput (add nodes), huge groups (fan-out cost → switch to "pull" for channels with 10k+ members).
- **Failures:**
- Gateway dies → clients reconnect to another one and re-sync by `seq`; nothing lost.
- Redis Pub/Sub message lost → client sync covers it.
- ScyllaDB node down → RF=3 + quorum keeps working.
- Push provider down → retry with backoff; messages are still in history.
- **Monitoring:** connected sockets per gateway, send→deliver latency p99, Kafka consumer lag, push failure rate, reconnect storms.
- **Next:** E2E encryption (Signal protocol: server stores only ciphertext), multi-region with users homed to a region, media pipeline.

↑ TOP

## Interview checklist

- Separate the **stateful gateway** (WebSockets) from **stateless** chat logic; estimate connections per server.
- Store the message **before** acking "sent"; real-time push is best-effort and backed by **sync from last seq**.
- **Per-conversation sequence numbers** for ordering, not client clocks; no need for global order.
- Explain cross-server fan-out: **session registry (user → gateway)** + Redis Pub/Sub per gateway (or a broker), with the trade-off.
- Pick ScyllaDB/Cassandra with **partition = conversation (+ time bucket), clustering = seq DESC**; mention the Postgres partitioning alternative.
- Receipts as **last_delivered_seq / last_read_seq** per member, not per message.
- Offline users → APNs/FCM push; presence with heartbeats + TTL and limited fan-out.
- Idempotency with `client_msg_id` for client retries.

↑ Back to top ☰ Contents

DESIGN PROBLEMS

# Notification System

ON THIS PAGE

1. 1. Requirements · p. 364
2. 2. Estimates · p. 365
3. 3. API · p. 365
4. 4. Data model · p. 365
5. 5. High-level design · p. 366
6. 6. Deep dives · p. 367
7. 7. Wrap-up · p. 368
8. Interview checklist · p. 368

Design a service that other teams call to send notifications by **push (mobile), SMS and email** — for example "deposit confirmed", "order filled", OTP codes, marketing campaigns. Follows the 7-step framework.

## 1. Requirements

↑ TOP

**Functional:**

- Internal services send a notification: `user_id`, `type` (template), params, channels (or "use user preferences").
- Channels: **push** (APNs/FCM), **SMS** (e.g. Kavenegar / a local SMS gateway), **email** (SMTP provider / SES).
- **Templates** with variables and languages (fa/en).
- **User preferences:** opt out per category/channel (marketing off, security always on), quiet hours.
- Bulk campaigns: send to millions of users.
- Status tracking: queued / sent / delivered / failed.

**Non-functional:**

- **Reliability:** transactional messages (OTP, withdrawal alerts) must not be lost; **at-least-once** delivery with deduplication → effectively once.
- **Priority:** an OTP must not wait behind a 5M-user marketing campaign. OTP p99 < 5 s end to end.
- **Respect provider limits** (SMS gateway may allow ~100–500 msg/s) and protect users from spam.
- Scalable and horizontally extendable with new channels.

**Out of scope:** in-app notification feed UI (mention it), A/B testing, building our own SMS/push infrastructure.

↑ TOP

## 2. Estimates

- 10M users. Transactional: ~10 notifications/user/day across channels → **100M/day ≈ 1,200/s avg**, peak ~4k/s (market crash = many order fills).
- Campaigns: 5M users in one campaign, to be sent within ~1 hour → **~1,400/s** extra.
- By channel (rough): push 80%, email 15%, SMS 5% → SMS ≈ 5M/day ≈ 60/s avg, but **cost** matters more than load (SMS costs money per message).
- Storage: notification log row ~500 B × 100M/day = **50 GB/day**; keep 90 days hot ≈ 4.5 TB → partition by day, archive older.

Conclusion: load is moderate; the hard parts are **reliability, priority, provider limits and dedup**, not raw throughput.

## 3. API

↑ TOP

```
POST /v1/notifications
Header: Idempotency-Key: deposit-confirmed:tx_9f3a     (required)
{
  "user_id": 42,
  "template": "deposit_confirmed",
  "params": {"amount": "0.5", "asset": "BTC"},
  "channels": ["push", "sms"],          // optional; default = from preferences
  "priority": "high"                    // high | normal | low(bulk)
}
→ 202 Accepted {notification_id, status: "queued"}

GET  /v1/notifications/{id}               → status per channel
POST /v1/campaigns {template, segment_id, schedule_at}   → {campaign_id}
PUT  /v1/users/{id}/preferences {category, channel, enabled}
POST /v1/webhooks/{provider}              ← delivery reports from SMS/email/push providers
```

`202` because sending is async. The **idempotency key is required** — the caller (e.g. deposit service) may retry.

## 4. Data model

↑ TOP

Postgres (moderate write rate, needs uniqueness + transactions):

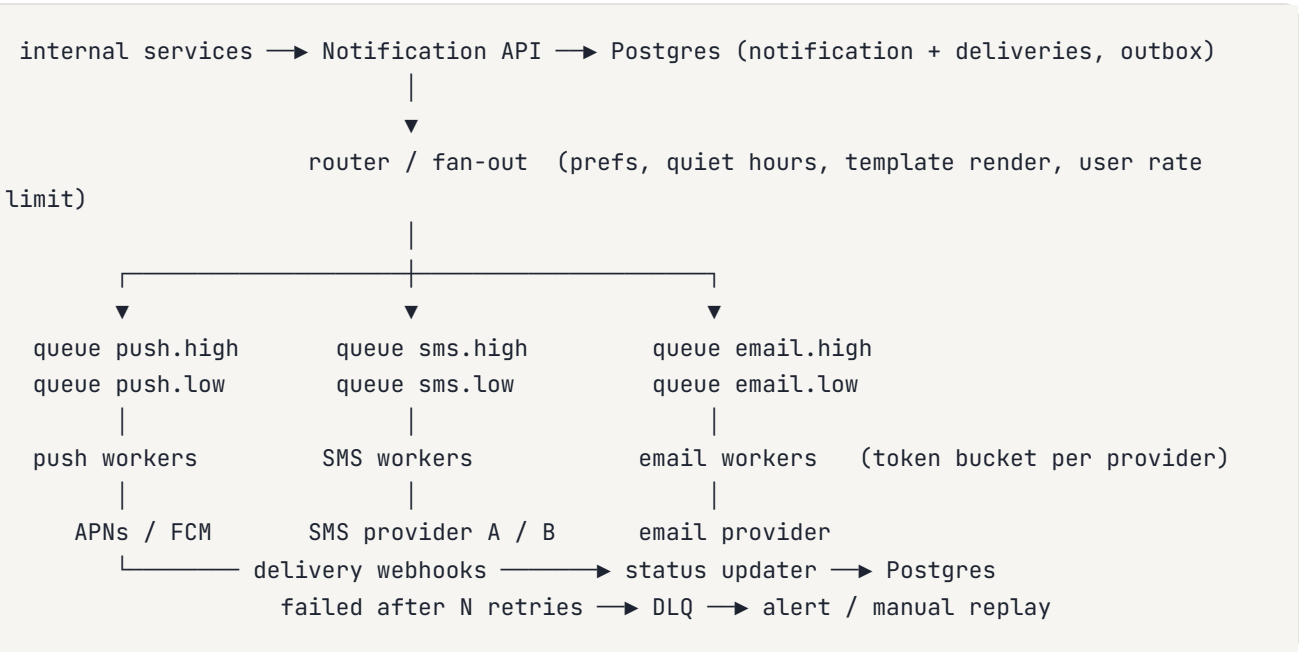
| Table | Key columns | Notes |
|---|---|---|
| notifications | `id` PK, `idempotency_key` **UNIQUE**, user_id, template, params JSONB, priority, created_at | partitioned by day |

| Table | Key columns | Notes |
|---|---|---|
| deliveries | `id` PK, notification_id, channel, status, provider, provider_msg_id, attempts, last_error, updated_at | index `(provider_msg_id)` for webhooks |
| templates | `(name, channel, locale, version)` PK, body, subject | cached in memory |
| preferences | `(user_id, category, channel)` PK, enabled, quiet_hours | cached in Redis |
| devices | `(user_id, device_token)` PK, platform, last_seen | remove token when APNs/FCM says invalid |

Why Postgres: the unique constraint on `idempotency_key` gives dedup for free, and status updates are transactional. Trade-off: at much higher volume the log table could move to Cassandra/ClickHouse, but 1–4k/s writes is fine for Postgres with partitioning.

## 5. High-level design

↑ TOP




**Flow:**

1. API validates, inserts `notifications` row (unique `idempotency_key`; duplicate → return the existing ID). In the same transaction, writes an **outbox** row.
2. Outbox relay publishes to the router queue (no lost message if the API crashes after commit).
3. Router loads **preferences** (Redis cache), drops channels the user disabled (except mandatory security types), applies quiet hours (delay low priority), checks per-user frequency limits, renders the template, and enqueues one job per channel into the right **priority queue**.
4. Channel workers send via the provider, store `provider_msg_id`, mark `sent`.
5. Provider webhook → mark `delivered` / `failed`.

↑ TOP

# 6. Deep dives

## 6.1 Queue per channel and priority

**Problem:** one shared queue → a slow SMS provider or a 5M campaign blocks OTPs and push.

**Choice: separate queues per channel and per priority** (`sms.high`, `sms.low`, ...). Each channel has its own worker pool, so a broken email provider never delays push. Workers always drain `high` first (or use more workers for `high`).

**Broker: RabbitMQ** — this is a classic task queue: per-message ack, retries with TTL, dead-letter exchanges, priority queues are built in (why). Kafka would work for the event intake, but per-message retry/delay is harder there. Trade-off: RabbitMQ has lower max throughput than Kafka — fine at ~5k/s. (A Celery + Redis setup is also reasonable for a smaller team.)

## 6.2 Retries, DLQ and provider failover

- **Retryable errors** (timeout, 5xx, 429 from provider): retry with **exponential backoff + jitter** (e.g. 10 s, 30 s, 2 min, 10 min) using a delay/retry queue (RabbitMQ TTL queue → dead-letters back into the main queue).
- **Non-retryable** (invalid phone, unregistered device token): mark `failed`, clean the token, no retry.
- After **N attempts** → **DLQ** + alert. An admin tool can replay DLQ messages after a fix.
- **Provider failover for SMS:** keep two providers; a **circuit breaker** per provider. If provider A's error rate passes 20%, route to B. Important for OTP.
- **Expiry:** OTPs and price alerts become useless after a few minutes → set a TTL; don't send stale messages after an outage.

## 6.3 Idempotency (no double SMS)

Duplicates can come from three places:

1. **Caller retries** → unique `idempotency_key` in `notifications` returns the existing row.
2. **Queue redelivery** (worker crashed after sending, before ack) → before sending, the worker does an atomic state change: `UPDATE deliveries SET status='sending' WHERE id=? AND status IN ('queued','retry')` — 0 rows updated means someone else handled it. Optionally `SET NX` a Redis key `sent:{delivery_id}` with TTL.
3. **Provider ambiguity** (timeout, did it send?) → pass our `delivery_id` as the provider's idempotency/reference ID when supported.

Trade-off: there is still a small window (sent, then crash before marking `sent`) where one duplicate is possible — true exactly-once to an external provider is impossible. We minimize it and accept it, and say so.

### 6.4 User preferences and rate limiting

- Categories: `security` (OTP, login, withdrawal — **cannot opt out**), `transactional` (fills, deposits), `marketing` (opt-out, quiet hours 22:00–08:00 local time).
- Preferences cached in Redis (`prefs:{user}`), invalidated on update; DB is the source of truth.
- **Per-user limit** (e.g. max 3 marketing pushes/day, max 5 SMS/hour) with a Redis counter — protects users and SMS cost (rate limiter).
- **Per-provider limit:** a token bucket in workers matching the provider's contract (e.g. 200 SMS/s). Excess waits in the queue — that's what queues are for.
- **Campaigns:** don't enqueue 5M jobs at once; a campaign scheduler pages through the segment and enqueues in batches at a controlled rate, into `low` queues only.

## 7. Wrap-up

↑ TOP

- **Bottlenecks:** provider throughput and cost (SMS), not our servers. Postgres log table growth → daily partitions, drop/archive old ones.
- **Failures:** provider down → circuit breaker + failover; RabbitMQ node down → quorum queues; worker crash → message redelivered, dedup protects user; router down → outbox keeps messages until it's back.
- **Monitoring:** queue depth and age per queue (alert if `sms.high` oldest message > 10 s), send success rate per provider, delivery rate from webhooks, DLQ size, cost per day per channel, opt-out rate after campaigns.
- **Next:** in-app inbox (store and show in the app), smart channel fallback (push not opened in 10 min → SMS for important messages), analytics on opens/clicks.

## Interview checklist

↑ TOP

- Accept with **202** and process async; make the **idempotency key required** for callers.
- **Queue per channel and per priority** so OTPs never wait behind a campaign.
- Retries with **exponential backoff + jitter**, separate retryable vs non-retryable errors, **DLQ** with replay.
- Explain the three sources of duplicates and how each is handled; admit exactly-once to an external provider isn't possible.
- **Preferences** with mandatory security category, quiet hours, per-user frequency caps.
- Respect **provider rate limits** with a token bucket; **circuit breaker + second SMS provider**.
- Outbox pattern between the API's DB write and the queue.
- Track delivery status via provider **webhooks**.

↑ Back to top ☰ Contents

DESIGN PROBLEMS

# Order Book and Matching Engine

ON THIS PAGE

1. 1. Requirements · p. 369
2. 2. Estimates · p. 369
3. 3. API · p. 370
4. 4. Data model · p. 370
5. 5. High-level design · p. 371
6. 6. Deep dives · p. 372
7. 7. Wrap-up · p. 374
8. Interview checklist · p. 374

Design the core of a crypto exchange (like Nobitex): users place limit/market orders on markets like `BTC-USDT` or `USDT-IRT`; the engine matches them by **price-time priority**, produces trades, and streams market data. Follows the 7-step framework.

## 1. Requirements

↑ TOP

**Functional:**

- Place **limit** and **market** orders (buy/sell); cancel orders.
- Match by **price-time priority**: best price first; at the same price, earlier order first.
- Partial fills; order status (open, partially filled, filled, cancelled).
- Funds must be **held** before an order enters the book (no selling coins you don't have).
- Publish trades and order-book updates (market data) to clients in real time.

**Non-functional:**

- **Deterministic and correct:** same input sequence → same trades, always. No lost or duplicated fills.
- **Low latency:** matching in microseconds; order accept → ack p99 < 50 ms end to end (retail exchange, not HFT).
- **Throughput:** ~50k orders/s peak across all markets; one hot market (USDT-IRT, BTC-USDT) can get ~10k/s.
- **Recoverable:** after a crash, rebuild the exact book state with no data loss.
- Fairness: strict ordering, no one jumps the queue.

**Out of scope:** stop/OCO orders (they are a trigger layer on top), margin/futures, KYC, the full ledger (wallet design).

## 2. Estimates

↑ TOP

- 300 markets, 2M active traders. Normal: ~5k orders+cancels/s total; volatile peak **50k/s** (bots cancel/replace a lot — cancels are often > 50% of messages).
- Order message ≈ 200 B → 50k/s × 200 B = **10 MB/s** into the log → Kafka handles this easily (~hundreds of MB/s per broker).

- Trades: ~10% of orders match → ~5k trades/s peak → ~100M+ trades/day at peak days.
- Book size in memory: a hot market may have ~100k open orders × ~100 B ≈ 10 MB. All markets ⩽ 1 GB → **the whole state fits in RAM** on one machine.
- Market data: 100k WebSocket clients; each update fanned out → must **batch/throttle** (e.g. depth snapshot every 100 ms) or it's millions of messages/s.

Conclusion: matching itself is cheap CPU work in memory; the design is about **ordering, determinism, durability and fan-out**.

## 3. API

↑ TOP

```
POST   /v1/orders   Header: Idempotency-Key (client_order_id)
       {market: "BTC-USDT", side: "buy", type: "limit", price: "65000.00", qty: "0.25",
tif: "GTC"}
       → 202 {order_id, status: "accepted"}        (final status comes via events)
DELETE /v1/orders/{order_id}                        → 202 {status: "cancel_requested"}
GET    /v1/orders?market=&status=open               → my orders
GET    /v1/markets/{m}/depth?limit=50               → {bids: [[price, qty]], asks: [...],
seq}
GET    /v1/markets/{m}/trades?limit=100             → recent trades

WebSocket /ws
  subscribe {channel: "depth" | "trades" | "orders" (private), market}
  ← depth_update {market, seq, bids: [[p, q]], asks: [[p, q]]}   (q = 0 means remove level)
  ← trade {market, trade_id, price, qty, taker_side, ts}
  ← order_update {order_id, status, filled_qty, avg_price}
```

`seq` on market data lets clients detect gaps and re-fetch a snapshot.

## 4. Data model

↑ TOP

**In memory (engine, per market):**

```
OrderBook
  bids: sorted map price → PriceLevel   (highest first)
  asks: sorted map price → PriceLevel   (lowest first)
  orders: hash map order_id → *Order     (O(1) cancel)
PriceLevel
  price, total_qty
  queue: doubly linked list of Orders   (FIFO = time priority)
Order
  id, user_id, side, price, remaining_qty, seq, prev/next pointers
```

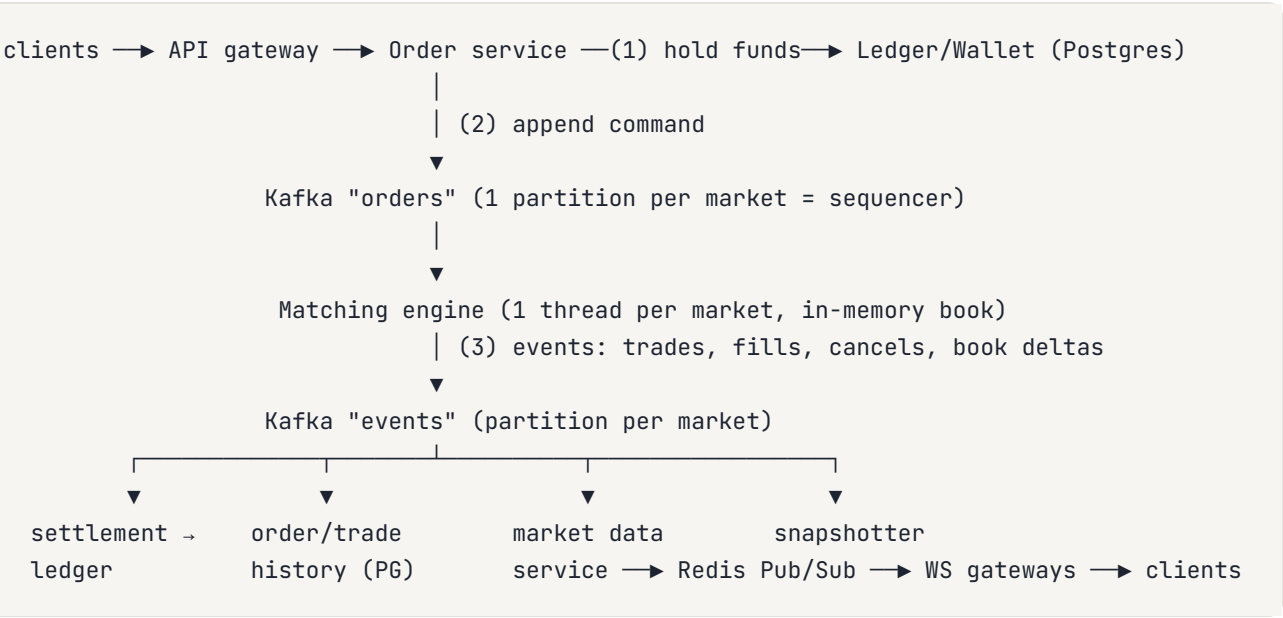
**Durable:**

| Store | Content | Why |
|---|---|---|
| Kafka `orders.{market}` (input log) | every accepted command (new/cancel), in sequence | source of truth for replay; ordered per partition |
| Kafka `events.{market}` (output log) | trades, order updates, book deltas | many consumers: ledger, market data, history |
| Postgres `orders`, `trades` | queryable history for users/API | written by consumers, not by the engine |
| Snapshots (object storage / disk) | full book state at seq N | fast recovery |

Prices and quantities as **fixed-point integers** (price in 1e-8 units), never floats — determinism and exact money.

## 5. High-level design

↑ TOP




**Place order flow:**

1. Order service validates (tick size, min qty, market open), checks idempotency (`client_order_id`).
2. **Holds funds** in the ledger: buy 0.25 BTC @ 65,000 → hold 16,250 USDT (+ max fee). Insufficient → reject immediately.
3. Appends `NewOrder` to Kafka partition of that market. The partition offset **is** the sequence number.
4. Engine consumes in order, matches against the opposite side, emits `Trade` and `OrderUpdate` events, rests any remainder in the book.
5. Settlement consumer applies trades to the ledger (move held quote → seller, base → buyer, fees; release unused hold), idempotent by `trade_id`.

6. Market data service builds depth/trades streams and fans out.

**Cancel:** same path (`CancelOrder` command in the same partition), so a cancel and a fill can never race — whichever is first in the log wins.

## 6. Deep dives

↑ TOP

### 6.1 Matching algorithm and data structures

Incoming **buy limit** at price P, qty Q:

```
while Q > 0 and asks not empty and best_ask.price ⩽ P:
    level = best_ask
    while Q > 0 and level.queue not empty:
        maker = level.queue.head
        fill = min(Q, maker.remaining)
        emit Trade(price = level.price, qty = fill, maker, taker)   # trade at the MAKER's
price
        Q -= fill; maker.remaining -= fill
        if maker.remaining == 0: remove maker (pop head, delete from orders map)
    if level empty: remove level
if Q > 0: rest order at P in bids (append to tail of level P)   # GTC; IOC → cancel rest
```

Market order = same loop without the price check (with a protection limit to avoid sweeping the whole book in a thin market).

**Complexity:**

| Operation | Structure | Cost |
|---|---|---|
| Best bid/ask | sorted map (red-black tree / skip list) min/max, or cached pointer | O(1)–O(log L) |
| Add at existing level | hash map price → level, append to list tail | O(1) |
| New price level | tree insert | O(log L), L = number of levels |
| Cancel | `orders[id]` → unlink from doubly linked list | **O(1)** |
| Match one fill | pop list head | O(1) |

Why these structures: matching only ever touches the **best** level, and cancels (most messages) need O(1) lookup. An array indexed by price ticks is even faster for markets with a narrow price range, but wastes memory for wide ranges. Trade-off: tree = general and compact; array = fastest for hot markets. In Go: a B-tree/skip list for levels + `container/list`-style intrusive linked list; avoid allocations on the hot path (object pools).

### 6.2 Single-threaded matching per market (sequencer)

**Problem:** if two threads match the same book, we need locks, and results depend on thread timing → **non-deterministic** and slower under contention.

**Choice: one single-threaded engine per market**, consuming a **single ordered input** (the Kafka partition = the sequencer). Like the LMAX architecture: no locks, CPU-cache friendly, millions of simple ops/s per core. Different markets are independent → run many markets in parallel (one goroutine/thread each, or spread across machines). Shard key = `market`.

Trade-off: one market's throughput is capped by **one core** (~100k–1M+ orders/s with a careful implementation — far above our 10k/s). A hot market can't be split, but none needs to be at this scale.

Determinism rules: no wall-clock reads in matching logic (timestamps come from the command), no randomness, no map iteration order dependence. Then **replaying the same log yields the same book and the same trades**.

### 6.3 Balance holds before matching

**Problem:** the engine must not check balances itself (that would need a DB call per order → slow and breaks determinism). But it must never match an order the user can't pay for.

**Choice: hold first, then match.** The order service moves funds from `available` → `held` in the ledger (atomic conditional update) before the order goes into the log. The engine assumes every order is funded. After fills, settlement moves held funds to the counterparty; on cancel/expiry, the remainder is released.

Trade-off: an extra synchronous DB write before each order (~a few ms). Bigger exchanges keep balances **in the engine's memory** (risk engine in the same sequenced pipeline) to avoid it — much faster but much more complex. For a retail exchange, the ledger hold is the right default.

Edge case: a buy limit that fills at a better price → hold was larger than needed → release the difference at settlement.

### 6.4 Persistence, recovery and market data

**Recovery via replay:**

- Input log (Kafka, replication factor 3, `acks=all`) is the source of truth.
- The engine writes a **snapshot** of each book every N seconds (e.g. every 1M messages) with the last processed offset.
- On crash: load the latest snapshot → replay commands from offset+1 → identical state. Recovery takes seconds.
- **Hot standby:** a second engine consumes the same partition and stays in sync (deterministic), ready to take over. Output events carry the input offset; consumers dedup by `(market, seq)` so a takeover that re-emits events is harmless.

**Idempotent downstream:** settlement uses `trade_id` (derived from market + input offset + fill index) as the ledger idempotency key → re-processed events never double-settle.

**Market data fan-out:**

- Engine emits per-level deltas `(price, new_total_qty)` with a sequence number.
- Market data service keeps a copy of the book, publishes **throttled** depth updates (e.g. every 100 ms, top 50 levels) and every trade.
- Fan-out to 100k WebSocket clients via gateways subscribed to Redis Pub/Sub channels per market. Client gets a REST snapshot with `seq`, then applies deltas; on a gap, re-fetch the snapshot.
- Trade-off: throttling hides some intermediate states (fine for retail UIs); professional clients can get a raw full-depth feed.

## 7. Wrap-up

↑ TOP

- **Bottlenecks:** the ledger hold before each order (DB write rate), market-data fan-out (batch + throttle), one core per hot market (far from the limit).
- **Failures:**
- Engine crash → standby takes over / snapshot + replay; no order lost because the input was durable before ack.
- Kafka broker down → RF=3, leader election; order intake pauses for seconds.
- Settlement lag → balances update late but correctly; alert on consumer lag.
- Bad input (bug) → **halt the market** (circuit breaker), fix, replay; exchanges prefer halting to wrong trades.
- **Monitoring:** order → ack latency p99, engine processing time per message, Kafka lag per market, trades/s, settlement lag, reconciliation (sum of held funds = sum of open order values), WebSocket gap/resubscribe rate.
- **Next:** stop orders, self-trade prevention, per-user rate limits for bots (rate limiter), co-located low-latency API for market makers.

## Interview checklist

↑ TOP

- Define **price-time priority** and trade at the **maker's price**.
- Data structures: **sorted price levels + FIFO doubly linked list per level + order_id hash map** (O(1) cancel); give complexities.
- **Single-threaded engine per market** fed by **one ordered log** (sequencer) → no locks, deterministic.
- **Hold funds before matching**; release/adjust after fills or cancel.
- Input log as source of truth; **snapshot + replay** for recovery; hot standby.
- Trades and events via **Kafka**; downstream idempotent by `trade_id`.
- Market data: deltas with **seq numbers**, snapshot + delta protocol, throttling, Redis Pub/Sub fan-out.

- **Fixed-point integers**, no floats; no wall-clock or randomness in matching.

↑ Back to top ☰ Contents

DESIGN PROBLEMS

# Rate Limiter

ON THIS PAGE

1. 1. Requirements · p. 376
2. 2. Estimates · p. 376
3. 3. API · p. 377
4. 4. Data model · p. 377
5. 5. High-level design · p. 378
6. 6. Deep dives · p. 378
7. 7. Wrap-up · p. 380
8. Interview checklist · p. 380

Design a distributed rate limiter that protects an API (for example an exchange's public and trading APIs) from abuse and overload. Follows the 7-step framework.

## 1. Requirements

↑ TOP

**Functional:**

- Limit requests per **key** (user ID, API key, or IP) per **rule** (e.g. `POST /orders`: 10/s per user; public market data: 1,200/min per IP).
- Rules are configurable without a deploy.
- Rejected requests get **HTTP 429** with headers telling the client when to retry.

**Non-functional:**

- **Low overhead:** < 1–2 ms added latency per request.
- **Distributed:** the limit is global across all API servers, not per server.
- **Highly available:** if the limiter fails, the API must keep working (**fail open** for most routes).
- **Accurate enough:** small overshoot (a few %) is acceptable; big bursts at window edges are not.

**Out of scope:** DDoS protection at the network layer (that's the CDN/WAF job), billing quotas per month.

## 2. Estimates

↑ TOP

- 10M DAU, peak **50k requests/s** hitting the API gateway.
- Each request = **1 Redis call** (one Lua script) → 50k ops/s. One Redis node handles ~100k simple ops/s, but Lua scripts are heavier → use a **Redis Cluster with 3–6 shards** for headroom and HA.
- Memory: active keys ≈ 10M users × 2 rules × ~100 B (token bucket = 2 fields) ≈ **2 GB**. A sliding-window **log** would be much bigger: 100 req/min limit × 16 B per entry × 10M = 16 GB+ — one reason to avoid it at scale.
- Latency budget: Redis round trip in the same DC ≈ 0.5 ms → fits.

↑ TOP

## 3. API

The limiter is a library/middleware in the API gateway, not a public API. Internal interface:

```
allow(key="user:42", rule="orders_create") → {allowed: bool, remaining: int, reset_at: ts,
retry_after_s: int}
```

Responses to the client:

```
200 OK
X-RateLimit-Limit: 10
X-RateLimit-Remaining: 3
X-RateLimit-Reset: 1759740000        (unix seconds when the window/bucket is full again)

429 Too Many Requests
Retry-After: 2                        (seconds; standard HTTP header)
X-RateLimit-Limit: 10
X-RateLimit-Remaining: 0
```

(The IETF draft standardizes `RateLimit-Limit/Remaining/Reset`; many APIs still use the `X-` names.)

## 4. Data model

↑ TOP

**Rules** (Postgres or a YAML in a config service, cached in each gateway, reloaded every ~30 s):

| Field | Example |
|---|---|
| rule_id | `orders_create` |
| match | `POST /api/v1/orders` |
| key_type | `user` / `api_key` / `ip` |
| algorithm | `token_bucket` |
| capacity / refill_rate | 20 tokens, 10 tokens/s |
| fail_mode | `open` / `closed` |

**Counters** (Redis, ephemeral, with TTL):

- Token bucket: hash `rl:{rule}:{key}` → `{tokens, ts}`, TTL = time to fully refill.
- Sliding window counter: `rl:{rule}:{key}:{window_start}` → count, TTL = 2 × window.

**Why Redis:** in-memory (sub-ms), atomic Lua scripts, built-in TTL to clean up idle keys. Trade-off: it's another hop and a dependency; we must decide what happens when it's down.

↑ TOP

## 5. High-level design

```
client ──► CDN/WAF ──► API gateway (rate-limit middleware) ──► backend services
                            │    ▲
                            │    │ rules (cached, refreshed every 30 s)
                            │  config service / Postgres
                            ▼
                      Redis Cluster (counters, Lua scripts)
```

**Flow:**

1. Request arrives at the gateway; middleware finds matching rules and builds the key (`user:42`, or IP if unauthenticated).
2. Calls `EVALSHA` with the token-bucket script on Redis → gets `allowed, remaining, retry_after`.
3. Allowed → forward to backend, add `X-RateLimit-*` headers. Denied → return **429** + `Retry-After` immediately (no backend work).
4. Redis timeout (> 5 ms) or error → apply the rule's `fail_mode` (open by default), increment a metric.

Putting it in the **gateway** (not in each service) means one place for rules and rejected requests never reach the backend.

## 6. Deep dives

↑ TOP

### 6.1 Choosing the algorithm

| Algorithm | How | Bursts | Memory per key | Accuracy | Note |
|---|---|---|---|---|---|
| **Fixed window** | counter per minute `INCR` + `EXPIRE` | **2× burst at the window edge** (limit at 0:59 and again at 1:00) | 1 int | low at edges | simplest |
| **Sliding window log** | sorted set of timestamps; drop old, count rest | none | O(limit) — one entry per request | exact | too much memory at high limits |
| **Sliding window counter** | current + previous window counts, weighted: `prev × (1 - elapsed/window) + curr` | small | 2 ints | ~approximate (assumes even spread) | good general default |
| **Token bucket** | bucket of N tokens, refills at r/s; each request takes 1 | **allows controlled bursts** up to capacity | 2 fields | exact for its model | used by AWS, Stripe |

| Algorithm | How | Bursts | Memory per key | Accuracy | Note |
|---|---|---|---|---|---|
| **Leaky bucket** | requests enter a queue, processed at a fixed rate | smooths output, no bursts | queue | exact | shapes traffic; adds waiting, not a pure reject |

**Choice: token bucket** for user/API-key limits: it allows short bursts (a trading bot sends 5 orders at once) but enforces the average rate, and needs only 2 fields. **Sliding window counter** is a fine choice for simple "N per minute" public limits. Trade-off: token bucket has two knobs (capacity, rate) that are a bit harder to explain to API users than "100 per minute".

### 6.2 Atomic implementation in Redis (Lua)

**Problem:** "read tokens → compute → write tokens" from two gateways at the same time is a **race**: both read 1 token, both allow. `GET` + `SET` is not atomic.

**Options:** `WATCH/MULTI` (optimistic, retries under contention), a distributed lock (slow), or a **Lua script** (Redis runs it atomically, single-threaded). **Choice: Lua.**

LUA

```lua
-- KEYS[1] = bucket key; ARGV = capacity, refill_per_sec, now_ms, cost
local cap    = tonumber(ARGV[1])
local rate   = tonumber(ARGV[2])
local now    = tonumber(ARGV[3])
local cost   = tonumber(ARGV[4])

local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(b[1]) or cap
local ts     = tonumber(b[2]) or now

-- refill based on elapsed time
tokens = math.min(cap, tokens + (now - ts) / 1000 * rate)

local allowed = 0
local retry_ms = 0
if tokens >= cost then
  tokens = tokens - cost
  allowed = 1
else
  retry_ms = math.ceil((cost - tokens) / rate * 1000)
end

redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(cap / rate * 1000))
return {allowed, math.floor(tokens), retry_ms}
```

Notes:

- Load once with `SCRIPT LOAD`, call with `EVALSHA` (less bandwidth).

- **Clock:** gateways' clocks may differ. Safer: use Redis time inside the script (`redis.call('TIME')`), so one clock decides. (Allowed since Redis 5 with effect replication.)
- In Redis Cluster, a script may only touch keys in **one slot** — fine here (one key). For multi-key scripts, use hash tags `rl:{user:42}:...`.
- `cost` lets heavy endpoints consume more tokens (e.g. a batch order = 5).

### 6.3 Distributed concerns

- **Per-server local limits** (no Redis) are fast but wrong: with 20 gateways, a user can get 20× the limit if the LB spreads requests. Only OK as a coarse first layer.
- **Hot keys:** one abusive IP at 20k/s hits one Redis shard. Add a **local in-memory pre-limiter** in each gateway (e.g. deny an IP locally for 1 s after Redis says it is far over the limit). Cuts Redis load for attackers.
- **Batching / approximate mode:** for very high QPS, each gateway can take tokens in chunks (lease 10 tokens at once) and count locally. Fewer Redis calls, small overshoot. Trade-off: accuracy vs load.
- **Multi-region:** a global exact limit needs cross-region coordination (~100 ms) — too slow. Use **per-region limits** (global limit ÷ regions) or async sync between regions; accept some overshoot.
- **Fail open vs fail closed:** if Redis is down, fail **open** for read/public routes (availability first), but for sensitive routes (login, withdrawal, OTP) fail **closed** or use a strict local limit — abuse there is more costly than a short outage.

### 6.4 Client experience

- Always return `Retry-After` on 429, so well-behaved clients back off instead of retrying in a tight loop.
- Different limits per tier (free vs VIP market maker) come from the rule lookup by API key.
- Log rejections with the key and rule — support needs to answer "why am I blocked?".

## 7. Wrap-up

↑ TOP

- **Bottleneck:** Redis throughput on hot keys → local pre-limiter + cluster sharding by key.
- **Failures:** Redis shard down → replica promotion (seconds); during that time `fail_mode` decides. Config service down → gateways keep the last cached rules.
- **Monitoring:** 429 rate per rule (spikes = attack or a bad client release), limiter latency p99, Redis CPU/ops, fail-open count (alert if > 0 for long).
- **Next:** adaptive limits based on backend health (load shedding), per-endpoint cost weights.

## Interview checklist

↑ TOP

- Compare **all five algorithms** in one table and name the fixed-window **edge burst** problem.
- Pick **token bucket** (bursts OK, 2 fields per key) and say why not sliding log (memory).

- Show the **race condition** and solve it with an **atomic Lua script**; mention `EVALSHA` and single-slot keys in Redis Cluster.
- Use **one clock** (Redis `TIME`) to avoid skew between servers.
- Put the limiter in the **gateway**; return **429 + Retry-After + X-RateLimit-*** headers.
- Discuss **fail open vs fail closed** per route (login/withdraw closed, market data open).
- Mention hot keys, local pre-limiting, and multi-region trade-offs.

---

↑ Back to top ☰ Contents

DESIGN PROBLEMS

# URL Shortener

ON THIS PAGE

1. 1. Requirements · p. 382
2. 2. Estimates · p. 382
3. 3. API · p. 383
4. 4. Data model · p. 383
5. 5. High-level design · p. 384
6. 6. Deep dives · p. 384
7. 7. Wrap-up · p. 386
8. Interview checklist · p. 386

Design a service like bit.ly: turn a long URL into a short link (`sho.rt/aZ3kQ9x`) and redirect visitors to the long URL. Follows the 7-step framework.

## 1. Requirements

↑ TOP

**Functional:**

- Create a short link for a long URL (optional custom alias, optional expiry).
- Redirect `GET /{code}` to the long URL.
- Basic click analytics per link (count, by day, by country/referrer).

**Non-functional:**

- **Read-heavy:** ~100 redirects per 1 create.
- **Low latency redirect:** p99 < 50 ms server-side.
- **High availability** for redirects (a dead link breaks every page that uses it). Creation can be slightly less available.
- Short codes must be **unique** and **not easy to guess in order** (no one should scan all links).
- Durability: a created link must never be lost.

**Out of scope:** user accounts/billing, link editing, spam/malware scanning (mention it), custom domains.

## 2. Estimates

↑ TOP

- **Writes:** 100M new links/month ≈ 3.3M/day → 3.3M / $10^5$ s ≈ **~40 creates/s**, peak ~100/s.
- **Reads:** 100:1 → **~4,000 redirects/s avg**, peak ~10–12k/s.
- **Storage:** one row ≈ 500 B (code, long URL ~ 200 B, timestamps, owner). 100M/month × 12 × 5 years = 6B links × 500 B ≈ **3 TB** over 5 years.
- **Code length:** base62 (a–z, A–Z, 0–9). $62^6$ ≈ 57B, $62^7$ ≈ 3.5T. **7 chars** is plenty for 6B links (and leaves room for sparse/random IDs).
- **Bandwidth:** redirect response is tiny (~500 B) → 12k/s × 0.5 KB ≈ 6 MB/s. Not a problem.
- **Cache:** 80/20 rule — 20% of links get most traffic. Hot set of ~10M links × 500 B ≈ **5 GB** → fits one Redis node (with a replica).

- **Analytics events:** = redirects → ~4k events/s avg, ~350M/day × ~100 B ≈ 35 GB/day raw.

Conclusion: writes are tiny (one Postgres primary is enough); reads need a cache; analytics needs an async pipeline.

## 3. API

↑ TOP

```
POST /api/v1/links          {long_url, custom_alias?, expires_at?}
                            Header: Idempotency-Key (optional)
                            → 201 {code, short_url, long_url, expires_at}
GET  /{code}                → 302 Location: <long_url>   (404 if unknown, 410 if expired)
GET  /api/v1/links/{code}/stats?from=&to=  → {total_clicks, by_day[], top_countries[]}
DELETE /api/v1/links/{code} → 204
```

Rate-limit `POST` per API key / IP (see rate limiter) to stop abuse.

## 4. Data model

↑ TOP

**links** (Postgres):

| Column | Type | Note |
|---|---|---|
| code | varchar(10) | **PK** |
| long_url | text | |
| owner_id | bigint | nullable; index `(owner_id, created_at)` for "my links" |
| created_at | timestamptz | |
| expires_at | timestamptz | nullable |

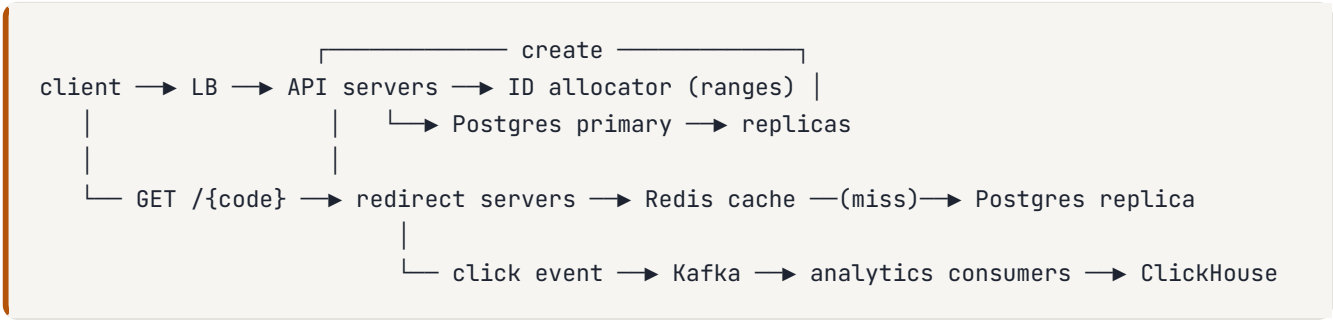
**click_events** (ClickHouse): `(code, ts, country, referrer, user_agent_family)`, ordered by `(code, ts)`, partitioned by month.

**DB choice:**

- **Postgres** for links: 40 writes/s and a 3 TB table is fine with one primary + read replicas; the access pattern is a single PK lookup. A key-value store (DynamoDB/Cassandra) would also work and scales writes more easily, but we don't need it at this size. Trade-off: if we grow 100×, we shard by `code` (hash), which is easy because every query is by `code`.
- **ClickHouse** for analytics: columnar, fast `GROUP BY` over billions of rows, cheap compression. Trade-off: not for point updates, but click events are append-only.

↑ TOP

## 5. High-level design




**Create flow:**

1. API validates the URL (scheme http/https, length ≤ 2 KB, optional blocklist check).
2. Gets the next numeric ID from its **local ID range** (see deep dive), encodes it to base62.
3. Inserts into Postgres. Custom alias → insert with that code; unique PK violation → `409 Conflict`.
4. Optionally writes the new link into Redis (write-through), returns the short URL.

**Redirect flow:**

1. Look up `code` in Redis (`GET link:{code}`). Hit (~95%+) → respond.
2. Miss → read from a Postgres replica, put into Redis with TTL (e.g. 24 h), respond.
3. Not found → cache a short "negative" entry (60 s) so bots scanning random codes don't hammer the DB.
4. Publish a click event to Kafka **asynchronously** (fire-and-forget from a local buffer); the redirect never waits for analytics.

## 6. Deep dives

↑ TOP

### 6.1 Generating unique short codes

**Problem:** 40–100 creates/s across many API servers; codes must be unique, short (7 chars), and not trivially enumerable.

**Options:**

| Option | How | Pros | Cons |
|---|---|---|---|
| Hash of URL (MD5/SHA → first 7 base62 chars) | `base62(md5(url))[:7]` | stateless, same URL → same code (dedup) | **collisions** (must check DB and retry with salt); dedup is wrong if two users want separate stats |

| Option | How | Pros | Cons |
|---|---|---|---|
| Random 7 chars | `secrets.choice` × 7 | simple, not guessable | collision check on every insert (rare at 6B/3.5T ≈ 0.2%, but grows) |
| Single DB auto-increment + base62 | `nextval()` → base62 | no collisions, short | one sequence = bottleneck/ SPOF; codes are **sequential = guessable** |
| Snowflake (64-bit: time + machine + seq) | per-server generator | no coordination, time-ordered | 64-bit number → **11 base62 chars**, too long for a short link |
| **Counter ranges** | each server leases a block (e.g. 1,000 IDs) from a coordinator (Postgres row / Redis `INCRBY` / ZooKeeper) | no collisions, no per-request coordination | lost IDs when a server dies (gaps are fine); sequential inside a block |

**Choice: counter ranges**. A server does `UPDATE id_ranges SET next = next + 1000 RETURNING next` (or Redis `INCRBY ids 1000`) once per 1,000 links — that is < 1 coordinator call per second. To make codes **not guessable**, pass the counter through a reversible bit-shuffle (e.g. a small Feistel cipher / format-preserving permutation over the ID space) before base62. Unique in, unique out — no collision check needed.

**Trade-off:** we need a small coordinator and a little crypto-ish code; in return we get guaranteed uniqueness and short codes. For custom aliases we still rely on the PK unique constraint.

### 6.2 301 vs 302 redirect

| | 301 Moved Permanently | 302 Found (or 307) |
|---|---|---|
| Browser caching | browser caches it; next click **never reaches us** | every click hits our server |
| Load on us | lower | higher |
| Analytics | **lost** after the first click | accurate |
| Can change/expire link | hard (browsers remember) | easy |

**Choice: 302** (or 307), because analytics and expiry are requirements. Add `Cache-Control: private, max-age=0`. Trade-off: more traffic to us (~4k/s), which the cache handles easily. If a customer doesn't want analytics, 301 is a valid option to save cost.

### 6.3 Caching the redirect path

- **Cache-aside** with Redis, key `link:{code}` → `long_url|expires_at`, TTL 24 h; LRU eviction (`allkeys-lru`).
- 5 GB hot set fits one node; one replica for failover. Redis does ~100k ops/s → 12k/s peak is ~12% load.

- **Hot link** (viral link, 50k/s on one key): add a tiny **in-process LRU** (e.g. 10k entries, TTL 60 s) in each redirect server; that removes most Redis traffic for the hottest keys.
- **Delete/expiry:** on delete, `DEL link:{code}`; the in-process cache TTL bounds staleness to 60 s. Trade-off: a deleted link may still redirect for up to a minute — acceptable here, and we say it out loud.
- Optional: put a **CDN** in front of redirects with a short TTL for the very hottest links (costs analytics accuracy; the CDN logs can feed analytics instead).

### 6.4 Analytics without slowing redirects

**Problem:** 4k–12k click events/s; writing each to a DB synchronously would add latency and load.

**Choice:** redirect servers push events to **Kafka** (topic `clicks`, key = `code`). Why Kafka over RabbitMQ: high throughput, retention, and multiple consumers (analytics, fraud, billing) can read the same stream (comparison). Consumers **batch-insert** into ClickHouse every ~1 s or 10k rows (ClickHouse prefers big batches). A pre-aggregated table (`clicks_per_code_per_day`, materialized view) serves the stats API fast.

**Trade-off:** stats are eventually consistent (seconds behind) and at-least-once — a consumer crash can double-count a batch. Acceptable for click stats; if exactness matters, add an event ID and deduplicate (ReplacingMergeTree).

## 7. Wrap-up

↑ TOP

- **Bottlenecks:** the redirect path is all reads → Redis + replicas scale it. Writes are tiny. At 100× growth: shard Postgres by hash(code) or move links to a KV store.
- **Failures:**
- Redis down → fall back to Postgres replicas (protect them with a circuit breaker + in-process cache).
- Postgres primary down → creates fail (promote replica); redirects keep working from cache + replicas.
- Kafka down → buffer events locally for a short time, then drop them; never block redirects.
- ID coordinator down → servers keep using their current range; only new leases fail.
- **Abuse:** rate limit creation, scan URLs against a malware/phishing list, allow takedown.
- **Monitoring:** redirect p99 latency, cache hit ratio (alert < 90%), 404 rate (scanning bots), Kafka consumer lag, create error rate.

## Interview checklist

↑ TOP

- Do the math: **7 base62 chars ≈ 3.5 trillion codes**; 100:1 read:write; hot set fits in RAM.
- Compare ID strategies (hash / random / counter / Snowflake) and explain why **counter ranges + shuffle** avoid both collisions and guessable codes.
- Explain **301 vs 302** and connect it to the analytics requirement.

- Cache-aside on the redirect path, plus **negative caching** for unknown codes and an in-process cache for hot links.
- Analytics is **async via Kafka** → the redirect never waits; stats are eventually consistent.
- Mention abuse: rate limiting, malicious URL checks, expiry.
- Say clearly that one Postgres primary is enough for writes — don't shard prematurely.

↑ Back to top ☰ Contents

DESIGN PROBLEMS

# Wallet and Ledger (Crypto Exchange)

ON THIS PAGE

1. 1. Requirements · p. 388
2. 2. Estimates · p. 388
3. 3. API · p. 389
4. 4. Data model · p. 389
5. 5. High-level design · p. 390
6. 6. Deep dives · p. 391
7. 7. Wrap-up · p. 392
8. Interview checklist · p. 393

Design the wallet/ledger of a crypto exchange (like Nobitex): users hold balances in many assets (IRT, USDT, BTC, ...), deposit and withdraw on-chain, and trade. Money must never be created, lost or double-spent. Follows the 7-step framework.

## 1. Requirements

↑ TOP

**Functional:**

- Per user, per asset balance: **available** and **held** (locked for open orders / pending withdrawals).
- **Deposits:** detect on-chain transactions to the user's deposit address, credit after N confirmations.
- **Withdrawals:** user requests → hold funds → risk checks → sign and broadcast → mark done after confirmations (or refund on failure).
- **Internal transfers:** trade settlement (from the matching engine), fees, transfers between users.
- Transaction history for the user; full audit trail for finance/regulators.

**Non-functional:**

- **Correctness over everything:** no negative balances, no double spend, no lost money. Strong consistency → CP, not AP.
- **Idempotent** operations: every retry is safe.
- **Auditable:** immutable history; any balance can be rebuilt from entries.
- Latency: balance update p99 < 50 ms (trading depends on it).
- Availability 99.95%; short write unavailability is better than wrong balances.

**Out of scope:** KYC, fiat bank integration details, the matching engine itself (separate design), cold storage key management (mention HSM/multisig).

## 2. Estimates

↑ TOP

- 10M users, ~1M active/day. ~20 assets → up to 200M balance rows, most are zero → store only non-zero (~20M rows).

- Ledger writes: trades are the main source. 2M trades/day → each trade touches 4 accounts (buyer base/quote, seller base/quote) + fee accounts → ~6 entries/trade → **12M entries/day** + deposits/withdrawals (~500k/day) → ~13M/day ≈ **150 entries/s avg, peak ~2k/s** (volatile market = 10× normal).
- Storage: entry ≈ 200 B → 13M × 200 B ≈ 2.6 GB/day ≈ **~1 TB/year**. Keep forever (audit) → partition by month.
- Reads: balance reads ~5k/s (app, order placement checks).

Conclusion: a single well-tuned **Postgres** primary handles this write rate; the hard part is correctness under concurrency, not scale.

## 3. API

↑ TOP

Internal (called by trading, deposit and withdrawal services):

```
POST /v1/transfers                Header: Idempotency-Key: trade:8812:settle
  {entries: [{account, asset, amount(+/-)}...], type: "trade_settlement", ref_id}
  → 201 {transfer_id}  |  409 duplicate (returns original)  |  422 insufficient_funds

POST /v1/holds    {user_id, asset, amount, reason, ref_id}  → {hold_id}
POST /v1/holds/{id}/release
POST /v1/holds/{id}/capture  {to_account}                   (hold → real transfer)
```

User-facing:

```
GET  /v1/balances                         → [{asset, available, held}]
GET  /v1/transactions?asset=&cursor=      → history
POST /v1/withdrawals  Header: Idempotency-Key   {asset, amount, address, network, otp}
GET  /v1/deposits/address?asset=&network=      → deposit address (+ memo/tag)
```

Amounts are **integers in the smallest unit** (satoshi, wei, rial) or `NUMERIC(38,18)` — **never float**.

## 4. Data model

↑ TOP

Postgres (ACID transactions, row locks, constraints — exactly what a ledger needs):

| Table | Key columns | Notes |
|---|---|---|
| accounts | `id` PK, owner_id (user or system), asset, type (`user_available`, `user_held`, `fee`, `hot_wallet`, `deposit_suspense`…) | UNIQUE `(owner_id, asset, type)` |
| balances | `account_id` PK, amount NUMERIC, allow_negative bool | `CHECK (allow_negative OR amount ⩾ 0)`: only system accounts may go negative |

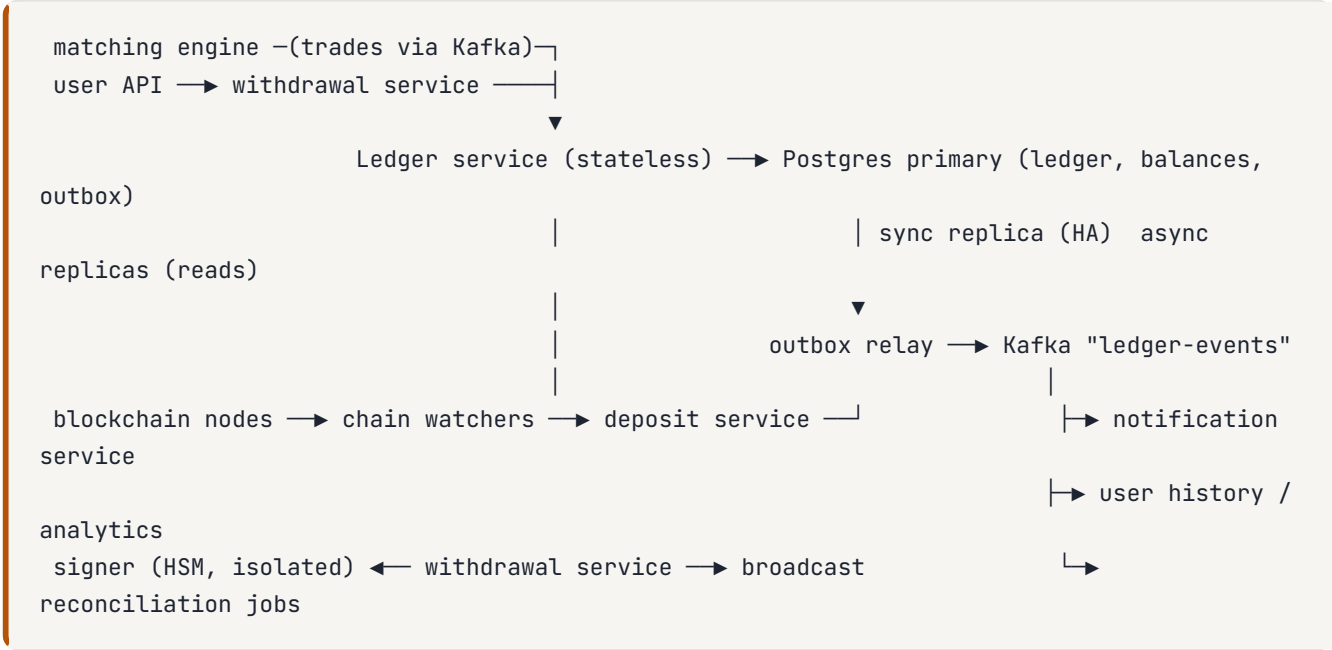
| Table | Key columns | Notes |
|---|---|---|
| transfers | `id` PK, `idempotency_key` **UNIQUE**, type, ref_id, created_at | one business operation |
| entries | `id` PK, transfer_id, account_id, amount (+ credit / − debit), created_at | append-only; index `(account_id, id)`; partitioned by month |
| deposits | `(chain, tx_hash, output_index)` **UNIQUE**, user_id, asset, amount, confirmations, status | dedup of chain events |
| withdrawals | `id` PK, user_id, asset, amount, address, status, tx_hash, hold_id | state machine |
| outbox | `id` PK, topic, payload, created_at, published_at | see deep dive |

**Double-entry rule:** every transfer's entries **sum to zero per asset**. Money only moves between accounts; it's never created. Simple sign convention: system accounts like `hot_wallet` mirror the outside world and go negative when money enters. A 0.5 BTC deposit is `hot_wallet -0.5` and `user_available +0.5` → sum = 0. So `-hot_wallet` always equals what the exchange should hold on-chain, which is exactly what reconciliation checks.

Holds are just transfers between `user_available` and `user_held` accounts — same rules, same audit trail.

## 5. High-level design

↑ TOP




**Trade settlement:** ledger consumes a trade event, builds the 4–6 entries, runs them in **one DB transaction** keyed by `idempotency_key = trade:{trade_id}`.

**Withdrawal:** create hold (available → held) → risk/AML + limits + OTP check → signer signs, broadcast → watcher sees N confirmations → capture hold (`user_held -X`, `hot_wallet +X`). Failure/rejection → release hold.

**Deposit:** chain watcher sees tx to a user's address → record `deposits` row (status `pending`) → after N confirmations → credit user in one transaction.

## 6. Deep dives

↑ TOP

### 6.1 Updating balances safely under concurrency

**Problem:** a user places two orders and a withdrawal at the same time; each checks "balance ≥ amount" then subtracts. Without care, both pass → **negative balance / double spend**.

**Options:**

1. **Pessimistic lock:** `SELECT ... FROM balances WHERE account_id IN (...) ORDER BY account_id FOR UPDATE`, check, update, insert entries, commit. Lock rows **in a fixed order (by id)** to avoid deadlocks.
2. **Atomic conditional update:** `UPDATE balances SET amount = amount - :x WHERE account_id = :a AND amount ≥ :x` → 0 rows updated = insufficient funds. No read-then-write gap; the row lock is held only for the statement + transaction.
3. **Optimistic lock:** `UPDATE ... SET amount = :new, version = version + 1 WHERE account_id = :a AND version = :v` → retry on conflict. Bad under contention (hot accounts like `fee`).
4. **Single-writer per account** (shard accounts to in-memory processors, like the matching engine): very fast, but much more complex; used by the biggest exchanges.

**Choice: (2) conditional updates inside one transaction**, plus `CHECK (amount ≥ 0)` as a final safety net. Simple, fast, correct, and the DB does the hard work.

**Hot accounts:** the `fee` account is credited by every trade → lock contention. Fix: split into N sub-accounts (`fee_0`..`fee_15`, pick by `trade_id % 16`) and sum them for reporting. Trade-off: reads must aggregate.

**Isolation:** `READ COMMITTED` is enough with conditional updates/row locks; `SERIALIZABLE` also works but adds retries.

### 6.2 Idempotency

Every ledger call has an **idempotency key** derived from the business event (`trade:{id}`, `deposit:{chain}:{tx_hash}:{vout}`, `withdrawal:{id}:capture`). It's inserted into `transfers` **in the same transaction** as the entries. A retry hits the UNIQUE constraint → we return the original result. Because it's one transaction, there's no state where entries exist without the key or vice versa.

Kafka consumers (trade settlement) are at-least-once, so this is what makes re-processing safe.

### 6.3 Outbox pattern for events

**Problem:** after committing a transfer we must publish `balance_changed` / `withdrawal_completed` to Kafka. If we commit then crash before publishing → event lost. If we publish then the commit fails → ghost event.

**Choice: transactional outbox.** Insert the event into the `outbox` table **in the same transaction** as the entries. A relay process (polling `WHERE published_at IS NULL ORDER BY id`, or CDC with Debezium reading the WAL) publishes to Kafka and marks it published. Delivery is at-least-once → consumers dedup by event ID. Trade-off: a small delay (ms to ~1 s) and one more component, in exchange for **no lost or ghost events** (Kafka notes).

### 6.4 Deposits, confirmations and reconciliation

**Confirmations:** credit only after N confirmations to be safe from **chain reorgs** (e.g. BTC ~2–3 for small amounts, up to 6 for large; ETH waits for finality ~2 epochs ≈ 13 min, or a block depth like 12–64; Tron ~19–20). Show as "pending" before that; optionally let users **trade** but not withdraw a pending deposit (exchange takes the reorg risk).

**Reorg handling:** watcher tracks block hashes; if a block with a credited deposit is orphaned before N confirmations, the pending deposit is cancelled. Unique `(chain, tx_hash, output_index)` prevents crediting the same output twice.

**Reconciliation (daily + continuous):**

- **Internal:** for each asset, sum of all entries = 0; `balances.amount` = sum of that account's entries (rebuild check).
- **External:** sum of user liabilities ≤ on-chain holdings (hot + cold wallets, from node queries) — this is the base of **proof of reserves**.
- Withdrawals: every `completed` withdrawal has a matching on-chain tx; every outgoing on-chain tx has a withdrawal record (detects stolen hot-wallet funds).
- Mismatch → alert and **pause withdrawals** for that asset automatically.

**Auditability:** entries are **append-only** (no UPDATE/DELETE; corrections are new reversing transfers), DB role for the app has no DELETE on `entries`, and every transfer stores who/what caused it (`type`, `ref_id`, actor).

## 7. Wrap-up

↑ TOP

- **Bottlenecks:** hot accounts (split them), single Postgres primary (shard by `user_id` later — but cross-shard transfers then need sagas; stay on one primary as long as possible).
- **Failures:**
- Primary down → failover to a **synchronous replica** (no data loss, RPO = 0).
- Ledger service crash mid-request → transaction rolls back; the client retries with the same key.
- Kafka down → outbox accumulates, nothing lost.
- Blockchain node lagging → deposits delayed (alert on node height vs public explorers).
- **Security:** signer isolated with HSM, most funds in cold storage/multisig, withdrawal limits + manual review for large amounts, address allowlists.
- **Monitoring:** reconciliation diff (must be 0), negative-balance check failures, withdrawal queue age, pending deposits by confirmations, outbox lag, hot-wallet balance vs threshold.

↑ TOP

## Interview checklist

- **Double-entry**: every transfer sums to zero; balances are a cache of entries and can be rebuilt.
- **Never use float**; integer smallest units or `NUMERIC`.
- Prevent double spend with **atomic conditional UPDATE** (or `SELECT FOR UPDATE` in fixed order) + `CHECK (amount ⩾ 0)`.
- **Idempotency key UNIQUE in the same transaction** as the entries.
- **Outbox pattern** to publish events reliably.
- Holds as transfers available → held; withdrawals as a **state machine** with refund on failure.
- Deposits after **N confirmations**, reorg handling, unique `(tx_hash, output_index)`.
- **Reconciliation** internal and against the blockchain; auto-pause withdrawals on mismatch.

↑ Back to top ☰ Contents

# Index of Terms


## 0

0-RTT — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133

## 1

1-RTT — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133

103 Early Hints — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133

## 2

2PC — V2·7 Hotel Reservation System p. 307, V2·12 Digital Wallet p. 342

## 3

301 vs 302 redirect — V1·8 URL Shortener p. 222

3Sum — Two Sum / Complement Lookup p. 84

## 4

40001 — Transactions & Isolation Levels p. 116

401 vs 403 — Django REST Framework p. 178

## 6

64-bit ID — V1·7 Unique ID Generator p. 218

## A

A record — Web & Networking Fundamentals p. 137

A* search — V2·3 Google Maps p. 279

ABC — OOP Internals & Memory Model p. 42, OOP Internals & Memory Model p. 42

abstractmethod — OOP Internals & Memory Model p. 42, OOP Internals & Memory Model p. 42

ACID — Transactions & Isolation Levels p. 116

ack — Kafka vs RabbitMQ p. 96, V2·4 Distributed Message Queue p. 285

ACK — Web & Networking Fundamentals p. 137

acks early — Celery with Redis as Broker p. 107

action decorator — Django REST Framework p. 178

ad click aggregation — V2·6 Ad Click Event Aggregation p. 300

adaptive ETA — V2·3 Google Maps p. 279

AddConstraintNotValid — Transactions & Migrations p. 172

AddIndexConcurrently — Transactions & Migrations p. 172

advisory lock — Locking & Concurrency Control p. 121

aggregate — Django ORM in Depth p. 158

aggregation — V1·13 Search Autocomplete p. 248

Airbnb — V2·7 Hotel Reservation System p. 307

alert manager — V2·5 Metrics Monitoring and Alerting p. 293
alerting — V2·5 Metrics Monitoring and Alerting p. 293
Alex Xu — The 4-Step Framework (Alex Xu, with a News Feed Example) p. 100
aliasing — Python Gotchas (Predict the Output) p. 36
ALPN — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
Alt-Svc — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
amortized analysis — Big-O Cheat Sheet p. 80
analytics — V1·10 Notification System p. 232
annotate — Django ORM in Depth p. 158
API design — How to Run a System Design Interview p. 92
API gateway — V1·4 Rate Limiter p. 201
APNs — V1·10 Notification System p. 232
AppConfig.ready — Request Lifecycle, Middleware & Signals p. 166
application loop — V2·13 Stock Exchange p. 349
apps.get_model — Transactions & Migrations p. 172
as_view — Request Lifecycle, Middleware & Signals p. 166
ASGI — Request Lifecycle, Middleware & Signals p. 166
asking for a hint — How to Solve a Coding Question p. 86
assertNumQueries — Django ORM in Depth p. 158
async — Async / Await p. 16
async ORM — Django in Production: Performance, Security, Async p. 185
async views — Django in Production: Performance, Security, Async p. 185
asynchronous replication — SQL vs NoSQL & Scaling Databases p. 126
asyncio — Async / Await p. 16
asyncio.create_task — Async / Await p. 16
asyncio.gather — Async / Await p. 16
asyncio.Lock — Async / Await p. 16
asyncio.Queue — Python Data Structures Map p. 53
asyncio.sleep(0) — Async / Await p. 16
asyncio.to_thread — Async / Await p. 16, Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
at-most-once — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105
atomic = False — Transactions & Migrations p. 172
atomic update — Locking & Concurrency Control p. 121
ATOMIC_REQUESTS — Transactions & Migrations p. 172
atomicity — Transactions & Isolation Levels p. 116
attachment store — V2·8 Distributed Email Service p. 313
attribute lookup — Class vs Instance Attributes & Methods p. 19
augmented assignment — Python Gotchas (Predict the Output) p. 36
authentication — Django REST Framework p. 178
authoritative name server — Web & Networking Fundamentals p. 137
auto_increment — V1·7 Unique ID Generator p. 218
autocommit — Transactions & Migrations p. 172
await — Async / Await p. 16
AWS Lambda — V2·10 Real-time Gaming Leaderboard p. 329

## B

B-tree — Indexes p. 111
back queues — V1·9 Web Crawler p. 227
back-of-the-envelope — How to Run a System Design Interview p. 92
backing array — Go Essentials & Gotchas p. 143
backpressure — Kafka vs RabbitMQ p. 96
backtracking — Pattern Recognition Map p. 68
balance transfer — V2·12 Digital Wallet p. 342
banker's rounding — Python Gotchas (Predict the Output) p. 36
base 62 — V1·8 URL Shortener p. 222
batched — Generators & Iterators p. 26
BFS — Pattern Recognition Map p. 68
BFS vs DFS — V1·9 Web Crawler p. 227
big-o — Big-O Cheat Sheet p. 80
Bigtable — V2·8 Distributed Email Service p. 313

binary framing — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
binary search — Pattern Recognition Map p. 68
binary search on the answer — Pattern Recognition Map p. 68
bisect — Python Data Structures Map p. 53
Bitmap Heap Scan — Indexes p. 111
BLMOVE — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105
blob storage — V1·14 YouTube p. 253, V2·9 S3-like Object Storage p. 321
block server — V1·15 Google Drive p. 259
blocking call — Async / Await p. 16
bloom filter — V1·6 Key-Value Store p. 211, V1·8 URL Shortener p. 222, V1·9 Web Crawler p. 227
booking — V2·7 Hotel Reservation System p. 307
__bool__ — OOP Internals & Memory Model p. 42
bottleneck — How to Run a System Design Interview p. 92
BRIN — Indexes p. 111
broker — V2·4 Distributed Message Queue p. 285
browser cache — V1·13 Search Autocomplete p. 248
BRPOP — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105, Celery with Redis as Broker p. 107
brute force — How to Solve a Coding Question p. 86
bucket — V2·9 S3-like Object Storage p. 321
buffered channel — Go Essentials & Gotchas p. 143
bulk_create — Django ORM in Depth p. 158
bulk_update — Django ORM in Depth p. 158
buy-in — The 4-Step Framework (Alex Xu, with a News Feed Example) p. 100

## C

C3 linearization — OOP Internals & Memory Model p. 42
cache — V1·8 URL Shortener p. 222
cache invalidation — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105, Django in Production: Performance, Security, Async p. 185
cache misses — V1·5 Consistent Hashing p. 207
cache stampede — Django in Production: Performance, Security, Async p. 185
cache.get_or_set — Django in Production: Performance, Security, Async p. 185
cache_page — Django in Production: Performance, Security, Async p. 185
candlestick — V2·13 Stock Exchange p. 349
CAP — How to Run a System Design Interview p. 92
CAP theorem — V1·6 Key-Value Store p. 211
capacity estimation — How to Run a System Design Interview p. 92
captureOnCommitCallbacks — Transactions & Migrations p. 172
car wash — Python Data Structures Map p. 53
cardinality — Indexes p. 111
Cassandra — SQL vs NoSQL & Scaling Databases p. 126, V1·5 Consistent Hashing p. 207, V1·6 Key-Value Store p. 211, V2·2 Nearby Friends p. 273, V2·3 Google Maps p. 279
CDC — V2·7 Hotel Reservation System p. 307
CDN — V1·11 News Feed System p. 237, V1·14 YouTube p. 253, V2·3 Google Maps p. 279
celebrity problem — V1·11 News Feed System p. 237
Celery — Celery with Redis as Broker p. 107
Celery best practices — Django in Production: Performance, Security, Async p. 185
certificate — Web & Networking Fundamentals p. 137
chain — Generators & Iterators p. 26
channel — Go Essentials & Gotchas p. 143
chat system — V1·12 Chat System p. 242
check --deploy — Django in Production: Performance, Security, Async p. 185
checksum — V2·9 S3-like Object Storage p. 321
chunk_size — Django ORM in Depth p. 158
Citus — SQL vs NoSQL & Scaling Databases p. 126
clarifying questions — How to Solve a Coding Question p. 86
class attribute — Class vs Instance Attributes & Methods p. 19

class-based views — Request Lifecycle, Middleware & Signals p. 166
classmethod — Class vs Instance Attributes & Methods p. 19
clickjacking — Django in Production: Performance, Security, Async p. 185
clock synchronization — V1·7 Unique ID Generator p. 218
close — Go Essentials & Gotchas p. 143
closure — Closures, Decorators & Context Managers p. 31
cloud storage — V1·15 Google Drive p. 259
CNAME — Web & Networking Fundamentals p. 137
coding interview — How to Solve a Coding Question p. 86
cold storage — V1·15 Google Drive p. 259
collections.deque — Python Data Structures Map p. 53
collectstatic — Django in Production: Performance, Security, Async p. 185
compaction — V2·9 S3-like Object Storage p. 321
compare-and-swap — Locking & Concurrency Control p. 121
compensating transaction — V2·12 Digital Wallet p. 342
competing consumers — Kafka vs RabbitMQ p. 96
complement lookup — Two Sum / Complement Lookup p. 84
completion queue — V1·14 YouTube p. 253
composite index — Indexes p. 111
compression — V1·15 Google Drive p. 259
concurrency vs parallelism — Async / Await p. 16, Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
concurrent map writes — Go Essentials & Gotchas p. 143
concurrent.futures — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
CONN_HEALTH_CHECKS — Django in Production: Performance, Security, Async p. 185
CONN_MAX_AGE — Django in Production: Performance, Security, Async p. 185
connection ID — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
connection migration — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
connection pooling — SQL vs NoSQL & Scaling Databases p. 126, Django in Production: Performance, Security, Async p. 185
connection.cursor — Django ORM in Depth p. 158
consistency — Transactions & Isolation Levels p. 116
consistent hashing — SQL vs NoSQL & Scaling Databases p. 126, V1·5 Consistent Hashing p. 207, V2·2 Nearby Friends p. 273
consumer group — Kafka vs RabbitMQ p. 96, V2·4 Distributed Message Queue p. 285
content seen — V1·9 Web Crawler p. 227
context — Go Essentials & Gotchas p. 143
context manager — Closures, Decorators & Context Managers p. 31
context.WithCancel — Go Essentials & Gotchas p. 143
context.WithTimeout — Go Essentials & Gotchas p. 143
contextlib.contextmanager — Closures, Decorators & Context Managers p. 31
conversation threads — V2·8 Distributed Email Service p. 313
cookies — Web & Networking Fundamentals p. 137
copy.deepcopy — Python Gotchas (Predict the Output) p. 36
coroutine — Async / Await p. 16
correlation_id — Kafka vs RabbitMQ p. 96
CORS — Web & Networking Fundamentals p. 137
countdown — Celery with Redis as Broker p. 107
Counter — Python Data Structures Map p. 53
covering index — Indexes p. 111
CPU bound — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
CPU pinning — V2·13 Stock Exchange p. 349
CQRS — V2·12 Digital Wallet p. 342
CRC32 — V1·8 URL Shortener p. 222
CREATE INDEX CONCURRENTLY — Indexes p. 111
CSRF — Django in Production: Performance, Security, Async p. 185
CursorPagination — Django REST Framework p. 178

custom epoch — V1·7 Unique ID Generator p. 218

## D

DAG — V1·14 YouTube p. 253
data gathering — V1·13 Search Autocomplete p. 248
data migration — Transactions & Migrations p. 172
data model — How to Run a System Design Interview p. 92
data sampling — V1·13 Search Autocomplete p. 248
data store — V2·9 S3-like Object Storage p. 321
database constraint — V2·7 Hotel Reservation System p. 307
dataclass — OOP Internals & Memory Model p. 42, Python Data Structures Map p. 53
db_default — Transactions & Migrations p. 172
dead letter queue — Kafka vs RabbitMQ p. 96, V2·11 Payment System p. 335
deadlock — Locking & Concurrency Control p. 121
deadlock_timeout — Locking & Concurrency Control p. 121
decorator — Closures, Decorators & Context Managers p. 31
decorator factory — Closures, Decorators & Context Managers p. 31
decorator with arguments — Closures, Decorators & Context Managers p. 31
dedupe — V1·10 Notification System p. 232
deduplication — V1·15 Google Drive p. 259
deep copy — Python Gotchas (Predict the Output) p. 36
deep dive — How to Run a System Design Interview p. 92, The 4-Step Framework (Alex Xu, with a News Feed Example) p. 100
default_factory — OOP Internals & Memory Model p. 42
defaultdict — Python Data Structures Map p. 53
DefaultRouter — Django REST Framework p. 178
defer — Go Essentials & Gotchas p. 143, Django ORM in Depth p. 158
delay_on_commit — Transactions & Migrations p. 172
deliverability — V2·8 Distributed Email Service p. 313
delivery semantics — V2·4 Distributed Message Queue p. 285
delta encoding — V2·5 Metrics Monitoring and Alerting p. 293
delta sync — V1·15 Google Drive p. 259
denormalization — SQL vs NoSQL & Scaling Databases p. 126, V2·8 Distributed Email Service p. 313
deque — Python Data Structures Map p. 53
deque cost — Big-O Cheat Sheet p. 80
design scope — The 4-Step Framework (Alex Xu, with a News Feed Example) p. 100
DFS — Pattern Recognition Map p. 68
diamond inheritance — OOP Internals & Memory Model p. 42
dict — Python Data Structures Map p. 53
dict cost — Big-O Cheat Sheet p. 80
dict ordering — Python Gotchas (Predict the Output) p. 36
__dict__ — Class vs Instance Attributes & Methods p. 19
digital wallet — V2·12 Digital Wallet p. 342
Dijkstra — Python Data Structures Map p. 53, V2·3 Google Maps p. 279
direct exchange — Kafka vs RabbitMQ p. 96
dirty read — Transactions & Isolation Levels p. 116
DISABLE_SERVER_SIDE_CURSORS — Django in Production: Performance, Security, Async p. 185
dispatch — Request Lifecycle, Middleware & Signals p. 166
dispatch_uid — Request Lifecycle, Middleware & Signals p. 166
distributed hash table — V1·6 Key-Value Store p. 211
distributed ID — V1·7 Unique ID Generator p. 218
distributed rate limiting — V1·4 Rate Limiter p. 201
distributed transaction — V2·12 Digital Wallet p. 342
Django async questions — Django Interview Questions p. 152
Django deployment questions — Django Interview Questions p. 152

Django interview questions — Django Interview Questions p. 152
Django mixin order — OOP Internals & Memory Model p. 42
Django quiz — Django Interview Questions p. 152
Django REST Framework — Django REST Framework p. 178
Django security questions — Django Interview Questions p. 152
django-debug-toolbar — Django ORM in Depth p. 158
django-filter — Django REST Framework p. 178
django-redis — Django in Production: Performance, Security, Async p. 185
django-storages — Django in Production: Performance, Security, Async p. 185
django_migrations — Transactions & Migrations p. 172
DLQ — Kafka vs RabbitMQ p. 96
DNS — Web & Networking Fundamentals p. 137
DNS cache — V1·9 Web Crawler p. 227
document store — SQL vs NoSQL & Scaling Databases p. 126
double booking — V2·7 Hotel Reservation System p. 307
double spend — Locking & Concurrency Control p. 121
double-entry ledger — V2·11 Payment System p. 335
downsampling — V2·5 Metrics Monitoring and Alerting p. 293
DP — Pattern Recognition Map p. 68
DRF — Django REST Framework p. 178
DRF questions — Django Interview Questions p. 152
DRM — V1·14 YouTube p. 253
Dropbox — V1·15 Google Drive p. 259
dunder methods — OOP Internals & Memory Model p. 42
duplicate content — V1·9 Web Crawler p. 227
durability — Transactions & Isolation Levels p. 116, V2·9 S3-like Object Storage p. 321
dynamic programming — Pattern Recognition Map p. 68
Dynamo — V1·5 Consistent Hashing p. 207, V1·6 Key-Value Store p. 211
DynamoDB — V2·10 Real-time Gaming Leaderboard p. 329

## E

edge cases — How to Solve a Coding Question p. 86
Elasticsearch — V2·8 Distributed Email Service p. 313
email — V1·10 Notification System p. 232
email service — V2·8 Distributed Email Service p. 313
encoding — V1·14 YouTube p. 253
__enter__ — Closures, Decorators & Context Managers p. 31
__eq__ — OOP Internals & Memory Model p. 42
erasure coding — V2·9 S3-like Object Storage p. 321
Erlang — V2·2 Nearby Friends p. 273
errgroup — Go Essentials & Gotchas p. 143
ETA — Celery with Redis as Broker p. 107, V2·3 Google Maps p. 279
etcd — V2·2 Nearby Friends p. 273
event loop — Async / Await p. 16
event sourcing — V2·12 Digital Wallet p. 342, V2·13 Stock Exchange p. 349
event streaming — Kafka vs RabbitMQ p. 96, V2·4 Distributed Message Queue p. 285
eventual consistency — V1·6 Key-Value Store p. 211
exactly once — V2·4 Distributed Message Queue p. 285
exactly-once — Kafka vs RabbitMQ p. 96, V2·6 Ad Click Event Aggregation p. 300, V2·11 Payment System p. 335
exchange — Kafka vs RabbitMQ p. 96
Exists — Django ORM in Depth p. 158
__exit__ — Closures, Decorators & Context Managers p. 31
expand and contract — Transactions & Migrations p. 172
EXPLAIN — Indexes p. 111
EXPLAIN ANALYZE — Indexes p. 111
expression index — Indexes p. 111

extra — Django in Production: Performance, Security, Async p. 185

## F

F expression — Django ORM in Depth p. 158
F expression race — Transactions & Migrations p. 172
Facebook feed — V1·11 News Feed System p. 237
factory_boy — Django in Production: Performance, Security, Async p. 185
failover — SQL vs NoSQL & Scaling Databases p. 126
fan-out — V2·2 Nearby Friends p. 273
fanout — The 4-Step Framework (Alex Xu, with a News Feed Example) p. 100
fanout exchange — Kafka vs RabbitMQ p. 96
fanout on read — V1·11 News Feed System p. 237
fanout on write — V1·11 News Feed System p. 237
FCM — V1·10 Notification System p. 232
file sync — V1·15 Google Drive p. 259
file versions — V1·15 Google Drive p. 259
FIN — Web & Networking Fundamentals p. 137
fire-and-forget — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105
FIX — V2·13 Stock Exchange p. 349
fixed window — V1·4 Rate Limiter p. 201
Flickr — V1·7 Unique ID Generator p. 218
Flink — V2·6 Ad Click Event Aggregation p. 300
float precision — Python Gotchas (Predict the Output) p. 36
floor division — Python Gotchas (Predict the Output) p. 36
FOR NO KEY UPDATE — Locking & Concurrency Control p. 121
FOR SHARE — Locking & Concurrency Control p. 121
free variable — Closures, Decorators & Context Managers p. 31
free-threaded — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
freshness — V1·9 Web Crawler p. 227
front queues — V1·9 Web Crawler p. 227
frozen — OOP Internals & Memory Model p. 42
frozenset — Python Data Structures Map p. 53
function-based views — Request Lifecycle, Middleware & Signals p. 166
functional index — Indexes p. 111
functional requirements — How to Run a System Design Interview p. 92
functools.partial — Closures, Decorators & Context Managers p. 31
functools.wraps — Closures, Decorators & Context Managers p. 31
Future — Async / Await p. 16

## G

garbage collection — V2·9 S3-like Object Storage p. 321
garbage collector — OOP Internals & Memory Model p. 42
gc — OOP Internals & Memory Model p. 42
generator — Generators & Iterators p. 26
generator expression — Generators & Iterators p. 26
GenericViewSet — Django REST Framework p. 178
geocoding — V2·3 Google Maps p. 279
geofencing — V2·1 Proximity Service p. 266
geohash — V2·1 Proximity Service p. 266, V2·3 Google Maps p. 279
geohash channel — V2·2 Nearby Friends p. 273
geospatial index — V2·1 Proximity Service p. 266
get_or_create — Django ORM in Depth p. 158
get_queryset — Django REST Framework p. 178
get_response — Request Lifecycle, Middleware & Signals p. 166
__getitem__ — OOP Internals & Memory Model p. 42
GIL — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21

GIN — Indexes p. 111
GiST — Indexes p. 111
global interpreter lock — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
Gmail — V2·8 Distributed Email Service p. 313
GMP scheduler — Go Essentials & Gotchas p. 143
Go — Go Essentials & Gotchas p. 143
Go 1.22 — Go Essentials & Gotchas p. 143
Google Drive — V1·15 Google Drive p. 259
Google Maps — V2·3 Google Maps p. 279
Google S2 — V2·1 Proximity Service p. 266
GOP — V1·14 YouTube p. 253
goroutine — Go Essentials & Gotchas p. 143
goroutine leak — Go Essentials & Gotchas p. 143
gossip protocol — V1·6 Key-Value Store p. 211
Grafana — V2·5 Metrics Monitoring and Alerting p. 293
graph database — SQL vs NoSQL & Scaling Databases p. 126, V1·11 News Feed System p. 237
greedy — Pattern Recognition Map p. 68
group chat — V1·12 Chat System p. 242
groupby — Generators & Iterators p. 26
gRPC — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133, Web & Networking Fundamentals p. 137
gthread — Django in Production: Performance, Security, Async p. 185
gunicorn — Request Lifecycle, Middleware & Signals p. 166
gunicorn workers — Django in Production: Performance, Security, Async p. 185

## H

h2c — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
has_object_permission — Django REST Framework p. 178
hash collision — V1·8 URL Shortener p. 222
hash collision True 1 — Python Gotchas (Predict the Output) p. 36
hash index — Indexes p. 111
hash map — Pattern Recognition Map p. 68
hash ring — V1·5 Consistent Hashing p. 207, V2·2 Nearby Friends p. 273
hash set — Pattern Recognition Map p. 68, Two Sum / Complement Lookup p. 84
hash slot — V2·10 Real-time Gaming Leaderboard p. 329
__hash__ — OOP Internals & Memory Model p. 42
hashValue length — V1·8 URL Shortener p. 222
head-of-line blocking — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
heap — Python Data Structures Map p. 53, Pattern Recognition Map p. 68
heapq — Python Data Structures Map p. 53
heapq cost — Big-O Cheat Sheet p. 80
heartbeat — V1·12 Chat System p. 242
high-level design — How to Run a System Design Interview p. 92, The 4-Step Framework (Alex Xu, with a News Feed Example) p. 100
Hilbert curve — V2·1 Proximity Service p. 266
hinted handoff — V1·6 Key-Value Store p. 211
HLS — V1·14 YouTube p. 253
HOL blocking — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
horizontal scaling — V1·5 Consistent Hashing p. 207
hosted payment page — V2·11 Payment System p. 335
hot partition — SQL vs NoSQL & Scaling Databases p. 126
HOT update — Indexes p. 111
hot-warm — V2·13 Stock Exchange p. 349
hotel reservation — V2·7 Hotel Reservation System p. 307
hotkey — V1·11 News Feed System p. 237
hotspot — V1·5 Consistent Hashing p. 207, V2·6 Ad Click Event Aggregation p. 300
how many queries — Django Interview Questions p. 152
HPACK — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
HSTS — Django in Production: Performance, Security, Async p. 185
HTTP 429 — V1·4 Rate Limiter p. 201

HTTP status codes — Web & Networking Fundamentals p. 137
HTTP/1.0 — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
HTTP/1.1 — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
HTTP/2 — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
HTTP/3 — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
hybrid fanout — V1·11 News Feed System p. 237
hydration — V1·11 News Feed System p. 237

## I

I/O bound — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
idempotency key — Locking & Concurrency Control p. 121, Django REST Framework p. 178, V2·7 Hotel Reservation System p. 307, V2·11 Payment System p. 335
idempotent — Web & Networking Fundamentals p. 137
idempotent tasks — Celery with Redis as Broker p. 107
IMAP — V2·8 Distributed Email Service p. 313
in-sync replicas — V2·4 Distributed Message Queue p. 285
INCLUDE — Indexes p. 111
INCR EXPIRE — V1·4 Rate Limiter p. 201
index — Indexes p. 111
index only scan — Indexes p. 111
Index Scan — Indexes p. 111
InfluxDB — V2·5 Metrics Monitoring and Alerting p. 293
__init__ — OOP Internals & Memory Model p. 42
input size limits — Big-O Cheat Sheet p. 80
Instagram feed — V1·11 News Feed System p. 237
instance attribute — Class vs Instance Attributes & Methods p. 19
interface — OOP Internals & Memory Model p. 42
interning — Python Gotchas (Predict the Output) p. 36
intervals — Pattern Recognition Map p. 68
is vs == — Python Gotchas (Predict the Output) p. 36
islice — Generators & Iterators p. 26
isolation — Transactions & Isolation Levels p. 116
isolation level — Transactions & Isolation Levels p. 116
ISR — V2·4 Distributed Message Queue p. 285
__iter__ — Generators & Iterators p. 26, OOP Internals & Memory Model p. 42
iterable — Generators & Iterators p. 26
iterator — Generators & Iterators p. 26, Django ORM in Depth p. 158
iterator protocol — Generators & Iterators p. 26
itertools — Generators & Iterators p. 26

## J

job queue — Locking & Concurrency Control p. 121
joinedload — SQL vs NoSQL & Scaling Databases p. 126
JWT — Web & Networking Fundamentals p. 137

## K

k-nearest — V2·1 Proximity Service p. 266
Kafka — Kafka vs RabbitMQ p. 96, V2·3 Google Maps p. 279, V2·4 Distributed Message Queue p. 285, V2·5 Metrics Monitoring and Alerting p. 293, V2·6 Ad Click Event Aggregation p. 300
Kahn's algorithm — Pattern Recognition Map p. 68
kappa architecture — V2·6 Ad Click Event Aggregation p. 300
keep-alive — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
key-value — SQL vs NoSQL & Scaling Databases p. 126
key-value store — V1·6 Key-Value Store p. 211, V1·12 Chat System p. 242

Kombu — Celery with Redis as Broker p. 107

KRaft — Kafka vs RabbitMQ p. 96

## L

lambda — Python Gotchas (Predict the Output) p. 36

lambda architecture — V2·6 Ad Click Event Aggregation p. 300

lambda in loop — Closures, Decorators & Context Managers p. 31

late binding — Closures, Decorators & Context Managers p. 31, Python Gotchas (Predict the Output) p. 36

latency numbers — How to Run a System Design Interview p. 92

lazy evaluation — Generators & Iterators p. 26, Django ORM in Depth p. 158

LBS — V2·1 Proximity Service p. 266

leaderboard — V2·10 Real-time Gaming Leaderboard p. 329

leaking bucket — V1·4 Rate Limiter p. 201

ledger — Locking & Concurrency Control p. 121

LeetCode 1882 — Python Data Structures Map p. 53

leftmost prefix — Indexes p. 111

__len__ — OOP Internals & Memory Model p. 42

limit order — V2·13 Stock Exchange p. 349

LimitOffsetPagination — Django REST Framework p. 178

linked list — Python Data Structures Map p. 53

list — Python Data Structures Map p. 53

list comprehension — Generators & Iterators p. 26

list multiplication — Python Gotchas (Predict the Output) p. 36

list operations cost — Big-O Cheat Sheet p. 80

load balancer — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133

location cache — V2·2 Nearby Friends p. 273

location sharing — V2·2 Nearby Friends p. 273

location-based service — V2·1 Proximity Service p. 266

lock ordering — Locking & Concurrency Control p. 121

lock_timeout — Locking & Concurrency Control p. 121, Transactions & Migrations p. 172

locking — Locking & Concurrency Control p. 121

long polling — Web & Networking Fundamentals p. 137, V1·12 Chat System p. 242, V1·15 Google Drive p. 259

long tail — V1·14 YouTube p. 253

loop variable capture — Go Essentials & Gotchas p. 143

lost update — Transactions & Isolation Levels p. 116

low-level cache API — Django in Production: Performance, Security, Async p. 185

LPUSH — Celery with Redis as Broker p. 107

LRU cache — Python Data Structures Map p. 53

lru_cache — Python Data Structures Map p. 53

LSM tree — V2·8 Distributed Email Service p. 313

Lua script — V1·4 Rate Limiter p. 201

## M

m2m_changed — Request Lifecycle, Middleware & Signals p. 166

magic methods — OOP Internals & Memory Model p. 42

makemigrations — Transactions & Migrations p. 172

ManifestStaticFilesStorage — Django in Production: Performance, Security, Async p. 185

map iteration order — Go Essentials & Gotchas p. 143

map rendering — V2·3 Google Maps p. 279

map tiles — V2·3 Google Maps p. 279

MapReduce — V2·6 Ad Click Event Aggregation p. 300

mark_safe — Django in Production: Performance, Security, Async p. 185

market data — V2·13 Stock Exchange p. 349

Marriott — V2·7 Hotel Reservation System p. 307

matching engine — V2·13 Stock Exchange p. 349

max-heap — Python Data Structures Map p. 53

MD5 — V1·8 URL Shortener p. 222

memoization — Pattern Recognition Map p. 68

memory — Generators & Iterators p. 26
merge intervals — Pattern Recognition Map p. 68
Merkle tree — V1·6 Key-Value Store p. 211
message broker — Kafka vs RabbitMQ p. 96
message ID — V1·12 Chat System p. 242
message queue — Kafka vs RabbitMQ p. 96, V1·10 Notification System p. 232, V2·4 Distributed Message Queue p. 285
message sync queue — V1·12 Chat System p. 242
messenger — V1·12 Chat System p. 242
Meta indexes — Django ORM in Depth p. 158
metadata database — V1·15 Google Drive p. 259, V2·8 Distributed Email Service p. 313
metadata store — V2·9 S3-like Object Storage p. 321
metrics collector — V2·5 Metrics Monitoring and Alerting p. 293
metrics monitoring — V2·5 Metrics Monitoring and Alerting p. 293
middleware — Request Lifecycle, Middleware & Signals p. 166
MIDDLEWARE order — Request Lifecycle, Middleware & Signals p. 166
middleware questions — Django Interview Questions p. 152
migrate — Transactions & Migrations p. 172
migration dependencies — Transactions & Migrations p. 172
min-heap — Python Data Structures Map p. 53
mixin — OOP Internals & Memory Model p. 42
mmap — V2·12 Digital Wallet p. 342, V2·13 Stock Exchange p. 349
ModelSerializer — Django REST Framework p. 178
ModelViewSet — Django REST Framework p. 178
modulo hashing — V1·5 Consistent Hashing p. 207
modulo negative — Python Gotchas (Predict the Output) p. 36
MongoDB — SQL vs NoSQL & Scaling Databases p. 126
monotonic stack — Pattern Recognition Map p. 68
MPEG-DASH — V1·14 YouTube p. 253
MRO — Class vs Instance Attributes & Methods p. 19, OOP Internals & Memory Model p. 42
multi-device sync — V1·12 Chat System p. 242
multi-master replication — V1·7 Unique ID Generator p. 218
multicast — V2·13 Stock Exchange p. 349
multicolumn index — Indexes p. 111
multipart upload — V2·9 S3-like Object Storage p. 321
multiple inheritance — OOP Internals & Memory Model p. 42
multiplexing — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
multiprocessing — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
mutable default argument — Class vs Instance Attributes & Methods p. 19, Python Gotchas (Predict the Output) p. 36
MVCC — Transactions & Isolation Levels p. 116
MX record — V2·8 Distributed Email Service p. 313

## N

N+1 — Django ORM in Depth p. 158, Django REST Framework p. 178
N+1 query — SQL vs NoSQL & Scaling Databases p. 126
namedtuple — Python Data Structures Map p. 53
navigation — V2·3 Google Maps p. 279
nearby friends — V2·2 Nearby Friends p. 273
nearby search — V2·1 Proximity Service p. 266
Neo4j — SQL vs NoSQL & Scaling Databases p. 126
nested atomic — Transactions & Migrations p. 172
nested serializer — Django REST Framework p. 178
Netflix — V1·14 YouTube p. 253
__new__ — OOP Internals & Memory Model p. 42
news feed — The 4-Step Framework (Alex Xu, with a News Feed Example) p. 100, V1·11 News Feed System p. 237
news feed cache — V1·11 News Feed System p. 237
next greater element — Pattern Recognition Map p. 68
__next__ — Generators & Iterators p. 26
nil channel — Go Essentials & Gotchas p. 143
nil interface — Go Essentials & Gotchas p. 143

no_key — Transactions & Migrations p. 172
non-functional requirements — How to Run a System Design Interview p. 92
non-repeatable read — Transactions & Isolation Levels p. 116
nonlocal — Closures, Decorators & Context Managers p. 31
normalization — SQL vs NoSQL & Scaling Databases p. 126
NoSQL — SQL vs NoSQL & Scaling Databases p. 126
notification system — V1·10 Notification System p. 232
notification template — V1·10 Notification System p. 232
NOWAIT — Locking & Concurrency Control p. 121
nowait — Transactions & Migrations p. 172
NTP — V1·7 Unique ID Generator p. 218

## O

object storage — V2·9 S3-like Object Storage p. 321
observability — V2·5 Metrics Monitoring and Alerting p. 293
of — Transactions & Migrations p. 172
offset — Kafka vs RabbitMQ p. 96, V2·4 Distributed Message Queue p. 285
only — Django ORM in Depth p. 158
opt-out — V1·10 Notification System p. 232
optimisation — How to Solve a Coding Question p. 86
optimistic locking — Locking & Concurrency Control p. 121, V2·7 Hotel Reservation System p. 307
OPTIONS — Web & Networking Fundamentals p. 137
order book — V2·13 Stock Exchange p. 349
order manager — V2·13 Stock Exchange p. 349
OrderedDict — Python Data Structures Map p. 53
ordering — Kafka vs RabbitMQ p. 96
OrderingFilter — Django REST Framework p. 178
ORM — SQL vs NoSQL & Scaling Databases p. 126
ORM questions — Django Interview Questions p. 152
OuterRef — Django ORM in Depth p. 158
Outlook — V2·8 Distributed Email Service p. 313
overbooking — V2·7 Hotel Reservation System p. 307

## P

p99 latency — V2·13 Stock Exchange p. 349
PageRank — V1·9 Web Crawler p. 227
pagination — Django REST Framework p. 178
partial index — Indexes p. 111
partition — Kafka vs RabbitMQ p. 96, V2·4 Distributed Message Queue p. 285
partitioning — SQL vs NoSQL & Scaling Databases p. 126, V1·5 Consistent Hashing p. 207
pass by object reference — OOP Internals & Memory Model p. 42
path converters — Request Lifecycle, Middleware & Signals p. 166
pattern recognition — Pattern Recognition Map p. 68
pay-in — V2·11 Payment System p. 335
pay-out — V2·11 Payment System p. 335
payment system — V2·11 Payment System p. 335
PCI DSS — V2·11 Payment System p. 335
PEP 703 — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
percentile — V2·10 Real-time Gaming Leaderboard p. 329
permissions — Django REST Framework p. 178
persistent connection — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
pessimistic locking — Locking & Concurrency Control p. 121, V2·7 Hotel Reservation System p. 307
pg_advisory_xact_lock — Locking & Concurrency Control p. 121
pg_trgm — Indexes p. 111
pg_try_advisory_lock — Locking & Concurrency Control p. 121
PgBouncer — SQL vs NoSQL & Scaling Databases p. 126

pgbouncer — Django in Production: Performance, Security, Async p. 185
phantom read — Transactions & Isolation Levels p. 116
pipelining — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
placement service — V2·9 S3-like Object Storage p. 321
politeness — V1·9 Web Crawler p. 227
POP — V2·8 Distributed Email Service p. 313
post_save — Request Lifecycle, Middleware & Signals p. 166
pre-signed URL — V1·14 YouTube p. 253
pre_save — Request Lifecycle, Middleware & Signals p. 166
Prefetch — Django ORM in Depth p. 158
prefetch_related — SQL vs NoSQL & Scaling Databases p. 126, Django ORM in Depth p. 158
prefix sum — Pattern Recognition Map p. 68
prefix tree — V1·13 Search Autocomplete p. 248
preflight — Web & Networking Fundamentals p. 137
presence — V1·12 Chat System p. 242
primary — SQL vs NoSQL & Scaling Databases p. 126
priority queue — Python Data Structures Map p. 53, Kafka vs RabbitMQ p. 96
problem solving — How to Solve a Coding Question p. 86
process_exception — Request Lifecycle, Middleware & Signals p. 166
process_template_response — Request Lifecycle, Middleware & Signals p. 166
process_view — Request Lifecycle, Middleware & Signals p. 166
ProcessPoolExecutor — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
Prometheus — V2·5 Metrics Monitoring and Alerting p. 293
Promise.all — Async / Await p. 16
property — OOP Internals & Memory Model p. 42
Protobuf — Web & Networking Fundamentals p. 137
Protocol — OOP Internals & Memory Model p. 42
proximity service — V2·1 Proximity Service p. 266
PSP — V2·11 Payment System p. 335
PSUBSCRIBE — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105
pub/sub — Kafka vs RabbitMQ p. 96
PUBLISH — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105
pull vs push — V2·4 Distributed Message Queue p. 285, V2·5 Metrics Monitoring and Alerting p. 293
push notification — V1·10 Notification System p. 232, V1·12 Chat System p. 242
push vs pull — V1·11 News Feed System p. 237
put get — V1·6 Key-Value Store p. 211
PUT vs PATCH — Web & Networking Fundamentals p. 137
pytest-django — Django in Production: Performance, Security, Async p. 185
python3.13t — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21

## Q

Q object — Django ORM in Depth p. 158
QPACK — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
QPS — How to Run a System Design Interview p. 92
quadtree — V2·1 Proximity Service p. 266
query service — V1·13 Search Autocomplete p. 248
QuerySet — Django ORM in Depth p. 158
queryset cache — Django ORM in Depth p. 158
queue — Python Data Structures Map p. 53
queue.Queue — Python Data Structures Map p. 53
QUIC — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
quorum — V1·6 Key-Value Store p. 211
quorum queue — Kafka vs RabbitMQ p. 96

## R

RabbitMQ — Kafka vs RabbitMQ p. 96

RabbitMQ streams — Kafka vs RabbitMQ p. 96

race condition — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21, Locking & Concurrency Control p. 121, V1·4 Rate Limiter p. 201, V2·7 Hotel Reservation System p. 307

race detector — Go Essentials & Gotchas p. 143

Raft — V2·12 Digital Wallet p. 342, V2·13 Stock Exchange p. 349

ranking — V2·10 Real-time Gaming Leaderboard p. 329

rate limiter — V1·4 Rate Limiter p. 201

rate limiting — V1·10 Notification System p. 232

raw SQL — Django ORM in Depth p. 158

RawSQL — Django ORM in Depth p. 158, Django in Production: Performance, Security, Async p. 185

Read Committed — Transactions & Isolation Levels p. 116

read replica — SQL vs NoSQL & Scaling Databases p. 126

read replicas — V2·1 Proximity Service p. 266

Read Uncommitted — Transactions & Isolation Levels p. 116

read unread — V2·8 Distributed Email Service p. 313

read-heavy — V1·8 URL Shortener p. 222

read-your-writes — SQL vs NoSQL & Scaling Databases p. 126

real-time bidding — V2·6 Ad Click Event Aggregation p. 300

rebalancing — V2·4 Distributed Message Queue p. 285

reconciliation — V2·6 Ad Click Event Aggregation p. 300, V2·11 Payment System p. 335

recursion stack — Big-O Cheat Sheet p. 80

recursive resolver — Web & Networking Fundamentals p. 137

redirect — V1·8 URL Shortener p. 222

Redis — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105, SQL vs NoSQL & Scaling Databases p. 126, V1·4 Rate Limiter p. 201, V2·10 Real-time Gaming Leaderboard p. 329

Redis broker — Celery with Redis as Broker p. 107

Redis cache — V2·1 Proximity Service p. 266, V2·7 Hotel Reservation System p. 307

Redis cluster — V2·10 Real-time Gaming Leaderboard p. 329

Redis List queue — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105

Redis Pub/Sub — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105

Redis pub/sub — V2·2 Nearby Friends p. 273

Redis Streams — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105

RedisCache — Django in Production: Performance, Security, Async p. 185

reference counting — OOP Internals & Memory Model p. 42

reference cycle — OOP Internals & Memory Model p. 42

rehashing problem — V1·5 Consistent Hashing p. 207

relational — SQL vs NoSQL & Scaling Databases p. 126

Repeatable Read — Transactions & Isolation Levels p. 116

replay — Kafka vs RabbitMQ p. 96

replica — SQL vs NoSQL & Scaling Databases p. 126

replication — SQL vs NoSQL & Scaling Databases p. 126, V2·9 S3-like Object Storage p. 321

replication lag — SQL vs NoSQL & Scaling Databases p. 126

reply_to — Kafka vs RabbitMQ p. 96

__repr__ — OOP Internals & Memory Model p. 42

reproducibility — V2·12 Digital Wallet p. 342

request lifecycle — Request Lifecycle, Middleware & Signals p. 166

request/reply — Kafka vs RabbitMQ p. 96

request_finished — Request Lifecycle, Middleware & Signals p. 166

requirements — How to Run a System Design Interview p. 92

rerouting — V2·3 Google Maps p. 279

resharding — SQL vs NoSQL & Scaling Databases p. 126

REST — Web & Networking Fundamentals p. 137

result backend — Celery with Redis as Broker p. 107
resumable upload — V1·14 YouTube p. 253, V1·15 Google Drive p. 259
retry — Transactions & Isolation Levels p. 116, V1·10 Notification System p. 232
retry queue — V2·11 Payment System p. 335
RETURNING — Locking & Concurrency Control p. 121
robots.txt — V1·9 Web Crawler p. 227
RocksDB — V2·12 Digital Wallet p. 342
room type inventory — V2·7 Hotel Reservation System p. 307
root server — Web & Networking Fundamentals p. 137
round half to even — Python Gotchas (Predict the Output) p. 36
router — Django REST Framework p. 178
routing key — Kafka vs RabbitMQ p. 96
routing tiles — V2·3 Google Maps p. 279
RPC — Kafka vs RabbitMQ p. 96
RTB — V2·6 Ad Click Event Aggregation p. 300
run_in_executor — Async / Await p. 16
RunPython — Transactions & Migrations p. 172
runtime_checkable — OOP Internals & Memory Model p. 42

## S

S3 — V1·15 Google Drive p. 259, V2·8 Distributed Email Service p. 313, V2·9 S3-like Object Storage p. 321
safe methods — Web & Networking Fundamentals p. 137
saga — V2·7 Hotel Reservation System p. 307, V2·12 Digital Wallet p. 342
same-origin policy — Web & Networking Fundamentals p. 137
savepoint — Transactions & Migrations p. 172
scatter-gather — V2·10 Real-time Gaming Leaderboard p. 329
scheduling — Pattern Recognition Map p. 68
ScopedRateThrottle — Django REST Framework p. 178
ScyllaDB — SQL vs NoSQL & Scaling Databases p. 126
search autocomplete — V1·13 Search Autocomplete p. 248
SearchFilter — Django REST Framework p. 178
SECRET_KEY — Django in Production: Performance, Security, Async p. 185
SECURE_PROXY_SSL_HEADER — Django in Production: Performance, Security, Async p. 185
SECURE_SSL_REDIRECT — Django in Production: Performance, Security, Async p. 185
segment — V2·4 Distributed Message Queue p. 285
select — Go Essentials & Gotchas p. 143
SELECT FOR UPDATE — Locking & Concurrency Control p. 121, V2·7 Hotel Reservation System p. 307
select_for_update — Transactions & Migrations p. 172
select_related — SQL vs NoSQL & Scaling Databases p. 126, Django ORM in Depth p. 158
selectinload — SQL vs NoSQL & Scaling Databases p. 126
selectivity — Indexes p. 111
send_robust — Request Lifecycle, Middleware & Signals p. 166
Sentry — Django in Production: Performance, Security, Async p. 185
SeparateDatabaseAndState — Transactions & Migrations p. 172
Seq Scan — Indexes p. 111
sequence number — V1·7 Unique ID Generator p. 218
sequencer — V2·13 Stock Exchange p. 349
Serializable — Transactions & Isolation Levels p. 116
serialization failure — Transactions & Isolation Levels p. 116
Serializer — Django REST Framework p. 178
SerializerMethodField — Django REST Framework p. 178
server push — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
Server-Sent Events — Web & Networking Fundamentals p. 137
service discovery — V1·12 Chat System p. 242, V2·2 Nearby Friends p. 273, V2·5 Metrics Monitoring and Alerting p. 293
service layer — Request Lifecycle, Middleware & Signals p. 166

session — Web & Networking Fundamentals p. 137
set — Python Data Structures Map p. 53
set cost — Big-O Cheat Sheet p. 80
setter — OOP Internals & Memory Model p. 42
settlement file — V2·11 Payment System p. 335
setUpTestData — Django in Production: Performance, Security, Async p. 185
SHA-1 — V1·5 Consistent Hashing p. 207, V1·8 URL Shortener p. 222
shadowing — Class vs Instance Attributes & Methods p. 19
shallow copy — Python Gotchas (Predict the Output) p. 36
shard key — SQL vs NoSQL & Scaling Databases p. 126
shard map manager — V1·13 Search Autocomplete p. 248
sharding — SQL vs NoSQL & Scaling Databases p. 126, V1·5 Consistent Hashing p. 207, V2·7 Hotel Reservation System p. 307, V2·9 S3-like Object Storage p. 321, V2·10 Real-time Gaming Leaderboard p. 329
short URL — V1·8 URL Shortener p. 222
signals — Request Lifecycle, Middleware & Signals p. 166
singleton — OOP Internals & Memory Model p. 42
skip list — V2·10 Real-time Gaming Leaderboard p. 329
SKIP LOCKED — Locking & Concurrency Control p. 121
skip_locked — Transactions & Migrations p. 172
slice append — Go Essentials & Gotchas p. 143
sliding window — Pattern Recognition Map p. 68, V2·6 Ad Click Event Aggregation p. 300
sliding window counter — V1·4 Rate Limiter p. 201
sliding window log — V1·4 Rate Limiter p. 201
sloppy quorum — V1·6 Key-Value Store p. 211
__slots__ — OOP Internals & Memory Model p. 42
small int cache — Python Gotchas (Predict the Output) p. 36
SMS — V1·10 Notification System p. 232
SMTP — V2·8 Distributed Email Service p. 313
snapshot — V2·12 Digital Wallet p. 342
snapshot isolation — Transactions & Isolation Levels p. 116
SNI — Web & Networking Fundamentals p. 137
soft real-time — V1·10 Notification System p. 232
sort complexity — Big-O Cheat Sheet p. 80
sortable ID — V1·7 Unique ID Generator p. 218
sorted set — V2·10 Real-time Gaming Leaderboard p. 329
sortedcontainers — Python Data Structures Map p. 53
space complexity — Big-O Cheat Sheet p. 80
SPF DKIM DMARC — V2·8 Distributed Email Service p. 313
spider — V1·9 Web Crawler p. 227
spider trap — V1·9 Web Crawler p. 227
SQL — SQL vs NoSQL & Scaling Databases p. 126
SQL injection — Django in Production: Performance, Security, Async p. 185
squashmigrations — Transactions & Migrations p. 172
SSE — Web & Networking Fundamentals p. 137
SSI — Transactions & Isolation Levels p. 116
SSTable — V1·6 Key-Value Store p. 211
stack — Python Data Structures Map p. 53
stacked decorators — Closures, Decorators & Context Managers p. 31
star schema — V2·6 Ad Click Event Aggregation p. 300
stateful servers — V2·2 Nearby Friends p. 273
staticmethod — Class vs Instance Attributes & Methods p. 19
stock exchange — V2·13 Stock Exchange p. 349
StopIteration — Generators & Iterators p. 26
STORAGES — Django in Production: Performance, Security, Async p. 185
__str__ — OOP Internals & Memory Model p. 42
stream — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
stream processing — V2·6 Ad Click Event Aggregation p. 300
string concatenation — Big-O Cheat Sheet p. 80
string immutability — Python Gotchas (Predict the Output) p. 36
Stripe — V2·11 Payment System p. 335
strong consistency — V1·15 Google Drive p. 259

structural typing — OOP Internals & Memory Model p. 42
structured logging — Django in Production: Performance, Security, Async p. 185
stuck — How to Solve a Coding Question p. 86
Subquery — Django ORM in Depth p. 158
SUBSCRIBE — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105
super() — OOP Internals & Memory Model p. 42
suppress exception — Closures, Decorators & Context Managers p. 31
SYN — Web & Networking Fundamentals p. 137
sync conflict — V1·15 Google Drive p. 259
sync.Map — Go Essentials & Gotchas p. 143
sync.Mutex — Go Essentials & Gotchas p. 143
sync.Once — Go Essentials & Gotchas p. 143
sync.RWMutex — Go Essentials & Gotchas p. 143
sync.WaitGroup — Go Essentials & Gotchas p. 143
sync_to_async — Django in Production: Performance, Security, Async p. 185
synchronous replication — SQL vs NoSQL & Scaling Databases p. 126
SynchronousOnlyOperation — Django in Production: Performance, Security, Async p. 185
system design framework — How to Run a System Design Interview p. 92, The 4-Step Framework (Alex Xu, with a News Feed Example) p. 100

## T

task priority — Celery with Redis as Broker p. 107
task_acks_late — Celery with Redis as Broker p. 107
TC/C — V2·12 Digital Wallet p. 342
TCP — Web & Networking Fundamentals p. 137
tee — Generators & Iterators p. 26
template fragment cache — Django in Production: Performance, Security, Async p. 185
TestCase — Django in Production: Performance, Security, Async p. 185
testing — How to Solve a Coding Question p. 86
think out loud — How to Solve a Coding Question p. 86
threading — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
threading.Lock — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
ThreadPoolExecutor — Concurrency Models (GIL, Threads, Processes, asyncio) p. 21
three-way handshake — Web & Networking Fundamentals p. 137
throttling — Django REST Framework p. 178, V1·4 Rate Limiter p. 201
ticket server — V1·7 Unique ID Generator p. 218
time allocation — The 4-Step Framework (Alex Xu, with a News Feed Example) p. 100
time complexity — Big-O Cheat Sheet p. 80
time-series database — V2·5 Metrics Monitoring and Alerting p. 293
time-space tradeoff — Two Sum / Complement Lookup p. 84
time.After — Go Essentials & Gotchas p. 143
TIME_WAIT — Web & Networking Fundamentals p. 137
timeline — V1·11 News Feed System p. 237
timestamp bits — V1·7 Unique ID Generator p. 218
TinyURL — V1·8 URL Shortener p. 222
TLD — Web & Networking Fundamentals p. 137
TLS 1.3 — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133, Web & Networking Fundamentals p. 137
TLS handshake — Web & Networking Fundamentals p. 137
TLS termination — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133
to_attr — Django ORM in Depth p. 158
to_internal_value — Django REST Framework p. 178
to_representation — Django REST Framework p. 178
token bucket — V1·4 Rate Limiter p. 201
top k — V1·13 Search Autocomplete p. 248
top K — V2·10 Real-time Gaming Leaderboard p. 329
top N — V2·6 Ad Click Event Aggregation p. 300
top-k — Python Data Structures Map p. 53, Pattern Recognition Map p. 68
topic — V2·4 Distributed Message Queue p. 285

topic exchange — Kafka vs RabbitMQ p. 96
topological sort — Pattern Recognition Map p. 68
trade-offs — How to Run a System Design Interview p. 92
transaction — Transactions & Isolation Levels p. 116
transaction questions — Django Interview Questions p. 152
transaction.atomic — Transactions & Migrations p. 172
transaction.on_commit — Transactions & Migrations p. 172
TransactionManagementError — Transactions & Migrations p. 172
TransactionTestCase — Django in Production: Performance, Security, Async p. 185
transcoding — V1·14 YouTube p. 253
trending queries — V1·13 Search Autocomplete p. 248
trie — V1·13 Search Autocomplete p. 248
trie cache — V1·13 Search Autocomplete p. 248
try finally return — Python Gotchas (Predict the Output) p. 36
try-confirm-cancel — V2·12 Digital Wallet p. 342
TTL — Kafka vs RabbitMQ p. 96, Web & Networking Fundamentals p. 137, V2·2 Nearby Friends p. 273
tumbling window — V2·6 Ad Click Event Aggregation p. 300
tuple += — Python Gotchas (Predict the Output) p. 36
Twitter snowflake — V1·7 Unique ID Generator p. 218
Twitter timeline — V1·11 News Feed System p. 237
two pointers — Pattern Recognition Map p. 68, Two Sum / Complement Lookup p. 84
two sum — Two Sum / Complement Lookup p. 84
two-phase commit — V2·12 Digital Wallet p. 342
typeahead — V1·13 Search Autocomplete p. 248
typed nil — Go Essentials & Gotchas p. 143
typing.Protocol — OOP Internals & Memory Model p. 42

## U

UDP — HTTP/1.1 vs HTTP/2 vs HTTP/3 p. 133, Web & Networking Fundamentals p. 137
unacked — Celery with Redis as Broker p. 107
unbuffered channel — Go Essentials & Gotchas p. 143
unhashable — OOP Internals & Memory Model p. 42
union-find — Python Data Structures Map p. 53, Pattern Recognition Map p. 68
unique ID generator — V1·7 Unique ID Generator p. 218, V1·8 URL Shortener p. 222
UniqueConstraint — Django ORM in Depth p. 158
update vs save — Django ORM in Depth p. 158
update_or_create — Django ORM in Depth p. 158
URL frontier — V1·9 Web Crawler p. 227
URL resolver — Request Lifecycle, Middleware & Signals p. 166
URL seen — V1·9 Web Crawler p. 227
URL shortener — V1·8 URL Shortener p. 222
urlconf — Request Lifecycle, Middleware & Signals p. 166
use case — The 4-Step Framework (Alex Xu, with a News Feed Example) p. 100
UUID — V1·7 Unique ID Generator p. 218
uvicorn — Request Lifecycle, Middleware & Signals p. 166
uvicorn worker — Django in Production: Performance, Security, Async p. 185

## V

VACUUM — Transactions & Isolation Levels p. 116
validate — Django REST Framework p. 178
validate_field — Django REST Framework p. 178
validation order — Django REST Framework p. 178
values — Django ORM in Depth p. 158
values_list — Django ORM in Depth p. 158
vector clock — V1·6 Key-Value Store p. 211
vector tiles — V2·3 Google Maps p. 279
version column — Locking & Concurrency Control p. 121
versioning — Django REST Framework p. 178, V2·9 S3-like Object Storage p. 321

video streaming — V1·14 YouTube p. 253
video upload — V1·14 YouTube p. 253
ViewSet — Django REST Framework p. 178
virtual nodes — V1·5 Consistent Hashing p. 207
visibility map — Indexes p. 111
visibility_timeout — Celery with Redis as Broker p. 107
vnodes — V1·5 Consistent Hashing p. 207

## W

WAL — Transactions & Isolation Levels p. 116
wallet — Locking & Concurrency Control p. 121
watermark — V2·6 Ad Click Event Aggregation p. 300
web crawler — V1·9 Web Crawler p. 227
Web Mercator — V2·3 Google Maps p. 279
webhook — V2·11 Payment System p. 335
webmail — V2·8 Distributed Email Service p. 313
WebSocket — Web & Networking Fundamentals p. 137, V1·12 Chat System p. 242, V2·2 Nearby Friends p. 273, V2·8 Distributed Email Service p. 313
WebSocket fan-out — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105
what happens when you type a URL — Web & Networking Fundamentals p. 137
what's wrong with this code — Django Interview Questions p. 152
WhatsApp — V1·12 Chat System p. 242
WhiteNoise — Django in Production: Performance, Security, Async p. 185
wide-column — SQL vs NoSQL & Scaling Databases p. 126
with — Closures, Decorators & Context Managers p. 31
worker_prefetch_multiplier — Celery with Redis as Broker p. 107
workers — V1·10 Notification System p. 232
wrap up — The 4-Step Framework (Alex Xu, with a News Feed Example) p. 100
writable nested — Django REST Framework p. 178
write amplification — Indexes p. 111
write sharding — V2·10 Real-time Gaming Leaderboard p. 329
write skew — Transactions & Isolation Levels p. 116
write-ahead log — V2·4 Distributed Message Queue p. 285
WSGI — Request Lifecycle, Middleware & Signals p. 166

## X

X_FRAME_OPTIONS — Django in Production: Performance, Security, Async p. 185
XACK — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105
XADD — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105
XAUTOCLAIM — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105
xmax — Transactions & Isolation Levels p. 116
xmin — Transactions & Isolation Levels p. 116
XREADGROUP — Redis Pub/Sub vs RabbitMQ vs Kafka p. 105
XSS — Django in Production: Performance, Security, Async p. 185

## Y

Yelp — V2·1 Proximity Service p. 266
yield — Generators & Iterators p. 26
yield from — Generators & Iterators p. 26
YouTube — V1·14 YouTube p. 253

## Z

zero-downtime migration — Transactions & Migrations p. 172

ZINCRBY — V2·10 Real-time Gaming Leaderboard p. 329

ZooKeeper — V1·12 Chat System p. 242, V2·2 Nearby Friends p. 273

zoom levels — V2·3 Google Maps p. 279

ZREVRANGE — V2·10 Real-time Gaming Leaderboard p. 329

ZREVRANK — V2·10 Real-time Gaming Leaderboard p. 329